Análisis y Diseño de Algoritmos

Dr. José Martínez Carranza

Otoño 2026 - Bryan Ezequiel Violante Arriaga

Índice

1. Introducción al Curso **3**

2. Funciones asintóticas **7**

3. Continuación de funciones asintóticas **15**

4. Anexo: caracterización por límites **21**

5. Recurrencias **24**

6. Métodos para resolver recurrencias **26**

7. Heaps **33**

8. Mochila fraccional, actividades y divide y vencerás **38**

9. Multiplicación de enteros grandes **42**

10. Programación dinámica **46**

11. Grafos **55**

12. Teoría de grafos: terminología, conexidad y Euler **67**

13. Hamilton, bipartitos, cliques y árboles **77**

14. Representaciones, BFS y DFS a fondo **89**

15. Aplicaciones de DFS **103**

16. Árboles de expansión mínima **115**

17. Caminos más cortos desde una fuente **124**

18. Demostraciones y problemas de caminos más cortos **137**

19. Caminos más cortos entre todos los pares **148**

20. Flujo máximo: Ford-Fulkerson y Edmonds-Karp **160**

21. Emparejamiento bipartito máximo **172**

22.Empujar y reetiquetar **181**

1. Introducción al Curso

18 de agosto de 2026

Resumen rápido. Sesión inicial: temario, criterios de evaluación y datos de contacto. El profesor comparó bubble sort contra quick sort a partir del video de Obama y de ahí saltó a por qué sigue valiendo la pena estudiar estructuras de datos y algoritmos en la era de la IA. Cerró con computable, decidible e indecidible, y con P contra NP.

Info general

Temario:

1. Introducción
2. Notación asintótica
3. Recurrencias
4. Ordenamiento
5. Algoritmos voraces
6. Divide y vencerás
7. Programación dinámica
8. *(falta anotar este tema)*
9. Algoritmos aleatorizados
10. Teoría de complejidad

Criterios de evaluación:

- 90 % exámenes, tres parciales de 30 % cada uno
- 10 % asistencia

Contacto y material:

- Sitio del curso: `http://ccc.inaoep.mx/~carranza/alg.html`
- Correo: `carranza@inaoep.mx`

Ojito...

Todo correo al profesor lleva `ALG2026` en el asunto. Si no, se pierde en el SPAM.

El objetivo del curso es desarrollar pensamiento computacional para resolver problemas. No se trata tanto de programar sino de analizar y diseñar algoritmos eficientes.

Ordenamiento (por arribita)

A raíz del vídeo de Obama, en el cual le preguntan cómo ordenar un millón de enteros, se da una embarradita de algoritmos de ordenamiento. Son los dos ejemplos con los que el profesor mostró que dos algoritmos correctos para el mismo problema no cuestan lo mismo.

Bubble sort

La idea es ir burbujeando los elementos. Se recorre el arreglo comparando pares vecinos y se intercambian cuando están en desorden, así que en cada pasada el mayor de los que quedan sube hasta su posición final. Se caracteriza por usar 2 ciclos anidados, lo que denota su complejidad cuadrática.

Mejor caso, caso promedio y peor caso son los tres $\mathcal{O}(n^2)$. La versión simple recorre los dos ciclos anidados completos sin importar cómo venga el arreglo, así que un arreglo ya ordenado le cuesta lo mismo que uno al revés.

Quick sort

Parte el problema. Elige un pivote, acomoda a la izquierda lo menor y a la derecha lo mayor, y se llama a sí mismo sobre cada mitad. Como las dos mitades ya no dependen una de la otra, el algoritmo es paralelizable.

Algoritmo	**Mejor**	**Promedio**	**Peor**	**Memoria**
Bubble sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$
Quick sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$

Tabla 1: Costo de los dos algoritmos vistos en clase.

Idea clave

Dos algoritmos igual de correctos para el mismo problema pueden diferir en órdenes de magnitud. Escoger el algoritmo es una decisión de diseño, no un detalle de implementación.

Por qué estudiar DSA

Saber como funcionan los algoritos y como usar las estructuras de datos nos permite

- Romper problemas en pasos pequeños.
- Elegir la estructura de datos correcta.
- Analizar la complejidad en tiempo y en memoria, porque siempre se busca la solución óptima.
- Comparar diseños y sus trade-offs.

Se espera que al terminar el curso uno pueda determinar la estrategia correcta para resolver un problema y dar soluciones óptimas, escalables y eficientes en tiempo y espacio.

Conceptos clave

- **Algoritmo:** secuencia finita y no ambigua de pasos que, a partir de una entrada, produce una salida en un número finito de operaciones.
- **Problema computable:** el problema para el que sí se puede estructurar un programa que obtenga un resultado. Saber cuál es el último número natural no lo es: no hay tal número, así que no hay programa que lo devuelva.
- **Problema decidible:** existe un algoritmo que, para cualquier entrada, se detiene y entrega una respuesta.
- **Problema indecidible:** no existe tal algoritmo. Ninguna solución general lo resuelve.

Verificación

Si desarrollo un algoritmo que va a resolver un problema, la pregunta inmediata es cómo sé que esa solución es válida. Esa es la verificación, y es un paso aparte de escribir el código.

DSA en tiempos de la IA

Conocer estructuras de datos y algoritmos sirve para analizar el código que la IA genera. El vibe coding sirve para prototipar rápido, pero *abstraction eventually leaks*: tarde o temprano hay que bajar al detalle que la abstracción escondía. Los dos sirven mejor en conjunto.

Ojito...

El código generado siempre lo juzga un humano, para detectar áreas de oportunidad y cosas por optimizar. Sin ese criterio no hay forma de saber si lo que salió es bueno.

Cómo convertirse en experto en DSA

Antes era a la antigua: tirar código hasta que la intuición apareciera sola y uno supiera qué estructura de datos usar en cada caso. Eso ya cambió. Ahora se aprende la teoría primero, se entienden los fundamentos, se estudian las matemáticas detrás de DSA y luego se practica con ejemplos.

El ciclo de trabajo que espera el profesor:

1. Intentarlo por mi cuenta.
2. Pedir una pista, no la solución.

3. Implementar y probar.
4. Preguntarle a la IA por mi razonamiento.
5. Explicarle la solución a la IA.
6. Resolver un problema similar sin IA.

Idea clave

La práctica hace al maestro. La IA entra como quien revisa, no como quien resuelve.

El santo grial: P contra NP

P es la clase de los problemas que se resuelven en tiempo polinomial. NP es la de los problemas cuya solución propuesta se *verifica* en tiempo polinomial, aunque encontrarla parezca costar tiempo exponencial. Todo problema de P está en NP, porque resolverlo ya es una forma de verificarlo. La pregunta abierta es la contraria: si algo se verifica rápido, ¿también se resuelve rápido?

Demostrar $P = NP$ implicaría que miles de problemas que hoy se atacan por aproximación tienen algoritmo eficiente y nadie lo ha encontrado. Demostrar $P \neq NP$ cerraría la puerta de una vez. Nadie ha hecho ninguna de las dos.

La IA agentica vamos para allá... (La IA que resuelve problemas por sí misma, es decir, puede fijar metas, planificar y ejecutar tareas de forma humana para alcanzar un objetivo específico, con poca supervisión humana) pero eso nos podría hacer tratar todo como una caja negra... pero eso no es lo ideal. Siempre queremos entender cómo funcionan las cosas, y el estudio de DSA nos da esas herramientas.

2. Funciones asintóticas

20 de agosto de 2026

Resumen rápido. En esta clase vimos una intro a la notación asintótica y a las funciones de crecimiento, asi como un ejemplo/tecnica de como se puede analizar la complejidad de un algoritmo. Finalizamos la clase con un poco de histria, hablando sobre el Debate Lighthill de 1973, "The general purpose robot is a mirage"

Qué es una función?

Definición 2.1. Una función es una relación entre dos conjuntos, donde a cada elemento del primer conjunto (*dominio*) le corresponde un único elemento del segundo conjunto (*codominio*). conjunto.

COnceptos importantes:

- Dominio: conjunto de entrada, todos los valores de entrada permitidos, por ejemplo, los reales.
- Codominio: conjunto de llegada, donde viven las salidas.
- Imagen: conjunto de llegada que realmente se obtiene. SUbconjunto del codominio.

Ojito...

Rango y recorrido son sinónimos de imagen.

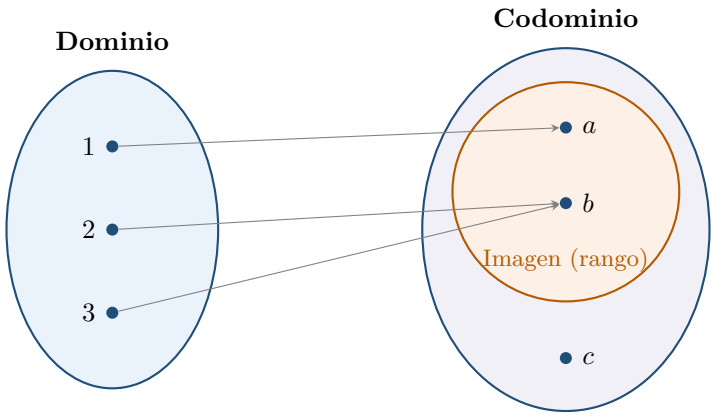


Figura 1: El elemento c pertenece al codominio pero no a la imagen.

Una funcion de los reales a los reales por ejemplo se denota con

$$f : \mathbb{R} \to \mathbb{R}$$

que basicamente dice que es una funcion que toma un real y devuelve un real. Si esta funcion se define como $f(x) = x^2$, entonces el dominio es $\mathbb{R}$, el codominio es $\mathbb{R}$ y la imagen es $\mathbb{R}_{\geq 0}$.

Analisis Asintotico

Supongamos que tenemos un arreglo con n elementos desordenados y queremos ordenarlos, como ya sabemos tenemos varias opciones que hacen la misma tarea, pero se comportaran diferente dependiendo del tamaño del arreglo.

Ordenamiento Burbuja

El arreglo va creciendo y como se observa en la grafica de abajo, el tiempo que tarda en ordenarse tambien

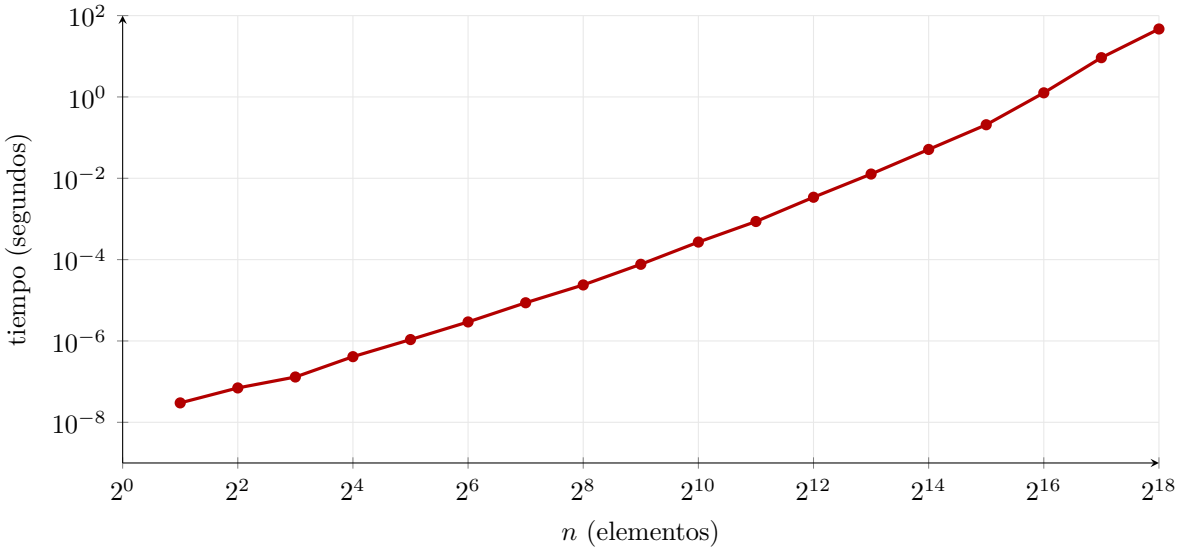


Figura 2: Tiempo de bubble sort sobre arreglos aleatorios de 2^1 a 2^{18} elementos.

Estos datos son discretos, no son continuos pero podemos aproximar una funcion, interpolar, usar alguna tecnica de algoritmos evolutivos o hasta con intuicion

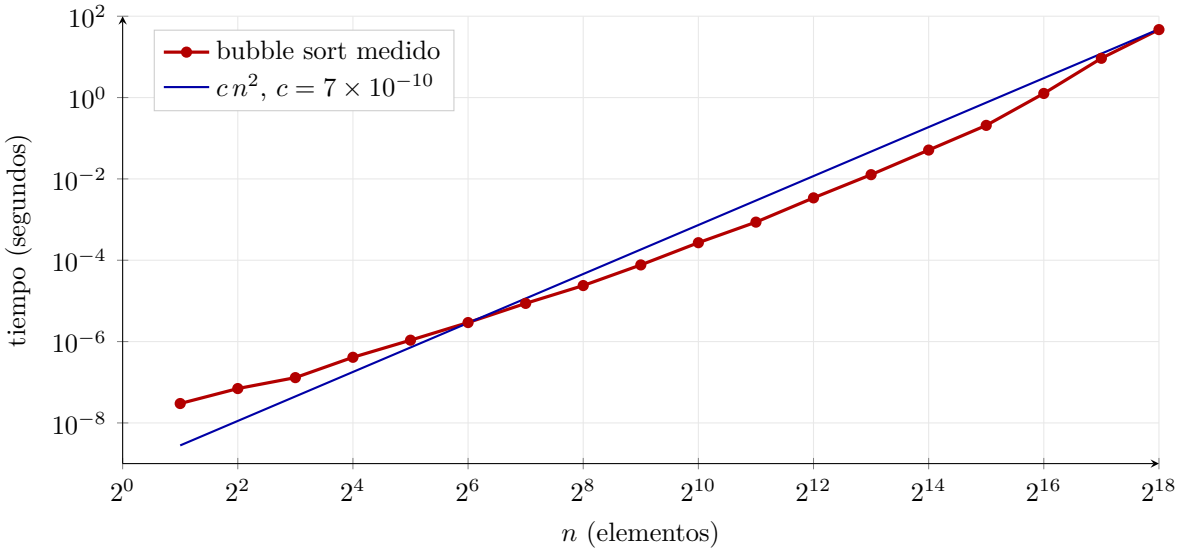


Figura 3: Tiempos medidos de bubble sort contra la curva $c\,n^2$.

Se puede ver que la curva $c\,n^2$ está encima de los puntos medidos, así que podemos decir que el tiempo de bubble sort es $\mathcal{O}(n^2)$.

Pendiente por resolver

La curva $c\,n^2$ es una cota superior, pero no es la mejor. ¿Cómo encontrar la mejor cota?... LO digo pq como que no acota todos los puntos y ademas se ve como una linea recta pero quiza sea por la escala
UPDATE: No toca los puntos por lo dicho abajo, para valores pequeños es irrelevante, el problema es que no se eligio un buen n_0 pero le pregunto al profe.

Quick sort

Ahora el mismo experimento con quick sort, los datos (resultados) se muestran a continuacion:

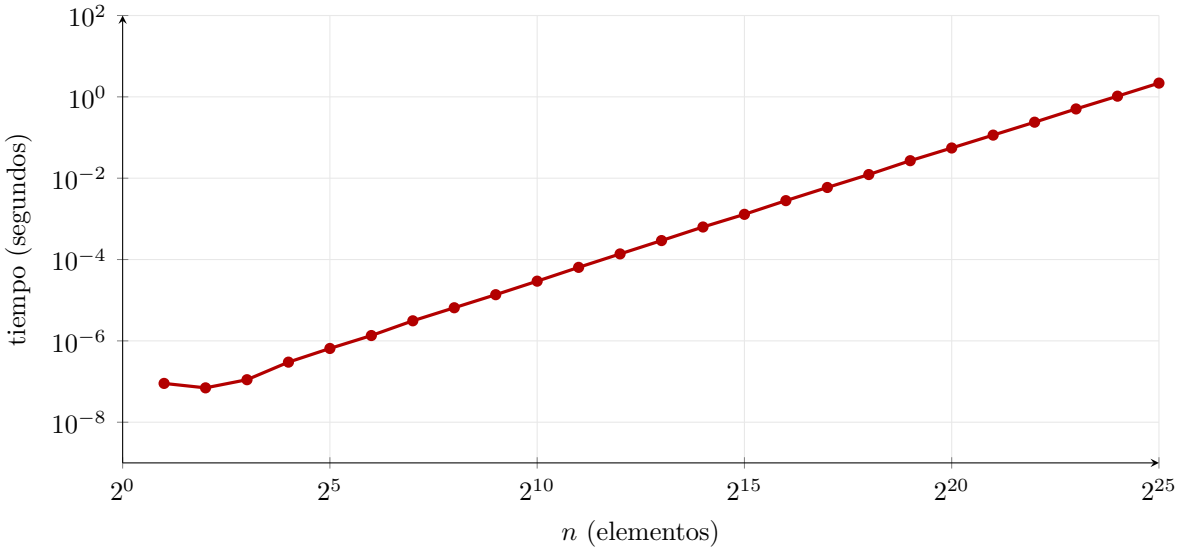


Figura 4: Tiempo de quick sort sobre arreglos aleatorios de 2^1 a 2^{25} elementos.

Al tratar de determianr la funcion analitica a ojo de buen cubero...

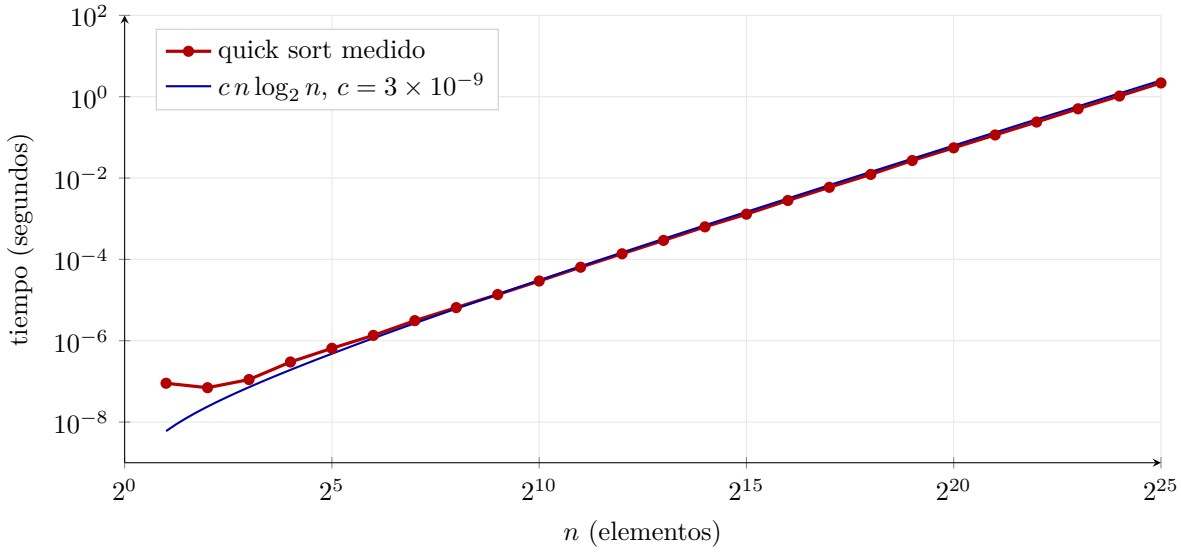


Figura 5: Tiempos medidos de quick sort contra la curva $c\,n\log_2 n$.

La curva $c\,n\log_2 n$ acota mas o menos bien los puntos medidos, asi que podemos decir que el tiempo de quick sort es $\mathcal{O}(n\log n)$.

Pendiente por resolver

Igual que en burbuja, por que no acota todos los puntos??
UPDATE: No toca los puntos por lo dicho abajo, para valores pequeños es irrelevante, el problema es que no se eligio un buen n_0 pero le pregunto al profe.

Brubuja vs Quick

Al comparar ambos algoritmos en la sig grafica...

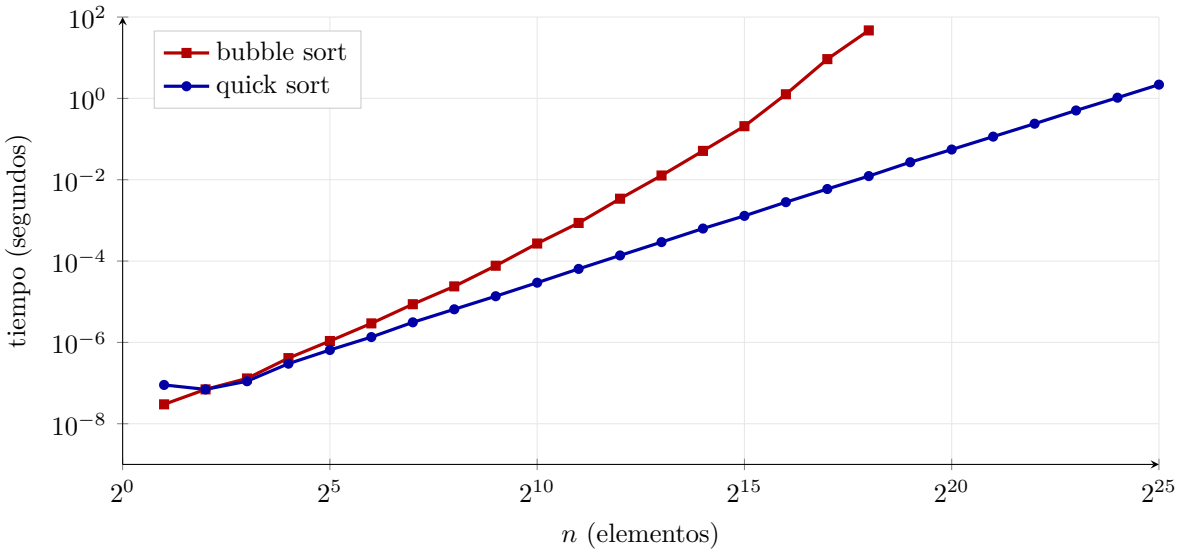


Figura 6: Bubble sort contra quick sort sobre los mismos arreglos. segundos.

Se puede ver que quick sort es mucho mas rapido que burbuja, y la diferencia se hace mas evidente a medida que el tamaño del arreglo crece. Por ejemplo, burbuja lo detuve en 2^{18} elementos porque ya tardaba demasiado, mientras que quick sort lo corrí hasta 2^{25} elementos y seguía siendo razonable. Entonces podemos concluir que quick sort es mucho mas eficiente que burbuja.

Ojito...

Se puede hacer esta comparacion ya que los arreglos que se usan para probar ambos algoritmos fueron los mismos y se testeo en la misma PC. El codigo se encuentra en https://github.com/zxxz456/inaoe/tree/main/ada/dummy/ComparaOrdenamientos.c

Notacion Big O

En los ejemplos de arriba mencione que el tiempo de bubble sort es $\mathcal{O}(n^2)$ y el tiempo de quick sort es $\mathcal{O}(n \log n)$.

Lo que quiere decir esto es que la funcion empirica encontrada del ALGORITMO de ordenamiento burbuja $f(n)$, pertenece al conjunto de funciones o a la familia de funciones $\mathcal{O}(n^2)$

Y analogamente la funcion empirica encontrada del ALGORITMO de ordenamiento quick sort $f(n)$, pertenece al conjunto de funciones o a la familia de funciones $\mathcal{O}(n \log n)$

De forma mas general, nuestra funcion empirica $f(n)$ la queremos modelar como una funcion continua $c \cdot g(n)$ y dada esta funcion $g(n)$ denotamos $\mathcal{O}(g(n))$ al conjunto de funciones tales que existen constntes positivas c y n_0 tales que para todo $n \geq n_0$ se cumple que $f(n) \leq c \cdot g(n)$

Definición 2.2. Sea $g(n)$ una función positiva. Decimos que una función $f(n)$ pertenece a $\mathcal{O}(g(n))$ si existen constantes positivas c y n_0 tales que

$$0 \leq f(n) \leq c \cdot g(n) \qquad \text{para todo } n \geq n_0.$$

Equivalentemente,

$$\mathcal{O}(g(n)) = \{\, f(n) : \exists\, c > 0,\ \exists\, n_0 > 0 \text{ tales que } 0 \leq f(n) \leq c \cdot g(n) \ \ \forall\, n \geq n_0 \,\}.$$

Y lo que dice basicamente es que $f(n)$ esta en el conjunto de funciones $\mathcal{O}(g(n))$ y que crece a lo mucho como esa $g(n)$

Ejemplo 2.3. Sean $f(n) = 2n^2 + 3n + 1$ y $g(n) = n^2$. Tomando $c = 6$ y $n_0 = 1$ se cumple

$$2n^2 + 3n + 1 \ \leq\ 6n^2 \iff 4n^2 - 3n - 1 \geq 0 \iff (4n+1)(n-1) \geq 0,$$

lo cual es cierto para todo $n \geq 1$. Por lo tanto $f(n) \in \mathcal{O}(n^2)$.

El par (c, n_0) no es único: con $c = 3$ la desigualdad $2n^2 + 3n + 1 \leq 3n^2$ equivale a $n^2 - 3n - 1 \geq 0$, que se satisface a partir de $n \geq \frac{3+\sqrt{13}}{2} \approx 3.30$, de modo que $n_0 = 4$ también funciona. La definición solo exige que *exista* al menos un par de constantes.

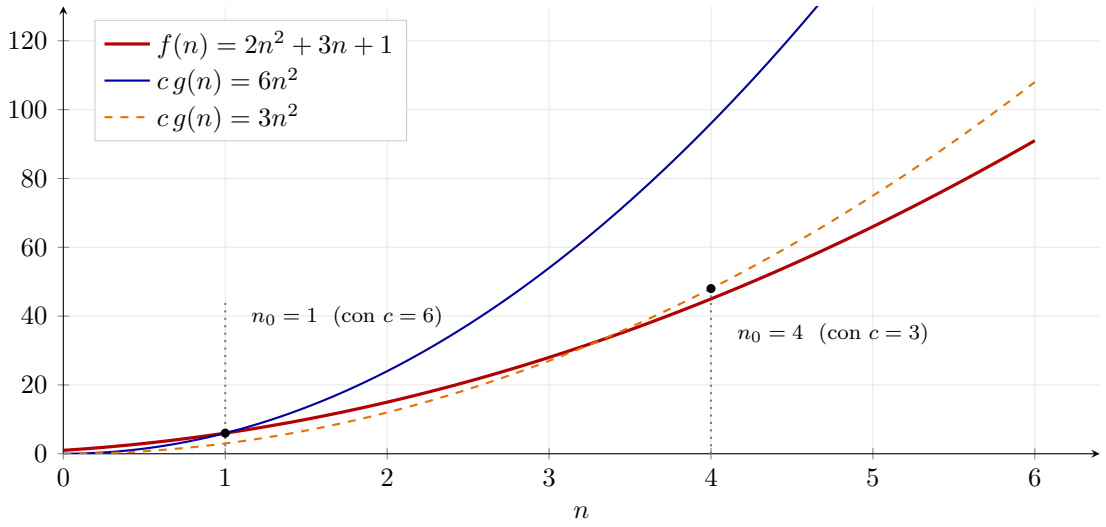


Figura 7: Para $n \geq 1$ la curva $6n^2$ domina a $f(n)$; a partir de $n \geq 4$ basta con $3n^2$.

Ojito...

El "para todo $n \geq n_0$" es lo que hace que la def sea asintotica. El comportamiento en entradas chiquitas es irrelevante, la curva f puede estar por encima de $cg(n)$ tanto como sea antes del umbral.

Ojito...

La Big O da una cota superior q no tiene que ser ajustada. Es perfectamente cierto que $n = \mathcal{O}(n^5)$, aunque sea una cota muy floja. Cuando quieres afirmar que la cota es ajustada por arriba y por abajo, usas $\Theta(g(n))$, definida con dos constantes: $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$.

Ojito...

Como $\mathcal{O}(g(n))$ es un conjunto, lo correcto sería escribir $f(n) \in \mathcal{O}(g(n))$. La costumbre de escribir $f(n) = \mathcal{O}(g(n))$ es un abuso de notación establecido, y por eso el signo no es simétrico: puedes escribir $2n^2 = \mathcal{O}(n^3)$, pero nunca $\mathcal{O}(n^3) = 2n^2$.

Crecimiento de funciones

Determinar la eficiencia asintótica de un algoritmo pasa por modelarlo: encontrar la función que describe su costo y ver contra qué se puede acotar. Muchas funciones admiten cota asintótica, y en la práctica casi todo termina comparándose contra tres familias: polinomios, logaritmos y exponenciales.

Operación elemental

Definición 2.4. Una operación elemental es aquella cuyo tiempo de ejecución está acotado por una constante que no depende del tamaño de la entrada. Una comparación, una asignación o una suma cuentan como una sola, sin importar qué tan grande sea n

Un algoritmo casi nunca ejecuta un solo tipo de operación. Sumar dos matrices de $n \times n$ hace sumas, pero también compara índices, incrementa contadores y accede a memoria. Se escoge una operación representativa, la que se ejecuta al menos tan seguido como cualquier otra, y se cuenta esa nada más: las demás quedan absorbidas por un factor constante

Principio de invarianza

Dos implementaciones del mismo algoritmo, en lenguajes distintos o sobre máquinas distintas, no tardan lo mismo, pero sus tiempos difieren a lo mucho por un factor constante. Esa es la razón de que la constante se descarte: lo que cambia al cambiar de máquina es ella, mientras que la clase asintótica es propiedad del algoritmo y sobrevive al cambio

Mejor caso, peor caso y caso promedio

El costo no depende solo de n, también de cómo venga la entrada, así que se analizan tres escenarios. Quick sort es el ejemplo de la tabla 1: en promedio cuesta $\mathcal{O}(n \log n)$, pero si el pivote cae siempre en un extremo las particiones quedan desbalanceadas y degenera a $\mathcal{O}(n^2)$. La burbuja simple no distingue, sus tres casos son $\mathcal{O}(n^2)$ porque recorre los ciclos completos sin importar cómo venga el arreglo

Idea clave

Siempre conviene optimizar. Es la única forma de llegar a los límites de lo que las arquitecturas actuales pueden dar

3. Continuación de funciones asintóticas

25 de agosto de 2026

Resumen rápido. Continuación de la clase anterior. Las propiedades de las tres notaciones, con la regla del límite que las decide de un jalón. $\Omega(\cdot)$ como cota inferior, $\Theta(\cdot)$ como las dos a la vez y el teorema que las relaciona. Cerramos con las notaciones chicas, los órdenes de complejidad y los consejos para contar operaciones elementales.

Propiedades de $\mathcal{O}(\cdot)$

Sean f, g, h, f_1 y f_2 funciones positivas.

1. Para cualquier función f se tiene $f \in \mathcal{O}(f)$.
2. $f \in \mathcal{O}(g) \Rightarrow \mathcal{O}(f) \subset \mathcal{O}(g)$.
3. $\mathcal{O}(f) = \mathcal{O}(g) \iff f \in \mathcal{O}(g)$ y $g \in \mathcal{O}(f)$.
4. Si $f \in \mathcal{O}(g)$ y $g \in \mathcal{O}(h) \Rightarrow f \in \mathcal{O}(h)$.
5. Si $f \in \mathcal{O}(g)$ y $f \in \mathcal{O}(h) \Rightarrow f \in \mathcal{O}(\min(g, h))$.
6. **Regla de la suma:** si $f_1 \in \mathcal{O}(g)$ y $f_2 \in \mathcal{O}(h) \Rightarrow f_1 + f_2 \in \mathcal{O}(\max(g, h))$.
7. **Regla del producto:** si $f_1 \in \mathcal{O}(g)$ y $f_2 \in \mathcal{O}(h) \Rightarrow f_1 \cdot f_2 \in \mathcal{O}(g \cdot h)$.
8. Si existe $\lim_{n\to\infty} \dfrac{f(n)}{g(n)} = k$, entonces según el valor de k:
   - Si $k \neq 0$ y $k < \infty$, entonces $\mathcal{O}(f) = \mathcal{O}(g)$.
   - Si $k = 0$, entonces $f \in \mathcal{O}(g)$, es decir $\mathcal{O}(f) \subset \mathcal{O}(g)$, pero se verifica que $g \notin \mathcal{O}(f)$.

Idea clave

La propiedad del límite es la herramienta práctica. En lugar de buscar c y n_0 a mano, se calcula el cociente f/g y se ve a dónde tiende: si tiende a una constante distinta de cero, las dos funciones son del mismo orden; si tiende a cero, f crece estrictamente más despacio.

Notación Omega

$f(n) = \Omega(g(n))$ quiere decir que $g(n)$ es una cota asintótica inferior de $f(n)$.

Definición 3.1. Sea $g(n)$ una función positiva. Decimos que una función $f(n)$ pertenece a $\Omega(g(n))$ si existen constantes positivas c y n_0 tales que

$$0 \leq c \cdot g(n) \leq f(n) \qquad \text{para todo } n \geq n_0.$$

Equivalentemente,

$$\Omega(g(n)) = \{\, f(n) : \exists\, c > 0,\ \exists\, n_0 > 0 \text{ tales que } 0 \leq c \cdot g(n) \leq f(n) \ \ \forall\, n \geq n_0 \,\}.$$

Ojito...

$f(n) \in \Omega(g(n))$ si y solo si $g(n) \in \mathcal{O}(f(n))$. Las dos notaciones son la misma relación vista desde cada lado.

Propiedades de $\Omega(\cdot)$

1. Para cualquier función f se tiene $f \in \Omega(f)$.
2. $f \in \Omega(g) \Rightarrow \Omega(f) \subset \Omega(g)$.
3. $\Omega(f) = \Omega(g) \iff f \in \Omega(g)$ y $g \in \Omega(f)$.
4. Si $f \in \Omega(g)$ y $g \in \Omega(h) \Rightarrow f \in \Omega(h)$.
5. Si $f \in \Omega(g)$ y $f \in \Omega(h) \Rightarrow f \in \Omega(\text{máx}(g, h))$.
6. **Regla de la suma:** si $f_1 \in \Omega(g)$ y $f_2 \in \Omega(h) \Rightarrow f_1 + f_2 \in \Omega(g + h)$.
7. **Regla del producto:** si $f_1 \in \Omega(g)$ y $f_2 \in \Omega(h) \Rightarrow f_1 \cdot f_2 \in \Omega(g \cdot h)$.
8. Si existe $\lim_{n\to\infty} \dfrac{f(n)}{g(n)} = k$, entonces según el valor de k:
   - Si $k \neq 0$ y $k < \infty$, entonces $\Omega(f) = \Omega(g)$.
   - Si $k = 0$, entonces $g \in \Omega(f)$, es decir $\Omega(g) \subset \Omega(f)$, pero se verifica que $f \notin \Omega(g)$.

Ojito...

En $\mathcal{O}(\cdot)$ la propiedad 5 usa el mínimo y la regla de la suma el máximo. En $\Omega(\cdot)$ la 5 usa el máximo y la suma da $g + h$. No son simétricas, conviene no copiarlas de memoria.

Por qué importan las cotas inferiores

Una cota inferior dice cuándo ya no se puede mejorar dentro de un modelo de cómputo. El ejemplo canónico es el ordenamiento por comparaciones: en el modelo general de comparaciones, cualquier algoritmo de ordenamiento necesita del orden de $n \log n$ comparaciones en el peor caso, o sea $\Omega(n \log n)$.

Idea clave

Si un algoritmo ya logra $\mathcal{O}(n \log n)$ y el problema tiene cota inferior $\Omega(n \log n)$, entonces ese algoritmo es asintóticamente óptimo en ese modelo. $\Omega(\cdot)$ contesta cuál es el trabajo mínimo que hay que hacer.

Notación Theta

$f(n) = \Theta(g(n))$ quiere decir que $c_2\, g(n)$ y $c_1\, g(n)$ son las cotas asintóticas de $f(n)$, superior e inferior respectivamente. Diremos que $f(n) \in \Theta(g(n))$ si $f(n)$ pertenece tanto a $\mathcal{O}(g(n))$ como a $\Omega(g(n))$.

Definición 3.2. Sea $g(n)$ una función positiva. Decimos que una función $f(n)$ pertenece a $\Theta(g(n))$ si existen constantes positivas c_1, c_2 y n_0 tales que

$$0 < c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \qquad \text{para todo } n \geq n_0.$$

Equivalentemente,

$$\Theta(g(n)) = \{\, f(n) : \exists\, c_1 > 0,\ \exists\, c_2 > 0,\ \exists\, n_0 > 0 \text{ tales que } 0 < c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \ \ \forall\, n \geq n_0 \,\}.$$

Propiedades de $\Theta(\cdot)$

1. Para cualquier función f se tiene $f \in \Theta(f)$.
2. $f \in \Theta(g) \Rightarrow \Theta(f) \subset \Theta(g)$.
3. $\Theta(f) = \Theta(g) \iff f \in \Theta(g)$ y $g \in \Theta(f)$.
4. Si $f \in \Theta(g)$ y $g \in \Theta(h) \Rightarrow f \in \Theta(h)$.
5. **Regla de la suma:** si $f_1 \in \Theta(g)$ y $f_2 \in \Theta(h) \Rightarrow f_1 + f_2 \in \Theta(\max(g, h))$.
6. **Regla del producto:** si $f_1 \in \Theta(g)$ y $f_2 \in \Theta(h) \Rightarrow f_1 \cdot f_2 \in \Theta(g \cdot h)$.
7. Si existe $\lim_{n\to\infty} \dfrac{f(n)}{g(n)} = k$, entonces según el valor de k:
   - Si $k \neq 0$ y $k < \infty$, entonces $\Theta(f) = \Theta(g)$.
   - Si $k = 0$, entonces los órdenes exactos de f y g son distintos.

Teorema 3.3. *$f(n) = \Theta(g(n))$ si y solo si $f(n) = \mathcal{O}(g(n))$ y $f(n) = \Omega(g(n))$.*

Demostración. De ida es inmediato: la definición de $\Theta(\cdot)$ contiene las dos desigualdades, así que c_2 y n_0 sirven de testigo para $\mathcal{O}(\cdot)$, y c_1 y n_0 para $\Omega(\cdot)$.

De regreso, si $f \in \mathcal{O}(g)$ existen c_2 y n_1 con $f(n) \leq c_2\, g(n)$ para $n \geq n_1$, y si $f \in \Omega(g)$ existen c_1 y n_2 con $c_1\, g(n) \leq f(n)$ para $n \geq n_2$. Tomando $n_0 = \max\{n_1, n_2\}$ las dos valen a la vez a partir de n_0, que es justo la definición de $\Theta(g(n))$. □

Una cota puede ser correcta y a la vez imprecisa

Sea $T(n) = 3n^2 + 7n + 20$. Todas estas afirmaciones son matemáticamente ciertas:

$$T(n) \in \mathcal{O}\left(n^2\right), \quad T(n) \in \mathcal{O}\left(n^3\right), \quad T(n) \in \mathcal{O}\left(n^4\right), \quad T(n) \in \mathcal{O}(2^n)\,.$$

Cada una de esas funciones se puede escalar por alguna constante positiva de modo que termine quedando por encima de $T(n)$. Por eso Big O por sí sola no identifica la verdadera tasa de crecimiento. $\Theta(n^2)$ sí: dice que el crecimiento cuadrático es el orden asintótico correcto, salvo factores constantes.

Ojito...

$\mathcal{O}(\cdot)$ no es sinónimo de peor caso, ni $\Omega(\cdot)$ de mejor caso. Son relaciones matemáticas de acotamiento. Primero se elige qué función de tiempo se está analizando y después se le aplica la notación que sea. Se puede decir $T_{\text{mejor}}(n) = \Theta(n)$, o $T_{\text{peor}}(n) = \Omega(n \log n)$, sin ninguna contradicción.

Notaciones chicas

$f(n) = o(g(n))$ quiere decir que $g(n)$ es una cota superior de $f(n)$ que no es asintóticamente justa (*tight*). Y $f(n) = \omega(g(n))$, que $g(n)$ es una cota inferior de $f(n)$ que tampoco lo es.

Definición 3.4. Sea $g(n)$ una función positiva. Decimos que una función $f(n)$ pertenece a $o(g(n))$ si para toda constante positiva c existe $n_0 > 0$ tal que

$$0 \leq f(n) < c \cdot g(n) \qquad \text{para todo } n \geq n_0.$$

Equivalentemente,

$$o(g(n)) = \{\, f(n) : \forall\, c > 0,\ \exists\, n_0 > 0 \text{ tales que } 0 \le f(n) < c \cdot g(n) \ \ \forall\, n \ge n_0 \,\}.$$

Definición 3.5. Sea $g(n)$ una función positiva. Decimos que una función $f(n)$ pertenece a $\omega(g(n))$ si para toda constante positiva c existe $n_0 > 0$ tal que

$$0 \le c \cdot g(n) < f(n) \qquad \text{para todo } n \ge n_0.$$

Equivalentemente,

$$\omega(g(n)) = \{\, f(n) : \forall\, c > 0,\ \exists\, n_0 > 0 \text{ tales que } 0 \le c \cdot g(n) < f(n) \ \ \forall\, n \ge n_0 \,\}.$$

La distinción precisa no es tanto lo justo o no justo, sino el cuantificador sobre la constante:

- $f(n) = \mathcal{O}(g(n)) \iff \exists\, c > 0,\ \exists\, n_0 :\ f(n) \le c\, g(n),\ \forall\, n \ge n_0.$
- $f(n) = o(g(n)) \iff \forall\, c > 0,\ \exists\, n_0 :\ f(n) < c\, g(n),\ \forall\, n \ge n_0.$

Con $\Omega(\cdot)$ y $\omega(\cdot)$ pasa lo mismo, volteando la desigualdad. Las grandes permiten que las dos funciones sean del mismo orden; las chicas lo prohíben.

Propiedad	$\mathcal{O}(g(n))$	$o(g(n))$
Tipo de cota superior	No estricta	Estricta
Condición sobre c	$\exists\, c > 0$	$\forall\, c > 0$
Permite el mismo orden	Sí	No
Puede ser $\Theta(g(n))$	Sí	No
Cociente $f(n)/g(n)$	Acotado basta	Tiende a 0

Tabla 2: Big O contra o chica.

El punto de vista del límite

Cuando el límite existe, el cociente deja la distinción clarísima:

$$f = o(g) \iff \frac{f(n)}{g(n)} \to 0, \qquad f = \Theta(g) \iff \frac{f(n)}{g(n)} \to L \text{ con } 0 < L < \infty, \qquad f = \omega(g) \iff \frac{f(n)}{g(n)} \to \infty.$$

$\mathcal{O}(\cdot)$ y $\Omega(\cdot)$ son relaciones más amplias: solo piden una cota por factor constante, no un límite específico. Por ejemplo $n^2 \neq o(n^2)$, porque el cociente vale 1 y no 0; en cambio $n = o(n^2)$, porque $n/n^2 = 1/n \to 0$.

Jerarquía de crecimiento

$$1 \ll \log n \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n$$

Leída con o chica: $n = o(n \log n)$, $n \log n = o(n^2)$, $n^2 = o(n^3)$. Y al revés con omega chica: $n \log n = \omega(n)$, $n^2 = \omega(n \log n)$, $n^3 = \omega(n^2)$.

Idea clave

$o(\cdot) < \mathcal{O}(\cdot) - \Theta(\cdot) - \Omega(\cdot) < \omega(\cdot)$. Las grandes admiten crecimiento del mismo orden, las chicas lo prohíben. Esa es toda la diferencia.

Notación	Crecimiento	Tipo de cota
$o(g(n))$	Estrictamente más lento	Superior estricta
$\mathcal{O}(g(n))$	No más rápido	Superior
$\Theta(g(n))$	Del mismo orden	Ajustada
$\Omega(g(n))$	No más lento	Inferior
$\omega(g(n))$	Estrictamente más rápido	Inferior estricta

Tabla 3: Las cinco relaciones asintóticas de un vistazo.

Orden de complejidad

La familia $\mathcal{O}(f(n))$, $\Omega(f(n))$, $\Theta(f(n))$ define un orden de complejidad, y como representante del orden se elige la función $f(n)$ más sencilla de la familia. Se identifican diferentes familias:

Orden	Nombre
$\mathcal{O}(c)$	Constante
$\mathcal{O}(\log n)$	Logarítmico
$\mathcal{O}(n)$	Lineal
$\mathcal{O}(n \log n)$	Casi lineal
$\mathcal{O}(n^2)$	Cuadrático
$\mathcal{O}(n^3)$	Cúbico
$\mathcal{O}(n^c)$	Polinómico de grado c
$\mathcal{O}(2^n)$	Exponencial
$\mathcal{O}(n!)$	Factorial

Tabla 4: Órdenes de complejidad.

Consejos para identificar $f(n)$

Son los consejos para dar con la $f(n)$ que representa el número de operaciones elementales (OE) de un algoritmo.

Consejo 1: la base

Se asume que el tiempo de una OE es de orden 1, o sea que la constante c del principio de invarianza se toma como 1 por fines prácticos. El tiempo de ejecución de una secuencia de instrucciones, elementales o no, se obtiene sumando los tiempos de cada instrucción.

Instrucción `case`

Para `switch(n)` con casos $S_1, \ldots, S_k$:

$$f(n) = f(c) + \max\{f(S_1), \ldots, f(S_k)\},$$

donde $f(c)$ considera el tiempo de comparar n contra cada uno de los casos.

Instrucción `if`

Para `if C then` S_1 `else` S_2:

$$f(n) = f(C) + \max\{f(S_1), f(S_2)\}.$$

```
if (n == 0)
    for (i = 0; i < n; i++)
        r += i;
else
    r = 2;
```

Instrucción `while`

Para `while (c) { S }`:

$$f(n) = f(c) + (\#\text{iteraciones}) \cdot (f(c) + f(S)).$$

Ojito...

Las instrucciones `for`, `repeat` y `loop` son equivalentes a un `while`, así que se analizan con la misma fórmula.

Llamado a funciones no recursivas

El tiempo de una llamada $F(A_1, \ldots, A_s)$ es 1 por el llamado, más el tiempo de evaluar los parámetros, más el tiempo de ejecutar el cuerpo de la función:

$$f(A_1, \ldots, A_s) = 1 + f(A_1) + \cdots + f(A_s) + f(S).$$

Funciones recursivas

El tiempo de ejecución de una función recursiva se calcula a través de la solución de funciones de recurrencia, que es el tema que sigue.

4. Anexo: caracterización por límites

Herramienta de diagnóstico para $\mathcal{O}(\cdot)$, $\Omega(\cdot)$, $\Theta(\cdot)$, $o(\cdot)$ y $\omega(\cdot)$.

La idea que hay detrás

Todas estas notaciones comparan el tamaño de dos funciones, y la manera natural de comparar tamaños es dividir. Si f y g son positivas para n suficientemente grande, toda la información relevante está en el cociente $f(n)/g(n)$.

Traducción de cada notación al cociente

$f \in \mathcal{O}(g)$ significa que el cociente está *acotado por arriba* en la cola. $f \in \Omega(g)$ significa que está *acotado lejos de cero*. $f \in \Theta(g)$ significa ambas cosas. Las versiones minúsculas son las variantes estrictas: $o(\cdot)$ pide que el cociente *colapse* a cero, y $\omega(\cdot)$ que *explote* a infinito.

Conviene ver por qué la definición de $\mathcal{O}(\cdot)$ es exactamente un enunciado de acotación. La condición $f(n) \leq c\, g(n)$ para todo $n \geq n_0$ equivale, dividiendo entre $g(n) > 0$, a $f(n)/g(n) \leq c$ para todo $n \geq n_0$. Es decir: la constante c de la definición es literalmente una cota superior del cociente, y el umbral n_0 es el permiso para ignorar un tramo inicial finito. Esa lectura es la que hace transparente todo lo demás.

Criterio con el límite ordinario

Este es el caso cómodo y cubre la gran mayoría de los ejercicios, porque polinomios, logaritmos y exponenciales son funciones bien portadas cuyos cocientes convergen.

Supóngase que existe

$$L \;=\; \lim_{n\to\infty} \frac{f(n)}{g(n)}$$

admitiendo el valor $+\infty$. Entonces se tiene la clasificación siguiente.

Valor de L	Relación más fuerte	Relaciones implicadas
$L = 0$	$f \in o(g)$	$f \in \mathcal{O}(g)$
$0 < L < \infty$	$f \in \Theta(g)$	$f \in \mathcal{O}(g)$ y $f \in \Omega(g)$
$L = \infty$	$f \in \omega(g)$	$f \in \Omega(g)$

Tabla 5: Clasificación según el límite del cociente.

En el primer renglón fallan $\Omega(\cdot)$, $\Theta(\cdot)$ y $\omega(\cdot)$. En el segundo fallan $o(\cdot)$ y $\omega(\cdot)$. En el tercero fallan $\mathcal{O}(\cdot)$, $\Theta(\cdot)$ y $o(\cdot)$. Y si el límite no existe, el criterio no dice nada y hay que pasar a la sección siguiente.

Observación. El primer renglón es la fuente de un error frecuente. Que $L = 0$ no excluye a $\mathcal{O}(\cdot)$: como $o(g) \subsetneq \mathcal{O}(g)$, la afirmación $n \in \mathcal{O}(n^2)$ es perfectamente cierta, solo que floja. Simétricamente, $L = \infty$ no excluye a $\Omega(\cdot)$.

Criterio general: límite superior e inferior

Cuando el cociente oscila, el límite ordinario no existe y hace falta una herramienta que siempre esté definida. Esa herramienta es el par $\limsup$, $\liminf$.

Definición

Dada una sucesión a_n, para cada entero N se considera el supremo de la cola que empieza en N:

$$s_N \;=\; \sup\{\, a_n \;:\; n \geq N \,\}, \qquad i_N \;=\; \text{ínf}\{\, a_n \;:\; n \geq N \,\}.$$

Al aumentar N se le quitan elementos al conjunto, de modo que su supremo no puede crecer y su ínfimo no puede decrecer: la sucesión s_N es no creciente y la sucesión i_N es no decreciente. Toda sucesión monótona tiene límite en la recta extendida, y por eso las dos definiciones siguientes tienen sentido siempre, sin hipótesis adicionales:

$$\limsup_{n\to\infty} a_n \;=\; \lim_{N\to\infty} s_N, \qquad \liminf_{n\to\infty} a_n \;=\; \lim_{N\to\infty} i_N.$$

Lectura informal

El límite superior es el valor más alto al que la sucesión regresa infinitas veces; el límite inferior, el más bajo. Coinciden si y solo si la sucesión converge, y en ese caso su valor común es el límite ordinario.

Los criterios

Con $f, g > 0$ eventualmente y $a_n = f(n)/g(n)$:

Relación	Criterio equivalente
$f \in \mathcal{O}(g)$	$\limsup_{n\to\infty} a_n < \infty$
$f \in \Omega(g)$	$\liminf_{n\to\infty} a_n > 0$
$f \in \Theta(g)$	$\liminf_{n\to\infty} a_n > 0$ y $\limsup_{n\to\infty} a_n < \infty$
$f \in o(g)$	$\lim_{n\to\infty} a_n = 0$
$f \in \omega(g)$	$\lim_{n\to\infty} a_n = \infty$

Tabla 6: Criterios con límite superior e inferior.

Nótese la asimetría de los dos últimos renglones: ahí aparece el límite ordinario y no el superior o el inferior. La razón está en la forma de las definiciones. Las de $\mathcal{O}(\cdot)$ y $\Omega(\cdot)$ empiezan con *existe* una constante, y basta con controlar la cola por un lado, lo cual es justamente lo que mide un lím sup o un lím inf. Las de $o(\cdot)$ y $\omega(\cdot)$ empiezan con *para toda* constante, y esa cuantificación universal obliga a que la cola entera se comprima, no solo una subsucesión. Comprimirse entera es exactamente converger.

Relaciones estructurales

Las inclusiones siguientes son útiles para no reportar una relación más débil de la que se tiene.

- $o(g) \subsetneq \mathcal{O}(g)$ y $\omega(g) \subsetneq \Omega(g)$: las minúsculas son las cotas estrictas.
- $\Theta(g) = \mathcal{O}(g) \cap \Omega(g)$.
- $f \in \mathcal{O}(g)$ si y solo si $g \in \Omega(f)$; la misma dualidad relaciona $o(\cdot)$ con $\omega(\cdot)$.
- $\mathcal{O}(\cdot)$ y $\Omega(\cdot)$ pueden fallar ambos a la vez, y entonces las funciones son incomparables.
- $o(\cdot)$ y $\omega(\cdot)$ nunca se cumplen simultáneamente.

Un ejemplo donde el límite ordinario no sirve

Sea $f(n) = n^2$ y sea

$$g(n) \;=\; \begin{cases} n & \text{si } n \text{ es impar,} \\ n^3 & \text{si } n \text{ es par.} \end{cases}$$

El cociente vale n en los impares y $1/n$ en los pares. Fijado cualquier N, la cola contiene impares arbitrariamente grandes, de modo que no está acotada superiormente y $s_N = \infty$; y contiene pares con $1/n$ tan pequeño como se quiera, todos positivos, de modo que $i_N = 0$. Por lo tanto

$$\limsup_{n\to\infty} \frac{f(n)}{g(n)} = \infty, \qquad \liminf_{n\to\infty} \frac{f(n)}{g(n)} = 0,$$

y aplicando los criterios se concluye que $f \notin \mathcal{O}(g)$ y $f \notin \Omega(g)$. Las dos funciones son incomparables.

El punto que conviene retener del cálculo anterior es que quitar una cantidad finita de términos iniciales no cambia nada, porque siempre sobran infinitos impares e infinitos pares. Esa es la misma idea que expresa el umbral n_0 de la definición original: n_0 concede el derecho a ignorar el principio de la sucesión, y aquí se ve que ejercer ese derecho no ayuda.

Señal de alarma

Una función definida por casos, típicamente por paridad o por rangos, casi siempre está construida para romper la monotonía. El reflejo correcto ante ella es evaluar cada rama por separado antes de intentar probar nada, porque el desenlace habitual es la incomparabilidad.

5. Recurrencias

10 de septiembre de 2026

Resumen rápido. El tema que quedó abierto al final de la clase pasada. Qué es una recurrencia y de dónde sale, el caso de quick sort que explica por qué el mismo algoritmo aparece con dos complejidades distintas en la tabla, y los cuatro métodos para resolverlas: sustitución, iteración, árboles de recursión y el método maestro.

De dónde salen

Las recurrencias se dan cuando un algoritmo se llama a sí mismo. El trabajo que hace sobre una entrada de tamaño n se parte en dos piezas: lo que cuesta acomodar el problema en este nivel, y lo que cuestan las llamadas sobre entradas más chicas. Esa segunda parte no se puede contar directamente, porque es la misma función otra vez. Entonces el tiempo de ejecución no queda como una fórmula cerrada, queda definido en términos de sí mismo.

Definición 5.1. Una recurrencia es una ecuación o desigualdad que describe una función en términos de su valor para entradas más pequeñas.

Ojito...

Una recurrencia sin caso base no define nada. La ecuación sola se desdobla para siempre; es el caso base el que detiene el descenso y le da un valor a todo lo demás. Cuando se omite, como se hace seguido, se sobreentiende que $T(n)$ es constante para n suficientemente chico.

Ejemplo: quick sort

Quick sort escoge un pivote, particiona el arreglo alrededor de él y se llama a sí mismo sobre cada lado. La partición recorre el arreglo una vez, así que cuesta $\mathcal{O}(n)$. Lo que cambia todo es cómo queda el corte, y eso depende del pivote.

Si el pivote parte el arreglo por la mitad, cada llamada genera dos subproblemas de tamaño $n/2$:

$$T(n) = 2\,T\left(\frac{n}{2}\right) + \mathcal{O}(n)\,.$$

Si el pivote resulta ser el mínimo o el máximo, un lado se queda vacío y el otro con todo menos el pivote:

$$T(n) = T(n-1) + \mathcal{O}(n)\,.$$

Idea clave

Son dos recurrencias del mismo algoritmo, y resuelven a $\mathcal{O}(n \log n)$ y $\mathcal{O}(n^2)$ respectivamente. Ahí está la explicación del renglón de quick sort en la tabla 1: la diferencia entre el caso promedio y el peor caso no está en el código, está en qué tan parejo queda el corte.

Los cuatro métodos, de un vistazo

Resolver una recurrencia es encontrarle una forma cerrada, o al menos una cota asintótica. Se nombraron cuatro caminos, que se desarrollan en la clase siguiente.

Método	**En qué consiste**
Sustitución	Adivinar una cota y probarla por inducción matemática
Iteración	Expandir la recurrencia hasta expresarla como una sumatoria en términos de n y las condiciones iniciales
Árbol	Convertir la recurrencia en un árbol cuyos nodos son los costos de los distintos niveles de la recursión
Maestro	Receta para las recurrencias de la forma $T(n) = aT(n/b) + f(n)$. Requiere memorizar tres casos

6. Métodos para resolver recurrencias

15 de septiembre de 2026

Resumen rápido. Los cuatro métodos de la clase pasada, ahora cada uno con un ejemplo resuelto paso a paso. Sustitución con su prueba por inducción fuerte y el problema de las condiciones de frontera, iteración desenrollando hasta una serie geométrica, árboles de recursión sumando por niveles, y el teorema maestro con sus tres casos y el hueco que dejan.

Método de sustitución

Consta de dos pasos, en este orden:

1. **Adivinar** la forma de la solución.
2. **Sustituir** en la recursión donde n decrece, asumir de manera inductiva que la cota es cierta ahí, y probarla para el siguiente valor de n.

Es un método poderoso, pero solo se aplica cuando es fácil adivinar la forma de la respuesta. Sirve tanto para cotas superiores como para inferiores.

Ejemplo resuelto

Ejemplo 6.1. Resolver $T(n) = 2\,T(\lfloor n/2 \rfloor) + n$.

Paso 1. Adivinamos $T(n) = \mathcal{O}(n \lg n)$. Escrito como lo va a usar la inducción, por probar que existe $c > 0$ con

$$T(n) \leq c\,n \lg n.$$

Paso 2. Supongamos que la cota vale para $\lfloor n/2 \rfloor$, o sea

$$T(\lfloor n/2 \rfloor) \leq c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor).$$

Sustituyendo en la recurrencia:

$$\begin{aligned} T(n) &\leq 2\,(c\lfloor n/2 \rfloor \lg(\lfloor n/2 \rfloor)) + n \\ &\leq c\,n \lg(n/2) + n && \text{quitando el piso, que solo achica} \\ &= c\,n \lg n - c\,n \lg 2 + n && \lg(n/2) = \lg n - \lg 2 \\ &= c\,n \lg n - c\,n + n && \lg 2 = 1 \\ &\leq c\,n \lg n. \end{aligned}$$

El último paso pide que $-c\,n + n \leq 0$, es decir $\boxed{c \geq 1}$.

Idea clave

Fíjate en de dónde sale todo el trabajo: el $-c\,n$ aparece solo porque $\lg 2 = 1$, y es justo lo que se come el $+n$ que la recurrencia agrega en cada nivel. Si el término suelto fuera n^2 en vez de n, ese $-c\,n$ ya no alcanzaría y la conjetura $\mathcal{O}(n \lg n)$ se caería.

Las condiciones de frontera

La inducción no está completa hasta probar que la cota vale en la frontera. Y ahí aparece un detalle que tumba la prueba si se pasa por alto.

Ojito...

Con $T(1) = 1$, la cota exige $T(1) \leq c \cdot 1 \cdot \lg 1 = 0$, o sea $1 \leq 0$, que es falso **para cualquier** c. La recurrencia no funciona para $T(1)$.

La salida es que la notación asintótica solo exige que la desigualdad valga de un n_0 en adelante, así que se escoge $n_0 = 2$ y se verifican $n = 2$ y $n = 3$ como casos base:

Valor	De la recurrencia	Lo que pide la cota
$T(2)$	$2T(1) + 2 = 4$	$c\,2\lg 2 = 2c \geq 4 \;\Rightarrow\; c \geq 2$
$T(3)$	$2T(1) + 3 = 5$	$c\,3\lg 3 = 4.755c \geq 5$

Con $c \geq 2$ los dos se cumplen, y como el paso inductivo solo pedía $c \geq 1$, esa misma constante sirve para toda la prueba.

Ojito...

La constante c tiene que ser **una sola** para toda la prueba. No se vale escoger una para el paso inductivo y otra para cada caso base: se elige la más exigente de todas y se usa en todos lados.

Por qué la inducción tiene que ser fuerte

Hay una razón de fondo para que aquí no sirva la inducción ordinaria. La recurrencia manda de $n+1$ a $\lfloor (n+1)/2 \rfloor$, que no es n sino un valor mucho más chico. Suponer la cota solo para n no sirve de nada.

Definición 6.2. En la **inducción fuerte** la hipótesis no es que la propiedad vale para n, sino que vale para *todos* los valores desde el caso base hasta n. Con eso se puede aplicar a cualquier tamaño menor que aparezca.

Ejemplo 6.3. La misma recurrencia, escrita con la hipótesis fuerte.

Hipótesis. Para todo k con $2 \leq k \leq n$,

$$T(k) \leq c\,k \log_2 k.$$

Por demostrar. $T(n+1) \leq c\,(n+1)\log_2(n+1)$.

Paso inductivo. Sea $m = \lfloor (n+1)/2 \rfloor$. Para $n + 1 \geq 4$ se cumple $2 \leq m \leq n$, así que la hipótesis aplica a $T(m)$:

$$\begin{aligned}
T(n+1) &= 2\,T(m) + (n+1) \\
&\leq 2c\,m \log_2 m + (n+1) && \text{por la hipótesis} \\
&\leq c\,(n+1)\log_2\!\left(\tfrac{n+1}{2}\right) + (n+1) && \text{porque } 2m \leq n+1 \\
&= c\,(n+1)\,[\log_2(n+1) - 1] + (n+1) \\
&= c\,(n+1)\log_2(n+1) + (n+1)(1-c).
\end{aligned}$$

Si $c \geq 1$ entonces $(n+1)(1-c) \leq 0$, y por lo tanto

$$T(n+1) \leq c\,(n+1)\log_2(n+1).$$

Idea clave

Los casos base $n = 2$ y $n = 3$ no son un capricho. Se necesita $m \geq 2$ para poder aplicar la hipótesis, y $m = 1$ ocurre justo cuando $n + 1 = 3$. Cubrir 2 y 3 a mano deja el paso general trabajando siempre con m dentro del rango.

Cómo adivinar la cota

Es la debilidad del método: **no hay una manera general** de adivinar la solución correcta. Requiere experiencia y creatividad. Dos heurísticas que sí ayudan:

- Dibujar un árbol de recursión para generar un buen valor inicial, y luego probarlo con sustitución.
- Si la recurrencia se parece a otra ya resuelta, probar con una solución parecida.

Método de iteración

No requiere adivinar la respuesta, y esa es toda su gracia. Se expande la recurrencia dentro de sí misma una y otra vez hasta expresarla como una sumatoria de términos que dependen solo de n y de las condiciones iniciales. Después se acota esa sumatoria.

El costo es el álgebra: puede ponerse pesado rápido. A veces se itera solo lo suficiente para intuir la forma de la solución y de ahí se sigue con sustitución.

Ejemplo 6.4. Resolver $T(n) = 3\,T(\lfloor n/4 \rfloor) + n$.

Paso 1. Desenrollar.

$$\begin{aligned} T(n) &= n + 3\,T(\lfloor n/4 \rfloor) \\ &= n + 3\,(\lfloor n/4 \rfloor + 3\,T(\lfloor n/16 \rfloor)) \\ &= n + 3\,(\lfloor n/4 \rfloor + 3\,(\lfloor n/16 \rfloor + 3\,T(\lfloor n/64 \rfloor))) \\ &= n + 3\lfloor n/4 \rfloor + 9\lfloor n/16 \rfloor + 27\,T(\lfloor n/64 \rfloor). \end{aligned}$$

Paso 2. Leer el patrón. El término i-ésimo de la serie es $3^i \lfloor n/4^i \rfloor$.

Paso 3. Saber dónde parar. La iteración llega al caso base cuando $\lfloor n/4^i \rfloor = 1$, o sea cuando i alcanza $\log_4 n$.

Paso 4. Acotar la suma. Usando $\lfloor n/4^i \rfloor \leq n/4^i$:

$$\begin{aligned} T(n) &\leq n + \tfrac{3n}{4} + \tfrac{9n}{16} + \tfrac{27n}{64} + \cdots + 3^{\log_4 n}\,\Theta(1) \\ &\leq n \sum_{i=0}^{\infty} \left(\tfrac{3}{4}\right)^i + \Theta\left(n^{\log_4 3}\right) \\ &= 4n + o(n) \\ &= \mathcal{O}(n)\,. \end{aligned}$$

Idea clave

Los dos pasos que hacen el truco. Primero, la serie $\sum (3/4)^i$ es geométrica decreciente, así que converge a $1/(1 - 3/4) = 4$ aunque se extienda al infinito: acotar de más no cuesta nada y ahorra manejar el límite exacto. Segundo, $n^{\log_4 3} \approx n^{0.79}$ crece más lento que n, así que es $o(n)$ y desaparece dentro del $\mathcal{O}(n)$.

Árboles de recursión

Es la iteración, pero dibujada. Cada nodo representa el costo de un subproblema dentro del conjunto de llamadas recursivas. Se suman los costos por nivel y luego se suman los niveles. Sirve sobre todo para **generar una buena conjetura**, que después se comprueba con sustitución.

Ejemplo 6.5. Resolver $T(n) = 3\,T(\lfloor n/4 \rfloor) + \Theta(n^2)$. Se trabaja con $T(n) = 3\,T(n/4) + c\,n^2$, con $c > 0$, y se supone que n es potencia exacta de 4 para no cargar con los pisos.

Paso 1. Cuántos niveles hay. El subproblema a profundidad i tiene tamaño $n/4^i$. Llega a 1 cuando $n/4^i = 1$, o sea $i = \log_4 n$. El árbol tiene entonces $\log_4 n + 1$ niveles, contando el 0.

Paso 2. Cuánto cuesta cada nivel. Cada nodo se abre en tres, así que a profundidad i hay 3^i nodos, y cada uno cuesta $c\,(n/4^i)^2$. Multiplicando:

$$3^i \cdot c\left(\frac{n}{4^i}\right)^2 = \frac{3^i\,c\,n^2}{16^i} = \left(\frac{3}{16}\right)^i c\,n^2.$$

Nivel	Nodos	Tamaño	Costo del nivel
0	1	n	$c\,n^2$
1	3	$n/4$	$(3/16)\,c\,n^2$
2	9	$n/16$	$(3/16)^2 c\,n^2$
i	3^i	$n/4^i$	$(3/16)^i c\,n^2$

Paso 3. Las hojas. El último nivel, a profundidad $\log_4 n$, tiene $3^{\log_4 n} = n^{\log_4 3}$ hojas, cada una con costo $T(1)$, para un total de $\Theta\left(n^{\log_4 3}\right)$.

Paso 4. Sumar todo.

$$\begin{aligned} T(n) &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &< \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &= \frac{1}{1 - 3/16}\,c\,n^2 + \Theta\left(n^{\log_4 3}\right) \\ &= \frac{16}{13}\,c\,n^2 + \Theta\left(n^{\log_4 3}\right) = \mathcal{O}\left(n^2\right). \end{aligned}$$

Y como la raíz sola ya cuesta $c\,n^2$, la cota inferior también es n^2, así que $T(n) = \Theta(n^2)$.

La identidad del exponente

En el paso 3 se usó $3^{\log_4 n} = n^{\log_4 3}$, que parece un truco y no lo es. Es un caso de $a^{\log_c b} = b^{\log_c a}$, y sale de escribir los dos lados como exponenciales:

$$3^{\log_4 n} = 3^{\frac{\ln n}{\ln 4}} = \left(e^{\ln 3}\right)^{\frac{\ln n}{\ln 4}} = e^{\frac{(\ln 3)(\ln n)}{\ln 4}}, \qquad n^{\log_4 3} = e^{\frac{(\ln n)(\ln 3)}{\ln 4}}.$$

Los exponentes son iguales porque la multiplicación es conmutativa.

La serie geométrica finita

La otra pieza que se usó. Para $S = 1 + r + r^2 + \cdots + r^{k-1}$:

$$\begin{aligned} S &= 1 + r + r^2 + \cdots + r^{k-1} \\ r\,S &= \quad r + r^2 + \cdots + r^{k-1} + r^k. \end{aligned}$$

Restando la segunda de la primera se cancelan todos los términos de en medio y queda $S - r\,S = 1 - r^k$, de donde

$$S = \frac{1 - r^k}{1 - r}.$$

Cuando $|r| < 1$ y la suma es infinita, $r^k \to 0$ y se llega a $\sum_{k=0}^{\infty} r^k = 1/(1-r)$, que es la forma que se usó arriba.

Idea clave

Que $r = 3/16 < 1$ es lo que decide el resultado: el costo baja geométricamente conforme se desciende, así que **manda la raíz**. Si r fuera mayor que 1 mandarían las hojas, y si fuera exactamente 1 todos los niveles costarían lo mismo y habría que multiplicar por el número de niveles. Esos tres escenarios son exactamente los tres casos del método maestro.

Método maestro

Es una receta, sin inducción ni desenrollado. Aplica a las recurrencias de la forma

$$T(n) = a\,T(n/b) + f(n), \qquad a \geq 1, \quad b > 1,$$

con a y b constantes y $f(n)$ asintóticamente positiva. Describe un algoritmo que parte un problema de tamaño n en a subproblemas de tamaño n/b, los resuelve recursivamente en tiempo $T(n/b)$, y paga $f(n)$ por dividir y combinar. Da igual si n/b se lee como $\lfloor n/b \rfloor$ o como $\lceil n/b \rceil$.

Teorema 6.6 (Teorema maestro)**.** *Con $T(n) = a\,T(n/b) + f(n)$:*

1. *Si $f(n) = \mathcal{O}\left(n^{\log_b a - \epsilon}\right)$ para alguna constante $\epsilon > 0$, entonces $T(n) = \Theta\left(n^{\log_b a}\right)$.*
2. *Si $f(n) = \Theta\left(n^{\log_b a}\right)$, entonces $T(n) = \Theta\left(n^{\log_b a} \lg n\right)$.*
3. *Si $f(n) = \Omega\left(n^{\log_b a + \epsilon}\right)$ para alguna constante $\epsilon > 0$, y además $a\,f(n/b) \leq c\,f(n)$ para alguna constante $c < 1$ y todo n suficientemente grande, entonces $T(n) = \Theta(f(n))$.*

Idea clave

Los tres casos son la misma pregunta: comparar $f(n)$, que es lo que cuesta la raíz, contra $n^{\log_b a}$, que es lo que cuestan las hojas. **Gana la mayor de las dos.** Si empatan, todos los niveles cuestan lo mismo y se multiplica por el factor logarítmico, que no es otra cosa que el número de niveles.

La letra chica

Ojito...

En los casos 1 y 3 no basta con que una función sea mayor que la otra: tiene que serlo **polinómicamente**, o sea por un factor de n^ϵ con $\epsilon > 0$. Y el caso 3 pide además la condición de regularidad $a\,f(n/b) \le c\,f(n)$, que se cumple para casi cualquier polinomio pero hay que verificarla.
La consecuencia es que **los tres casos no cubren todas las posibilidades**. Hay recurrencias que caen en los huecos y para las que el teorema simplemente no dice nada.

Los cuatro ejemplos

Ejemplo 6.7 (Caso 1)**.** $T(n) = 9\,T(n/3) + n$. Aquí $a = 9$, $b = 3$, $f(n) = n$, y $n^{\log_b a} = n^{\log_3 9} = n^2$. Como $f(n) = n = \mathcal{O}(n^{2-\epsilon})$ con $\epsilon = 1$, las hojas dominan:

$$T(n) = \Theta\left(n^2\right).$$

Ejemplo 6.8 (Caso 2)**.** $T(n) = T(2n/3) + 1$. Aquí $a = 1$, $b = 3/2$, $f(n) = 1$, y $n^{\log_{3/2} 1} = n^0 = 1$. Como $f(n) = \Theta(1)$ empata con $n^{\log_b a}$, todos los niveles cuestan lo mismo:

$$T(n) = \Theta(\lg n).$$

Ejemplo 6.9 (Caso 3)**.** $T(n) = 3\,T(n/4) + n \lg n$. Aquí $a = 3$, $b = 4$, $f(n) = n \lg n$, y $n^{\log_4 3} = \mathcal{O}(n^{0.793})$. Como $f(n) = \Omega\left(n^{\log_4 3 + \epsilon}\right)$ con $\epsilon \approx 0.2$, falta verificar la regularidad:

$$a\,f(n/b) = 3\,\frac{n}{4}\lg\frac{n}{4} \le \frac{3}{4}\,n \lg n = c\,f(n) \quad \text{con } c = 3/4 < 1.$$

Se cumple, así que manda la raíz:

$$T(n) = \Theta(n \lg n).$$

Ejemplo 6.10 (Ninguno de los tres)**.** $T(n) = 2\,T(n/2) + n \lg n$. Tiene la forma correcta, con $a = 2$, $b = 2$, $f(n) = n \lg n$ y $n^{\log_2 2} = n$.

Parecería el caso 3, porque $n \lg n$ sí es asintóticamente mayor que n. El problema es que **no lo es polinómicamente**: el cociente

$$\frac{f(n)}{n^{\log_b a}} = \frac{n \lg n}{n} = \lg n$$

es asintóticamente *menor* que n^ϵ para cualquier $\epsilon > 0$ positivo. La recurrencia cae en el hueco entre el caso 2 y el caso 3, y el teorema maestro no la resuelve.

Ojito...

Ese último ejemplo es el que hay que tener presente: que una recurrencia tenga la forma $a\,T(n/b) + f(n)$ **no garantiza** que el método maestro aplique. Siempre hay que comprobar cuál de los tres casos se cumple, y aceptar que a veces ninguno lo hace. Ahí toca volver a los árboles de recursión y a la sustitución.

Cuál usar

Método	Qué pide	Qué entrega
Sustitución	Una conjetura previa	Prueba formal
Iteración	Aguante algebraico	La forma cerrada
Árbol	Dibujar y sumar niveles	Una conjetura
Maestro	Que encaje en un caso	La cota, directo

Idea clave

En la práctica se usan en pareja. El árbol da la conjetura, la sustitución la prueba. El maestro se intenta primero porque si aplica ahorra todo el trabajo, pero hay que verificar el caso, no suponerlo.

7. Heaps

17 de septiembre de 2026

Resumen rápido. Se cerró el método maestro y arrancó ordenamiento. El heap: qué es, cómo se guarda un árbol binario en un arreglo plano, la propiedad que lo define y su altura. Luego las operaciones, con Heapify como la pieza central, y la construcción del heap, que resulta costar $\mathcal{O}(n)$ y no $\mathcal{O}(n \lg n)$ como parecería.

Qué es un heap

Definición 7.1. Un **heap** es una estructura de datos binaria: un arreglo que representa un árbol binario **completo**, o sea lleno en todos sus niveles salvo tal vez el último, que se llena de izquierda a derecha.

Cada nodo del árbol es un elemento del arreglo y guarda su valor. El arreglo lleva dos atributos:

$\mathit{length}[A]$	Número de elementos del arreglo A
$\mathit{heap\text{-}size}[A]$	Número de elementos que de verdad forman parte del heap

Siempre se cumple $\mathit{heap\text{-}size}[A] \leq \mathit{length}[A]$, y la raíz del árbol es $A[1]$.

Ojito...

Los dos atributos no son lo mismo y la diferencia importa. El arreglo puede ser más grande que el heap que contiene, y en heapsort esa diferencia es justo el mecanismo: el heap se va encogiendo dentro del mismo arreglo mientras la parte de atrás guarda lo ya ordenado.

Navegar el árbol con aritmética

Lo que hace útil al heap es que no necesita apuntadores. Dado el índice i de un nodo:

Padre(i)	**devolver** $\lfloor i/2 \rfloor$
Izq(i)	**devolver** $2i$
Der(i)	**devolver** $2i + 1$

Idea clave

Estas tres fórmulas funcionan **porque** el árbol es completo. Al recorrer por niveles, el nivel k ocupa las posiciones 2^k a $2^{k+1} - 1$, así que los hijos de i caen siempre en $2i$ y $2i + 1$ sin dejar huecos. Si el árbol tuviera agujeros, la cuenta se rompería y habría que volver a los apuntadores.

Ejemplo 7.2. Sobre el arreglo $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$ con $\mathit{heap\text{-}size} = 10$:

i	$A[i]$	**Padre**	**Izq**	**Der**
1	16	–	$A[2] = 14$	$A[3] = 10$
2	14	$A[1] = 16$	$A[4] = 8$	$A[5] = 7$
5	7	$A[2] = 14$	$A[10] = 1$	no existe
8	2	$A[4] = 8$	no existe	no existe

El índice 5 solo tiene hijo izquierdo porque $2 \cdot 5 + 1 = 11 > 10$. Los índices del 6 en adelante son hojas.

La propiedad de heap

Definición 7.3. Un **MAX-HEAP** cumple $A[\text{Padre}(i)] \geq A[i]$ para todo nodo i distinto de la raíz. Un **MIN-HEAP** cumple lo contrario, $A[\text{Padre}(i)] \leq A[i]$.

Ojito...

La propiedad solo relaciona a cada nodo con su **padre**. No dice nada entre hermanos ni entre ramas distintas, así que un heap **no es** un árbol binario de búsqueda y recorrerlo en orden no da la secuencia ordenada. Lo único garantizado es que el máximo está en la raíz.

Altura

La **altura de un nodo** es el número de arcos de la ruta simple más larga que baja de ese nodo a una hoja. La **altura del árbol** es la de su raíz.

Como el árbol es completo, cada nivel duplica al anterior, así que con x niveles caben

$$n = \underbrace{2 \cdot 2 \cdots 2}_{x} = 2^x \quad \Longrightarrow \quad x = \log_2 n.$$

Por lo tanto la altura del árbol es $\Theta(\lg n)$, y ese logaritmo es el que va a aparecer en el costo de casi todas las operaciones.

Operaciones sobre un MAX-HEAP

Máximo

Está en la raíz por definición, así que se lee $A[1]$ y ya: $\mathcal{O}(1)$.

Inserción

Primero hay que **respetar la forma** del heap, o sea que siga siendo un árbol completo. Eso obliga a meter el elemento nuevo en la única posición que no abre huecos: al final del arreglo.

1. Insertar al final del arreglo: $\mathcal{O}(1)$.
2. Verificar si se mantiene la propiedad de heap: $\mathcal{O}(1)$, es una sola comparación contra el padre.
3. Si no se mantiene, intercambiar con el padre y repetir hacia arriba, hasta la raíz si hace falta.

El paso 3 sube a lo más la altura del árbol, así que la inserción cuesta $\mathcal{O}(\lg n)$.

Eliminación de la raíz

Eliminar el último nodo es $\mathcal{O}(1)$. Eliminar la raíz, que es lo que de verdad se necesita, se resuelve con un truco:

1. Intercambiar el último nodo con la raíz: $\mathcal{O}(1)$.
2. Eliminar el último nodo, que ahora contiene el valor que queríamos sacar: $\mathcal{O}(1)$.
3. La raíz quedó con un valor que probablemente viola la propiedad. Hay que restaurarla, y para eso está Heapify.

Heapify

Es la operación central. El escenario es muy específico:

- **Entrada:** un arreglo A y un índice i.
- **Supuesto:** los subárboles con raíz en Izq(i) y Der(i) *ya son heaps* válidos.
- **Problema:** $A[i]$ puede ser menor que alguno de sus hijos, y eso viola la propiedad de MAX-HEAP.

La solución es buscar el mayor de los dos hijos e intercambiarlo con el padre. Pero el hijo que recibió el valor viejo puede ahora violar la propiedad en su propio subárbol, así que se repite el procedimiento ahí, con ese hijo como raíz.

Algoritmo 7.1 Max-Heapify(A, i)

```
1: l ← Izq(i)
2: r ← Der(i)
3: si l ≤ heap-size[A] y A[l] > A[i] entonces
4:     mayor ← l
5: si no
6:     mayor ← i
7: fin si
8: si r ≤ heap-size[A] y A[r] > A[mayor] entonces
9:     mayor ← r
10: fin si
11: si mayor ≠ i entonces
12:     intercambiar A[i] ↔ A[mayor]
13:     Max-Heapify(A, mayor)
14: fin si
```

Ejemplo 7.4. Sobre $A = [16, 4, 10, 14, 7, 9, 3, 2, 8, 1]$ con *heap-size* $= 10$, llamando Max-Heapify$(A, 2)$. El nodo $A[2] = 4$ es menor que sus hijos, así que viola la propiedad.

Paso	i	Hijos	Acción
1	2	$A[4] = 14$, $A[5] = 7$	mayor es 14, intercambiar con $A[2]$
2	4	$A[8] = 2$, $A[9] = 8$	mayor es 8, intercambiar con $A[4]$
3	9	sin hijos	es hoja, se detiene

El arreglo queda $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$. El 4 bajó dos niveles, de la posición 2 a la 9, empujando hacia arriba a los mayores.

Costo de Heapify

En un subárbol de tamaño n con raíz en i, el trabajo son dos cosas: comparar $A[i]$ contra sus dos hijos, que es $\Theta(1)$, más la llamada recursiva sobre uno de los hijos. La recurrencia es

$$T(n) = T(n/2) + \Theta(1)\,,$$

que por el método maestro cae en el caso 2 con $a = 1$, $b = 2$, $n^{\log_2 1} = n^0 = 1$, y da

$$T(n) = \mathcal{O}(\lg n)\,.$$

Dicho de otro modo, Heapify sobre un nodo de altura h cuesta $\mathcal{O}(h)$: baja un nivel por llamada y no puede bajar más que la altura.

Observación. La cota fina es $T(n) \leq T(2n/3) + \Theta(1)$, no $T(n/2)$. El peor caso ocurre cuando el último nivel está lleno exactamente a la mitad, y ahí el subárbol izquierdo alcanza $2n/3$ nodos en vez de $n/2$. El resultado no cambia, porque $n^{\log_{3/2} 1} = 1$ igual y el caso 2 sigue dando $\mathcal{O}(\lg n)$, pero conviene saber de dónde sale ese $2/3$ cuando aparezca en el libro.

Construcción del heap

La forma obvia es insertar los elementos uno por uno: meter al final, verificar la propiedad, subir si hace falta. Son n inserciones de $\mathcal{O}(\lg n)$ cada una, o sea $\mathcal{O}(n \lg n)$.

Se puede mejor. La idea es aplicar Heapify de abajo hacia arriba sobre la mitad de los nodos, apoyándose en dos observaciones:

- Los elementos $A[\lfloor n/2 \rfloor + 1 \ldots n]$ son **hojas** del árbol, y una hoja ya es un heap válido de un solo nodo. No hay nada que hacer con ellas.
- Si se recorren los nodos **en orden decreciente** de índice, cuando le toca a i los subárboles de sus hijos ya fueron procesados y son heaps. Eso es exactamente el supuesto que Heapify necesita.

Algoritmo 7.2 Build-Max-Heap(A)

```
1: heap-size[A] ← length[A]
2: para i ← ⌊length[A]/2⌋ bajando hasta 1 hacer
3:     Max-Heapify(A, i)
4: fin para
```

Ejemplo 7.5. Construir el heap sobre $A = [4, 1, 3, 2, 16, 9, 10, 14, 8, 7]$, con $n = 10$. El ciclo va de $i = \lfloor 10/2 \rfloor = 5$ hasta 1.

i	**Qué pasa**	**Arreglo después**
5	$A[5] = 16$ contra su hijo $A[10] = 7$. Ya cumple, no se mueve nada.	4 1 3 2 16 9 10 14 8 7
4	$A[4] = 2$ contra 14 y 8. Sube el 14.	4 1 3 14 16 9 10 2 8 7
3	$A[3] = 3$ contra 9 y 10. Sube el 10.	4 1 10 14 16 9 3 2 8 7
2	$A[2] = 1$ contra 14 y 16. Sube el 16; el 1 sigue bajando y cambia con el 7.	4 16 10 14 7 9 3 2 8 1
1	$A[1] = 4$ contra 16 y 10. Sube el 16; el 4 baja hasta la posición 9.	16 14 10 8 7 9 3 2 4 1

Fíjate en los renglones de $i = 2$ y $i = 1$: ahí Heapify no hizo un solo intercambio sino una cadena de ellos, porque el valor desplazado siguió violando la propiedad más abajo.

Por qué cuesta $\mathcal{O}(n)$ y no $\mathcal{O}(n \lg n)$

La cuenta rápida dice: hay $\mathcal{O}(n)$ llamadas a Heapify y cada una cuesta $\mathcal{O}(\lg n)$, así que $\mathcal{O}(n \lg n)$. Esa cota es **correcta pero no justa**, y el motivo es que casi todas las llamadas son sobre nodos bajitos, donde Heapify apenas trabaja.

La cota fina sale de contar por altura, apoyándose en una propiedad del árbol completo: en un heap de n elementos hay a lo más $\lceil n/2^{h+1} \rceil$ nodos de altura h. Como Heapify en un nodo de altura h cuesta $\mathcal{O}(h)$, el total es

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \mathcal{O}(h) = \mathcal{O}\left(n \cdot \frac{1}{2} \sum_{h=0}^{\lfloor \lg n \rfloor} \frac{h}{2^h} \right).$$

Falta evaluar esa sumatoria, y el truco es derivar la serie geométrica. Partiendo de $\sum_{k=0}^{\infty} x^k = 1/(1-x)$ para $|x| < 1$, se derivan los dos lados:

$$\frac{d}{dx}\left(\sum_{k=0}^{\infty} x^k \right) = \frac{d}{dx}\left(\frac{1}{1-x} \right) \quad \Longrightarrow \quad \sum_{k=0}^{\infty} k\, x^{k-1} = \frac{1}{(1-x)^2}.$$

Multiplicando ambos lados por x:

$$\sum_{k=0}^{\infty} k\, x^k = \frac{x}{(1-x)^2}.$$

En nuestro caso $k = h$ y $x = 1/2$, así que

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1-1/2)^2} = \frac{1/2}{1/4} = 2.$$

Sustituyendo, el costo de Build-Max-Heap es

$$\mathcal{O}\Big(n \cdot \tfrac{1}{2} \cdot 2 \Big) = \mathcal{O}(n)\,.$$

Idea clave

De dónde sale la ganancia: la mitad de los nodos son hojas y cuestan cero, un cuarto tiene altura 1 y cuesta uno, un octavo tiene altura 2, y así. Los nodos caros son poquísimos, y la serie $\sum h/2^h$ converge a 2 en vez de crecer con $\lg n$. Por eso construir el heap de golpe es $\mathcal{O}(n)$, mientras que construirlo insertando uno por uno es $\mathcal{O}(n \lg n)$.

Resumen

Operación	**Costo**
Encontrar el máximo	$\mathcal{O}(1)$
Insertar	$\mathcal{O}(\lg n)$
Eliminar la raíz, vía Heapify	$\mathcal{O}(\lg n)$
Heapify en un nodo de altura h	$\mathcal{O}(h)$
Construir insertando uno por uno	$\mathcal{O}(n \lg n)$
Build-Max-Heap	$\mathcal{O}(n)$

8. Mochila fraccional, actividades y divide y vencerás *8 de octubre de 2026*

Resumen rápido. Dos problemas clásicos que se resuelven con algoritmos voraces: la mochila fraccional y la selección de actividades cuyos horarios se traslapan. Para cada uno, el criterio voraz que funciona, un ejemplo completo, por qué da el óptimo y qué criterios parecidos fallan. Al final arrancó la técnica de divide y vencerás.

Repaso: algoritmos voraces

Un algoritmo voraz (*greedy*) resuelve un problema de optimización tomando, en cada paso, la opción que se ve mejor en ese momento, sin revisar nunca una decisión ya tomada. Suelen ser lineales o polinomiales. El riesgo es que lo mejor en el momento no siempre lleva al óptimo global, así que cada algoritmo voraz necesita un argumento de por qué su criterio funciona en ese problema en particular. Los dos de esta clase son el ejemplo de cuándo sí, y una variante de la mochila es el ejemplo de cuándo no.

Mochila fraccional

Definición 8.1. Hay n objetos; el objeto i pesa w_i y vale v_i. La mochila aguanta un peso máximo W. De cada objeto se puede tomar una fracción $x_i \in [0, 1]$, como si fuera polvo de oro y no un lingote. Se busca

$$\text{máx} \sum_{i=1}^{n} v_i\, x_i \qquad \text{sujeto a} \qquad \sum_{i=1}^{n} w_i\, x_i \le W.$$

El criterio voraz

Lo que importa no es el valor de un objeto sino cuánto vale cada unidad de peso que ocupa: su **densidad** v_i/w_i. El algoritmo ordena los objetos de mayor a menor densidad y los va metiendo completos mientras quepan. Cuando el siguiente ya no cabe completo, mete la fracción que alcance a llenar la mochila y termina.

Algoritmo 8.1 Mochila-Fraccional(w, v, W)

```
ordenar los objetos de mayor a menor v_i/w_i
libre ← W,    valor ← 0
para i ← 1 hasta n hacer
    si w_i ≤ libre entonces
        x_i ← 1
    si no
        x_i ← libre/w_i
    fin si
    libre ← libre − x_i w_i
    valor ← valor + x_i v_i
    si libre = 0 entonces
        terminar
    fin si
fin para
devolver valor
```

Ordenar cuesta $\mathcal{O}(n \lg n)$ y la pasada es $\mathcal{O}(n)$, así que el algoritmo es $\mathcal{O}(n \lg n)$.

Ejemplo 8.2. Mochila de $W = 50$ con tres objetos:

Objeto	Peso	Valor	Densidad
1	10	60	6
2	20	100	5
3	30	120	4

Ya están en orden de densidad. Paso a paso:

Objeto	Qué se toma	Libre después	Valor acumulado
1	completo, 10 de peso	40	60
2	completo, 20 de peso	20	160
3	no cabe completo: $20/30 = 2/3$	0	$160 + \frac{2}{3} \cdot 120 = 240$

El valor máximo es 240.

Por qué da el óptimo

Supóngase una solución que deja espacio sin usar de un objeto denso mientras carga un poco de uno menos denso. Si se saca una unidad de peso del menos denso y se mete una del más denso, el peso no cambia y el valor sube, o se queda igual si las densidades empatan. Repitiendo ese intercambio, cualquier solución se puede convertir en la del algoritmo sin perder valor. Por eso ninguna solución vale más que la voraz.

Ojito...

Lo que hace funcionar al criterio es poder partir los objetos. En la **mochila 0/1**, donde cada objeto se toma completo o no se toma, el mismo criterio falla. Con el ejemplo de arriba, el voraz mete los objetos 1 y 2, y el 3 ya no cabe: vale 160. Pero los objetos 2 y 3 juntos pesan exactamente 50 y valen 220. Al no poder partir el objeto 3, el espacio que deja libre el voraz se desperdicia. La mochila 0/1 se resuelve con programación dinámica, no con un algoritmo voraz.

Selección de actividades

Es el problema de los horarios que se traslapan.

Definición 8.3. Hay n actividades que quieren usar el mismo recurso, por ejemplo un salón, y solo una puede usarlo a la vez. La actividad a_i ocupa el intervalo $[s_i, f_i)$: empieza en s_i y termina en f_i. Dos actividades son **compatibles** si sus intervalos no se traslapan, o sea si una termina antes de que empiece la otra, $f_i \leq s_j$ o $f_j \leq s_i$. Se busca el mayor número posible de actividades compatibles entre sí.

El criterio voraz

Elegir siempre la actividad que **termina primero** entre las que todavía son compatibles con lo ya elegido. La razón: terminar pronto deja libre el mayor tiempo posible para todas las demás.

Algoritmo 8.2 Seleccion-Actividades(s, f)

```
ordenar las actividades por tiempo de fin f_i creciente
A ← {a_1},   ultima ← 1
para i ← 2 hasta n hacer
    si s_i ≥ f_ultima entonces
        A ← A ∪ {a_i}
        ultima ← i
    fin si
fin para
devolver A
```

Igual que la mochila: $\mathcal{O}(n \lg n)$ por el ordenamiento y $\mathcal{O}(n)$ por la pasada.

Ejemplo 8.4. Once actividades, ya ordenadas por su tiempo de fin:

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

Se recorren en orden, comparando el inicio de cada una contra el fin de la última elegida:

Actividad	**Intervalo**	**Fin de la última**	**Decisión**
a_1	$[1, 4)$	—	se toma
a_2	$[3, 5)$	4	$3 < 4$, se traslapa
a_3	$[0, 6)$	4	se traslapa
a_4	$[5, 7)$	4	$5 \geq 4$, se toma
a_5	$[3, 9)$	7	se traslapa
a_6	$[5, 9)$	7	se traslapa
a_7	$[6, 10)$	7	se traslapa
a_8	$[8, 11)$	7	$8 \geq 7$, se toma
a_9	$[8, 12)$	11	se traslapa
a_{10}	$[2, 14)$	11	se traslapa
a_{11}	$[12, 16)$	11	$12 \geq 11$, se toma

Resultado: $\{a_1, a_4, a_8, a_{11}\}$, cuatro actividades. Revisando todos los subconjuntos posibles, ninguno compatible tiene cinco.

Por qué da el óptimo

Sea a_1 la actividad que termina primero, y tómese cualquier solución óptima. Si no empieza con a_1, se le quita su primera actividad y se pone a_1 en su lugar. Como a_1 termina antes o al mismo tiempo, no choca con nada de lo que seguía, y la solución sigue teniendo el mismo número de actividades. Así que siempre existe una solución óptima que empieza con a_1. Después de elegirla, lo que queda es el mismo problema con las actividades que empiezan después de f_1, y el argumento se repite.

Criterios que parecen razonables y fallan

- **La que empieza primero.** Con $[0, 10)$, $[1, 2)$ y $[3, 4)$ elige $[0, 10)$, que choca con las otras dos: se queda con una sola. El óptimo es dos, $[1, 2)$ y $[3, 4)$.

- **La más corta.** Con $[0,5)$, $[4,6)$ y $[5,10)$ elige $[4,6)$, que dura 2 pero se traslapa con las otras dos: otra vez una sola. El óptimo es $[0,5)$ y $[5,10)$.

Las dos fallan por lo mismo: miran una propiedad de la actividad que no dice cuánto tiempo deja libre. El tiempo de fin es justo eso.

Observación. Hay un problema hermano que también se resuelve con un voraz: en lugar de elegir cuántas actividades caben en un salón, programarlas *todas* usando el menor número de salones. Ahí se ordenan por tiempo de *inicio* y cada actividad va a un salón que ya esté libre, o se abre uno nuevo si no hay.

Divide y vencerás

La idea es simple: dado un problema grande, se divide en subproblemas parecidos de menor tamaño, se resuelven, y con sus soluciones se arma la del problema general. Formalmente son tres pasos:

1. **Dividir** el problema en a subproblemas del mismo tipo, cada uno de tamaño n/b.
2. **Vencer**: resolver cada subproblema recursivamente. Cuando uno ya es lo bastante chico, el caso base, se resuelve directo.
3. **Combinar** las soluciones de los subproblemas en la solución del problema original.

Si dividir y combinar cuestan $f(n)$, el tiempo total cumple

$$T(n) = a\,T(n/b) + f(n),$$

que es justo la forma que resuelve el método maestro de la clase del 15 de septiembre. Por eso las recurrencias se vieron antes: son la herramienta para medir estos algoritmos.

Ejemplo 8.5 (Ordenamiento por mezcla)**.** Para ordenar $[5,2,4,7,1,3,2,6]$ se parte a la mitad hasta llegar a arreglos de un elemento, que ya están ordenados, y luego se van mezclando de dos en dos:

Etapa	**Arreglos**
dividir	$[5,2,4,7]$ $[1,3,2,6]$
dividir	$[5,2]$ $[4,7]$ $[1,3]$ $[2,6]$
caso base	$[5]$ $[2]$ $[4]$ $[7]$ $[1]$ $[3]$ $[2]$ $[6]$
combinar	$[2,5]$ $[4,7]$ $[1,3]$ $[2,6]$
combinar	$[2,4,5,7]$ $[1,2,3,6]$
combinar	$[1,2,2,3,4,5,6,7]$

Mezclar dos arreglos ya ordenados cuesta $\Theta(n)$: basta comparar sus primeros elementos y sacar el menor, una y otra vez. Son dos subproblemas de la mitad del tamaño:

$$T(n) = 2\,T(n/2) + \Theta(n)\,.$$

Con $a = 2$ y $b = 2$, $n^{\log_2 2} = n$, que es del mismo orden que $f(n)$: caso 2 del método maestro, $T(n) = \Theta(n \lg n)$.

Idea clave

La diferencia con un algoritmo voraz está en cuántos caminos se siguen. El voraz toma una decisión y se queda con un solo subproblema, el que deja esa decisión. Divide y vencerás resuelve *todos* los subproblemas en que partió el original y después combina sus respuestas.

9. Multiplicación de enteros grandes

13 de octubre de 2026

Resumen rápido. Divide y vencerás aplicado a multiplicar dos enteros de n dígitos. El método de la escuela cuesta $\Theta(n^2)$. Partir cada número a la mitad da cuatro multiplicaciones de la mitad del tamaño y el mismo $\Theta(n^2)$: dividir por sí solo no ahorra nada. El truco de Karatsuba obtiene el término de en medio con una sola multiplicación en lugar de dos, y con tres subproblemas en lugar de cuatro el costo baja a $\Theta\left(n^{\log_2 3}\right) \approx \Theta(n^{1.585})$.

El problema

Multiplicar dos enteros x y y de n dígitos cada uno. El procesador multiplica números de 64 bits en una instrucción, pero la criptografía trabaja con números mucho más grandes: una llave RSA de 2048 bits tiene unos 617 dígitos decimales. Las bibliotecas de enteros de tamaño arbitrario, como los `int` de Python, tienen que multiplicar números así con operaciones sobre dígitos. El costo se mide en operaciones de un dígito.

El método de la escuela

Se multiplica x por cada dígito de y, se recorre cada renglón un lugar a la izquierda y se suman.

Ejemplo 9.1 (1234×5678)**.** Los cuatro renglones y su suma:

$$\begin{array}{rr} 1234 \times 8 = & 9872 \\ 1234 \times 7 = & 8638 \\ 1234 \times 6 = & 7404 \\ 1234 \times 5 = & 6170 \\ \hline \text{suma:} & 7006652 \end{array}$$

Cada uno de los 4 dígitos de y se multiplica por cada uno de los 4 de x: 16 productos de un dígito.

Con n dígitos son n^2 productos de un dígito, más sumar n renglones de unos $2n$ dígitos. El método es $\Theta(n^2)$. Duplicar el tamaño de los números cuadruplica el tiempo.

Primer intento: partir a la mitad

Con $m = n/2$, cada número se parte en su mitad alta y su mitad baja:

$$x = a \cdot 10^m + b, \qquad y = c \cdot 10^m + d.$$

Para 1234 y 5678 con $m = 2$: $a = 12$, $b = 34$, $c = 56$, $d = 78$. Multiplicando,

$$x\,y = ac \cdot 10^{2m} + (ad + bc) \cdot 10^m + bd.$$

Ejemplo 9.2.

$$\begin{aligned} ac &= 12 \cdot 56 = 672, & ad &= 12 \cdot 78 = 936, \\ bc &= 34 \cdot 56 = 1904, & bd &= 34 \cdot 78 = 2652. \end{aligned}$$

$$x\,y = 672 \cdot 10^4 + (936 + 1904) \cdot 10^2 + 2652 = 6\,720\,000 + 284\,000 + 2\,652 = 7\,006\,652.$$

Multiplicar por 10^k es agregar k ceros, un corrimiento que cuesta $\Theta(n)$, y las sumas también son $\Theta(n)$. Lo caro son las cuatro multiplicaciones de números de $n/2$ dígitos, que se resuelven recursivamente:

$$T(n) = 4\,T(n/2) + \Theta(n)\,.$$

Con $a = 4$ y $b = 2$, $n^{\log_2 4} = n^2$ crece polinomialmente más rápido que $f(n) = n$: caso 1 del método maestro, $T(n) = \Theta(n^2)$.

Ojito...

Dividir no ahorró nada. Cuatro problemas de la mitad del tamaño cuestan $4 \cdot (n/2)^2 = n^2$, lo mismo que el original. Divide y vencerás no es más rápido por partir el problema; es más rápido cuando los subproblemas, juntos, son menos trabajo que el original. Para eso hay que reducir *cuántos* son.

El truco de Karatsuba

En 1960 Kolmogorov propuso en un seminario que multiplicar no se podía hacer en menos de $\Theta(n^2)$. Anatoli Karatsuba, un estudiante de 23 años, encontró en una semana cómo hacerlo con tres multiplicaciones en lugar de cuatro.

La observación es que no hacen falta ad y bc por separado, solo su suma. Y

$$(a+b)(c+d) = ac + ad + bc + bd,$$

así que

$$ad + bc = (a+b)(c+d) - ac - bd.$$

ac y bd ya se iban a calcular de todos modos. Basta con tres multiplicaciones:

$$p_1 = ac, \qquad p_2 = bd, \qquad p_3 = (a+b)(c+d),$$

$$x\,y = p_1 \cdot 10^{2m} + (p_3 - p_1 - p_2) \cdot 10^m + p_2.$$

Ejemplo 9.3 (1234×5678 otra vez)**.** $p_1 = 672$ y $p_2 = 2652$ como antes. El tercero:

$$p_3 = (12+34)(56+78) = 46 \cdot 134 = 6164,$$

$$p_3 - p_1 - p_2 = 6164 - 672 - 2652 = 2840,$$

el mismo $ad + bc = 936 + 1904$ de antes, con una multiplicación menos. El resultado sale igual: $7\,006\,652$.

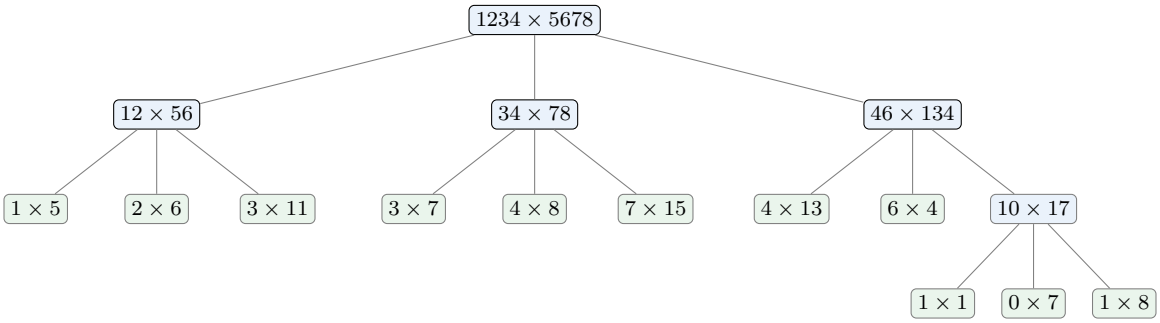
Se cambió una multiplicación por tres sumas y restas extra. Las sumas cuestan $\Theta(n)$ y las multiplicaciones mucho más, así que el cambio conviene.

Algoritmo 9.1 KARATSUBA(x, y)

si $x < 10$ **o** $y < 10$ **entonces**
  devolver $x \cdot y$ *// un factor de un dígito: directo*
fin si
$n \leftarrow$ número de dígitos del mayor de x y y
$m \leftarrow \lfloor n/2 \rfloor$
$a \leftarrow \lfloor x/10^m \rfloor$, $\quad b \leftarrow x \bmod 10^m$
$c \leftarrow \lfloor y/10^m \rfloor$, $\quad d \leftarrow y \bmod 10^m$
$p_1 \leftarrow$ KARATSUBA(a, c)
$p_2 \leftarrow$ KARATSUBA(b, d)
$p_3 \leftarrow$ KARATSUBA$(a+b,\ c+d)$
devolver $p_1 \cdot 10^{2m} + (p_3 - p_1 - p_2) \cdot 10^m + p_2$

La recursión completa de 1234×5678

Cada producto se parte en tres hasta que uno de los factores tiene un solo dígito. En verde, los casos base:



En cada nodo, los tres hijos son ac, bd y $(a+b)(c+d)$, en ese orden. Los nodos que no son base se arman así, de abajo hacia arriba:

Producto	m	$a, b \mid c, d$	p_1	p_2	p_3	$p_3 - p_1 - p_2$	**Resultado**
$10 \cdot 17$	1	$1, 0 \mid 1, 7$	1	0	8	7	$100 + 70 + 0 = 170$
$12 \cdot 56$	1	$1, 2 \mid 5, 6$	5	12	33	16	$500 + 160 + 12 = 672$
$34 \cdot 78$	1	$3, 4 \mid 7, 8$	21	32	105	52	$2100 + 520 + 32 = 2652$
$46 \cdot 134$	1	$4, 6 \mid 13, 4$	52	24	170	94	$5200 + 940 + 24 = 6164$
$1234 \cdot 5678$	2	$12, 34 \mid 56, 78$	672	2652	6164	2840	7 006 652

Ojito...

$a + b$ puede tener un dígito más que a y b: $56 + 78 = 134$. Por eso aparece $46 \cdot 134$, con factores de tamaños distintos, y por eso la rama de la derecha baja un nivel más. No cambia el análisis: el tercer subproblema es de tamaño $n/2 + 1$ en lugar de $n/2$, y ese $+1$ no altera el orden de crecimiento.

Observación. Con números de 4 dígitos Karatsuba no es más rápido que la escuela: hace casi las mismas multiplicaciones de un dígito y además sumas, restas y llamadas recursivas. La ventaja aparece con números grandes, que es lo que dice el análisis.

Análisis

Tres subproblemas de la mitad del tamaño, más sumas, restas y corrimientos lineales:

$$T(n) = 3\,T(n/2) + \Theta(n)\,.$$

Con $a = 3$, $b = 2$ y $f(n) = n$:

$$n^{\log_b a} = n^{\log_2 3} = n^{1.585}.$$

$f(n) = n = \mathcal{O}(n^{1.585-\varepsilon})$ con $\varepsilon \approx 0.585$, así que es el caso 1 del método maestro:

$$T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta\left(n^{1.585}\right).$$

Con el árbol de recursión

Nivel	Subproblemas	Tamaño de cada uno	Trabajo del nivel
0	1	n	cn
1	3	$n/2$	$3 \cdot cn/2 = \frac{3}{2}cn$
2	9	$n/4$	$9 \cdot cn/4 = \left(\frac{3}{2}\right)^2 cn$
i	3^i	$n/2^i$	$\left(\frac{3}{2}\right)^i cn$
$\log_2 n$	$3^{\log_2 n} = n^{\log_2 3}$	1	$n^{\log_2 3}$

El trabajo crece $3/2$ veces en cada nivel, así que manda el último: las hojas. El árbol tiene $\log_2 n$ niveles y en el último hay $3^{\log_2 n} = n^{\log_2 3}$ casos base. Con cuatro subproblemas serían $4^{\log_2 n} = n^2$ hojas; pasar de 4 a 3 es lo que cambia el exponente.

Cuánto se gana

n (dígitos)	n^2	$n^{1.585}$	Karatsuba es
10	100	38	2.6 veces menos
100	10 000	1 479	6.8 veces menos
1 000	1 000 000	56 871	17.6 veces menos
1 000 000	10^{12}	3.2×10^9	309 veces menos

La ventaja crece con n sin límite, porque la razón entre los dos es $n^{2-1.585} = n^{0.415}$.

Ojito...

La tabla ignora las constantes. Karatsuba hace más sumas, restas y llamadas por nivel, y con números chicos eso pesa más que la multiplicación que se ahorra. Por eso las implementaciones reales, como las de Python o de la biblioteca GMP, usan la escuela para números chicos y solo cambian a Karatsuba a partir de cierto tamaño, del orden de cientos de dígitos. Además trabajan en binario: se parte por bits, y multiplicar por 2^m es un corrimiento de bits.

La lección general

Idea clave

En $T(n) = a\,T(n/b) + f(n)$, cuando dividir y combinar son baratos, manda $n^{\log_b a}$, y lo que más importa es a: *cuántos* subproblemas se generan. Gastar un poco más en combinar para generar un subproblema menos es el truco que mejora el exponente.

El mismo truco aparece en otros problemas:

- **Strassen** (1969), para multiplicar matrices de $n \times n$: partidas en bloques de la mitad, el método directo hace 8 productos de bloques, $\Theta(n^3)$. Strassen lo hace con 7 y queda $\Theta\left(n^{\log_2 7}\right) \approx \Theta(n^{2.807})$.
- **Toom-Cook**: parte cada número en tres partes y se las arregla con 5 multiplicaciones en lugar de 9: $\Theta\left(n^{\log_3 5}\right) \approx \Theta(n^{1.465})$.
- Para números enormes se usan algoritmos basados en la transformada rápida de Fourier, como el de Schönhage y Strassen, $\mathcal{O}(n \log n \log\log n)$. En 2019 Harvey y van der Hoeven llegaron a $\mathcal{O}(n \log n)$, que se cree que es lo mejor posible.

10. Programación dinámica

15 de octubre de 2026

Resumen rápido. Programación dinámica resuelve un problema combinando soluciones de subproblemas, como divide y vencerás, pero para el caso en que los subproblemas se repiten: cada uno se resuelve una sola vez y su resultado se guarda en una tabla. Hace falta que el problema tenga subestructura óptima y subproblemas superpuestos. Se ve con Fibonacci, que pasa de exponencial a lineal; con el cambio de monedas, donde el voraz falla; y a fondo con la mochila 0/1, la que quedó pendiente en la clase del 8: el voraz daba 160 y la programación dinámica encuentra el óptimo, 220.

La idea

Divide y vencerás parte un problema en subproblemas **independientes**: en Karatsuba, ac, bd y $(a+b)(c+d)$ son tres multiplicaciones distintas que nunca se vuelven a pedir. En muchos problemas no es así. Al partir, los mismos subproblemas aparecen una y otra vez, y la recursión los resuelve desde cero cada vez.

Definición 10.1 (Programación dinámica)**.** Una técnica para resolver problemas, casi siempre de optimización, combinando soluciones de subproblemas, en la que cada subproblema distinto se resuelve **una sola vez** y su solución se guarda en una tabla para consultarla cada vez que se vuelva a necesitar.

Se aplica cuando el problema tiene dos propiedades:

- **Subestructura óptima.** Una solución óptima del problema contiene soluciones óptimas de sus subproblemas. Eso permite armar la solución grande a partir de las chicas.
- **Subproblemas superpuestos.** La recursión natural visita los mismos subproblemas muchas veces. Si no se repitieran, guardarlos no ahorraría nada, y bastaría con divide y vencerás.

Observación. El nombre es de Richard Bellman, en los años cincuenta. "Programación" no se refiere a escribir código sino a planear con tablas, como en "programación lineal". "Dinámica" porque las decisiones se toman en etapas, una tras otra.

	Divide y vencerás	**Voraz**	**Programación dinámica**
Subproblemas	independientes, se resuelven todos	queda uno solo, el que deja la decisión	se repiten; cada uno se resuelve una vez
Decisión	no hay que elegir	se elige lo mejor del momento y no se revisa	se prueban todas las opciones y se elige la mejor
Ejemplo	mergesort, Karatsuba	mochila fraccional	mochila 0/1

Primer ejemplo: Fibonacci

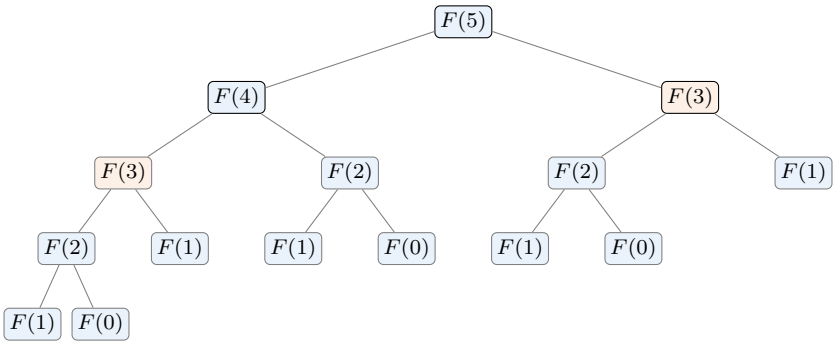
$$F(0) = 0, \qquad F(1) = 1, \qquad F(n) = F(n-1) + F(n-2).$$

La recursión directa copia la definición:

Algoritmo 10.1 Fib(n)

```
1: si n ≤ 1 entonces
2:     devolver n
3: fin si
4: devolver Fib(n − 1) + Fib(n − 2)
```

Para calcular $F(5)$ hace este árbol de llamadas:



Son 15 llamadas para obtener $F(5) = 5$, y casi todas repiten trabajo: $F(3)$ se calcula dos veces (en naranja), $F(2)$ tres veces, $F(1)$ cinco. Pero solo hay 6 subproblemas distintos, de $F(0)$ a $F(5)$.

El número de llamadas crece igual que la propia sucesión: para $F(n)$ son $2F(n+1) - 1$, que es $\Theta(\varphi^n)$ con $\varphi = (1 + \sqrt{5})/2 \approx 1.618$. Para $F(30) = 832\,040$ la recursión hace $2\,692\,537$ llamadas, cuando bastaba con 31 cálculos.

Dos formas de no repetir

Memoización, de arriba hacia abajo. Se deja la recursión igual, pero antes de calcular algo se revisa si ya está en una tabla M, y al calcularlo se guarda:

Algoritmo 10.2 Fib-Memo(n), con M inicializada en nulo

```
1: si M[n] ≠ nulo entonces
2:     devolver M[n]                          // ya se calculó: no se repite
3: fin si
4: si n ≤ 1 entonces
5:     M[n] ← n
6: si no
7:     M[n] ← Fib-Memo(n − 1) + Fib-Memo(n − 2)
8: fin si
9: devolver M[n]
```

Tabulación, de abajo hacia arriba. Se llena la tabla desde los casos base, en un orden en el que todo lo que hace falta ya está calculado:

Algoritmo 10.3 Fib-Tabla(n)

```
F[0] ← 0,    F[1] ← 1
para i ← 2 hasta n hacer
    F[i] ← F[i − 1] + F[i − 2]
fin para
devolver F[n]
```

i	0	1	2	3	4	5	6	7	8	9	10
$F[i]$	0	1	1	2	3	5	8	13	21	34	55

Las dos son $\Theta(n)$: cada uno de los $n+1$ subproblemas se resuelve una vez, en tiempo constante. La tabulación además deja ver que cada valor solo necesita los dos anteriores, así que basta con guardar dos números: espacio $\mathcal{O}(1)$.

Idea clave

El costo de un algoritmo de programación dinámica es casi siempre

$$\text{(subproblemas distintos)} \times \text{(trabajo por subproblema)},$$

donde el trabajo de cada uno se cuenta suponiendo que los demás ya están en la tabla. Para Fibonacci: $n+1$ subproblemas por $\mathcal{O}(1)$ cada uno.

El método en cuatro pasos

1. **Caracterizar la estructura de una solución óptima.** Qué decisión se toma y qué subproblema queda después de tomarla. De aquí sale qué hay que saber para describir un subproblema: el **estado**.

2. **Definir el valor óptimo de forma recursiva**, en términos de los subproblemas. Es la recurrencia.

3. **Calcular los valores de abajo hacia arriba**, llenando una tabla con un subproblema por casilla.

4. **Reconstruir la solución** con la información de la tabla: no solo cuánto vale el óptimo, sino cuál es.

Segundo ejemplo: cambio de monedas

Con monedas de 1, 3 y 4 pesos, ¿cuál es el menor número de monedas que suman 6? El voraz de la clase del 8 toma siempre la moneda más grande que quepa: $4+1+1$, tres monedas. Pero $3+3$ son dos.

Paso 1. La última moneda que se usa es de 1, de 3 o de 4. Si fue la de c pesos, lo que falta es pagar $a-c$ con el menor número de monedas: el mismo problema, más chico. El estado es solo la cantidad que falta pagar.

Paso 2. Sea $C[a]$ el menor número de monedas para pagar a:

$$C[0] = 0, \qquad C[a] = 1 + \min_{\substack{c \in \{1,3,4\} \\ c \le a}} C[a-c].$$

Se prueba cada moneda como la última y se queda la que deja el resto más barato.

Paso 3. De $a = 0$ a 6:

a	**Última** 1	**Última** 3	**Última** 4	$C[a]$	**Mejor última**
1	$1 + C[0] = 1$	—	—	1	1
2	$1 + C[1] = 2$	—	—	2	1
3	$1 + C[2] = 3$	$1 + C[0] = 1$	—	1	3
4	$1 + C[3] = 2$	$1 + C[1] = 2$	$1 + C[0] = 1$	1	4
5	$1 + C[4] = 2$	$1 + C[2] = 3$	$1 + C[1] = 2$	2	1
6	$1 + C[5] = 3$	$1 + C[3] = 2$	$1 + C[2] = 3$	2	3

Paso 4. Para 6 la mejor última moneda es la de 3; quedan 3, cuya mejor última es otra de 3; queda 0. Solución: $3 + 3$.

En $a = 6$ la tabla muestra justo dónde se equivoca el voraz: tomar 4 deja 2 pesos, que cuestan dos monedas; tomar 3 deja 3, que cuestan una. El voraz no compara, la tabla sí.

La mochila 0/1

Definición 10.2. Hay n objetos; el objeto i pesa w_i y vale v_i, y la mochila aguanta un peso W. Cada objeto se toma **completo o no se toma**: $x_i \in \{0, 1\}$. Se busca

$$\text{máx} \sum_{i=1}^{n} v_i\, x_i \qquad \text{sujeto a} \qquad \sum_{i=1}^{n} w_i\, x_i \le W.$$

Por qué no basta con lo que ya se tiene

- **Voraz.** En la clase del 8, con $W = 50$ y objetos de pesos $10, 20, 30$ y valores $60, 100, 120$, el voraz por densidad toma los dos primeros y vale 160; el tercero ya no cabe completo y sobran 20 de espacio. El óptimo es tomar el segundo y el tercero: 220.
- **Fuerza bruta.** Probar todos los subconjuntos de objetos funciona, pero son 2^n. Con 50 objetos son más de 10^{15}.

Para que la tabla quede chica, el ejemplo de esta sección es el mismo de la clase del 8 con los pesos divididos entre 10. Las decisiones no cambian:

Objeto i	**Peso** w_i	**Valor** v_i	**Densidad**
1	1	60	60
2	2	100	50
3	3	120	40

capacidad $W = 5$.

Paso 1: la estructura de una solución óptima

Hay que fijarse en una sola decisión: qué pasa con el **último** objeto, el n. En una solución óptima, o está o no está.

- **Si no está**, el resto de la solución es una forma de llenar la misma mochila de capacidad W con los objetos 1 a $n - 1$, y tiene que ser la mejor posible.
- **Si está**, ya ocupa w_n de espacio y aporta v_n. El resto es una forma de llenar el espacio que sobra, $W - w_n$, con los objetos 1 a $n - 1$, y también tiene que ser la mejor posible.

¿Por qué "tiene que ser la mejor posible"? Porque si el resto no fuera óptimo, se podría cambiar por uno mejor que pese lo mismo o menos, y la solución completa mejoraría. Eso contradice que era óptima. Es la subestructura óptima, y se demuestra siempre así: cortando la parte que no es óptima y pegando una mejor.

Idea clave

Aquí está la clave de todo el problema. Después de decidir sobre un objeto, el subproblema que queda no es solo "los objetos que faltan": también importa **cuánto espacio queda**. Por eso el estado necesita dos números,

$$(i,\ w) = (\text{cuántos objetos se pueden usar},\ \text{cuánta capacidad hay}).$$

El voraz falla justo por ignorar el segundo: decide sin preguntarse qué tan bien se va a poder aprovechar el espacio que deja.

Paso 2: la recurrencia

Sea

$$K[i][w] = \text{el mayor valor que se puede obtener usando solo los objetos } 1,\dots,i \text{ con capacidad } w.$$

La respuesta al problema es $K[n][W]$.

Casos base. Sin objetos, o sin capacidad, no se puede meter nada:

$$K[0][w] = 0, \qquad K[i][0] = 0.$$

Caso general. Para el objeto i hay dos opciones:

$$K[i][w] = \begin{cases} K[i-1][w] & \text{si } w_i > w \quad \text{(no cabe)}, \\ \max\Big(\underbrace{K[i-1][w]}_{\text{no tomarlo}},\ \underbrace{v_i + K[i-1][w-w_i]}_{\text{tomarlo}}\Big) & \text{si } w_i \le w. \end{cases}$$

- **No tomarlo**: la capacidad sigue siendo w y quedan los objetos anteriores. Vale lo mismo que la mejor solución de $1,\dots,i-1$ con capacidad w: la casilla de **arriba**.
- **Tomarlo**: se gana v_i, se pierden w_i de capacidad, y lo que sobra se llena lo mejor posible con los anteriores. Es la casilla del renglón de arriba, w_i **columnas a la izquierda**, más v_i.

Ojito...

Programar esta recurrencia tal cual, con recursión, hace dos llamadas por objeto: 2^n, lo mismo que la fuerza bruta. Pero solo hay $(n+1)(W+1)$ pares (i,w) distintos, así que con n grande y W chico casi todas las llamadas repiten un estado que ya se calculó. Son los subproblemas superpuestos, y la tabla los aprovecha.

Paso 3: llenar la tabla

La tabla tiene un renglón por cada $i = 0,\dots,3$ y una columna por cada capacidad $w = 0,\dots,5$. Cada casilla depende solo del renglón de arriba, así que se llena renglón por renglón.

Renglón 0**.** Sin objetos, todo vale 0.

Renglón 1, objeto 1 con peso 1 y valor 60. En $w = 0$ no cabe y queda 0. Desde $w = 1$ cabe, y tomarlo da $60 + K[0][w-1] = 60 + 0 = 60$ contra 0 de no tomarlo. Renglón: $0, 60, 60, 60, 60, 60$.

Renglón 2, objeto 2 con peso 2 y valor 100:

w	**¿Cabe?**	**No tomar** $K[1][w]$	**Tomar** $100 + K[1][w-2]$	$K[2][w]$	**Qué pasó**
0	no	0	—	0	se copia
1	no	60	—	60	se copia
2	sí	60	$100 + 0 = 100$	100	el 2 solo gana
3	sí	60	$100 + 60 = 160$	160	caben los dos
4	sí	60	$100 + 60 = 160$	160	igual
5	sí	60	$100 + 60 = 160$	160	igual

Renglón 3, objeto 3 con peso 3 y valor 120:

w	**¿Cabe?**	**No tomar** $K[2][w]$	**Tomar** $120 + K[2][w-3]$	$K[3][w]$	**Qué pasó**
0	no	0	—	0	se copia
1	no	60	—	60	se copia
2	no	100	—	100	se copia
3	sí	160	$120 + 0 = 120$	160	mejor 1 y 2
4	sí	160	$120 + 60 = 180$	180	mejor 3 y 1
5	sí	160	$120 + 100 = 220$	220	mejor 3 y 2

Vale la pena leer despacio las tres últimas filas. En $w = 3$ el objeto 3 cabe, pero tomarlo deja 0 de espacio y vale 120, menos que los objetos 1 y 2 juntos. En $w = 4$ tomarlo deja 1 de espacio, que el renglón de arriba sabe aprovechar con el objeto 1: $120 + 60 = 180$. En $w = 5$ deja 2, y lo mejor para 2 de espacio es el objeto 2: $120 + 100 = 220$.

La tabla completa, con la casilla de la respuesta y las dos de las que depende:

$i \setminus w$	0	1	2	3	4	5	
0	0	0	0	0	0	0	
1	0	60	60	60	60	60	objeto 1: peso 1, valor 60
2	0	60	100	160	160	160	objeto 2: peso 2, valor 100
3	0	60	100	160	180	220	objeto 3: peso 3, valor 120

$K[2][5] = 160$: no tomar el objeto 3 (la casilla de arriba)
$K[2][2] = 100$: tomarlo ($w_3 = 3$ columnas a la izquierda), $100 + 120 = 220$
$K[3][5] = \text{máx}(160,\ 220) = 220$: la respuesta

$K[3][5]$ compara la casilla de arriba, 160, con la que está $w_3 = 3$ columnas a la izquierda en el renglón de arriba, 100, más el valor del objeto, 120. Gana tomarlo: 220.

Paso 4: reconstruir qué objetos se tomaron

La tabla dice cuánto vale el óptimo, no cuáles objetos lo forman. Se recupera de atrás hacia adelante, empezando en $K[n][W]$. En cada renglón se pregunta si la casilla cambió respecto a la de arriba:

- Si $K[i][w] = K[i-1][w]$, el objeto i no hizo falta: se sube un renglón con la misma w.
- Si $K[i][w] \neq K[i-1][w]$, el valor solo se pudo alcanzar tomando el objeto i: se anota, se sube un renglón y se le restan w_i a la capacidad.

Paso	Casilla	Valor	Arriba	¿Cambió?	Decisión
1	$K[3][5]$	220	$K[2][5] = 160$	sí	se toma el 3; $w = 5 - 3 = 2$
2	$K[2][2]$	100	$K[1][2] = 60$	sí	se toma el 2; $w = 2 - 2 = 0$
3	$K[1][0]$	0	$K[0][0] = 0$	no	el 1 no

Se toman los objetos 2 y 3: pesan $2 + 3 = 5$, justo la capacidad, y valen $100 + 120 = 220$. Con los pesos originales es la respuesta de la clase del 8: los objetos de 20 y 30, con valor 220.

$i \setminus w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	60	60	60	60	60
2	0	60	100	160	160	160
3	0	60	100	160	180	220

camino: $K[3][5] \to K[2][2] \to K[1][0]$

En verde, el camino de la reconstrucción. De $K[3][5]$ a $K[2][2]$ se sube un renglón y se recorren $w_3 = 3$ columnas a la izquierda: se tomó el objeto 3. De $K[2][2]$ a $K[1][0]$, otro renglón y $w_2 = 2$ columnas: se tomó el 2. En $K[1][0]$ la casilla es igual a la de arriba, así que el objeto 1 no se tomó. Cada objeto tomado es un salto en diagonal; uno que no se toma sería un paso recto hacia arriba.

El algoritmo

Algoritmo 10.4 Mochila-01(w, v, n, W)

```
1: para c ← 0 hasta W hacer
2:     K[0][c] ← 0
3: fin para
4: para i ← 1 hasta n hacer
5:     para c ← 0 hasta W hacer
6:         K[i][c] ← K[i − 1][c]                          // no tomarlo
7:         si w_i ≤ c y v_i + K[i − 1][c − w_i] > K[i][c] entonces
8:             K[i][c] ← v_i + K[i − 1][c − w_i]          // tomarlo
9:         fin si
10:    fin para
11: fin para
12: devolver K
```

Algoritmo 10.5 Reconstruir(K, w, n, W)

```
1: S ← ∅,    c ← W
2: para i ← n hasta 1, de bajada hacer
3:     si K[i][c] ≠ K[i − 1][c] entonces
4:         S ← S ∪ {i},    c ← c − w_i
5:     fin si
6: fin para
7: devolver S
```

La tabla tiene $(n+1)(W+1)$ casillas y cada una se llena en tiempo constante: Mochila-01 es $\Theta(nW)$ en tiempo y en espacio. Reconstruir sube un renglón por paso: $\Theta(n)$.

Por qué $\Theta(nW)$ no es polinomial

Parece polinomial, pero W no es el tamaño de la entrada sino un número de la entrada. Escribir W ocupa $\log_2 W$ bits, y agregarle un solo bit duplica W y duplica la tabla. Respecto al tamaño de la entrada el algoritmo es exponencial; a esto se le llama **pseudopolinomial**.

Ojito...

Con $n = 100$ objetos y pesos en gramos hasta $W = 5\,000$, la tabla tiene medio millón de casillas: instantáneo. Con $W = 10^9$ tendría 10^{11} y ya no cabe en memoria. La mochila 0/1 es NP-difícil: no se conoce un algoritmo polinomial en el tamaño de la entrada, y la programación dinámica sirve cuando la capacidad es un número chico.

Segundo ejemplo, para practicar

Capacidad $W = 7$ y cuatro objetos:

Objeto	**A**	**B**	**C**	**D**
Peso	1	3	4	5
Valor	1	4	5	7
Densidad	1.00	1.33	1.25	1.40

El voraz por densidad toma D, el más denso; quedan 2 de espacio, donde ni B ni C caben, y toma A. Vale $7 + 1 = 8$.

La tabla de programación dinámica, con los objetos en el orden A, B, C, D:

$i \setminus w$	0	1	2	3	4	5	6	7
0 (ninguno)	0	0	0	0	0	0	0	0
1 (A: 1, 1)	0	1	1	1	1	1	1	1
2 (B: 3, 4)	0	1	1	4	5	5	5	5
3 (C: 4, 5)	0	1	1	4	5	6	6	**9**
4 (D: 5, 7)	0	1	1	4	5	7	8	**9**

La casilla clave es la última:

$$K[4][7] = \text{máx}\,(\underbrace{K[3][7]}_{\text{no tomar D}},\ \underbrace{7 + K[3][2]}_{\text{tomar D}}) = \text{máx}(9,\ 7+1) = 9.$$

Tomar D, lo que haría el voraz, deja 2 de espacio que solo valen 1. No tomarlo deja los 7 de capacidad para los objetos anteriores, que valen 9.

Reconstrucción: $K[4][7] = K[3][7]$, así que D no se toma. $K[3][7] = 9 \neq K[2][7] = 5$: se toma C y quedan $7 - 4 = 3$. $K[2][3] = 4 \neq K[1][3] = 1$: se toma B y quedan 0. Solución: B y C, que pesan 7 y valen 9.

Ahorrar memoria: una sola fila

Cada renglón solo usa el de arriba, así que basta un arreglo de $W + 1$ casillas que se reescribe para cada objeto:

Algoritmo 10.6 Mochila-01-Una-Fila(w, v, n, W)

```
1: K[0 ... W] ← 0
2: para i ← 1 hasta n hacer
3:     para c ← W hasta w_i, de bajada hacer
4:         K[c] ← máx (K[c], v_i + K[c − w_i])
5:     fin para
6: fin para
7: devolver K[W]
```

El espacio baja a $\Theta(W)$. Pero el orden del ciclo interno importa: va **de bajada**.

Ojito...

Al calcular $K[c]$ se lee $K[c - w_i]$, que debe ser el valor del renglón anterior, sin el objeto i. Recorriendo de bajada, $K[c - w_i]$ todavía no se ha reescrito. De subida ya se reescribió, y el objeto se podría tomar dos veces. Con el objeto 1 (peso 1, valor 60) y $W = 2$, de subida:

$$K[1] = \text{máx}(0,\ 60 + K[0]) = 60, \qquad K[2] = \text{máx}(0,\ 60 + K[1]) = 120,$$

como si hubiera dos copias del objeto. De bajada, $K[2]$ se calcula primero con $K[1] = 0$ y queda en 60. El recorrido de subida resuelve otro problema, la mochila **sin límite**, donde cada objeto se puede tomar cuantas veces se quiera.

Cómo reconocer un problema de programación dinámica

Idea clave

Las señales son tres: el problema pide un óptimo o un conteo; la solución se arma con una secuencia de decisiones; y al escribir la recursión, los mismos estados aparecen por distintos caminos. El trabajo difícil es el paso 1: encontrar el **estado**, lo mínimo que hay que saber para resolver lo que falta. En Fibonacci es un número; en las monedas, cuánto falta pagar; en la mochila, qué objetos quedan y cuánto espacio hay.

Problema	Estado	Recurrencia	Costo
Fibonacci	i	$F[i] = F[i-1] + F[i-2]$	$\Theta(n)$
Monedas	cantidad a	$C[a] = 1 + \text{mín}_c\, C[a-c]$	$\Theta(a \cdot k)$, k monedas
Mochila 0/1	objetos i, capacidad w	$\text{máx}(K[i-1][w],\ v_i + K[i-1][w-w_i])$	$\Theta(nW)$

11. Grafos

22 de octubre de 2026

Resumen rápido. Un grafo es un par de conjuntos, vértices y aristas, y modela cualquier cosa hecha de objetos y relaciones entre pares de ellos: ciudades y carreteras, páginas y ligas, estados de un juego y jugadas. Se representa con matriz o con listas de adyacencia, y se clasifica por sus aristas: dirigido o no, pesado o no, cíclico o acíclico, simple o no, encajado o topológico, implícito o explícito, etiquetado o no. Se vieron los dos recorridos, búsqueda en anchura (con cola) y en profundidad (con recursión o pila), y tres problemas: camino más corto con Dijkstra y Floyd, detección de ciclos con BFS y árbol de expansión mínima con Kruskal. Todos los ejemplos usan los mismos tres grafos.

Esta clase es la base de la parte de grafos: presenta cada tema con un ejemplo y la idea de por qué funciona. Las pruebas, las variantes y los temas que no cupieron están en las secciones de complemento, de la 12 a la 22.

Definición formal

Definición 11.1 (Grafo)**.** Un **grafo** es un par $G = (V, E)$ donde V es un conjunto finito de **vértices** y E un conjunto de **aristas**. En un grafo no dirigido cada arista es un par no ordenado de vértices distintos,

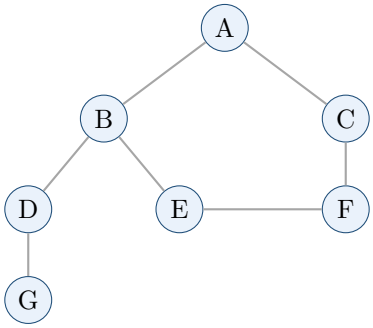
$$E \subseteq \{\{u, v\} : u, v \in V,\ u \neq v\}.$$

Se escribe $n = |V|$ y $m = |E|$.

Si $\{u, v\} \in E$, los vértices u y v son **adyacentes** y la arista es **incidente** en los dos. El **grado** de un vértice, $\mathrm{gr}(v)$, es el número de aristas incidentes en él. Otras fuentes, como Cormen, escriben $\deg(v)$.

Ejemplo 11.2 (El grafo G de esta clase)**.**

$$V = \{A, B, C, D, E, F, G\}, \qquad E = \{\{A, B\}, \{A, C\}, \{B, D\}, \{B, E\}, \{C, F\}, \{E, F\}, \{D, G\}\}.$$



$n = 7$ y $m = 7$. Los grados son $\mathrm{gr}(B) = 3$, $\mathrm{gr}(G) = 1$ y 2 para los demás. Este grafo se usa en toda la clase.

Observación. Cada arista suma 1 al grado de sus dos extremos, así que $\sum_{v \in V} \mathrm{gr}(v) = 2m$. En G: $2+3+2+2+2+2+1 = 14 = 2 \cdot 7$. Y como hay a lo más un par por cada dos vértices, $0 \leq m \leq n(n-1)/2$.

Matriz y listas de adyacencia

Definición 11.3 (Matriz de adyacencia)**.** Con los vértices numerados de 1 a n, la matriz de adyacencia es la matriz A de $n \times n$ con

$$A[i][j] = \begin{cases} 1 & \text{si } \{i, j\} \in E, \\ 0 & \text{si no.} \end{cases}$$

Para G, en el orden $A, \ldots, G$:

	A	B	C	D	E	F	G
A	0	1	1	0	0	0	0
B	1	0	0	1	1	0	0
C	1	0	0	0	0	1	0
D	0	1	0	0	0	0	1
E	0	1	0	0	0	1	0
F	0	0	1	0	1	0	0
G	0	0	0	1	0	0	0

En un grafo no dirigido la matriz es simétrica, la diagonal es cero y la suma de cada renglón es el grado del vértice.

La alternativa son las **listas de adyacencia**: para cada vértice, la lista de sus vecinos.

$A : B, C$
$B : A, D, E$
$C : A, F$
$D : B, G$
$E : B, F$
$F : C, E$
$G : D$

Operación	**Matriz**	**Listas**
Memoria	$\Theta(n^2)$	$\Theta(n+m)$
¿Es $\{u, v\}$ arista?	$\mathcal{O}(1)$	$\mathcal{O}(\text{gr}(u))$
Recorrer los vecinos de u	$\Theta(n)$	$\Theta(\text{gr}(u))$
Recorrer todo el grafo	$\Theta(n^2)$	$\Theta(n+m)$

Idea clave

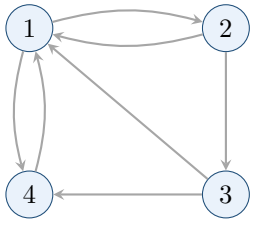
Casi todos los grafos reales son **dispersos**: m es mucho menor que n^2. Un mapa de carreteras con un millón de cruces tiene unos cuantos millones de tramos, no 10^{12}. Ahí la matriz gasta memoria en ceros y las listas ganan. La matriz conviene en grafos **densos** o cuando se pregunta mucho si una arista existe. Las complejidades de esta clase suponen listas.

Tipos de grafos

La clasificación es la de Skiena. Cada par describe una propiedad de las aristas o de cómo se guarda el grafo, y cambia qué algoritmo se puede usar.

Dirigido o no dirigido

En un **grafo dirigido** las aristas son pares ordenados, $E \subseteq V \times V$: la arista (u, v) va de u a v y no al revés. Cada vértice tiene **grado de entrada** $\text{gr}^-(v)$, las aristas que llegan a él, y **grado de salida** $\text{gr}^+(v)$, las que salen, y la matriz de adyacencia deja de ser simétrica. Una calle de un solo sentido o *A sigue a B* en una red social son dirigidas; una amistad no.

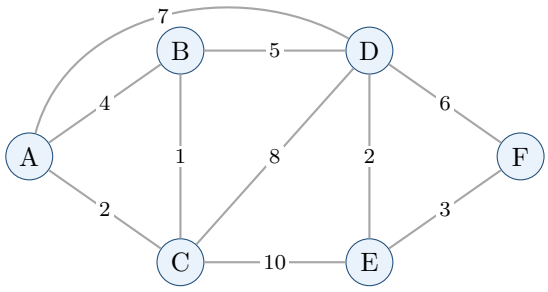


	1	2	3	4
1	0	1	0	1
2	1	0	1	0
3	1	0	0	1
4	1	0	0	0

$A[2][3] = 1$ pero $A[3][2] = 0$: de 2 se llega a 3 en un paso, de 3 a 2 no. El renglón de un vértice suma su grado de salida y su columna, el de entrada: el vértice 1 tiene $\text{gr}^+(1) = 2$ y $\text{gr}^-(1) = 3$. Este grafo se usa en Floyd.

Pesado o no pesado

En un **grafo pesado** cada arista tiene un número, $w : E \to \mathbb{R}$: distancia, tiempo, costo. En la matriz se guarda el peso en lugar del 1, y ∞ donde no hay arista. El 0 de la diagonal es la convención de Floyd: ir de un vértice a sí mismo cuesta 0.



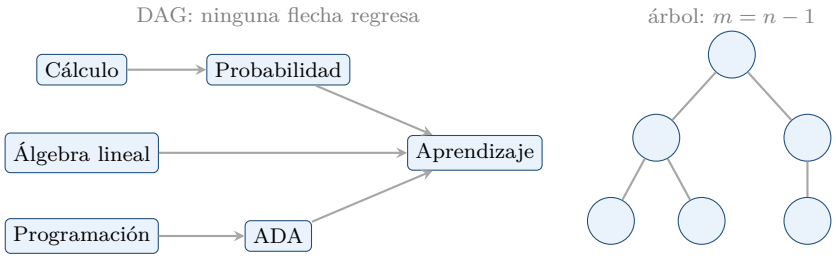
	A	B	C	D	E	F
A	0	4	2	7	∞	∞
B	4	0	1	5	∞	∞
C	2	1	0	8	10	∞
D	7	5	8	0	2	6
E	∞	∞	10	2	0	3
F	∞	∞	∞	6	3	0

En un grafo no pesado el camino más corto es el de menos aristas, y lo encuentra BFS. En uno pesado el de menos aristas puede no ser el más barato: de A a D la arista directa pesa 7, y A, C, B, D pesa $2 + 1 + 5 = 8$. Hacen falta Dijkstra o Floyd. Este grafo se usa en Dijkstra y en Kruskal.

Cíclico o acíclico

Un **camino** es una sucesión de vértices, cada uno adyacente al siguiente, que no repite vértices. Un **ciclo** es un camino cerrado: no repite vértices salvo el primero, que es también el último. G tiene uno: A, B, E, F, C, A. En un grafo dirigido el ciclo sigue el sentido de las flechas. La sección 12 completa la terminología (caminata, sendero, circuito).

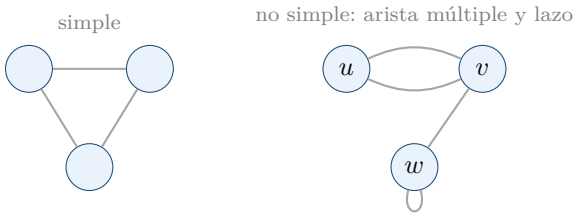
Un grafo sin ciclos es **acíclico**. Un no dirigido, conexo y acíclico es un **árbol**; un dirigido acíclico es un **DAG** (*directed acyclic graph*), como las seriaciones de un plan de estudios.



Un DAG siempre tiene un **orden topológico**: una forma de poner los vértices en fila con todas las flechas hacia adelante, el orden en que se pueden cursar las materias. Un grafo con un ciclo dirigido no lo tiene.

Simple o no simple

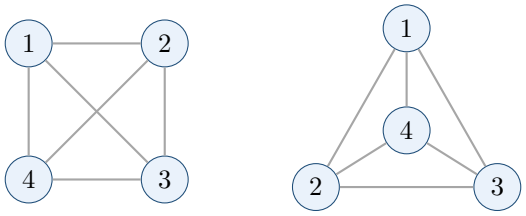
Un grafo es **simple** si no tiene **lazos** (aristas de un vértice a sí mismo) ni **aristas múltiples** (dos aristas entre el mismo par). La definición no dirigida de arriba, con conjuntos, solo admite grafos simples: $\{u, u\}$ no es un par y un conjunto no repite elementos. La dirigida, $E \subseteq V \times V$, sí admite lazos (u, u). Para tener aristas múltiples E se vuelve un multiconjunto.



Una arista múltiple aparece, por ejemplo, con dos vuelos distintos entre las mismas ciudades. Casi todos los algoritmos suponen grafos simples: con aristas múltiples pesadas basta quedarse con la más barata.

Encajado o topológico

Un grafo es **encajado** (*embedded*) si a sus vértices y aristas se les asignan posiciones geométricas: un punto en el plano para cada vértice y una curva para cada arista. Un grafo es **topológico** si solo importa quién está unido con quién. Los dos dibujos de abajo son el mismo grafo topológico, K_4, con cuatro vértices todos unidos entre sí, en dos encajes distintos:

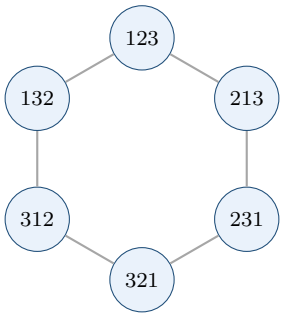


A la izquierda dos aristas se cruzan; a la derecha ninguna. Como grafo topológico es lo mismo; como grafo encajado no. En un mapa la geometría es parte del problema: la distancia entre dos cruces sale de sus coordenadas, y se puede saber si dos calles se cruzan.

Implícito o explícito

Un grafo **explícito** se construye completo y se guarda en una matriz o en listas antes de recorrerlo. Uno **implícito** no se guarda nunca: los vecinos de un vértice se generan cuando se necesitan, con una regla.

Ejemplo 11.4 (Permutaciones como grafo implícito)**.** Los vértices son las permutaciones de $1, 2, 3$, y dos son vecinas si una sale de la otra intercambiando dos elementos contiguos:



Los vecinos de 123 son 213 y 132, y se obtienen con la regla sin haber guardado nada. Con k elementos el grafo tiene $k!$ vértices: con $k = 10$ son $3\,628\,800$, y solo se generan los que el recorrido alcanza a visitar.

Los grafos implícitos aparecen en búsqueda: los estados de un rompecabezas, las posiciones de un juego, las soluciones parciales de un problema de backtracking.

Etiquetado o no etiquetado

En un grafo **etiquetado** cada vértice tiene un nombre propio, como A a G en G, y dos vértices con nombres distintos son distintos. En uno **no etiquetado** los nombres no importan, solo la estructura. Decidir si dos grafos son el mismo salvo los nombres es la **prueba de isomorfismo**; no se conoce un algoritmo polinomial para ella, y la sección 12 la define y la trabaja con invariantes.

Búsqueda en anchura (BFS)

Recorrer un grafo es visitar todos los vértices alcanzables desde uno inicial s, cada uno una sola vez. La **búsqueda en anchura** (*breadth-first search*) visita por capas: primero s, luego todos sus vecinos, luego los vecinos de esos que no se han visto, y así. Para respetar ese orden usa una **cola**: el primer vértice descubierto es el primero que se procesa.

Algoritmo 11.1 BFS(G, s)

```
1:  para todo v ∈ V hacer
2:      visto[v] ← falso,    d[v] ← ∞,    π[v] ← nulo
3:  fin para
4:  visto[s] ← verdadero,    d[s] ← 0
5:  Q ← cola vacía;  ENCOLAR(Q, s)
6:  mientras Q no está vacía hacer
7:      u ← DESENCOLAR(Q)
8:      para todo v en la lista de adyacencia de u hacer
9:          si no visto[v] entonces
10:             visto[v] ← verdadero,    d[v] ← d[u] + 1,    π[v] ← u
11:             ENCOLAR(Q, v)
12:         fin si
13:     fin para
14: fin mientras
```

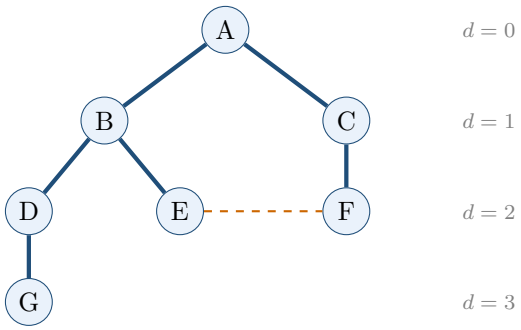
$d[v]$ es la distancia en aristas desde s y $\pi[v]$ el vértice desde el que se descubrió v, su **padre**.

Observación. Cormen escribe $v.d$, $v.\pi$ y NIL; aquí son $d[v]$, $\pi[v]$ y nulo. Sus tiempos de DFS $u.d$ y $u.f$ son aquí entra$[u]$ y sale$[u]$, porque $d[v]$ ya es la distancia de BFS. Su $O(V + E)$ es el $\mathcal{O}(n + m)$ de aquí.

Ejemplo 11.5 (BFS sobre G desde A)**.** Los vecinos se recorren en orden alfabético.

Paso	Sale	Descubre	Cola después
1	A	B, C	[B, C]
2	B	D, E (A ya visto)	[C, D, E]
3	C	F (A ya visto)	[D, E, F]
4	D	G (B ya visto)	[E, F, G]
5	E	nada: B y F ya vistos	[F, G]
6	F	nada: C y E ya vistos	[G]
7	G	nada	[ ]

Orden de visita: A, B, C, D, E, F, G. Las aristas por las que se descubrió cada vértice forman el **árbol BFS**:



Cada renglón es una capa: todos los vértices a la misma distancia de A. La arista $\{E, F\}$, punteada, no está en el árbol: cuando se procesó E, F ya estaba visto.

Cada vértice alcanzable entra y sale de la cola una sola vez, porque se marca al descubrirlo, y su lista se recorre una vez cuando sale; las listas suman $2m$ entradas. Con la inicialización, BFS cuesta $\mathcal{O}(n + m)$ con listas, y $\mathcal{O}(n^2)$ con matriz, porque encontrar los vecinos de cada vértice cuesta $\Theta(n)$.

Idea clave

En un grafo no pesado, $d[v]$ es la longitud del camino más corto de s a v, y siguiendo π de v hacia atrás se reconstruye ese camino. BFS ya resuelve el camino más corto cuando todas las aristas valen lo mismo. La prueba está en la sección 14.

Ojito...

En clase se dijo que la complejidad depende del problema, y que en un grafo completamente conectado es factorial. Hay que separar dos cosas. BFS procesa cada vértice una vez y cada arista desde sus dos extremos: en el grafo completo K_n, con $m = n(n-1)/2$, cuesta $\Theta(n^2)$, no factorial. Lo factorial aparece cuando el problema exige recorrer **caminos** y no vértices, como probar todas las rutas posibles: desde un vértice de K_5 salen 65 caminos, contando el trivial de longitud cero, y desde uno de K_{10}, 986 410, alrededor de $e \cdot (n-1)!$. O cuando el grafo es implícito y sus vértices son permutaciones, como el ejemplo de arriba: BFS sigue siendo lineal en el tamaño del grafo, pero el grafo mide $k!$.

Búsqueda en profundidad (DFS)

La **búsqueda en profundidad** (*depth-first search*) se mete lo más hondo que puede por un camino y solo regresa cuando se atora: cuando el vértice actual ya no tiene vecinos sin visitar. Se escribe de forma natural con recursión; la pila de llamadas hace el papel que la cola hace en BFS. La versión iterativa usa una pila explícita (sección 14).

Algoritmo 11.2 DFS-Visitar(G, u)

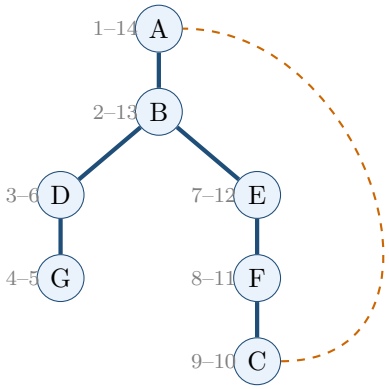
```
1: t ← t + 1,    entra[u] ← t,    visto[u] ← verdadero
2: para todo v en la lista de adyacencia de u hacer
3:     si no visto[v] entonces
4:         π[v] ← u
5:         DFS-Visitar(G, v)
6:     fin si
7: fin para
8: t ← t + 1,    sale[u] ← t
```

t es un reloj global que empieza en 0. entra[u] marca cuándo se descubrió u y sale[u] cuándo se terminó con todo lo que cuelga de él. Antes de la primera llamada se pone visto[v] $\leftarrow$ falso para todo v y $t \leftarrow 0$. Para recorrer un grafo desconectado se lanza DFS-Visitar desde cada vértice que siga sin visitar (sección 14).

Ejemplo 11.6 (DFS sobre G desde A)**.** Con los vecinos en orden alfabético, como en BFS:

- A $\to$ B $\to$ D $\to$ G. G no tiene vecinos nuevos: se termina, y se regresa a D, que tampoco tiene; se regresa a B.
- B $\to$ E $\to$ F $\to$ C. Los vecinos de C son A y F, los dos vistos: se termina C, luego F, E, B y A.

Orden de visita: A, B, D, G, E, F, C. El de BFS era A, B, C, D, E, F, G.



Junto a cada vértice, entra–sale. Este árbol DFS es largo y angosto; el BFS, ancho y bajo. La arista $\{A, C\}$, punteada, es una **arista de retroceso**: se explora primero desde C, cuando A sigue abierto, y va de C a un antecesor suyo.

DFS también cuesta $\mathcal{O}(n + m)$ con listas: DFS-Visitar se llama una vez por vértice, porque lo primero que hace es marcarlo, y cada llamada recorre su lista una vez.

	BFS	**DFS**
Estructura	cola	pila o recursión
Orden	por capas, de cerca a lejos	un camino hasta el fondo
Árbol	ancho y bajo	largo y angosto
Útil para	camino más corto sin pesos	ciclos dirigidos, orden topológico, componentes
Costo con listas	$\mathcal{O}(n + m)$	$\mathcal{O}(n + m)$

Las aplicaciones de DFS de la tabla se prueban en la sección 15.

Camino más corto: Dijkstra

Dado un grafo pesado con pesos **no negativos** y un vértice s, el algoritmo de Dijkstra (1959) encuentra la distancia $\delta(s, v)$ de s a todos los demás: el peso del camino más barato. Mantiene una estimación $d[v]$ que empieza en ∞ y nunca baja de $\delta(s, v)$, porque siempre es el peso de una caminata de s a v. En cada paso fija el vértice no fijado con la d más chica: esa ya es su distancia definitiva. Luego **relaja** las aristas que salen de él: si llegar a un vecino pasando por u es más barato que lo que se tenía, se actualiza.

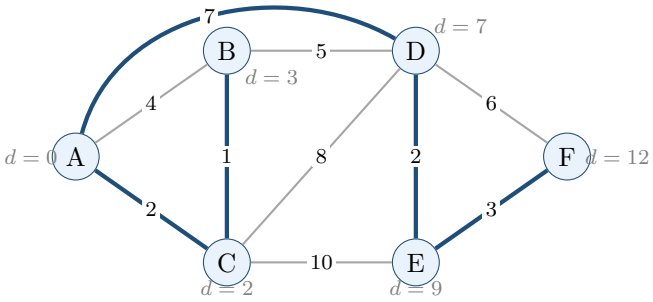
Algoritmo 11.3 Dijkstra(G, w, s)

```
1:  para todo v ∈ V hacer
2:      d[v] ← ∞,    π[v] ← nulo
3:  fin para
4:  d[s] ← 0
5:  Q ← montículo de mínimos con todos los vértices, con llave d
6:  mientras Q no está vacío hacer
7:      u ← Extraer-Min(Q)                          // d[u] queda fija
8:      para todo v adyacente a u hacer
9:          si d[u] + w(u, v) < d[v] entonces       // relajar
10:             d[v] ← d[u] + w(u, v),    π[v] ← u
11:             Decrementar-Llave(Q, v, d[v])
12:         fin si
13:     fin para
14: fin mientras
```

Ejemplo 11.7 (Dijkstra sobre el grafo pesado desde A)**.** Cada renglón muestra las d después de procesar el vértice que sale; en negritas, las ya fijas.

Paso	Sale	A	B	C	D	E	F	Relajaciones
0		0	∞	∞	∞	∞	∞	
1	A	**0**	4	2	7	∞	∞	B, C, D desde A
2	C	**0**	3	**2**	7	12	∞	B: $2+1=3<4$; D: 10, no
3	B	**0**	**3**	**2**	7	12	∞	D: $3+5=8$, no mejora
4	D	**0**	**3**	**2**	**7**	9	13	E: $7+2=9<12$
5	E	**0**	**3**	**2**	**7**	**9**	12	F: $9+3=12<13$
6	F	**0**	**3**	**2**	**7**	**9**	**12**	

B empezó en 4 por la arista directa y bajó a 3 pasando por C. Los padres quedan $\pi[B]=C$, $\pi[C]=A$, $\pi[D]=A$, $\pi[E]=D$, $\pi[F]=E$, y el camino a F se lee de atrás hacia adelante: F $\leftarrow$ E $\leftarrow$ D $\leftarrow$ A, o sea A, D, E, F, con $7+2+3=12$.



En azul, el **árbol de caminos más cortos**: las aristas $\{\pi[v], v\}$.

Por qué funciona. Supóngase que los vértices fijados antes que u tienen su distancia correcta, y sea P un camino más corto de s a u. Sea y el primer vértice de P sin fijar y x el anterior, ya fijo. Cuando x se fijó relajó la arista hacia y, así que $d[y]=\delta(s,y)$: hace falta que sea exacta, no basta con $d[y]\geq d[u]$. Como los pesos no restan, $d[y]=\delta(s,y)\leq\delta(s,u)\leq d[u]$, y como el algoritmo eligió u, $d[u]\leq d[y]$: todo es igual. Es un algoritmo voraz; la prueba completa está en la sección 17.

Costo. Cada vértice sale del montículo una vez y solo entonces se recorre su lista: $\mathcal{O}(m)$ relajaciones.

Con el montículo binario de la clase del 17 de septiembre, las n extracciones y los $\mathcal{O}(m)$ decrementos cuestan $\mathcal{O}(\log n)$ cada uno: $\mathcal{O}((n+m)\log n)$. Con un arreglo, buscar el mínimo cuesta $\Theta(n)$ y el total es $\Theta(n^2)$, mejor en grafos densos.

Ojito...

Con un peso negativo Dijkstra deja de ser confiable. Con las aristas dirigidas (A, B) de peso 2, (A, C) de peso 3, (C, B) de peso -2 y (B, D) de peso 2, desde A se fija B con $d = 2$ antes de procesar C, y B relaja (B, D): $d[D] = 4$. Al procesar C, $d[B]$ baja a $3 - 2 = 1$, pero nadie vuelve a relajar (B, D): D termina con 4, y el camino A, C, B, D cuesta 3. El argumento de arriba suponía que el tramo de y a u no resta. Para pesos negativos están Bellman-Ford (sección 17), Floyd y Johnson (sección 19).

Camino más corto entre todos los pares: Floyd

Dijkstra da las distancias desde un vértice. Para tener las de todos contra todos se puede correr n veces, una por fuente: $\mathcal{O}(n(n+m)\log n)$ con montículo binario. Floyd (Floyd-Warshall, 1962) lo resuelve con programación dinámica en tres ciclos, en $\Theta(n^3)$, y admite pesos negativos siempre que no haya ciclos de peso negativo. En grafos dispersos conviene correr Dijkstra n veces; en densos, Floyd.

Con los vértices numerados de 1 a n, un vértice **intermedio** de un camino es cualquiera salvo los dos extremos. Sea $d_{ij}^{(k)}$ el **peso** de un camino más corto de i a j cuyos intermedios están todos en $\{1, \ldots, k\}$. Para pasar de $k-1$ a k hay dos opciones: el camino no usa k, y queda igual; o lo usa una sola vez (sin ciclos negativos no hace falta repetir vértices), y k lo parte en dos tramos, de i a k y de k a j, con intermedios en $\{1, \ldots, k-1\}$. Los dos ya están calculados:

$$d_{ij}^{(k)} = \min\left(d_{ij}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kj}^{(k-1)}\right),$$

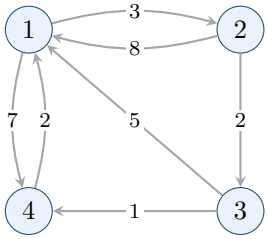
con $d_{ij}^{(0)} = w(i, j)$, 0 en la diagonal e ∞ si no hay arista. La respuesta es $D^{(n)}$. Si hay un ciclo negativo, al terminar algún $D[i][i]$ es negativo (sección 19).

Algoritmo 11.4 Floyd(W, n)

```
1: D ← W
2: para k ← 1 hasta n hacer
3:     para i ← 1 hasta n hacer
4:         para j ← 1 hasta n hacer
5:             D[i][j] ← mín(D[i][j], D[i][k] + D[k][j])
6:         fin para
7:     fin para
8: fin para
9: devolver D
```

Son $\Theta(n^3)$ operaciones y $\Theta(n^2)$ de memoria, porque la misma matriz se puede actualizar en su lugar: en la vuelta k el renglón k y la columna k no cambian.

Ejemplo 11.8 (Floyd sobre el grafo dirigido)**.** Los pesos de las aristas del grafo dirigido de arriba:



En negritas, las casillas que cambian en cada paso:

$$D^{(0)} = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{pmatrix} \qquad D^{(1)} = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \mathbf{15} \\ 5 & \mathbf{8} & 0 & 1 \\ 2 & \mathbf{5} & \infty & 0 \end{pmatrix} \qquad D^{(2)} = \begin{pmatrix} 0 & 3 & \mathbf{5} & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \mathbf{7} & 0 \end{pmatrix}$$

$$D^{(3)} = \begin{pmatrix} 0 & 3 & 5 & \mathbf{6} \\ \mathbf{7} & 0 & 2 & \mathbf{3} \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix} \qquad D^{(4)} = \begin{pmatrix} 0 & 3 & 5 & 6 \\ \mathbf{5} & 0 & 2 & 3 \\ \mathbf{3} & \mathbf{6} & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix}$$

Algunos cambios, uno por paso:

- $k = 1$: de 3 a 2 no había arista; pasando por 1, $d_{31} + d_{12} = 5 + 3 = 8$.
- $k = 2$: de 1 a 3, $d_{12} + d_{23} = 3 + 2 = 5$.
- $k = 3$: de 2 a 4 bajó de 15 a $d_{23} + d_{34} = 2 + 1 = 3$.
- $k = 4$: de 3 a 1 bajó de 5 a $d_{34} + d_{41} = 1 + 2 = 3$.

$D^{(4)}$ tiene todas las distancias. Por ejemplo $d_{32} = 6$ es el camino $3 \to 4 \to 1 \to 2$, con $1 + 2 + 3 = 6$. El pseudocódigo solo calcula pesos; para entregar el camino guarda predecesores (sección 19).

Encontrar ciclos con BFS

En un grafo no dirigido, BFS encuentra un ciclo en cuanto ve una arista que no es del árbol: al procesar u, un vecino v que ya fue visto y que no es el padre de u. El padre no cuenta porque la arista $\{u, \pi[u]\}$ es justo por la que se llegó a u. Basta cambiar el **si** de BFS:

Algoritmo 11.5 Cambio a BFS para detectar un ciclo

```
1: para todo v en la lista de adyacencia de u hacer
2:     si no visto[v] entonces
3:         ... igual que en BFS ...
4:     si no si v ≠ π[u] entonces
5:         devolver hay un ciclo que pasa por la arista {u, v}
6:     fin si
7: fin para
```

Ejemplo 11.9 (El ciclo de G)**.** En el paso 5 de BFS sale E y su vecino F ya está visto, pero $\pi[E] = B \neq F$. Hay ciclo. Para recuperarlo se sube por los padres desde los dos extremos hasta que se encuentran:

$$E \to B \to A, \qquad F \to C \to A.$$

Se juntan en A, y el ciclo es A, B, E, F, C, A: la arista punteada del árbol BFS cierra la vuelta.

Ojito...

En un grafo dirigido esta regla da falsos positivos. Con las aristas (A, B), (A, C) y (B, C), BFS desde A llega a C dos veces y la regla dice ciclo, pero no hay ningún ciclo dirigido. Para grafos dirigidos se usa DFS: hay un ciclo si y solo si encuentra una arista de retroceso, hacia un vértice que ya entró y no ha salido; no basta visto (sección 15). La otra opción es el algoritmo de Kahn: con una cola se van quitando los vértices de grado de entrada 0; si alguno nunca llega a 0, hay ciclo. En el grafo de Floyd ningún vértice empieza con $\text{gr}^-(v) = 0$, así que la cola nace vacía: tiene ciclos, como $1 \to 2 \to 1$.

Árbol de expansión mínima: Kruskal

Definición 11.10 (Árbol de expansión mínima)**.** En un grafo no dirigido, conexo y pesado, un **árbol de expansión** es un subconjunto de aristas que conecta todos los vértices sin formar ciclos; tiene exactamente $n - 1$ aristas. El de **expansión mínima** (MST, *minimum spanning tree*) es el de menor peso total.

Es el problema de conectar n ciudades con cable gastando lo menos posible. El algoritmo de Kruskal (1956) es voraz: ordena las aristas de la más ligera a la más pesada y las agrega en ese orden, saltándose las que cerrarían un ciclo.

Algoritmo 11.6 Kruskal(G, w)

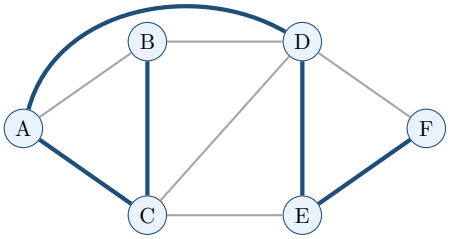
```
1:  T ← ∅
2:  para todo v ∈ V hacer
3:      Crear-Conjunto(v)
4:  fin para
5:  ordenar E por peso, de menor a mayor
6:  para todo {u, v} ∈ E, en ese orden hacer
7:      si Encontrar(u) ≠ Encontrar(v) entonces
8:          T ← T ∪ {{u, v}}
9:          Unir(u, v)
10:     fin si
11: fin para
12: devolver T
```

Para saber si una arista cierra un ciclo se usa una estructura de **conjuntos disjuntos** (*union-find*): cada vértice empieza en su propio conjunto, Encontrar dice en qué conjunto está un vértice y Unir junta dos. Si u y v están en el mismo conjunto, hay un camino entre ellos y la arista cerraría un ciclo.

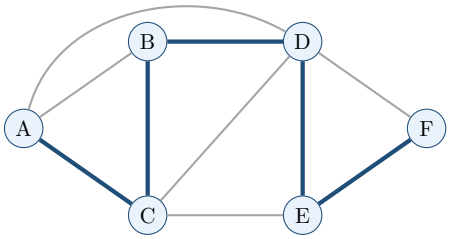
Ejemplo 11.11 (Kruskal sobre el grafo pesado)**.** Las aristas en orden de peso:

Arista	Peso	¿Entra?	Componentes después
$\{B, C\}$	1	sí	{A} {B, C} {D} {E} {F}
$\{A, C\}$	2	sí	{A, B, C} {D} {E} {F}
$\{D, E\}$	2	sí	{A, B, C} {D, E} {F}
$\{E, F\}$	3	sí	{A, B, C} {D, E, F}
$\{A, B\}$	4	no: A y B ya están juntos	
$\{B, D\}$	5	sí	{A, B, C, D, E, F}

Con $n - 1 = 5$ aristas el árbol está completo, y las que faltan ($\{D, F\}$, $\{A, D\}$, $\{C, D\}$, $\{C, E\}$) cerrarían ciclos. Peso total: $1 + 2 + 2 + 3 + 5 = 13$.



Caminos más cortos desde A: peso 15



Expansión mínima: peso 13

Ojito...

El árbol de caminos más cortos de Dijkstra y el árbol de expansión mínima no son lo mismo, aunque los dos tengan $n - 1$ aristas. El de Dijkstra minimiza la distancia de A a cada vértice; el MST, el peso total. En el MST el camino de A a D es A, C, B, D, con 8; en el de Dijkstra es la arista directa, con 7. Pero el MST pesa 13 y el de Dijkstra 15.

Por qué funciona. Se parte V en dos lados sin aristas aceptadas que crucen. La más ligera que cruza, e, es segura: si un MST T con las aceptadas no usa e, agregarla cierra un ciclo que cruza el corte por otra arista e', y cambiar e' por e no aumenta el peso. Kruskal siempre acepta la más ligera que sale del componente de un extremo: es la propiedad del corte (sección 16).

Costo. Domina ordenar las aristas, $\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$: con unión por rango y compresión de caminos, los conjuntos disjuntos cuestan $\mathcal{O}((n + m)\,\alpha(n))$, con $\alpha(n) \leq 4$ en la práctica.

Observación. El otro algoritmo clásico es el de Prim: hace crecer un solo árbol desde un vértice, agregando siempre la arista más ligera que sale de él. Es Dijkstra con llave $w(u, v)$ en lugar de $d[u] + w(u, v)$, actualizada solo si v sigue en Q (sección 16). Con montículo binario cuesta $\mathcal{O}(m \log n)$.

Resumen

Algoritmo	Resuelve	Estructura	Costo
BFS	recorrido por capas; camino más corto sin pesos; ciclos en no dirigidos	cola	$\mathcal{O}(n + m)$
DFS	recorrido en profundidad; ciclos dirigidos; orden topológico	pila o recursión	$\mathcal{O}(n + m)$
Dijkstra	caminos más cortos desde un vértice, pesos ≥ 0	montículo binario	$\mathcal{O}((n + m) \log n)$
		arreglo	$\Theta(n^2)$
Floyd	caminos más cortos entre todos los pares, sin ciclos negativos	matriz	$\Theta(n^3)$
Kruskal	árbol de expansión mínima	conjuntos disjuntos	$\mathcal{O}(m \log n)$
Prim	árbol de expansión mínima	montículo binario	$\mathcal{O}(m \log n)$

El resumen de toda la parte de grafos está al final de la sección 22.

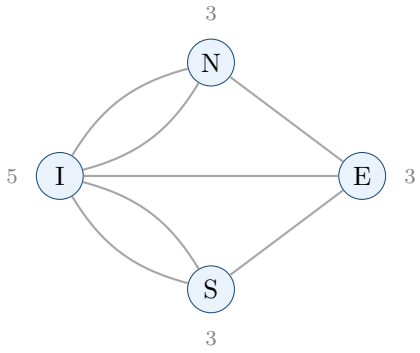
12. Teoría de grafos: terminología, conexidad y Euler *Complemento: Rosen, cap. 10*

Resumen rápido. Esta sección arma el vocabulario que el resto de la parte de grafos da por sabido. Parte del problema de los puentes de Königsberg y nombra lo que la clase del 22 de octubre dejó sin nombre: multigrafos y pseudografos, el grado con lazos y el teorema del apretón de manos, subgrafos, la matriz de incidencia y el isomorfismo. Fija la convención de caminata, sendero, camino, circuito y ciclo que usan todas las secciones siguientes, define conexidad, componentes, puentes y vértices de corte, y cierra con los grafos eulerianos: el teorema de Euler con su prueba completa, el caso dirigido y el algoritmo de Hierholzer, que construye el recorrido en $\Theta(n + m)$. Decidir si existe un recorrido que use cada arista una vez, y construirlo, cuesta lo mismo que leer el grafo.

La materia prima son las diapositivas *Grafos 1* de la Dra. Kelsey Ramírez, que siguen de cerca a Rosen (cap. 10). Las pruebas de suficiencia, el caso dirigido y el algoritmo de Hierholzer son agregados. Lo que ya está en la sección 11 no se repite: se cita y se complementa.

Motivación: los siete puentes de Königsberg

En el siglo XVIII la ciudad de Königsberg (hoy Kaliningrado) estaba partida por el río Pregel en cuatro regiones: la orilla norte N, la orilla sur S, la isla Kneiphof I y la región del este E. Siete puentes las unían. La pregunta era si se podía dar un paseo que cruzara cada puente exactamente una vez y regresara al punto de partida. Euler (1736) notó que solo importa qué regiones une cada puente: cada región se vuelve un vértice y cada puente una arista. Junto a cada vértice, su grado.



El grafo tiene $n = 4$ y $m = 7$, y no es simple: entre I y N hay dos puentes, y otros dos entre I y S. La pregunta de Euler se vuelve: ¿hay un recorrido cerrado que use cada arista exactamente una vez? La respuesta, al final de la sección, sale de los números junto a cada vértice.

Multigrafos, pseudografos y grado

La sección 11 define el grafo simple, que no admite lazos ni dos aristas entre el mismo par. Para admitirlos, E se vuelve un multiconjunto. Rosen les pone nombre según tres preguntas: ¿las aristas tienen dirección?, ¿puede haber varias entre el mismo par de vértices?, ¿puede haber lazos?

Definición 12.1 (Multigrafo y pseudografo)**.** Un **multigrafo** es un par $G = (V, E)$ en el que E es un multiconjunto de pares $\{u, v\}$ con $u \neq v$: dos aristas distintas pueden tener los mismos extremos, y se llaman **múltiples** o **paralelas**. Un **pseudografo** admite además **lazos**, aristas $\{v, v\}$ cuyos dos extremos son el mismo vértice.

Como dos aristas paralelas no se distinguen por sus extremos, en un multigrafo se les pone nombre, $e_1, \ldots, e_m$. Todo grafo simple es multigrafo y todo multigrafo es pseudografo, así que un teorema

probado para pseudografos vale para los tres. Con dirección, el **grafo dirigido** admite lazos pero no arcos múltiples, y el **multigrafo dirigido** admite los dos. Los arcos (c, d) y (d, c) **no** son múltiples: tienen sentidos opuestos. Múltiples son dos arcos con el mismo origen y el mismo destino.

El grado y el teorema del apretón de manos

En un pseudografo el grado necesita una precisión: un lazo toca a su vértice por los dos extremos.

Definición 12.2 (Grado)**.** El **grado** $\mathrm{gr}(v)$ de un vértice de un pseudografo es el número de extremos de aristas que caen en v: cada arista $\{u, v\}$ con $u \neq v$ suma 1 al grado de cada extremo, y cada lazo en v suma 2 al grado de v. Un vértice de grado 0 es **aislado** y uno de grado 1 es una **hoja**.

Teorema 12.3 (Apretón de manos)**.** *En todo pseudografo $G = (V, E)$,*

$$\sum_{v \in V} gr(v) = 2m.$$

Demostración. Se cuentan de dos maneras los pares (vértice, extremo de arista que cae en él). Contando por vértices, cada v aporta $\mathrm{gr}(v)$ pares, y el total es la suma de los grados. Contando por aristas, cada arista tiene exactamente dos extremos, sea o no un lazo, y aporta 2. Las dos cuentas cuentan lo mismo, así que coinciden. □

En el multigrafo de Königsberg, $\mathrm{gr}(I) = 5$ y las otras tres regiones tienen grado 3; suman $14 = 2 \cdot 7$, uno por cada extremo de los siete puentes.

Ojito...

Las diapositivas definen el grado como "el número de aristas incidentes". Leída al pie de la letra, esa definición cuenta un lazo una sola vez, y entonces el teorema falla: un grafo con un solo vértice y un lazo tendría suma de grados 1 y no 2.

Corolario 12.4. *En todo pseudografo, el número de vértices de grado impar es par.*

Sale de que la suma de los grados es par: una suma de impares es par solo con un número par de sumandos.

En un grafo dirigido, $\mathrm{gr}^-(v)$ es el **grado de entrada**, el número de arcos que llegan a v, y $\mathrm{gr}^+(v)$ el **grado de salida**; un lazo (v, v) suma 1 a cada uno. Como cada arco tiene exactamente un origen y un destino, $\sum_v \mathrm{gr}^-(v) = \sum_v \mathrm{gr}^+(v) = m$.

Familias con nombre y subgrafos

Algunos grafos tienen nombre propio. El **camino** P_n tiene n vértices en fila y $n - 1$ aristas; el **ciclo** C_n, con $n \geq 3$, cierra la fila con una arista más, y todos sus vértices tienen grado 2; el **completo** K_n une cada par de vértices, con $m = n(n-1)/2$, el máximo posible en un grafo simple; la **estrella** une un centro con k hojas, y la sección 13 vuelve a ella como el bipartito completo $K_{1,k}$.

Definición 12.5 (Subgrafo)**.** $H = (W, F)$ es un **subgrafo** de $G = (V, E)$ si $W \subseteq V$, $F \subseteq E$ y cada arista de F tiene sus dos extremos en W. Es el **subgrafo inducido** por W, y se escribe $G[W]$, si F contiene *todas* las aristas de E con los dos extremos en W. Es un **subgrafo generador** (o de expansión) si $W = V$.

$G - e$ es el grafo sin la arista e, con todos los vértices. $G - v$ es el grafo sin el vértice v *y sin todas las aristas que inciden en* v, porque una arista sin uno de sus extremos deja de ser arista. En el grafo G de

la sección 11, el subgrafo inducido por $\{A, B, C, E, F\}$ es el ciclo C_5 que la clase encuentra con BFS, y $G - \{E, F\}$ es un subgrafo generador: el árbol BFS de esa clase. Los árboles de BFS, de DFS y de expansión mínima son subgrafos generadores.

Matriz de incidencia

La matriz de adyacencia de la sección 11 relaciona vértices con vértices. La de incidencia relaciona vértices con aristas, y por eso admite aristas paralelas sin perder información.

Definición 12.6 (Matriz de incidencia)**.** Con los vértices $v_1, \ldots, v_n$ y las aristas $e_1, \ldots, e_m$ numerados, la matriz de incidencia es la matriz M de $n \times m$ con $M_{ij} = 1$ si v_i es extremo de e_j, y $M_{ij} = 0$ si no.

Proposición 12.7. *Sea M la matriz de incidencia de un multigrafo sin lazos y A su matriz de adyacencia, con A_{ij} igual al número de aristas entre v_i y v_j. Entonces cada columna de M tiene exactamente dos unos, la suma del renglón i es $gr(v_i)$, y*

$$MM^T = D + A,$$

donde D es la matriz diagonal con los grados.

Idea de la prueba. $(MM^T)_{ij} = \sum_k M_{ik}M_{jk}$ cuenta las aristas que inciden a la vez en v_i y en v_j: con $i = j$ es el grado de v_i, y con $i \neq j$ es A_{ij}. □

La matriz de incidencia ocupa $\Theta(nm)$ de memoria y preguntar si $\{u, v\}$ es arista cuesta $\Theta(m)$, así que casi ningún algoritmo la usa; su valor es teórico. La versión dirigida, con -1 en el origen y 1 en el destino, está en la sección 14.

Isomorfismo

La sección 11 distingue el grafo encajado del topológico. Cuando además los vértices tienen nombres distintos, "el mismo grafo" se precisa con una biyección.

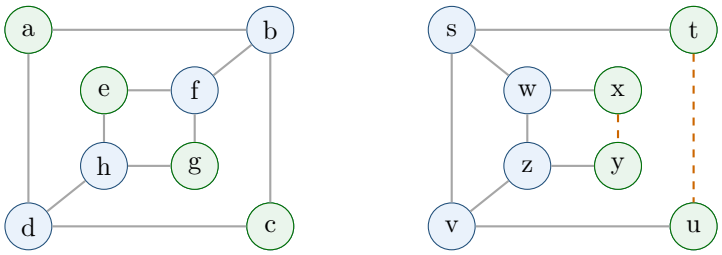
Definición 12.8 (Isomorfismo)**.** Dos grafos simples $G_1 = (V_1, E_1)$ y $G_2 = (V_2, E_2)$ son **isomorfos**, $G_1 \cong G_2$, si existe una biyección $f : V_1 \to V_2$ tal que

$$\{u, v\} \in E_1 \iff \{f(u), f(v)\} \in E_2.$$

Una propiedad que conserva todo isomorfismo es un **invariante**.

El número de vértices, el de aristas y la secuencia de grados son invariantes, y también lo son la conexidad, el número de ciclos de cada longitud y el grado de los vecinos de cada vértice: la biyección manda vecinos en vecinos. Si dos grafos difieren en un invariante, no son isomorfos. Si coinciden en todos los que se revisaron, no se sabe nada: para afirmar que lo son hay que exhibir la biyección.

Ejemplo 12.9 (Dos grafos que se parecen, diapositivas 48 a 51)**.** G y H tienen ocho vértices cada uno, un cuadrado exterior y uno interior unidos por dos aristas, diez aristas y la misma secuencia de grados, $2, 2, 2, 2, 3, 3, 3, 3$; en la figura, en verde, los vértices de grado 2. Pero un isomorfismo conserva el grado y la adyacencia, así que a, de grado 2, tendría que ir a un vértice de grado 2 de H cuyos dos vecinos tengan grado 3, como los de a. En H no hay ninguno: cada vértice de grado 2 tiene un vecino de grado 2 (punteadas, las aristas entre ellos). No existe a dónde mandar a, así que G y H **no son isomorfos**.



G *H*

No se conoce un algoritmo polinomial para decidir isomorfismo en general; el mejor resultado, de Babai (2015), es cuasipolinomial. En la práctica se descarta con invariantes baratos y solo se busca la biyección cuando todos coinciden.

Caminatas, senderos, caminos, circuitos y ciclos

La terminología cambia de un libro a otro; esta subsección fija la que usa toda la parte de grafos de la libreta.

Definición 12.10 (Caminata)**.** En un pseudografo, una **caminata** de longitud k de u a v es una secuencia alternada de vértices y aristas

$$v_0, e_1, v_1, e_2, \ldots, e_k, v_k$$

con $v_0 = u$, $v_k = v$ y cada e_i una arista de extremos v_{i-1} y v_i. Es **cerrada** si $v_0 = v_k$. Un solo vértice es una caminata de longitud 0.

En un grafo simple cada arista queda determinada por sus extremos, y la caminata se escribe solo con vértices, $v_0, v_1, \ldots, v_k$.

Definición 12.11 (Sendero, camino, circuito y ciclo)**.** Una caminata es

- un **sendero** si no repite aristas;
- un **camino** si no repite vértices;
- un **circuito** si es un sendero cerrado de longitud al menos 1;
- un **ciclo** si es un circuito en el que $v_0, v_1, \ldots, v_{k-1}$ son distintos: un camino cerrado, que no repite vértices salvo el primero y el último.

Todo camino es sendero: si no se repiten vértices, una arista no puede repetirse. Por lo mismo, todo ciclo es circuito. El recíproco falla: en dos triángulos pegados por un vértice (la "mariposa" de la diapositiva 119, más abajo) el recorrido c, a, b, c, d, e, c es un circuito que pasa dos veces por c, y no es ciclo. En un grafo simple todo ciclo tiene longitud al menos 3; en un multigrafo dos aristas paralelas forman un ciclo de longitud 2, y en un pseudografo un lazo es un ciclo de longitud 1.

Lema 12.12. *Si hay una caminata de u a v, hay un camino de u a v, y su longitud no es mayor.*

Idea de la prueba. Si la caminata repite un vértice x, se borra el tramo entre la primera y la última aparición de x, que es una vuelta cerrada; lo que queda es una caminata de u a v más corta. Se repite hasta que no quede ningún vértice repetido. □

En particular, la caminata más corta de u a v es un camino. Por eso la $d[v]$ de BFS es la longitud de un camino, y con pesos no negativos siempre hay un camino más corto que no repite vértices.

Ojito...

La profesora llama **circuito** a cualquier caminata cerrada (diapositiva 81), aunque repita aristas. Esta libreta sigue a Grimaldi, que exige que el circuito no repita aristas: así los nombres van en pares, abierto y cerrado (sendero y circuito, camino y ciclo), y un circuito euleriano es literalmente un circuito. Con los libros en inglés hay que traducir con cuidado: el *simple path* de Rosen es un sendero y el de Cormen es un camino.

Libreta	**Diapositivas**	**Grimaldi**	**Rosen**	**Cormen**
caminata	caminata	*walk*	*path*	*path*
sendero	sendero	*trail*	*simple path*	sin nombre
camino	camino, trayectoria	*path*	sin nombre	*simple path*
caminata cerrada	circuito	*closed walk*	*circuit*	*cycle* (dirigido)
circuito	circuito	*circuit*	*simple circuit*	sin nombre
ciclo	ciclo	*cycle*	sin nombre	*simple cycle*

Proposición 12.13. *Si A es la matriz de adyacencia de un multigrafo, con A_{ij} igual al número de aristas entre v_i y v_j, entonces $(A^k)_{ij}$ es el número de caminatas de longitud k de v_i a v_j.*

Idea de la prueba. Inducción sobre k: una caminata de longitud $k+1$ es una de longitud k hasta algún v_l más una arista de v_l a v_j, y sumar sobre l es el producto $A^k A$. □

La sección 19 usa la misma idea con pesos, cambiando la suma por un mínimo y el producto por una suma.

En un grafo dirigido las definiciones son las mismas con arcos: cada paso va de v_{i-1} a v_i siguiendo la flecha. En el grafo dirigido de la sección 11, $1 \to 2 \to 1$ es un ciclo de longitud 2: los arcos $(1,2)$ y $(2,1)$ son distintos, así que no se repite ninguno.

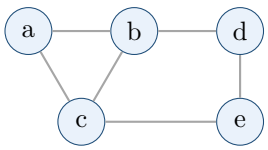
Conexidad

Definición 12.14 (Conexo)**.** Un grafo no dirigido es **conexo** si hay un camino entre cada par de vértices, y **desconectado** si no.

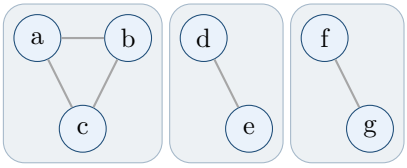
Por el lema 12.12 da lo mismo pedir caminos o caminatas. La relación "hay un camino de u a v" es de equivalencia: reflexiva por el camino de longitud 0, simétrica porque un camino se recorre al revés, y transitiva porque dos caminos seguidos forman una caminata, que el lema convierte en camino.

Definición 12.15 (Componente conexa)**.** Las **componentes conexas** de G son los subgrafos inducidos por las clases de esa relación. Equivalentemente, son los subgrafos conexos maximales: conexos, y sin poder agregar un vértice más sin dejar de serlo.

Ejemplo 12.16 (Diapositivas 95 y 100)**.** A la izquierda, un grafo conexo: a y e no son adyacentes, pero el camino a, c, e los une. A la derecha, un grafo con tres componentes, $\{a, b, c\}$, $\{d, e\}$ y $\{f, g\}$: no hay camino de a a d ni de e a g.



conexo



tres componentes

Las componentes parten a V, cada arista queda dentro de una sola componente, y G es conexo si y solo si tiene una sola. Un vértice aislado es una componente de un solo vértice.

Una BFS desde s visita exactamente la componente de s. Repetida desde cada vértice que siga sin visitar, etiqueta todas las componentes en $\Theta(n+m)$, porque cada vértice entra a la cola una sola vez en total y cada lista se recorre una sola vez, no una vez por componente. La sección 14 hace lo mismo con DFS.

En un grafo dirigido hay dos nociones. Es **débilmente conexo** si, olvidando el sentido de los arcos, el grafo no dirigido que queda es conexo; es **fuertemente conexo** si hay un camino dirigido de cada vértice a cada otro. El grafo dirigido de Floyd en la sección 11 es fuertemente conexo; el DAG de seriaciones solo débilmente, porque ninguna flecha regresa. Las componentes fuertemente conexas están en la sección 15.

Puentes y vértices de corte

Definición 12.17 (Puente y vértice de corte)**.** Una arista e es un **puente** (o arista de corte) si $G-e$ tiene más componentes conexas que G. Un vértice v es un **vértice de corte** (o punto de articulación) si $G-v$ tiene más componentes conexas que G.

En el grafo G de la sección 11, $\{B,D\}$ y $\{D,G\}$ son puentes y B y D son vértices de corte; ninguna arista del ciclo A,B,E,F,C,A es puente. Esa última observación es general.

Teorema 12.18. *Una arista es puente si y solo si no pertenece a ningún ciclo.*

Idea de la prueba. Si $e=\{u,v\}$ está en un ciclo, el resto del ciclo une a v con u sin usar e, y cualquier caminata que pase por e se desvía por ahí: quitarla no parte nada. Si no está en ningún ciclo, $G-e$ no tiene camino de u a v, porque ese camino seguido de e cerraría un ciclo; la componente de u y v se parte en dos. □

Toda arista que llega a una hoja es puente, y dos aristas paralelas nunca lo son, porque juntas forman un ciclo de longitud 2: el multigrafo de Königsberg no tiene ningún puente. Para vértices no hay una regla así: estar en un ciclo no protege a un vértice. Por la definición, encontrar todos los puentes cuesta una BFS por arista, $\Theta(m(n+m))$; una sola DFS con un valor más por vértice, low$[v]$, los encuentra todos en $\Theta(n+m)$, y también los vértices de corte (sección 15).

Grafos eulerianos

La diapositiva 111 plantea dos preguntas gemelas: ¿hay un recorrido que pase por cada *arista* exactamente una vez?, ¿hay uno que pase por cada *vértice* exactamente una vez? La primera es la de Euler y se resuelve aquí. La segunda es la de Hamilton, y está en la sección 13; se parecen tanto que uno esperaría resolverlas igual, y no.

Definición 12.19 (Recorridos eulerianos)**.** Un **sendero euleriano** de G es un sendero que usa cada arista de G exactamente una vez; un **circuito euleriano** es un sendero euleriano cerrado. G es **euleriano** si tiene circuito euleriano, **semi-euleriano** si tiene sendero euleriano pero no circuito, y **no euleriano** si no tiene ninguno.

La intuición está en un vértice intermedio. Cada vez que el recorrido pasa por él usa dos de sus aristas, una para entrar y otra para salir. Si el recorrido usa todas las aristas una vez, las de cada vértice quedan agrupadas en pares, salvo en los extremos de un recorrido abierto.

Ojito...

Las diapositivas y Rosen dicen "camino euleriano" (*Euler path*). Con la convención de esta libreta no es un camino, porque casi siempre repite vértices: el recorrido b, a, d, c, b, d de abajo pasa dos veces por b y por d. Grimaldi lo llama *Euler trail*, sendero euleriano, y es el nombre que se usa aquí.

El teorema de Euler

Teorema 12.20 (Euler)**.** *Sea G un pseudografo con al menos una arista, conexo salvo vértices aislados. G tiene un circuito euleriano si y solo si todos sus vértices tienen grado par.*

"Conexo salvo vértices aislados" significa que todas las aristas están en una misma componente. Los vértices aislados no estorban: no tienen aristas que recorrer.

Demostración. **Necesidad.** Sea $v_0, e_1, v_1, \ldots, e_m, v_m$ un circuito euleriano, con $v_m = v_0$, y sea $e_{m+1} = e_1$. Cada índice $i \in \{1, \ldots, m\}$ empareja dos extremos que caen en v_i: el final de e_i, por donde se entra, y el inicio de e_{i+1}, por donde se sale. Para $i = m$ el par es la última llegada a v_0 y la primera salida. Como cada arista aparece una sola vez, cada extremo que cae en un vértice v queda en exactamente un par, así que $\mathrm{gr}(v)$ es dos veces el número de apariciones de v entre $v_1, \ldots, v_m$. Un lazo en v también encaja: se sale y se regresa a v en el mismo paso, y sus dos extremos quedan en pares distintos.

Un sendero solo se atora en su inicio. Supóngase que todos los grados de un pseudografo son pares, y que desde un vértice s se camina usando cada vez una arista todavía no usada, mientras la haya. Estando en un $v \neq s$, el sendero ha entrado a v una vez más de las que ha salido, así que ha usado un número impar de los extremos de v (un lazo, entrado y salido en un paso, usa dos). Como $\mathrm{gr}(v)$ es par, queda un extremo libre y se puede seguir. Las aristas son finitas, así que la caminata se detiene, y solo puede hacerlo en s: el resultado es un circuito.

Suficiencia. Sea s un vértice con aristas. Por el paso anterior, desde s sale un circuito C. Si C usa todas las aristas, terminamos. Si no, alguna arista sin usar toca un vértice x de C. En efecto, si una arista sin usar $\{y, z\}$ no toca a C, se toma un camino de s a y, que existe porque todas las aristas están en una componente; las aristas de C unen vértices de C y y no es uno de ellos, así que la primera arista del camino que no está en C sale de un vértice x de C y está sin usar.

Sea R el grafo de las aristas sin usar. En R todos los grados siguen siendo pares, porque C usa en cada vértice un número par de extremos (es la necesidad aplicada a C). Desde x sale entonces un circuito C' en R, que regresa a x. Se **empalma** C' en C: en una aparición de x en C se inserta C' completo. El resultado es un circuito, porque C y C' no comparten aristas, y tiene más aristas que C. Se repite; como m es finito, al final el circuito usa todas las aristas. □

La prueba de la suficiencia es constructiva: camina, se atora, busca un vértice con aristas libres y empalma. Es exactamente el algoritmo de Hierholzer (1873), que se ve más abajo. Las diapositivas solo dan la necesidad.

Teorema 12.21 (Sendero euleriano)**.** *Sea G un pseudografo conexo salvo vértices aislados. G tiene un sendero euleriano que no es cerrado si y solo si tiene exactamente dos vértices de grado impar, y en ese caso todo sendero euleriano empieza en uno de ellos y termina en el otro.*

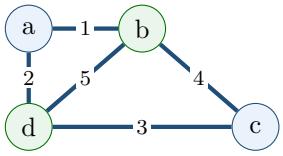
Idea de la prueba. La necesidad es la misma cuenta: en el inicio sobra la primera salida y en el final la última llegada. Para la suficiencia se agrega una arista nueva $e^* = \{u, v\}$ entre los dos impares; ahora

todos los grados son pares y hay circuito euleriano. Leído justo después de e^* y sin ella, es un sendero de u a v. □

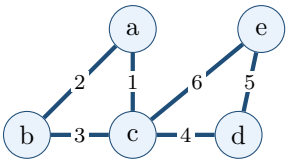
Con los dos teoremas y el corolario 12.4, la clasificación de un grafo conexo salvo aislados se lee contando vértices de grado impar:

Vértices impares	Tipo	Recorrido
0	euleriano	circuito desde cualquier vértice con aristas
2	semi-euleriano	sendero de un impar al otro
$4, 6, \ldots$	no euleriano	ninguno
$1, 3, 5, \ldots$	imposible	por el corolario 12.4

Ejemplo 12.22 (Diapositivas 115 y 119)**.** A la izquierda, el grafo con $E = \{ab, ad, bd, bc, dc\}$ tiene grados $\text{gr}(a) = \text{gr}(c) = 2$ y $\text{gr}(b) = \text{gr}(d) = 3$: es semi-euleriano, y el sendero b, a, d, c, b, d va de un impar (en verde) al otro. A la derecha, la mariposa con $E = \{ab, ac, bc, cd, ce, de\}$ tiene $\text{gr}(c) = 4$ y 2 en los demás: es euleriana, y c, a, b, c, d, e, c es un circuito euleriano. El número sobre cada arista es el orden en que se usa.



sendero b, a, d, c, b, d



circuito c, a, b, c, d, e, c

Ojito...

La hipótesis de conexidad no sobra, pero solo se pide entre los vértices que tienen aristas. Dos triángulos ajenos tienen todos sus grados pares y ningún circuito euleriano: no hay forma de pasar de uno al otro. En cambio, un triángulo más un vértice suelto sí tiene circuito euleriano.

Ya se puede contestar la pregunta de Euler.

Resultado

Los cuatro vértices del grafo de Königsberg tienen grado impar. No hay circuito euleriano ni sendero euleriano: no se pueden cruzar los siete puentes una sola vez cada uno, ni regresando al punto de partida ni sin regresar.

El caso dirigido

En un grafo dirigido el recorrido tiene que seguir las flechas, y el par entrada-salida de cada vértice intermedio se vuelve un arco que llega y uno que sale. La paridad se cambia por el balance.

Teorema 12.23 (Euler, caso dirigido)**.** *Sea G un grafo dirigido con al menos un arco, débilmente conexo salvo vértices aislados.*

1. *G tiene un circuito euleriano si y solo si $gr^-(v) = gr^+(v)$ para todo v.*
2. *G tiene un sendero euleriano no cerrado si y solo si hay un vértice s con $gr^+(s) - gr^-(s) = 1$, uno t con $gr^-(t) - gr^+(t) = 1$, y todos los demás balanceados. El sendero va de s a t.*

Idea de la prueba. Es la misma prueba. Cada paso por un vértice usa un arco que llega y uno que sale, así que un circuito deja a cada vértice balanceado. Al caminar, estando en $v \neq s$ se ha llegado a v una vez más de las que se ha salido, y si v está balanceado le queda un arco de salida libre. El sendero se reduce al circuito agregando el arco (t, s). □

El algoritmo de Hierholzer

Decidir el tipo cuesta lo mismo que leer el grafo: los grados salen de las longitudes de las listas, y una BFS desde un vértice con aristas revisa que todos los vértices con aristas queden en su componente. Si no quedan, no hay recorrido; si quedan, la tabla de arriba da la respuesta. En total, $\Theta(n + m)$.

Construir el recorrido también es lineal. La prueba del teorema 12.20 empalma subcircuitos; Hierholzer lo organiza con una pila para que el empalme salga solo, sin manipular listas.

Algoritmo 12.1 Hierholzer(G, s)

```
1:  procedimiento HIERHOLZER(G, s)
2:      para todo e ∈ E hacer
3:          usada[e] ← falso
4:      fin para
5:      para todo v ∈ V hacer
6:          sig[v] ← primera entrada de la lista de v
7:      fin para
8:      P ← pila vacía;  APILAR(P, s)
9:      C ← lista vacía
10:     mientras P no está vacía hacer
11:         u ← TOPE(P)
12:         mientras sig[u] apunta a una arista usada hacer
13:             avanzar sig[u]
14:         fin mientras
15:         si sig[u] apunta a una arista e = {u, w} entonces
16:             usada[e] ← verdadero
17:             APILAR(P, w)                          // seguir caminando
18:         si no                                     // u ya no tiene aristas libres
19:             DESAPILAR(P);  agregar u al frente de C
20:         fin si
21:     fin mientras
22:     devolver C
23: fin procedimiento
```

Si el grafo es euleriano se llama con cualquier s de grado positivo; si es semi-euleriano, **obligatoriamente** con uno de los dos impares. Las líneas que no son obvias:

- Líneas 12 a 14: sig$[u]$ es un apuntador que avanza sobre la lista de u y nunca retrocede. Cada arista $\{u, w\}$ aparece en la lista de u y en la de w, con el mismo identificador; al usarla desde u se marca, y la copia en la lista de w se salta cuando el apuntador de w pase por ella.
- Líneas 15 a 17: mientras el tope tenga aristas libres se camina, y la pila guarda la caminata actual.
- Línea 19: un vértice sin aristas libres ya no puede extender nada, y pasa al frente de C. Al

desapilar se regresa por la caminata; si aparece un vértice con aristas libres, desde él sale un subcircuito, cuyos vértices quedarán en C justo delante de ese vértice. Ese es el empalme de la prueba.

Ejemplo 12.24 (Hierholzer sobre la mariposa desde a)**.** Desde c, como en la diapositiva, la primera caminata ya usa todo y no se ve el empalme; desde a sí. Las listas, en orden alfabético: a: b, c; b: a, c; c: a, b, d, e; d: c, e; e: c, d.

Paso	**Tope**	**Acción**	**Pila** P	C
1	a	usa $\{a, b\}$ y apila b	a, b	
2	b	usa $\{b, c\}$ y apila c	a, b, c	
3	c	usa $\{c, a\}$ y apila a	a, b, c, a	
4	a	sin aristas libres: desapila	a, b, c	a
5	c	usa $\{c, d\}$ y apila d	a, b, c, d	a
6	d	usa $\{d, e\}$ y apila e	a, b, c, d, e	a
7	e	usa $\{e, c\}$ y apila c	a, b, c, d, e, c	a
8	c	sin aristas libres: desapila	a, b, c, d, e	c, a
9	e	desapila	a, b, c, d	e, c, a
10	d	desapila	a, b, c	d, e, c, a
11	c	desapila	a, b	c, d, e, c, a
12	b	desapila	a	b, c, d, e, c, a
13	a	desapila	vacía	a, b, c, d, e, c, a

En el paso 4 la caminata se atora en $a = s$, como predice la paridad, después de cerrar a, b, c, a y con $\{c, d\}$, $\{d, e\}$, $\{e, c\}$ sin usar. Al retroceder, c todavía tiene aristas libres, y de él sale el subcircuito c, d, e, c, que queda empalmado en c. El resultado, a, b, c, d, e, c, a, usa las seis aristas una vez cada una.

Costo. Cada arista se marca una sola vez y provoca un solo Apilar: hay $m + 1$ apilados contando s, y otros tantos desapilados. El trabajo de las líneas 12 a 14 se paga con el apuntador: sig$[u]$ recorre la lista de u una sola vez en todo el algoritmo, no una vez por visita a u, y las listas suman $2m$ entradas. En total, $\Theta(n + m)$ con listas de adyacencia. En un grafo dirigido el algoritmo es el mismo con la lista de arcos que salen de cada vértice, y ni siquiera hacen falta las marcas: cada arco aparece en una sola lista.

Ojito…

Hierholzer no comprueba nada: supone que el recorrido existe y que s es un inicio válido. Primero se clasifica el grafo y luego se elige s. En el grafo semi-euleriano del ejemplo 12.22, desde b (impar) devuelve b, a, d, b, c, d, un sendero euleriano igual de válido que el de la diapositiva. Desde a (par) devuelve a, b, c, d, b, a, que repite $\{a, b\}$ y omite $\{a, d\}$: la caminata de un vértice par se atora en el otro impar, no en su inicio, y el empalme queda mal hecho.

13. Hamilton, bipartitos, cliques y árboles *Complemento: Rosen, caps. 10 y 11*

Resumen rápido. La sección 12 buscó recorridos que usan cada arista una vez. Esta empieza con la pregunta gemela, pasar por cada vértice una vez, y muestra que el parecido engaña: Euler se decide en tiempo lineal y Hamilton es NP-completo. Se ven las condiciones necesarias, la suficiente de Dirac, la búsqueda con retroceso y el agente viajero con la programación dinámica de Held y Karp, que baja de $(n-1)!/2$ rutas a $\Theta(n^2 2^n)$ pasos. Luego los grafos bipartitos, que son exactamente los que no tienen ciclos impares, con un algoritmo de BFS que en $\Theta(n+m)$ entrega la partición o un ciclo impar como testigo, y las cliques, con el contraste entre maximal y máxima. La segunda mitad es de árboles: sus caracterizaciones, árboles con raíz, árboles k-arios llenos, casi completos y perfectos, el conteo de hojas y la altura, los tres recorridos y la evaluación de expresiones posfijas con una pila.

Caminos y ciclos hamiltonianos

Un recorrido euleriano pasa por cada **arista** exactamente una vez. Si se cambia arista por **vértice** sale otra pregunta, que lleva el nombre de W. R. Hamilton por un juego suyo de 1857: dar la vuelta a los 20 vértices de un dodecaedro sin repetir ninguno.

Definición 13.1 (Camino y ciclo hamiltonianos)**.** Sea $G = (V, E)$ con n vértices. Un **camino hamiltoniano** es un camino $v_1, v_2, \ldots, v_n$ que pasa por todos los vértices, cada uno exactamente una vez. Un **ciclo hamiltoniano** es un ciclo $v_1, v_2, \ldots, v_n, v_1$ que pasa por todos; el único repetido es v_1, por ser inicio y final. Un grafo con ciclo hamiltoniano es un **grafo hamiltoniano**.

Camino y ciclo van en el sentido de la sección 12: no repiten vértices. Un ciclo hamiltoniano usa exactamente n aristas, tenga el grafo las que tenga; un circuito euleriano las usa todas.

Condiciones necesarias y la suficiente de Dirac

Lo primero que se revisa antes de buscar un ciclo es lo que lo hace imposible.

Teorema 13.2 (Condiciones de grado)**.** *Sea G un grafo simple con $n \geq 3$ vértices.*

(a) Si G tiene ciclo hamiltoniano, $gr(v) \geq 2$ para todo v.
(b) Si G tiene más de dos vértices de grado 1, no tiene camino hamiltoniano.

Idea de la prueba. En un ciclo cada vértice usa dos aristas, una hacia su antecesor y otra hacia su sucesor. En un camino también, salvo los dos extremos, así que un vértice de grado 1 solo puede ser extremo. □

Teorema 13.3 (Vértices de corte)**.** *Si G tiene ciclo hamiltoniano, quitar cualquier conjunto S de $k \geq 1$ vértices deja a lo más k componentes conexas. En particular, un grafo con un vértice de corte no es hamiltoniano.*

Idea de la prueba. Quitar k vértices de un ciclo lo parte en a lo más k tramos, y cada tramo es un camino que sigue en $G - S$. Para caminos hamiltonianos sale un tramo más: a lo más $k+1$ componentes. □

Las condiciones de arriba solo descartan. La que garantiza un ciclo pide muchas aristas en cada vértice.

Teorema 13.4 (Dirac, 1952)**.** *Si G es simple, $n \geq 3$ y $gr(v) \geq n/2$ para todo v, entonces G tiene ciclo hamiltoniano.*

Idea de la prueba. Se prueba algo más fuerte, el teorema de Ore: basta $\operatorname{gr}(u) + \operatorname{gr}(v) \geq n$ para cada par no adyacente. En un contraejemplo con el máximo de aristas, agregar cualquier arista $\{u, v\}$ que falte crea un ciclo hamiltoniano, así que G tiene un camino hamiltoniano de u a v. Por la cota de grados, algún vecino de u en ese camino va justo después de un vecino de v, y con esas dos aristas el camino se cierra en un ciclo sin la arista nueva. □

En el ciclo C_5 cada vértice tiene grado $2 < 5/2$ y Dirac no dice nada, aunque el grafo es él mismo un ciclo hamiltoniano: la condición es suficiente, no necesaria.

Euler contra Hamilton. Euler se decide en $\Theta(n + m)$ contando grados impares (sección 12); para Hamilton no se conoce ningún algoritmo polinomial. La condición de Euler es **local**: como los vértices sí se pueden repetir, cada uno solo necesita emparejar las aristas por las que se entra con las que se sale, y dos circuitos que comparten un vértice se funden en uno. La de Hamilton es **global**: lo que se escoge en un vértice restringe a todos los demás, y fundir dos ciclos obliga a pasar dos veces por un vértice. Ninguna condición de grados cierra el hueco: el grafo de Petersen, 3-regular con $n = 10$ y sin vértices de corte, no es hamiltoniano. La prueba formal es la NP-completitud: un ciclo propuesto se verifica en tiempo lineal, y cualquier problema de NP se reduce a Hamilton (Karp, 1972; Cormen, teorema 34.13).

Búsqueda con retroceso

Sin una caracterización, lo que queda es buscar. La búsqueda con retroceso (*backtracking*) hace crecer un camino desde un vértice fijo, y cuando se atora deshace el último paso y prueba el siguiente vecino.

Algoritmo 13.1 Ciclo-Hamiltoniano(G): búsqueda con retroceso

```
1:  procedimiento Ciclo-Hamiltoniano(G)
2:      para todo v ∈ V hacer
3:          usado[v] ← falso
4:      fin para
5:      s ← cualquier vértice;  P ← [s];  usado[s] ← verdadero
6:      devolver Extender(G, P)
7:  fin procedimiento

8:  procedimiento Extender(G, P)
9:      u ← último vértice de P
10:     si |P| = n entonces
11:         devolver {u, P[1]} ∈ E                    // ¿cierra el ciclo?
12:     fin si
13:     para todo v adyacente a u con no usado[v] hacer
14:         agregar v al final de P;  usado[v] ← verdadero
15:         si Extender(G, P) entonces
16:             devolver verdadero
17:         fin si
18:         quitar v de P;  usado[v] ← falso           // retroceder
19:     fin para
20:     devolver falso
21: fin procedimiento
```

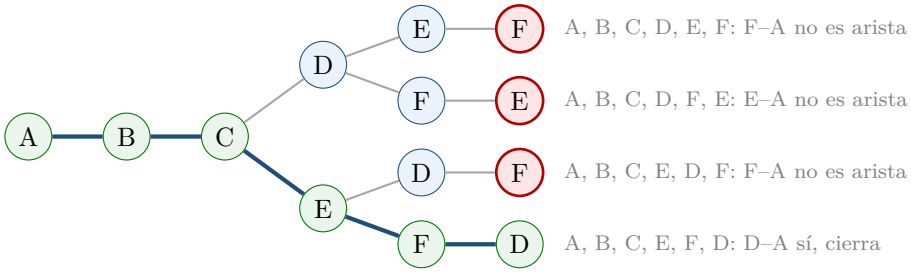
Fijar el inicio s no pierde nada: todo ciclo hamiltoniano pasa por s. **Costo.** Cada llamada a Extender

corresponde a un camino distinto que empieza en s, y el grafo con más caminos así es K_n:

$$\sum_{k=0}^{n-1} \frac{(n-1)!}{(n-1-k)!} = (n-1)! \sum_{j=0}^{n-1} \frac{1}{j!} \leq e \cdot (n-1)!.$$

Para K_5 son 65, el mismo número del ojito de la sección 11: es aquí donde ese factorial aparece de verdad. Cada llamada trabaja $\mathcal{O}(n)$ fuera de sus llamadas recursivas, así que el total es $\mathcal{O}(n!)$. Las podas de los teoremas 13.2 y 13.3 ahorran mucho en la práctica, pero no cambian el peor caso.

Ejemplo 13.5 (Retroceso sobre el grafo pesado, sin pesos)**.** Se busca un ciclo en el grafo pesado de la sección 11 ignorando los pesos, desde A y con los vecinos en orden alfabético. Cada nodo del árbol de búsqueda es el vértice que se agrega; las ramas se exploran de arriba hacia abajo.



Las dos ramas bajo D llegan a usar los seis vértices sin poder cerrar, porque A solo es adyacente a B, C y D. Se regresa a C y se prueba E: bajo E, la rama de D falla igual y la de F termina en D, que sí es vecino de A. El ciclo es A, B, C, E, F, D, A, tras 13 llamadas a Extender.

El agente viajero

Definición 13.6 (Problema del agente viajero)**.** Dadas n ciudades y el costo $w(u, v) \geq 0$ de ir de una a otra, el **problema del agente viajero** (*traveling salesman problem*, TSP) pide la ruta que visita cada ciudad una vez, regresa al inicio y tiene el menor costo total: un ciclo hamiltoniano de costo mínimo en un grafo completo y pesado.

Ojito...

"La ruta más corta" del agente viajero no es el camino más corto entre dos ciudades de las secciones 17 a 19, que se resuelve en tiempo polinomial, sino un ciclo que toca a todas.

Si el grafo no es completo, una arista que falta vale $w = \infty$. Es al menos tan difícil como Hamilton: con peso 0 en las aristas de G y 1 en las demás, G es hamiltoniano si y solo si hay una ruta de costo 0. La fuerza bruta fija la ciudad de salida y permuta las otras $n-1$: $(n-1)!/2$ rutas distintas, cada una evaluada en $\Theta(n)$, en total $\Theta(n!)$.

Programación dinámica (Bellman; Held y Karp, 1962). Con las ciudades numeradas $1, \ldots, n$ y salida en 1, sea $g[S][j]$ el costo del camino más barato que sale de 1, visita exactamente los vértices de $S \subseteq \{2, \ldots, n\}$ y termina en $j \in S$. El penúltimo vértice de ese camino es algún $i \in S$ distinto de j, y lo que viene antes es un camino de 1 a i por $S \setminus \{j\}$, que debe ser el más barato. Eso da

$$g[\{j\}][j] = w(1, j), \qquad g[S][j] = \min_{i \in S \setminus \{j\}} \big(g[S \setminus \{j\}][i] + w(i, j)\big),$$

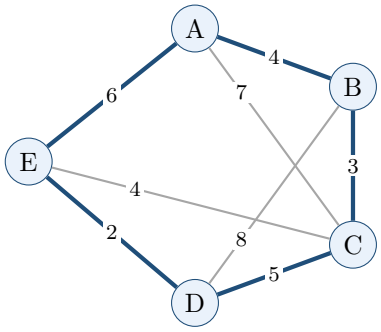
y la ruta óptima cuesta $\min_j \big(g[\{2, \ldots, n\}][j] + w(j, 1)\big)$. La mejor manera de terminar depende de qué falta y de dónde se está, no del orden en que se visitó S: hay $(n-1)!$ órdenes, pero solo 2^{n-1} conjuntos.

Costo. Recorrer los conjuntos por tamaño creciente garantiza que $g[S \setminus \{j\}][\cdot]$ ya está calculado. Hay $(n-1)\,2^{n-2}$ pares (S, j), cada uno con un mínimo sobre a lo más $n-2$ términos: $\Theta(n^2 2^n)$ en tiempo y $\Theta(n2^n)$ en memoria. Para $n = 20$ son unos 4.2×10^8 pasos contra 6.1×10^{16} rutas.

Algoritmo 13.2 Agente-Viajero(w, n): programación dinámica de Held y Karp

```
1:  procedimiento AGENTE-VIAJERO(w, n)
2:      g[{j}][j] ← w(1, j) para j = 2, ..., n
3:      para k ← 2 hasta n − 1 hacer                    // por tamaño creciente
4:          para todo S ⊆ {2, ..., n} con |S| = k hacer
5:              para todo j ∈ S hacer
6:                  g[S][j] ← mín_{i∈S\{j}} (g[S \ {j}][i] + w(i, j))
7:              fin para
8:          fin para
9:      fin para
10:     devolver mín_{2≤j≤n} (g[{2, ..., n}][j] + w(j, 1))
11: fin procedimiento
```

Ejemplo 13.7 (El agente viajero de las diapositivas)**.** Cinco ciudades con $w(A,B) = 4$, $w(A,C) = 7$, $w(A,E) = 6$, $w(B,C) = 3$, $w(B,D) = 8$, $w(C,D) = 5$, $w(C,E) = 4$ y $w(D,E) = 2$.



óptimo: A, B, C, D, E, A, con costo 20

Faltan A–D y B–E, que valen ∞; de las 12 rutas de K_5 solo existen cuatro. La tabla de Held y Karp, con salida en A, da el óptimo sin enumerarlas. Cada renglón es un conjunto S y cada columna el vértice final j; un guion indica $j \notin S$.

S	$j=B$	$j=C$	$j=D$	$j=E$
$\{B\}$	4	–	–	–
$\{C\}$	–	7	–	–
$\{D\}$	–	–	∞	–
$\{E\}$	–	–	–	6
$\{B,C\}$	10	7	–	–
$\{B,D\}$	∞	–	12	–
$\{B,E\}$	∞	–	–	∞
$\{C,D\}$	–	∞	12	–
$\{C,E\}$	–	10	–	11
$\{D,E\}$	–	–	8	∞

S	$j=B$	$j=C$	$j=D$	$j=E$
$\{B,C,D\}$	20	17	12	–
$\{B,C,E\}$	13	∞	–	11
$\{B,D,E\}$	16	–	∞	14
$\{C,D,E\}$	–	13	13	14
$\{B,C,D,E\}$	16	18	13	14
$+\,w(j,A)$	20	25	∞	20

Por ejemplo, $g[\{B,C\}][C] = g[\{B\}][B] + w(B,C) = 7$, porque la única opción es A, B, C. Y

$g[\{B, C, D, E\}][E] = 14$ es A, B, C, D, E, con $4 + 3 + 5 + 2$; cerrar con $w(E, A) = 6$ da 20. Por la columna de B también sale 20: es la misma ruta al revés.

Grafos bipartitos

Definición 13.8 (Grafo bipartito)**.** Un grafo no dirigido $G = (V, E)$ es **bipartito** si V se parte en dos conjuntos disjuntos, $V = V_1 \cup V_2$, de modo que toda arista tiene un extremo en V_1 y el otro en V_2. El **bipartito completo** $K_{p,q}$ tiene $|V_1| = p$, $|V_2| = q$ y cada vértice de V_1 unido con todos los de V_2: tiene pq aristas.

Modela cualquier relación entre dos tipos de objetos: estudiantes y proyectos, trabajadores y tareas. Asignar a cada estudiante un proyecto distinto es el emparejamiento de la sección 21. Las diapositivas y Rosen escriben $K_{m,n}$; aquí los lados se llaman p y q porque $n = |V|$ y $m = |E|$.

La caracterización por ciclos impares

Un ciclo en un bipartito va de un lado al otro en cada paso, así que para volver al lado de salida necesita un número par de pasos. Lo notable es que eso basta: lo único que impide partir un grafo en dos lados es un ciclo impar.

Teorema 13.9 (Caracterización de los bipartitos)**.** *Un grafo no dirigido es bipartito si y solo si no contiene ciclos de longitud impar.*

Demostración. ($\Rightarrow$) Sean V_1, V_2 los lados y $v_0, v_1, \ldots, v_k = v_0$ un ciclo. Cada arista cambia de lado, así que v_i está en el lado de v_0 exactamente cuando i es par. Como $v_k = v_0$, k es par.

($\Leftarrow$) Basta partir cada componente conexa por separado: no hay aristas entre componentes, así que la unión de las particiones sirve. Sea entonces G conexo y sin ciclos impares. Se corre BFS desde cualquier s y se pone

$$V_1 = \{v : d[v] \text{ es par}\}, \qquad V_2 = \{v : d[v] \text{ es impar}\}.$$

Falta ver que ninguna arista queda dentro de un lado. Supóngase que $\{u, v\} \in E$ con $d[u]$ y $d[v]$ de la misma paridad.

Primero, $d[u] = d[v]$. En BFS, $d[v] = \delta(s, v)$, el número de aristas del camino más corto (sección 11; la prueba está en la 14). Un camino más corto a u seguido de la arista $\{u, v\}$ llega a v, así que $\delta(s, v) \leq \delta(s, u) + 1$, y por simetría $|d[u] - d[v]| \leq 1$. Con la misma paridad, la diferencia es 0.

Luego, el ciclo. Sea w el ancestro común más profundo de u y v en el árbol BFS; existe, porque s es ancestro de todos. Cada arista del árbol baja exactamente un nivel, así que el camino del árbol de w a u mide $d[u] - d[w]$, y el de w a v mide $d[v] - d[w] = d[u] - d[w]$. Por ser w el más profundo, los dos caminos solo comparten a w; además $w \neq u$, porque u no puede ser ancestro de v estando en el mismo nivel. La arista $\{u, v\}$ no es del árbol, porque une dos vértices del mismo nivel. Los dos caminos y la arista forman entonces un ciclo de longitud

$$2\big(d[u] - d[w]\big) + 1,$$

impar. Contradicción. □

El algoritmo: dos lados con BFS

La prueba es un algoritmo: BFS asigna a cada vértice el lado que dice la paridad de su distancia, y una arista dentro de un lado da el ciclo impar subiendo por los padres desde sus dos extremos. El lado

se guarda en lado$[v] \in \{0, 1\}$, para no chocar con color$[v]$ de la sección 14. Las secciones 14 y 21 usan este algoritmo tal cual.

Algoritmo 13.3 Ciclo-Impar(u, v): el testigo de que no es bipartito

```
1: procedimiento Ciclo-Impar(u, v)                    // al llamarse, d[u] = d[v]
2:     x ← u,    y ← v,    P ← [u],    R ← [v]
3:     mientras x ≠ y hacer                           // suben a la par
4:         x ← π[x];  agregar x al final de P
5:         y ← π[y];  agregar y al final de R
6:     fin mientras
7:     devolver P seguida de R al revés, sin repetir x
8: fin procedimiento
```

Algoritmo 13.4 Bipartito(G): dos lados con BFS

```
1: procedimiento Bipartito(G)
2:     para todo v ∈ V hacer
3:         lado[v] ← nulo,    π[v] ← nulo
4:     fin para
5:     para todo s ∈ V hacer                          // una BFS por componente
6:         si lado[s] = nulo entonces
7:             lado[s] ← 0,    d[s] ← 0
8:             Q ← cola vacía;  Encolar(Q, s)
9:             mientras Q no está vacía hacer
10:                u ← Desencolar(Q)
11:                para todo v en la lista de adyacencia de u hacer
12:                    si lado[v] = nulo entonces
13:                        lado[v] ← 1 − lado[u],    d[v] ← d[u] + 1,    π[v] ← u
14:                        Encolar(Q, v)
15:                    si no si lado[v] = lado[u] entonces
16:                        devolver falso, Ciclo-Impar(u, v)
17:                    fin si
18:                fin para
19:            fin mientras
20:        fin si
21:    fin para
22:    devolver verdadero, V1 = {v : lado[v] = 0}, V2 = {v : lado[v] = 1}
23: fin procedimiento
```

El lado es la paridad de $d[v]$; d se guarda solo para que Ciclo-Impar sepa que los dos extremos están en el mismo nivel y puedan subir a la par. A diferencia de la detección de ciclos de la sección 11, no hace falta excluir al padre: está del otro lado y nunca dispara la condición. Una arista fuera del árbol entre niveles consecutivos cierra un ciclo **par** y es inofensiva; solo una arista dentro de un mismo nivel delata un ciclo impar.

Costo. Cada vértice recibe lado una sola vez, justo antes de encolarse, y la lista de cada vértice se recorre una vez, cuando sale de la cola; las listas suman $2m$ entradas. Total, $\Theta(n + m)$ con listas, y $\Theta(n^2)$ con matriz. Ciclo-Impar sube un nivel por iteración: $\mathcal{O}(n)$.

Idea clave

El algoritmo siempre entrega un testigo que se revisa a mano: la partición, si es bipartito, o un ciclo impar, si no. Es la diferencia con Hamilton, donde un "no" no viene con ningún certificado corto conocido.

Ejemplo 13.10 (El grafo G de la clase no es bipartito)**.** Se corre Bipartito sobre el grafo G de la sección 11, con los vecinos en orden alfabético.

Paso	Sale	Asigna o revisa	Cola después
0		A: 0	[A]
1	A	B: 1, C: 1	[B, C]
2	B	A bien; D: 0, E: 0	[C, D, E]
3	C	A bien; F: 0	[D, E, F]
4	D	B bien; G: 1	[E, F, G]
5	E	B bien; F tiene 0, **igual** que E	conflicto

En el paso 5 sale E, del lado 0, y su vecino F también está en el lado 0: $d[E] = d[F] = 2$, la arista E–F queda dentro de un nivel. Ciclo-Impar(E, F) sube $E \to B \to A$ y $F \to C \to A$, se encuentra en A y devuelve E, B, A, C, F; con la arista F–E es el ciclo A, B, E, F, C, A, de longitud 5. Es el mismo ciclo que la clase encontró con BFS, y ahora se sabe además que es impar. G no es bipartito.

En un bipartito conexo la partición es única salvo intercambiar los lados: el lado de s fija el de todos los demás.

El mismo argumento de alternancia da una poda para Hamilton.

Corolario 13.11. *Sea G bipartito con lados V_1 y V_2. Si G tiene ciclo hamiltoniano, $|V_1| = |V_2|$. Si tiene camino hamiltoniano, $||V_1| - |V_2|| \leq 1$.*

El ciclo y el camino alternan de lado y pasan por todos los vértices. Esto liquida de un golpe la estrella $K_{1,5}$ (lados de 1 y 5: ni camino) y muestra que $K_{2,3}$ no es hamiltoniano.

Cliques

Definición 13.12 (Clique)**.** Una **clique** de G es un conjunto $C \subseteq V$ de vértices adyacentes dos a dos: si $u, v \in C$ y $u \neq v$, entonces $\{u, v\} \in E$. Su tamaño es $|C|$. El **número de clique** $\omega(G)$ es el tamaño de la clique más grande. Una clique es **maximal** si no se le puede agregar ningún vértice, porque ninguno fuera de ella es adyacente a todos los de adentro; es **máxima** si tiene tamaño $\omega(G)$.

C es clique exactamente cuando su subgrafo inducido $G[C]$ es isomorfo al completo $K_{|C|}$. Toda clique máxima es maximal; al revés no.

Ejemplo 13.13 (Las cliques del grafo pesado)**.** En el grafo pesado de la sección 11, sin pesos, los cuatro vértices A, B, C, D están unidos todos con todos. $\{A, B, C, D\}$ es una clique de tamaño 4, así que $\omega(G) \geq 4$. Para la igualdad hay que descartar una de tamaño 5: una K_5 necesita cinco vértices de grado al menos 4, y los grados son $3, 3, 4, 5, 3, 2$ para A a F; solo C y D llegan. Así que $\omega(G) = 4$. Las cliques maximales son tres: $\{A, B, C, D\}$, $\{C, D, E\}$ y $\{D, E, F\}$. La última es maximal, porque ningún otro vértice es adyacente a D, E y F a la vez, pero no máxima.

Ojito...

Exhibir una clique de tamaño k solo prueba $\omega(G) \geq k$; para la igualdad falta un argumento de que no hay una más grande, como el de grados de arriba.

La diferencia entre maximal y máxima es algorítmica. Una clique maximal se encuentra con un barrido voraz en $\mathcal{O}(n^2)$: se agrega cada vértice adyacente a todos los ya escogidos. El resultado depende del orden: en el grafo pesado, A, B, ..., F da la máxima, y F, E, ..., A da $\{F, E, D\}$, maximal y nada más. Decidir si $\omega(G) \geq k$ es el problema CLIQUE, NP-completo (Cormen, teorema 34.11): la misma brecha que entre Euler y Hamilton.

Árboles

Definición y caracterizaciones

Definición 13.14 (Árbol y bosque)**.** Un **árbol** es un grafo no dirigido, conexo y sin ciclos. Un **bosque** es un grafo sin ciclos, no necesariamente conexo: cada una de sus componentes es un árbol. Una **hoja** de un árbol es un vértice de grado 1.

Cormen dice *free tree* para distinguirlo del árbol con raíz. Todo lo que sigue sale de dos lemas.

Lema 13.15 (Hojas)**.** *Todo árbol con $n \geq 2$ vértices tiene al menos dos hojas.*

Idea de la prueba. Los extremos de un camino de longitud máxima son hojas: otro vecino cerraría un ciclo o alargaría el camino. □

Lema 13.16 (Aristas)**.** *Todo árbol con n vértices tiene $n - 1$ aristas. Un bosque con n vértices y c componentes tiene $n - c$.*

Idea de la prueba. Inducción sobre n, quitando una hoja y su arista: lo que queda sigue siendo un árbol. Para el bosque se suma sobre las componentes. □

Teorema 13.17 (Caracterizaciones de un árbol)**.** *Para un grafo no dirigido G con n vértices y m aristas, son equivalentes:*

(1) G es un árbol: conexo y sin ciclos.
(2) Entre cada par de vértices hay un único camino.
(3) G es conexo y $m = n - 1$.
(4) G no tiene ciclos y $m = n - 1$.
(5) G es conexo y cada arista es un puente.
(6) G no tiene ciclos y agregar cualquier arista nueva crea uno.

Idea de la prueba. Dos caminos distintos entre los mismos vértices cierran un ciclo, y un ciclo da dos caminos entre vecinos: (1) y (2). El lema 13.16 da (3) y (4); al revés, quitar aristas de ciclos de un conexo con $n - 1$ aristas dejaría un árbol con menos de $n - 1$, y un bosque con $m = n - 1$ tiene una sola componente. (5) es que ninguna arista está en un ciclo, y (6) que no hay dos componentes que una arista pueda unir sin cerrar uno. □

(5) y (6) dicen lo mismo desde los dos lados: un árbol es un grafo conexo con el mínimo de aristas y un grafo acíclico con el máximo. Kruskal vive de eso: por (6), una arista entre dos vértices que ya están unidos cerraría un ciclo, y por (4), se detiene a las $n - 1$ aristas (sección 16).

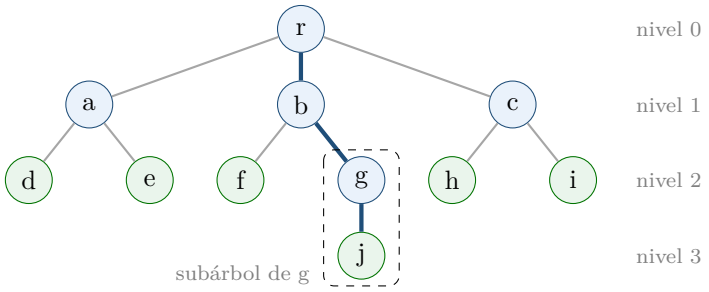
Árboles con raíz

Definición 13.18 (Árbol con raíz)**.** Un **árbol con raíz** (*rooted tree*) es un árbol con un vértice distinguido, la **raíz**; las aristas se leen de arriba hacia abajo. Para un vértice v:

- si v no es la raíz, su **padre** es el vecino que está en el camino de v a la raíz; v es **hijo** de su padre, y dos vértices con el mismo padre son **hermanos**;
- sus **ancestros** son los vértices del camino de la raíz a v, sin contar a v; sus **descendientes**, los vértices que tienen a v como ancestro;
- una **hoja** es un vértice sin hijos y un vértice **interno** uno con al menos un hijo;
- el **subárbol con raíz** v tiene a v, sus descendientes y las aristas entre ellos.

El **nivel** de v es la longitud del camino de la raíz a v; la raíz está en el nivel 0. La **altura** del árbol es el mayor nivel de sus hojas.

Ejemplo 13.19 (El vocabulario sobre un árbol)**.** El árbol de las diapositivas, con raíz r; en verde, las hojas.



El padre de g es b; los hijos de b son f y g; el hermano de h es i. Las hojas son d, e, f, h, i, j y los internos r, a, b, c, g. Los ancestros de j son r, b, g, el camino en azul; los descendientes de c son h e i. El subárbol con raíz g tiene los vértices g, j y la arista $\{g, j\}$. j está en el nivel 3 y la altura es 3.

Los árboles de los recorridos ya son árboles con raíz: en el árbol BFS de la sección 11, $\pi[v]$ es el padre de v y $d[v]$ es su nivel. Un árbol es **ordenado** si los hijos de cada vértice tienen un orden fijo de izquierda a derecha; en un binario ordenado se habla de hijo izquierdo e hijo derecho.

Ojito...

"Hoja" no significa lo mismo en el árbol libre y en el árbol con raíz. En el libre es grado 1; en el árbol con raíz, sin hijos. Difieren en la raíz: una raíz con un solo hijo tiene grado 1 y no es hoja.

Árboles k-arios: lleno, casi completo y perfecto

Definición 13.20 (Árboles k-arios)**.** Un árbol con raíz es **k-ario** si cada vértice tiene a lo más k hijos; **binario** si $k = 2$ y **ternario** si $k = 3$. Un árbol k-ario es:

- **lleno** si cada vértice interno tiene exactamente k hijos;
- **casi completo** si todos sus niveles están llenos salvo tal vez el último, que se llena de izquierda a derecha: es la forma del heap;
- **perfecto** si es lleno y todas sus hojas están en el mismo nivel.

Ojito...

"Completo" tiene tres significados según la fuente, y aquí se evita a propósito. Las diapositivas, siguiendo a Rosen (*full m-ary tree*), llaman árbol n-ario completo al que aquí es **lleno**. La clase de heaps (17 de septiembre) llama árbol binario completo al que aquí es **casi completo**, el *nearly complete* de Cormen. Y Cormen llama *complete k-ary tree* al que aquí es **perfecto**.

Teorema 13.21 (Conteo en árboles llenos)**.** *Un árbol k-ario lleno con i vértices internos tiene*

$$n = ki + 1 \text{ vértices} \qquad y \qquad \ell = (k-1)i + 1 \text{ hojas.}$$

Idea de la prueba. Todo vértice salvo la raíz es hijo de exactamente un interno, y cada interno tiene k hijos: $n - 1 = ki$. Las hojas son el resto. □

Las diapositivas escriben $m = ni+1$, con n la aridad. Un torneo de eliminación directa con 16 jugadores es un árbol binario lleno con $\ell = 16$ hojas: hay $i = \ell - 1 = 15$ partidos.

Definición 13.22 (Árbol balanceado)**.** Un árbol con raíz de altura h es **balanceado** si todas sus hojas están en los niveles h o $h-1$.

El del ejemplo 13.19 es balanceado: altura 3, hojas en los niveles 2 y 3. Todo casi completo es balanceado, y todo perfecto también. Balanceado, en este sentido, no es AVL: solo mira dónde están las hojas, no las alturas de los subárboles de cada vértice.

Teorema 13.23 (Altura contra hojas)**.** *(a) Un árbol k-ario de altura h tiene a lo más k^h hojas; por tanto $h \geq \lceil \log_k \ell \rceil$. (b) Si además es lleno y balanceado, $h = \lceil \log_k \ell \rceil$.*

Idea de la prueba. (a) Inducción sobre h: la raíz tiene a lo más k hijos, cada uno raíz de un subárbol con a lo más k^{h-1} hojas. (b) Lleno y balanceado, el nivel $h-1$ tiene exactamente k^{h-1} vértices y alguno es interno, así que $k^{h-1} < \ell \leq k^h$. □

La parte (a) da la cota inferior $\Omega(n \lg n)$ de ordenar por comparaciones: un árbol de decisión que ordene necesita $n!$ hojas, y su altura es al menos $\lg n! = \Theta(n \lg n)$. El torneo de 16 jugadores es lleno y balanceado: $\lceil \lg 16 \rceil = 4$ rondas.

Recorridos

Un **recorrido** de un árbol visita todos sus vértices de manera sistemática. En un árbol binario ordenado hay tres naturales, según cuándo se visita la raíz respecto a sus dos subárboles: **preorden** (raíz, izquierdo, derecho), **inorden** (izquierdo, raíz, derecho) y **postorden** (izquierdo, derecho, raíz).

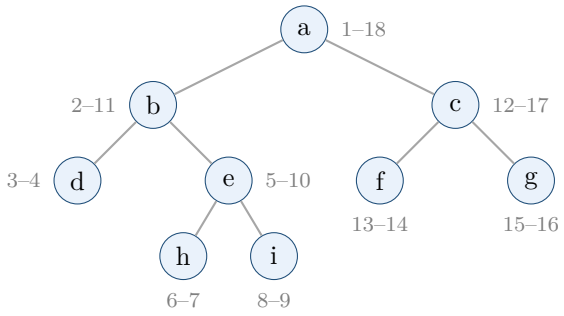
Los tres caben en un solo procedimiento, Recorrer: cada uno conserva una sola de sus tres líneas de visita, Preorden la 3, Inorden la 5 y Postorden la 7.

Costo. Cada llamada hace trabajo constante más dos llamadas recursivas. Con j vértices en el subárbol izquierdo, $T(n) = T(j) + T(n-j-1) + d$ y $T(0) = c$, y por sustitución $T(n) = (c+d)n + c$. Los tres son $\Theta(n)$, sin importar la forma del árbol (Cormen, teorema 12.1).

Algoritmo 13.5 Recorrer(x): los tres recorridos de un árbol binario

```
1: procedimiento Recorrer(x)
2:     si x ≠ nulo entonces
3:         visitar x                          // solo en preorden
4:         Recorrer(izq[x])
5:         visitar x                          // solo en inorden
6:         Recorrer(der[x])
7:         visitar x                          // solo en postorden
8:     fin si
9: fin procedimiento
```

Ejemplo 13.24 (Los tres recorridos y el reloj de DFS)**.** El árbol de las diapositivas, con entra–sale de DFS-Visitar junto a cada vértice, visitando primero el hijo izquierdo.

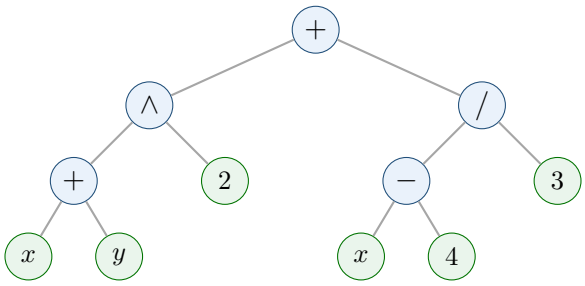


Recorrido	Resultado	Cómo sale del reloj
Preorden	a, b, d, e, h, i, c, f, g	orden creciente de entra
Inorden	d, b, h, e, i, a, f, c, g	(solo binarios)
Postorden	d, h, i, e, b, f, g, c, a	orden creciente de sale
Por niveles	a, b, c, d, e, f, g, h, i	BFS desde a

El preorden es el orden en que DFS descubre los vértices y el postorden el orden en que los termina. El inorden solo tiene sentido en binarios, porque necesita un "en medio" entre dos hijos.

Árboles de expresiones y notación posfija

Una expresión aritmética con operadores binarios es un árbol binario lleno: cada operador es un vértice interno con sus dos operandos como hijos, y los números y variables son las hojas. El árbol de $(x+y)^2+(x-4)/3$, con $\wedge$ para la potencia:



El preorden da la notación **prefija** (polaca), $+\wedge+\,x\;y\;2\;/\;-\;x\;4\;3$; el postorden, la **posfija** (polaca inversa), $x\;y\;+\;2\;\wedge\;x\;4\;-\;3\;/\;+$; y el inorden, la **infija**, que necesita paréntesis: $((x+y)\wedge 2)+((x-4)/3)$.

Sin ellos, $x + y \wedge 2 + x - 4/3$ se leería $x + y^2 + x - 4/3$, otra expresión. La prefija y la posfija no los necesitan: cada operador lleva exactamente dos operandos, así que la cadena determina el árbol.

La posfija se evalúa con una pila, de izquierda a derecha y sin mirar adelante: los operandos se apilan, y cada operador desapila sus dos operandos y apila el resultado.

Algoritmo 13.6 Evaluar-Posfija$(t_1, \ldots, t_k)$

```
1:  procedimiento EVALUAR-POSFIJA(t_1, ..., t_k)
2:      P ← pila vacía
3:      para i ← 1 hasta k hacer
4:          si t_i es un número o una variable entonces
5:              APILAR(P, valor(t_i))
6:          si no                                   // t_i es un operador binario ∘
7:              b ← DESAPILAR(P);  a ← DESAPILAR(P)
8:              APILAR(P, a ∘ b)
9:          fin si
10:     fin para
11:     devolver DESAPILAR(P)
12: fin procedimiento
```

Ejemplo 13.25 (Evaluar la posfija con $x = 1$, $y = 2$)**.** Contenido de la pila después de cada símbolo, con el fondo arriba:

x	y	$+$	2	$\wedge$	x	4	$-$	3	$/$	$+$
1	1	3	3	9	9	9	9	9	9	8
	2		2		1	1	-3	-3	-1	
						4		3		

El resultado es 8, y directamente $(1 + 2)^2 + (1 - 4)/3 = 9 - 1 = 8$.

El orden de los desapilamientos importa: el primero que sale es el operando **derecho**. En el $-$ del ejemplo sale primero el 4 y luego el 1, y el resultado es $1 - 4 = -3$, no $4 - 1$. Hay un apilamiento por símbolo y dos desapilamientos por operador: $\Theta(k)$. Funciona porque el postorden escribe cada operador justo después de sus dos subárboles completos, así que cuando llega el operador sus dos valores son los dos de arriba de la pila.

14. Representaciones, BFS y DFS a fondo *Complemento: Cormen, cap. 22.1 a 22.3*

Resumen rápido. La clase del 22 de octubre presentó las dos representaciones y los dos recorridos con un ejemplo cada uno. Esta sección los retoma con el rigor de Cormen. Primero, el costo de cada operación con listas y con matriz, con un grafo dirigido y su transpuesto de ejemplo y el sumidero universal como excepción en $\mathcal{O}(n)$. Después, BFS con colores: por qué su costo es $\mathcal{O}(n+m)$ y no $\Theta(n+m)$, la prueba de que calcula distancias, el árbol BFS, los ciclos impares y el diámetro de un árbol. Al final, DFS con colores y tiempos: el bosque DFS, los teoremas del paréntesis y del camino blanco, los cuatro tipos de arista, la versión iterativa con sus dos trampas y las componentes conexas. Es la maquinaria de pruebas sobre la que se apoyan las secciones que siguen.

La notación de atributos es la de la sección 11, que también da su equivalencia con la de Cormen: $d[v]$, $\pi[v]$, color$[v]$, y entra$[v]$ y sale$[v]$ para los tiempos de DFS.

Listas y matriz: el costo de cada operación

La clase del 22 de octubre definió las dos representaciones y comparó cuatro operaciones; aquí se completa la cuenta. Las **listas de adyacencia** son un arreglo Adj de n listas: $Adj[u]$ contiene los v con $(u,v) \in E$ en un dirigido, o con $\{u,v\} \in E$ en uno no dirigido, en un orden arbitrario. El orden no significa nada para el grafo, pero sí decide qué vecino descubren primero BFS y DFS; por eso cada ejemplo dice qué orden usa.

Lema 14.1 (Longitud total de las listas)**.** *En un grafo dirigido $\sum_{u \in V} |Adj[u]| = m$. En uno no dirigido la suma es $2m$, porque $\{u,v\}$ aparece como v en $Adj[u]$ y como u en $Adj[v]$.*

En un dirigido escribimos $\mathrm{gr}^+(u)$ para el **grado de salida** (el número de aristas (u,v)) y $\mathrm{gr}^-(u)$ para el **grado de entrada** (el de aristas (v,u)); el lema dice que cada una de las dos sumas de grados vale m. De ahí sale el $\Theta(n+m)$ de memoria de las listas: n cabezas más m o $2m$ nodos. El término n no se puede quitar, porque un grafo con vértices aislados sigue pagando su arreglo de cabezas.

Operación	**Listas**	**Matriz**
Memoria	$\Theta(n+m)$	$\Theta(n^2)$
¿Es (u,v) arista?	$\mathcal{O}(\mathrm{gr}^+(u))$, peor caso $\mathcal{O}(n)$	$\Theta(1)$
Recorrer los vecinos de u	$\Theta(1+\mathrm{gr}^+(u))$	$\Theta(n)$
Recorrer todas las aristas	$\Theta(n+m)$	$\Theta(n^2)$
Grado de salida de u	$\Theta(\mathrm{gr}^+(u))$; $\Theta(1)$ si la lista guarda su longitud	$\Theta(n)$
Grados de entrada de todos	$\Theta(n+m)$	$\Theta(n^2)$
Agregar una arista	$\mathcal{O}(1)$, al frente y sin revisar duplicados	$\Theta(1)$
Borrar la arista (u,v)	$\mathcal{O}(\mathrm{gr}^+(u))$; en no dirigido, también $\mathcal{O}(\mathrm{gr}(v))$	$\Theta(1)$
Grafo transpuesto	$\Theta(n+m)$	$\Theta(n^2)$

En un no dirigido, $\mathrm{gr}^+(u)$ es simplemente $\mathrm{gr}(u)$.

Grafos pesados y atributos. En listas, el peso $w(u,v)$ viaja en el nodo de v dentro de $Adj[u]$; en la matriz va en la casilla (u,v), y en las casillas sin arista se guarda lo que pida el problema: ∞ para caminos más cortos, 0 para capacidades. Los atributos de vértice, como $d[v]$ o color$[v]$, viven en arreglos indexados por vértice, con acceso $\mathcal{O}(1)$.

Ojito...

- $\Theta(n+m)$ no es $\Theta(m)$. Las dos coinciden cuando el grafo es no dirigido y conexo, porque entonces $m \geq n-1$, pero no en general.
- Cada arista no dirigida vive dos veces en las listas. Borrarla o cambiarle el peso obliga a tocar $Adj[u]$ y $Adj[v]$; olvidar una copia deja el grafo inconsistente.
- La matriz cobra $\Theta(n^2)$ desde que se crea, porque inicializarla ya cuesta eso. Un algoritmo que promete $\mathcal{O}(n+m)$ no puede construir una matriz por dentro, ni siquiera para consultar aristas rápido.

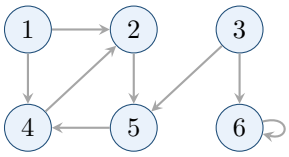
Grados y grafo transpuesto

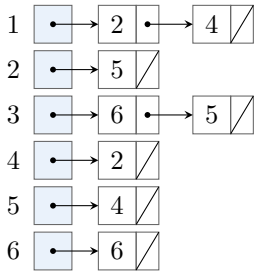
Los grados de salida y de entrada de todos los vértices salen de un solo recorrido de las listas: por cada $v \in Adj[u]$ se suma uno a $\text{gr}^+[u]$ y uno a $\text{gr}^-[v]$. Por el lema 14.1 cuesta $\Theta(n+m)$. Los de salida tienen atajo si cada lista guarda su longitud; los de entrada no, porque cualquier entrada de cualquier lista puede sumarle a cualquier vértice. Con matriz, $\text{gr}^+(u)$ es la suma del renglón u y $\text{gr}^-(u)$ la de la columna u: $\Theta(n^2)$ para todos.

Definición 14.2 (Grafo transpuesto)**.** El **transpuesto** de un grafo dirigido $G = (V, E)$ es $G^{\mathsf{T}} = (V, E^{\mathsf{T}})$ con $E^{\mathsf{T}} = \{(v, u) : (u, v) \in E\}$: el mismo grafo con todas las aristas invertidas. Su matriz es A^{T}.

Con listas se construye recorriendo u en orden creciente y, por cada $v \in Adj[u]$, agregando u al final de $Adj^{\mathsf{T}}[v]$: m inserciones de $\mathcal{O}(1)$, total $\Theta(n+m)$, y cada lista de G^{T} sale ordenada sin una sola comparación. Con matriz, transponer cuesta $\Theta(n^2)$. Las componentes fuertemente conexas de la sección 15 corren DFS sobre G y luego sobre G^{T}, y necesitan transponer en tiempo lineal.

Ejemplo 14.3 (Un grafo dirigido, sus grados y su transpuesto)**.** El grafo de la figura 22.2 de Cormen tiene seis vértices y ocho aristas: $(1,2)$, $(1,4)$, $(2,5)$, $(3,5)$, $(3,6)$, $(4,2)$, $(5,4)$ y el lazo $(6,6)$. A la izquierda el dibujo, al centro el arreglo Adj con sus listas ligadas y a la derecha la matriz.





	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Cada arista aparece una sola vez en las listas, que suman $8 = m$. La matriz no es simétrica ($a_{12} = 1$, $a_{21} = 0$) y el lazo pone un 1 en a_{66}. Los grados de salida son $2, 1, 2, 1, 1, 1$, las longitudes de las listas o las sumas de los renglones, y los de entrada $0, 2, 0, 2, 2, 2$, las sumas de las columnas. En G^{T} las listas de 1 y 3 quedan vacías, y las demás salen $Adj^{\mathsf{T}}[2] = 1, 4$, $Adj^{\mathsf{T}}[4] = 1, 5$, $Adj^{\mathsf{T}}[5] = 2, 3$ y $Adj^{\mathsf{T}}[6] = 3, 6$, todas en orden creciente.

Una excepción con matriz: el sumidero universal

Con la matriz casi toda pregunta sobre el grafo cuesta $\Omega(n^2)$. La excepción es el **sumidero universal**, un vértice con $\text{gr}^- = n-1$ y $\text{gr}^+ = 0$: si $k \neq j$, $a_{kj} = 1$ descarta a k y $a_{kj} = 0$ descarta a j. Con $n-1$ lecturas sobrevive un candidato, y verificar su renglón y su columna cuesta otras $2n-1$: $\mathcal{O}(n)$ sobre una entrada de tamaño n^2.

Búsqueda en anchura a fondo

La clase del 22 de octubre (sección 11) dio BFS con un arreglo booleano visto y lo corrió sobre G desde A. Aquí se usa la versión de Cormen, que pinta cada vértice de uno de tres colores. El resultado es el mismo; los colores sirven para las pruebas.

Definición 14.4 (Colores de BFS)**.** Un vértice es **blanco** mientras no se ha descubierto, **gris** desde que se descubre hasta que se termina de recorrer su lista, y **negro** después. Los grises forman la **frontera** entre lo descubierto y lo que falta.

Algoritmo 14.1 BFS(G, s) con colores (Cormen, sección 22.2)

```
1:  procedimiento BFS(G, s)
2:      para todo u ∈ V − {s} hacer
3:          color[u] ← BLANCO,    d[u] ← ∞,    π[u] ← NULO
4:      fin para
5:      color[s] ← GRIS,    d[s] ← 0,    π[s] ← NULO
6:      Q ← cola vacía;  ENCOLAR(Q, s)
7:      mientras Q no está vacía hacer
8:          u ← DESENCOLAR(Q)
9:          para todo v ∈ Adj[u] hacer
10:             si color[v] = BLANCO entonces                    // se descubre v
11:                 color[v] ← GRIS,    d[v] ← d[u] + 1,    π[v] ← u
12:                 ENCOLAR(Q, v)
13:             fin si
14:         fin para
15:         color[u] ← NEGRO                                     // lista de u agotada
16:     fin mientras
17: fin procedimiento
```

La línea 10 solo pregunta si v es blanco: gris y negro se tratan igual, así que un bit por vértice basta, y ese bit es el visto de la clase. La línea 11 fija color, d y π en el instante en que v se descubre, no cuando sale de la cola; eso garantiza que cada vértice entre a Q una sola vez. La línea 15 no cambia el resultado: existe para que los grises sean exactamente los vértices de Q, la frontera.

Costo: $\mathcal{O}(n+m)$, y por qué no $\Theta(n+m)$

El análisis es agregado: se cuenta cuántas veces puede pasar cada cosa en toda la ejecución.

1. *Inicialización* (líneas 2 a 6): $\Theta(n)$.
2. *Operaciones de cola.* Un vértice entra a Q solo si era blanco, y en ese momento se pinta de gris, color del que no regresa a blanco. Cada vértice se encola y se desencola a lo más una vez, con operaciones de $\mathcal{O}(1)$: $\mathcal{O}(n)$.
3. *Recorrido de listas.* La lista de u se recorre solo cuando u sale de la cola, a lo más una vez, y por el lema 14.1 todas suman m o $2m$: $\mathcal{O}(m)$.
4. *Suma:* $\Theta(n) + \mathcal{O}(n) + \mathcal{O}(m) = \mathcal{O}(n+m)$.

Por eso la clase escribe $\mathcal{O}(n+m)$ y no $\Theta(n+m)$. Si s no tiene aristas de salida, BFS inicializa los n vértices, saca a s, mira una lista vacía y termina en $\Theta(n)$, aunque m sea del orden de n^2: las listas de

los vértices inalcanzables nunca se leen. $\Theta(n+m)$ promete también una cota inferior, y esa falla en cuanto hay vértices inalcanzables con listas largas. En DFS no pasa, como se verá: su ciclo externo obliga a leer todas las listas. Las cotas suponen listas y una cola de $\mathcal{O}(1)$ por operación; con matriz BFS cuesta $\mathcal{O}(n^2)$, y con una lista de Python y `pop(0)` sube a $\mathcal{O}(n^2+m)$.

Distancias: por qué BFS las calcula

Definición 14.5 (Distancia de camino más corto)**.** $\delta(s,v)$ es el mínimo número de aristas de un camino de s a v, e ∞ si no hay ninguno. Un **camino más corto** de s a v es uno de longitud $\delta(s,v)$.

Aquí todas las aristas pesan 1; la sección 17 generaliza a pesos reales. La caja de idea de la clase afirma que BFS deja $d[v]=\delta(s,v)$. La prueba de Cormen tiene tres pasos: d nunca subestima, la cola está ordenada por d, y un vértice con la d equivocada lleva a una contradicción.

Lema 14.6 (Cormen, lema 22.1)**.** *Sea G dirigido o no y $s \in V$. Para toda arista $(u,v) \in E$, $\delta(s,v) \le \delta(s,u)+1$.*

Idea de la prueba. Un camino más corto a u seguido de (u,v) llega a v con $\delta(s,u)+1$ aristas; si u no es alcanzable, el lado derecho es ∞. □

En uno no dirigido cada arista sirve en los dos sentidos, y queda $|\delta(s,u)-\delta(s,v)| \le 1$ para toda arista $\{u,v\}$. En un dirigido la desigualdad va en un solo sentido.

Lema 14.7 (Cormen, lema 22.2)**.** *Al terminar BFS desde s, $d[v] \ge \delta(s,v)$ para todo $v \in V$.*

Idea de la prueba. Inducción sobre las operaciones Encolar. Si v se descubre desde u, $d[v]=d[u]+1 \ge \delta(s,u)+1 \ge \delta(s,v)$ por la hipótesis y el lema 14.6, y como v queda gris su d ya no cambia. □

Lema 14.8 (Cormen, lema 22.3)**.** *Si en algún momento $Q=\langle v_1,\dots,v_r\rangle$, con v_1 a la cabeza, entonces $d[v_r] \le d[v_1]+1$ y $d[v_i] \le d[v_{i+1}]$ para $i=1,\dots,r-1$.*

Idea de la prueba. Inducción sobre las operaciones de cola. Al desencolar, la nueva cabeza tiene d mayor o igual que la anterior. Al encolar v desde u, u acaba de salir como cabeza, así que $d[v]=d[u]+1 \le d[v_1]+1$ y $d[v_r] \le d[u]+1 = d[v]$. □

La consecuencia que conviene recordar: la cola contiene a lo más dos valores distintos de d, consecutivos y en orden no decreciente. La columna "d en Q" de la traza del ejemplo 14.13 lo muestra paso a paso.

Corolario 14.9 (Cormen, corolario 22.4)**.** *Si v_i se encola antes que v_j, entonces $d[v_i] \le d[v_j]$ en el momento en que v_j se encola.*

Es inmediato del lema 14.8 y de que cada vértice recibe un valor finito de d una sola vez.

Teorema 14.10 (Corrección de BFS; Cormen, teorema 22.5)**.** *Sea G dirigido o no y supóngase que BFS corre desde s. Entonces descubre todo vértice alcanzable desde s y, al terminar, $d[v]=\delta(s,v)$ para todo $v \in V$. Además, para todo $v \ne s$ alcanzable, un camino más corto de s a $\pi[v]$ seguido de la arista $(\pi[v],v)$ es un camino más corto de s a v.*

Demostración. Por contradicción. Supóngase que algún vértice termina con $d[v] \ne \delta(s,v)$, y de todos ellos tómese uno con $\delta(s,v)$ mínima. No es s, porque $d[s]=0=\delta(s,s)$. Por el lema 14.7, $d[v] > \delta(s,v)$, así que $\delta(s,v)$ es finita: v es alcanzable. Sea u el vértice anterior a v en un camino más corto de s

a v; entonces $\delta(s,v) = \delta(s,u) + 1$, y como $\delta(s,u)$ es menor que $\delta(s,v)$, la elección de v obliga a que $d[u] = \delta(s,u)$. Juntando,

$$d[v] > \delta(s,v) = \delta(s,u) + 1 = d[u] + 1. \qquad (*)$$

Como $d[u]$ es finita, u pasó por la cola. Considérese el instante en que u sale de ella. En ese momento v es blanco, gris o negro.

- Blanco: la línea 11 fija $d[v] = d[u] + 1$.
- Negro: v ya salió de la cola antes que u, así que entró antes, y por el corolario 14.9, $d[v] \leq d[u]$.
- Gris: v se descubrió al sacar algún w que salió antes que u, así que $d[v] = d[w] + 1$ con $d[w] \leq d[u]$ por el mismo corolario, y $d[v] \leq d[u] + 1$.

Los tres casos contradicen $(*)$. Entonces $d[v] = \delta(s,v)$ para todo v. Un alcanzable no descubierto tendría $d[v] = \infty > \delta(s,v)$, así que todos se descubren. Por último, si $\pi[v] = u$, la línea 11 dio $d[v] = d[u] + 1$, es decir, $\delta(s,v) = \delta(s,u) + 1$, y un camino más corto a u más la arista (u,v) mide exactamente eso. □

El árbol BFS y la impresión de caminos

Definición 14.11 (Subgrafo predecesor de BFS)**.** El **subgrafo predecesor** de BFS desde s es $G_\pi = (V_\pi, E_\pi)$ con $V_\pi = \{v : \pi[v] \neq \text{nulo}\} \cup \{s\}$ y $E_\pi = \{(\pi[v], v) : v \in V_\pi - \{s\}\}$. Es un **árbol BFS** si V_π son exactamente los vértices alcanzables desde s y, para cada $v \in V_\pi$, el único camino de s a v en G_π es un camino más corto en G.

Lema 14.12 (Cormen, lema 22.6)**.** *Aplicado a un grafo dirigido o no, BFS construye π de modo que G_π es un árbol BFS.*

Idea de la prueba. $\pi[v]$ deja de ser nulo exactamente cuando $v \neq s$ es alcanzable, así que V_π son los alcanzables; G_π es conexo y tiene $|V_\pi| - 1$ aristas, así que es un árbol. A lo largo de su camino de s a v cada arista sube d en 1, y por el teorema 14.10 el camino mide $\delta(s,v)$. □

Para leer el camino del árbol se sube por π y se imprime al regresar de la recursión:

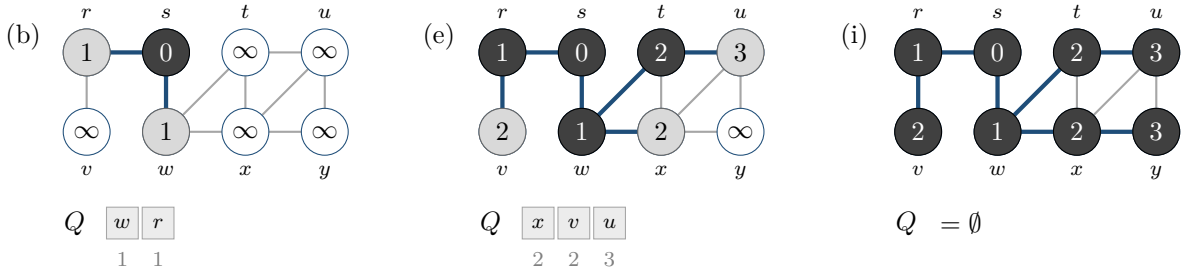
Algoritmo 14.2 Imprimir-Camino(G, s, v)

```
1:  procedimiento Imprimir-Camino(G, s, v)
2:      si v = s entonces
3:          imprimir s
4:      si no si π[v] = nulo entonces
5:          imprimir "no hay camino de s a v"
6:      si no
7:          Imprimir-Camino(G, s, π[v])
8:          imprimir v
9:      fin si
10: fin procedimiento
```

Supone que BFS ya corrió desde la misma s, y hace $\delta(s,v) + 1$ llamadas de costo $\mathcal{O}(1)$.

Observación. Las distancias no dependen del orden de las listas, porque $d[v] = \delta(s,v)$ solo depende de G. El árbol sí puede depender: en el ejemplo siguiente, si la lista de w es s, x, t en lugar de s, t, x, sale x antes que t y queda $\pi[u] = x$.

Ejemplo 14.13 (La figura 22.3 de Cormen)**.** El grafo es no dirigido, con $V = \{r, s, t, u, v, w, x, y\}$, diez aristas y fuente s. Cormen no imprime las listas; las que reproducen sus colas son r: s, v; s: w, r; t: w, x, u; u: t, x, y; v: r; w: s, t, x; x: w, t, u, y; y: x, u. La de s no es alfabética, porque la instantánea (b) tiene $Q = \langle w, r \rangle$. Van tres de las nueve instantáneas de Cormen, cada una al principio de una vuelta del ciclo: blancos con ∞, grises y negros con su d, y la cola con la d de cada vértice debajo; en azul, las aristas del árbol.



En la traza, B, G y N son el color del vecino en el momento en que se examina.

It.	Q **al inicio**	d **en** Q	**Sale**	**Examina**	**Descubre**
1	s	0	s	w B, r B	w, r con $d = 1$, $\pi = s$
2	w, r	1, 1	w	s N, t B, x B	t, x con $d = 2$, $\pi = w$
3	r, t, x	1, 2, 2	r	s N, v B	v con $d = 2$, $\pi = r$
4	t, x, v	2, 2, 2	t	w N, x G, u B	u con $d = 3$, $\pi = t$
5	x, v, u	2, 2, 3	x	w N, t N, u G, y B	y con $d = 3$, $\pi = x$
6	v, u, y	2, 3, 3	v	r N	nada
7	u, y	3, 3	u	t N, x N, y G	nada
8	y	3	y	x N, u N	nada

Quedan siete aristas de árbol, $n - 1$, y tres fuera: t–x, x–u y u–y. Las tres unen vértices del mismo nivel o de niveles contiguos, como exige $|\delta(s, u) - \delta(s, v)| \leq 1$. Imprimir-Camino$(G, s, y)$ imprime s, w, x, y. En las iteraciones 4, 5 y 7 aparece un vecino gris: son los casos que la prueba del teorema 14.10 trata aparte.

Ojito…

- Marcar al descubrir, no al sacar. Si el vértice se marca cuando sale de la cola, puede entrar varias veces, una por cada vecino que lo encuentre antes de que salga; la cola crece hasta $\Theta(m)$, se repite trabajo y, si d y π se sobrescriben al reencolar, se pierden los valores correctos.
- En un dirigido, alcanzable es en la dirección de las flechas: un vértice con una arista *hacia* s y ninguna que llegue a él termina blanco, con $d = \infty$. La propiedad $|d[u] - d[v]| \leq 1$ en cada arista es exclusiva del caso no dirigido; en un dirigido una arista puede regresar cualquier número de niveles.
- Un BFS solo visita lo alcanzable desde s. Para recorrer todo el grafo hace falta un ciclo externo sobre los vértices, como el del algoritmo bipartito o el de DFS.

Ciclos impares y grafos bipartitos

En un grafo no dirigido, el lema 14.6 dice que toda arista une dos niveles iguales o contiguos del árbol BFS. Si una arista fuera del árbol une u y v con $d[u] = d[v] = k$ y x es su ancestro común más bajo, los dos caminos del árbol de x a u y a v más la arista cierran un ciclo de $2(k - d[x]) + 1$ aristas: impar. En el grafo G de la clase, la arista E–F une dos vértices con $d = 2$, y el ciclo A, B, E, F, C, A tiene

cinco aristas: G no es bipartito. Es la prueba del algoritmo 13.4 de la sección 13, que colorea por la paridad de d.

Diámetro de un árbol con dos BFS

El **diámetro** de un árbol T es la mayor de todas las distancias, $\max_{u,v} \delta(u, v)$. Calcularlo par por par costaría n BFS. En un árbol bastan dos, gracias a esta propiedad:

Proposición 14.14. *En un árbol, desde cualquier s, el vértice a más lejano es extremo de algún diámetro.*

Idea de la prueba. Sea p, q un par que realiza el diámetro y z el vértice donde se separan los caminos de s a a y de s a q. Como a es el más lejano desde s, $\delta(z, a)$ es al menos la distancia de z a uno de los dos extremos, p o q, según dónde caiga z. Cambiar ese extremo por a da un par a distancia al menos $\delta(p, q)$. Todo descansa en que en un árbol el camino entre dos vértices es único. □

Entonces un BFS desde cualquier s da un extremo a de un diámetro, y un segundo BFS desde a encuentra el otro extremo:

Algoritmo 14.3 Diámetro(T) de un árbol con dos BFS

```
1: procedimiento Diámetro(T)
2:     s ← cualquier vértice de T
3:     BFS(T, s);  a ← un vértice con d[a] máxima
4:     BFS(T, a);  b ← un vértice con d[b] máxima
5:     devolver d[b]                    // el camino sale de Imprimir-Camino(T, a, b)
6: fin procedimiento
```

Son dos BFS de $\mathcal{O}(n)$, porque $m = n - 1$. El truco es exclusivo de árboles: en K_4 con vértices 1 a 4 sin la arista $\{3, 4\}$, desde 1 todos están a distancia 1; si se toma 2 como el más lejano, el segundo BFS vuelve a dar 1, pero $\delta(3, 4) = 2$.

Búsqueda en profundidad a fondo

La clase dio DFS-Visitar con visto, un reloj y los tiempos entra y sale. Cormen agrega dos piezas: un ciclo externo que vuelve a arrancar desde cada vértice que siga sin descubrir, de modo que el recorrido cubre todo el grafo y produce un bosque, y los tres colores: blanco antes de entra[u], gris entre entra[u] y sale[u], y negro después. El reloj t de la clase se llama aquí *tiempo*, porque uno de los ejemplos tiene un vértice t.

Definición 14.15 (Bosque DFS)**.** El subgrafo predecesor de DFS es $G_\pi = (V, E_\pi)$ con $E_\pi = \{(\pi[v], v) : v \in V,\ \pi[v] \neq \text{nulo}\}$. Es un **bosque DFS**, formado por **árboles DFS**, uno por cada raíz que elige el ciclo externo; sus aristas son las **aristas de árbol**.

A diferencia del de BFS, G_π contiene a todo V. Reproduce el árbol de llamadas recursivas: $\pi[v] = u$ si y solo si DFS-Visitar(G, v) se llamó desde el ciclo de u. En cada instante, los vértices grises son exactamente las llamadas activas, del más viejo al más reciente, y forman una cadena de descendientes en el bosque. De ahí sale la llave de casi todo lo que sigue: una arista que llega a un vértice gris llega a un antecesor.

Algoritmo 14.4 DFS(G) con colores y tiempos (Cormen, sección 22.3)

```
1:  procedimiento DFS(G)
2:      para todo u ∈ V hacer
3:          color[u] ← BLANCO,    π[u] ← NULO
4:      fin para
5:      tiempo ← 0
6:      para todo u ∈ V hacer
7:          si color[u] = BLANCO entonces
8:              DFS-VISITAR(G, u)                    // u es raíz de un árbol nuevo
9:          fin si
10:     fin para
11: fin procedimiento
```

Algoritmo 14.5 DFS-Visitar(G, u), la visita recursiva de DFS

```
1:  procedimiento DFS-VISITAR(G, u)
2:      tiempo ← tiempo + 1,    entra[u] ← tiempo
3:      color[u] ← GRIS
4:      para todo v ∈ Adj[u] hacer                   // explorar la arista (u, v)
5:          si color[v] = BLANCO entonces
6:              π[v] ← u
7:              DFS-VISITAR(G, v)
8:          fin si
9:      fin para
10:     color[u] ← NEGRO                             // u terminó
11:     tiempo ← tiempo + 1,    sale[u] ← tiempo
12: fin procedimiento
```

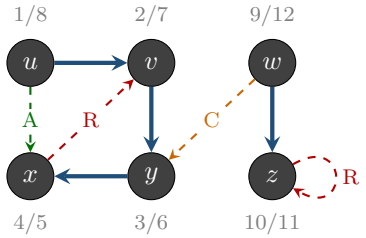
El reloj sube antes de asignar, así que el primer descubrimiento es el 1. Hay exactamente un descubrimiento y una finalización por vértice, de modo que los tiempos son los enteros de 1 a $2n$, todos distintos, y siempre entra$[u]$ < sale$[u]$. Un vértice gris o negro no se vuelve a visitar, pero la arista hacia él sí cuenta como explorada: eso es lo que permite clasificarla más adelante.

Costo: aquí sí es $\Theta(n+m)$. DFS-Visitar se llama exactamente una vez por vértice: a lo más una, porque solo se llama sobre un vértice blanco y lo primero que hace es pintarlo de gris; al menos una, porque el ciclo externo llama sobre todo vértice que siga blanco. Fuera de su ciclo cada llamada cuesta $\mathcal{O}(1)$, y el ciclo sobre u da $|Adj[u]|$ vueltas; por el lema 14.1 el total es $\Theta(n+m)$ en toda entrada, porque cada vértice y cada entrada de cada lista se tocan. Con matriz sube a $\Theta(n^2)$. La pila de recursión llega a profundidad n cuando el árbol es un camino, y en grafos grandes eso desborda la pila del sistema: es la razón práctica de la versión iterativa del final.

Ejemplo 14.16 (La figura 22.4 de Cormen, paso a paso)**.** $V = \{u, v, w, x, y, z\}$ y las aristas (u,v), (u,x), (v,y), (w,y), (w,z), (x,v), (y,x) y el lazo (z,z), con listas u: v, x; v: y; w: y, z; x: v; y: x; z: z. El ciclo externo toma los vértices en orden alfabético. La columna de grises va del fondo al tope y es literalmente la pila de llamadas.

tiempo	**Evento**	**Grises**	**Arista explorada**	**Color de v, tipo**
1	descubre u (raíz)	u	(u, v)	blanco: árbol
2	descubre v	u, v	(v, y)	blanco: árbol
3	descubre y	u, v, y	(y, x)	blanco: árbol
4	descubre x	u, v, y, x	(x, v)	gris: retroceso
5	termina x	u, v, y		
6	termina y	u, v		
7	termina v	u	(u, x)	negro, $1 < 4$: avance
8	termina u	vacía		
9	descubre w (raíz)	w	(w, y)	negro, $9 > 3$: cruce
			(w, z)	blanco: árbol
10	descubre z	w, z	(z, z)	gris: retroceso
11	termina z	w		
12	termina w	vacía		

El estado final, con entra/sale junto a cada vértice; las aristas punteadas llevan su tipo: R de retroceso, A de avance, C de cruce.



final, *tiempo* = 12

Quedan dos árboles, $\{u, v, y, x\}$ y $\{w, z\}$. El vértice y queda en el primero aunque w también le apunte: cuando w explora (w, y), y ya es negro. El ejemplo de la clase sobre G es una corrida del mismo algoritmo con una sola raíz, porque desde A se alcanza todo.

El teorema del paréntesis

Si se escribe "$(u$" en el instante entra$[u]$ y "$u)$" en sale$[u]$, la sucesión de eventos de una DFS es una expresión de paréntesis bien balanceada. En el ejemplo 14.16 es $(u\ (v\ (y\ (x\ x)\ y)\ v)\ u)\ (w\ (z\ z)\ w)$, y en el de la clase sobre G, $(A\ (B\ (D\ (G\ G)\ D)\ (E\ (F\ (C\ C)\ F)\ E)\ B)\ A)$. La razón es que la recursión es una pila: una llamada que empieza dentro de otra termina antes que ella.

Teorema 14.17 (Del paréntesis; Cormen, teorema 22.7)**.** *En cualquier DFS de un grafo, dirigido o no, para cualesquiera dos vértices u y v se cumple exactamente una de estas tres condiciones:*

1. *los intervalos* $[\text{entra}[u], \text{sale}[u]]$ *y* $[\text{entra}[v], \text{sale}[v]]$ *son disjuntos, y ninguno de los dos vértices desciende del otro en el bosque;*
2. *el intervalo de u está contenido en el de v, y u desciende de v;*
3. *el intervalo de v está contenido en el de u, y v desciende de u.*

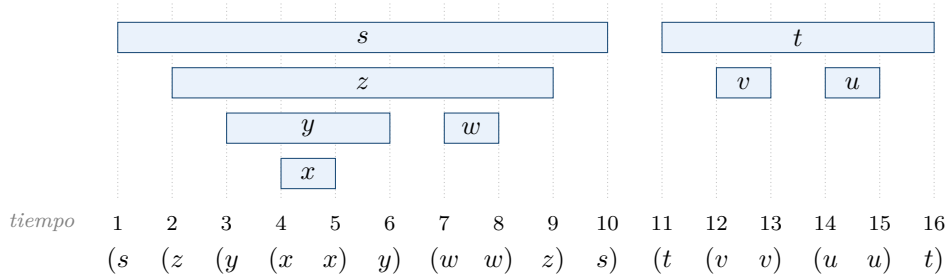
Demostración. Supóngase entra$[u] <$ entra$[v]$; el otro caso es simétrico. Si entra$[v] <$ sale$[u]$, v se descubrió mientras u era gris, es decir, durante la llamada sobre u, así que desciende de u. Como v es más reciente, su llamada termina antes de que la búsqueda regrese a u: sale$[v] <$ sale$[u]$, y el intervalo de v queda dentro del de u. Si en cambio sale$[u] <$ entra$[v]$, entonces entra$[u] <$ sale$[u] <$ entra$[v] <$ sale$[v]$:

los intervalos son disjuntos, y ninguno se descubrió con el otro gris, así que ninguno desciende del otro. □

Lo que el teorema prohíbe es el traslape parcial: dos intervalos se anidan o se separan, como los paréntesis.

Corolario 14.18 (Anidamiento; Cormen, corolario 22.8)**.** *v es descendiente propio de u si y solo si entra$[u] <$ entra$[v] <$ sale$[v] <$ sale$[u]$.*

Ejemplo 14.19 (La figura 22.5 de Cormen: los paréntesis como intervalos)**.** $V = \{s, t, u, v, w, x, y, z\}$ y trece aristas: (s, z), (s, w), (t, v), (t, u), (u, t), (u, v), (v, s), (v, w), (w, x), (x, z), (y, x), (z, y), (z, w). Cormen no da las listas; estas reproducen sus tiempos: s: z, w; t: v, u; u: t, v; v: s, w; w: x; x: z; y: x; z: y, w, con s primero y t segundo en el ciclo externo. Cada vértice es su intervalo, un renglón por nivel, con los paréntesis abajo.



Un rectángulo debajo de otro y dentro de su sombra es un descendiente: x dentro de y, dentro de z, dentro de s. Dos rectángulos sin traslape no tienen relación: y y w son hermanos, y s y t están en árboles distintos. Nunca hay traslape parcial. Los árboles son s, z, y, x, w, con $\pi[z] = s$, $\pi[y] = \pi[w] = z$ y $\pi[x] = y$, y t, v, u, con $\pi[v] = \pi[u] = t$.

El teorema del camino blanco

El paréntesis dice cómo reconocer a un descendiente a partir de los tiempos. El siguiente teorema dice cómo saber, en el instante en que se descubre u, quiénes van a terminar colgando de él.

Teorema 14.20 (Del camino blanco; Cormen, teorema 22.9)**.** *En un bosque DFS de un grafo dirigido o no, v es descendiente de u si y solo si, en el instante entra$[u]$, existe un camino de u a v formado solo por vértices blancos.*

Demostración. ($\Rightarrow$) Si $v = u$, el camino es solo u, que es blanco justo antes de que se fije entra$[u]$. Si v es descendiente propio, todos los vértices del camino del árbol de u a v descienden de u, así que por el corolario 14.18 se descubren después de entra$[u]$: en ese instante eran blancos.

($\Leftarrow$) Supóngase que en entra$[u]$ hay un camino blanco de u a v, pero v no desciende de u. Sea x el primer vértice del camino que no desciende de u, y w el anterior a él en el camino; w sí desciende de u (puede ser $w = u$), así que sale$[w] \leq$ sale$[u]$. Como x era blanco en entra$[u]$, se descubre después; y como w explora la arista (w, x) antes de terminar, x se descubre a más tardar en ese momento. Entonces

$$\text{entra}[u] < \text{entra}[x] < \text{sale}[w] \leq \text{sale}[u].$$

El intervalo de x empieza dentro del de u; por el teorema 14.17 no puede traslaparse a medias, así que queda contenido, y por el corolario 14.18 x desciende de u. Contradicción. □

La lectura útil: ser descendiente no depende de lo que pase después, sino de qué estaba blanco al descubrir u. Un camino de u a v que pase por un vértice ya gris o negro no obliga a nada.

Clasificación de aristas

Definición 14.21 (Tipos de arista)**.** Respecto del bosque DFS, una arista (u, v) es

1. **de árbol** si está en E_π: v se descubrió al explorarla;
2. **de retroceso** si une a u con un antecesor v en su árbol; los lazos cuentan como de retroceso;
3. **de avance** si no es de árbol y une a u con un descendiente v;
4. **de cruce** en cualquier otro caso: entre vértices del mismo árbol sin relación de ancestría, o entre árboles distintos.

Proposición 14.22 (Clasificación por el color)**.** *Cuando DFS explora (u, v) por primera vez, el tipo lo decide el color de v: blanco, árbol; gris, retroceso; negro, avance si entra$[u] <$ entra$[v]$ y cruce si entra$[u] >$ entra$[v]$.*

Idea de la prueba. Blanco: v se descubre justo ahora, desde u. Gris: los grises son la cadena de llamadas activas y u es el más reciente, así que v es antecesor de u (o u mismo, en un lazo). Negro: si entra$[u] <$ entra$[v]$, v se descubrió y terminó dentro de la llamada de u; si no, el teorema del paréntesis deja los intervalos disjuntos. □

Para separar avance de cruce se compara entra, no sale: cuando se explora la arista, sale$[u]$ todavía no existe. Para clasificar al vuelo basta cambiar el **si** del ciclo de DFS-Visitar por estos cuatro casos; el costo sigue siendo $\Theta(n + m)$, porque cada arista se clasifica en $\mathcal{O}(1)$.

Con los intervalos ya calculados también se clasifica sin repetir la búsqueda (Cormen, ejercicio 22.3-5).

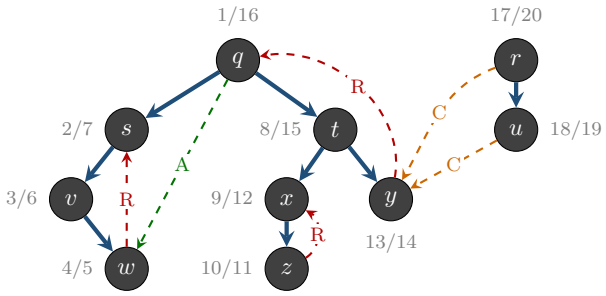
Proposición 14.23 (Clasificación por intervalos)**.** *La arista (u, v) es de árbol o de avance si y solo si entra$[u] <$ entra$[v] <$ sale$[v] <$ sale$[u]$; de retroceso si y solo si entra$[v] \leq$ entra$[u] <$ sale$[u] \leq$ sale$[v]$, con igualdad solo en un lazo; y de cruce si y solo si entra$[v] <$ sale$[v] <$ entra$[u] <$ sale$[u]$. Árbol y avance se separan con π: es de árbol si y solo si $\pi[v] = u$.*

Falta a propósito el caso entra$[u] <$ sale$[u] <$ entra$[v]$: al explorar (u, v) el vértice v habría estado blanco y se habría vuelto descendiente de u. La consecuencia que más se usa es esta: **toda arista que no es de retroceso cumple sale**$[v] <$ **sale**$[u]$. Es la desigualdad que hace funcionar el orden topológico de la sección 15.

Ejemplo 14.24 (Los cuatro tipos; Cormen, figura 22.6 y ejercicio 22.3-2)**.** $V = \{q, r, s, t, u, v, w, x, y, z\}$ y catorce aristas: (q, s), (q, t), (q, w), (r, u), (r, y), (s, v), (t, x), (t, y), (u, y), (v, w), (w, s), (x, z), (y, q), (z, x). Como pide el ejercicio, el ciclo externo y todas las listas van en orden alfabético.

Arista	*tiempo*	**Color de** v	**Tipo**	**Arista**	*tiempo*	**Color de** v	**Tipo**
(q, s)	1	blanco	árbol	(t, y)	12	blanco	árbol
(s, v)	2	blanco	árbol	(y, q)	13	gris	retroceso
(v, w)	3	blanco	árbol	(q, w)	15	negro	avance, $1 < 4$
(w, s)	4	gris	retroceso	(r, u)	17	blanco	árbol
(q, t)	7	blanco	árbol	(u, y)	18	negro	cruce, $18 > 13$
(t, x)	8	blanco	árbol	(r, y)	19	negro	cruce, $17 > 13$
(x, z)	9	blanco	árbol				
(z, x)	10	gris	retroceso				

Tiempos finales: q 1/16, s 2/7, v 3/6, w 4/5, t 8/15, x 9/12, z 10/11, y 13/14, r 17/20, u 18/19. Ocho de árbol, tres de retroceso, una de avance y dos de cruce. La figura redibuja el grafo como bosque: las de árbol y la de avance bajan, las de retroceso suben y las de cruce van de derecha a izquierda, hacia subárboles que ya terminaron.



el bosque DFS

Las tres de retroceso cierran tres ciclos: $s \to v \to w \to s$, $x \to z \to x$ y $q \to t \to y \to q$. Para saber que el grafo no es acíclico basta una; es el criterio que la sección 15 demuestra con el teorema del camino blanco.

Grafos no dirigidos: solo árbol y retroceso

En un no dirigido, $\{u, v\}$ aparece en las dos listas y se explora dos veces. Cormen la clasifica con el primero de los cuatro tipos que le aplique, lo que equivale a clasificarla por la dirección en que se explora primero (ejercicio 22.3-6). Con esa regla, dos de los tipos desaparecen.

Teorema 14.25 (Cormen, teorema 22.10)**.** *En una DFS de un grafo no dirigido, toda arista es de árbol o de retroceso.*

Idea de la prueba. Si entra$[u] <$ entra$[v]$, el camino blanco pone a v debajo de u. Si la arista se explora primero desde u, v sigue blanco y es de árbol; si se explora primero desde v, u sigue gris y es de retroceso. □

En el ejemplo de la clase, $\{A, C\}$ se explora primero desde C, en el tiempo 9, con A gris: es de retroceso. Para implementarlo, al explorar $\{u, v\}$ desde u: si v es blanco, árbol; si es gris y $v \neq \pi[u]$, retroceso; si $v = \pi[u]$, es la arista de árbol vista desde abajo; si v es negro, la arista ya se exploró desde v. En los dos últimos casos no se hace nada. Con aristas paralelas la condición $v \neq \pi[u]$ falla, porque descarta la segunda copia como si fuera la de árbol: hay que guardar la arista por la que se llegó a cada vértice y descartar solo esa.

Ojito…

- El resultado depende del orden. En la figura 22.4, si el ciclo externo empieza por w, sale w 1/10, y 2/7, x 3/6, v 4/5, z 8/9, u 11/12, y la de retroceso es (v, y) en lugar de (x, v). Lo que no depende del orden es *si hay* alguna de retroceso, porque eso equivale a tener un ciclo.
- Un bit basta para recorrer, no para clasificar. Si se quita la línea que pinta de negro, entra, sale y π salen idénticos (Cormen, ejercicio 22.3-4), así que el visto de la clase alcanza para el recorrido. Pero sin distinguir gris de negro una de retroceso y una de cruce se ven igual, y la detección de ciclos dirigidos da falsos positivos, como la regla con BFS que la clase advierte.

DFS iterativo con pila

La clase menciona que la versión iterativa usa una pila explícita, pero no la escribe. Esta es la versión corta, que sirve cuando solo interesan el orden de visita y π; sus dos decisiones de diseño son justo las dos formas naturales de escribirla mal.

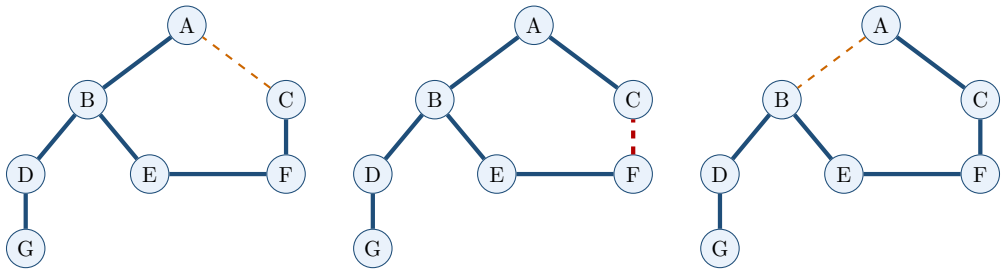
Algoritmo 14.6 DFS-Pila(G, s): orden de visita y padres

```
1:  procedimiento DFS-Pila(G, s)
2:      para todo v ∈ V hacer
3:          visto[v] ← falso
4:      fin para
5:      P ← pila vacía;  Apilar(P, (s, nulo))
6:      mientras P no está vacía hacer
7:          (u, p) ← Desapilar(P)
8:          si no visto[u] entonces                    // si no, es copia vieja
9:              visto[u] ← verdadero,    π[u] ← p
10:             para todo v ∈ Adj[u], del último al primero hacer
11:                 si no visto[v] entonces
12:                     Apilar(P, (v, u))
13:                 fin si
14:             fin para
15:         fin si
16:     fin mientras
17: fin procedimiento
```

Trampa 1: marcar al apilar. Aquí se marca al *sacar* (línea 9), y las copias viejas se descartan en la línea 8. Marcar al apilar, como en BFS, ya no es DFS: el vecino queda colgando del primero que lo ve. En G el orden de visita sale igual, pero C queda con $\pi[C] = A$ en lugar de F, y F–C uniría dos ramas distintas, cosa que el teorema 14.25 prohíbe.

Trampa 2: apilar en el orden de la lista. La pila invierte el orden, así que para visitar los vecinos en el orden de la lista hay que apilarlos del último al primero (línea 10). Apilados en el orden de la lista sale una DFS válida, pero la de las listas al revés: en G, A, C, F, E, B, D, G.

Ejemplo 14.26 (DFS-Pila sobre G desde A)**.** Con las listas en orden alfabético la visita es A, B, D, G, E, F, C, con los mismos padres que la recursión. La copia (C, F) sale antes que la (C, A) que A apiló al principio, y esta se descarta como copia vieja. Las tres versiones, lado a lado:



(a) recursión o DFS-Pila: C–A de retroceso (b) marcando al apilar: F–C sería de cruce (c) apilando en orden: otra DFS

Esta versión no da tiempos de salida, y su pila puede guardar copias: a lo más $m + 1$ apilados en un dirigido y $2m + 1$ en uno no dirigido; el tiempo es $\mathcal{O}(n + m)$, con $\mathcal{O}(m)$ de memoria. Para tener

tiempos de salida hay que simular la recursión de verdad (Cormen, ejercicio 22.3-7). La pila guarda solo vértices grises, y cada uno lleva un apuntador sig[u] al siguiente vecino por revisar. En cada vuelta se mira el tope u sin sacarlo: si sig[u] no es nulo, se toma ese vecino v, se avanza el apuntador y, si v es blanco, recibe $\pi[v] = u$ y entra[v], se pinta de gris y se apila; si es nulo, u se desapila, se pinta de negro y recibe sale[u]. La pila es exactamente la de llamadas, así que da los mismos tiempos y padres que la recursión. Sin sig[u], cada regreso de u al tope recorrería su lista desde el principio y el costo podría subir a $\mathcal{O}(nm)$; con él es $\Theta(n+m)$, con $\mathcal{O}(n)$ de pila.

Componentes conexas con DFS

En un grafo no dirigido, cada árbol del bosque DFS es exactamente una componente conexa (Cormen, ejercicio 22.3-12). Basta numerar los árboles.

Algoritmo 14.7 Cambio a DFS para numerar las componentes

```
1: k ← 0                                              // antes del ciclo externo
2: para todo u ∈ V hacer
3:     si color[u] = BLANCO entonces
4:         k ← k + 1                                  // árbol nuevo, componente nueva
5:         DFS-Visitar(G, u)
6:     fin si
7: fin para
```

En DFS-Visitar(G, u), junto con pintar a u de gris: cc[u] $\leftarrow k$.

Por qué funciona. Por inducción sobre los árboles: si los anteriores son componentes completas, la nueva raíz u no está en ninguna, y en el instante entra[u] toda su componente es blanca. Por el teorema del camino blanco, todo vértice de esa componente termina debajo de u, y nada más, porque no hay aristas hacia otras componentes. El costo es $\Theta(n+m)$. En un dirigido los árboles DFS no son componentes de nada: las componentes fuertemente conexas necesitan el algoritmo de dos DFS de la sección 15.

15. Aplicaciones de DFS

Complemento: Cormen, cap. 22.4, 22.5 y problemas

Resumen rápido. Los tiempos entra[v] y sale[v] del DFS de la sección 14 no son contabilidad del recorrido: son la estructura del grafo. Con ellos se resuelven, todos en $\Theta(n+m)$, el orden topológico de un DAG (por tiempos de salida decrecientes, o con el algoritmo de Kahn), las componentes fuertemente conexas (dos DFS, el segundo sobre el transpuesto) y los puntos de articulación, puentes y componentes biconexas de un grafo no dirigido (un DFS que calcula low[v]). Se cierra con el recorrido de Euler dirigido y la etiqueta mínima alcanzable desde cada vértice. Cada algoritmo trae su prueba o la idea de ella y su costo; el orden topológico, las componentes fuertes y las articulaciones traen además un ejemplo trabajado.

Lo que se usa del DFS

Todo corre sobre el DFS con colores y tiempos de la sección 14 (algoritmo 14.4), con un reloj t que sube en uno al descubrir y al terminar cada vértice, y sobre cuatro hechos de esa sección: el teorema del paréntesis, la caracterización de descendientes por intervalos anidados, el teorema del camino blanco y la clasificación de aristas por el color del destino. Para un conjunto de vértices U se escribe $\text{entra}(U) = \min_{u\in U} \text{entra}[u]$ y $\text{sale}(U) = \max_{u\in U} \text{sale}[u]$ (Cormen: $d(U)$ y $f(U)$). Con $u \rightsquigarrow v$ se indica que hay un camino de u a v, y $u \rightsquigarrow u$ siempre.

Orden topológico

La clase del 22 de octubre afirma que un DAG siempre tiene un orden topológico, pero no lo prueba. Aquí se prueba de dos maneras, y cada prueba trae su algoritmo lineal.

Definición 15.1 (Orden topológico)**.** Un **orden topológico** (*topological sort*) de un grafo dirigido $G = (V, E)$ es un orden lineal de todos sus vértices tal que, si $(u, v) \in E$, entonces u aparece antes que v.

Dibujados en fila según ese orden, todas las flechas apuntan a la derecha. El orden no es único, y si el grafo tiene un ciclo no existe: cada vértice del ciclo tendría que ir antes que su sucesor y, dando la vuelta, antes que sí mismo.

Ciclos y aristas de retroceso

Lema 15.2 (Cormen, lema 22.11)**.** *Un grafo dirigido G es acíclico si y solo si un DFS de G no produce aristas de retroceso.*

Demostración. Si hay una arista de retroceso (u, v), entonces v es antecesor de u en el bosque DFS, o $v = u$ si es un lazo. El camino de árbol de v a u seguido de la arista (u, v) es un ciclo.

Al revés, supóngase que G tiene un ciclo c. Sea v el primer vértice de c que se descubre y sea (u, v) la arista de c que llega a v. En el instante entra[v] los demás vértices de c siguen blancos, así que el tramo de c que va de v a u es un camino blanco. Por el teorema del camino blanco, u queda como descendiente de v. Cuando el DFS explora (u, v), desde dentro de la llamada de u, la llamada de v sigue abierta: v es gris y la arista es de retroceso. □

El lema vale para cualquier DFS, con cualquier orden de los vértices. La arista de retroceso, junto con los padres, entrega el ciclo mismo: se sube por π desde u hasta llegar a v.

El algoritmo por tiempos de salida

La idea cabe en una línea: correr DFS y listar los vértices en orden decreciente de sale. El teorema siguiente es la razón.

Teorema 15.3 (Cormen, teorema 22.12)**.** *Si G es un DAG, en cualquier DFS de G se cumple $sale[v] < sale[u]$ para toda arista (u, v). En consecuencia, los vértices en orden decreciente de sale forman un orden topológico.*

Demostración. Considérese el momento en que el DFS explora (u, v). En ese momento u es gris. Hay tres casos según el color de v.

- v gris: v sería antecesor de u y la arista sería de retroceso, lo que el lema 15.2 descarta en un DAG.
- v blanco: v se vuelve descendiente de u y, por la caracterización de descendientes, $\text{sale}[v] < \text{sale}[u]$.
- v negro: v ya terminó y $\text{sale}[v]$ ya tiene valor, mientras que u todavía no termina. El reloj solo avanza, así que $\text{sale}[v] < \text{sale}[u]$.

En los dos casos posibles u termina después que v, y queda antes que v en el orden decreciente. □

No hace falta ordenar: cada vértice que termina se inserta al frente de una lista. El algoritmo 15.1 es el DFS del algoritmo 14.4 con esa inserción y con la prueba del vértice gris, que Cormen no pone y que lo convierte en detector de ciclos.

Algoritmo 15.1 Orden-Topológico(G), con DFS y detección de ciclos

```
1:  procedimiento Orden-Topológico(G)
2:      para todo u ∈ V hacer
3:          color[u] ← blanco,    π[u] ← nulo
4:      fin para
5:      t ← 0;  L ← lista vacía
6:      para todo u ∈ V hacer
7:          si color[u] = blanco entonces
8:              Visitar-Orden(G, u)
9:          fin si
10:     fin para
11:     devolver L
12: fin procedimiento

13: procedimiento Visitar-Orden(G, u)
14:     t ← t + 1,    entra[u] ← t,    color[u] ← gris
15:     para todo v en la lista de adyacencia de u hacer
16:         si color[v] = blanco entonces
17:             π[v] ← u;  Visitar-Orden(G, v)
18:         si no si color[v] = gris entonces                  // arista de retroceso
19:             error: hay un ciclo, no existe orden topológico
20:         fin si
21:     fin para
22:     color[u] ← negro,    t ← t + 1,    sale[u] ← t
23:     insertar u al frente de L
24: fin procedimiento
```

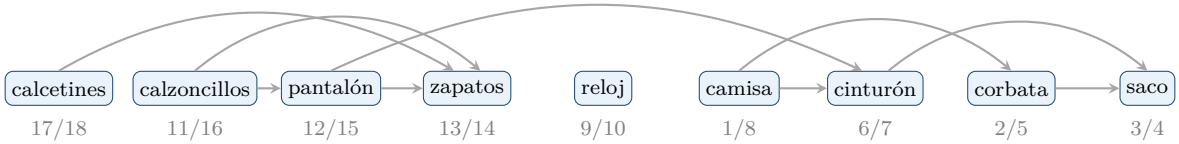
Las aristas hacia vértices negros, de avance o de cruce, se ignoran: esos vértices ya terminaron y quedarán a la derecha de u. **Costo.** Es un DFS más una inserción de $\mathcal{O}(1)$ por vértice: $\Theta(n+m)$ con listas de adyacencia y $\Theta(n^2)$ con matriz.

Ejemplo 15.4 (La ropa del profesor, con DFS)**.** Es la figura 22.7 de Cormen. Nueve prendas, donde la arista (u, v) significa que u se pone antes que v: calzoncillos → pantalón, calzoncillos → zapatos, pantalón → zapatos, pantalón → cinturón, camisa → cinturón, camisa → corbata, corbata → saco, cinturón → saco y calcetines → zapatos. El reloj está aislado. $n = 9$ y $m = 9$.

Cormen no dice en qué orden recorre los vértices; este orden reproduce exactamente sus tiempos. Ciclo exterior: camisa, corbata, saco, cinturón, reloj, calzoncillos, pantalón, zapatos, calcetines. En las listas, camisa tiene a corbata antes que a cinturón, y pantalón tiene a zapatos antes que a cinturón.

- camisa (1) → corbata (2) → saco (3); saco termina (4) y corbata (5). De regreso en camisa se descubre cinturón (6), cuyo único vecino, saco, ya es negro: cinturón termina (7) y camisa (8).
- reloj se descubre y termina sin vecinos: 9 y 10.
- calzoncillos (11) → pantalón (12) → zapatos (13), que termina (14). Cinturón ya es negro, así que pantalón termina (15). Zapatos ya es negro; calzoncillos termina (16).
- calcetines: 17 y 18; su vecino zapatos ya es negro.

Ninguna arista llega a un vértice gris: es un DAG. La figura pone las prendas en orden decreciente de sale, con entra/sale debajo de cada una.



Todas las flechas van a la derecha. El teorema 15.3 se comprueba arista por arista; por ejemplo, en cinturón → saco se tiene $7 > 4$, aunque cuando se exploró esa arista saco ya era negro.

Ojito...

Sin la prueba de gris (líneas 18 y 19) el algoritmo no avisa que hay ciclo. Si a la ropa se le agrega la arista saco → camisa, el DFS la explora cuando camisa sigue gris: es de retroceso. La versión de Cormen, que no tiene esa prueba, devuelve exactamente el mismo orden de antes, que pone a camisa antes que saco, en contra de la arista nueva.

El algoritmo de Kahn

La otra forma de construir el orden va de adelante hacia atrás. Un vértice sin aristas de entrada no depende de nada y puede ir primero; se saca, se borran sus aristas y se repite con lo que queda (Cormen, ejercicio 22.4-5). En lugar de borrar aristas se lleva un contador faltan[v], el número de predecesores de v que faltan por salir, que empieza en $\text{gr}^-(v)$ y solo baja.

Algoritmo 15.2 Kahn(G), orden topológico por grados de entrada

```
1:  procedimiento Kahn(G)
2:      faltan[v] ← 0 para todo v ∈ V
3:      para todo u ∈ V hacer
4:          para todo v en la lista de adyacencia de u hacer
5:              faltan[v] ← faltan[v] + 1
6:          fin para
7:      fin para
8:      Q ← cola con todos los v tales que faltan[v] = 0;  L ← lista vacía
9:      mientras Q no está vacía hacer
10:         u ← Desencolar(Q);  agregar u al final de L
11:         para todo v en la lista de adyacencia de u hacer
12:             faltan[v] ← faltan[v] − 1                    // se borra la arista (u, v)
13:             si faltan[v] = 0 entonces
14:                 Encolar(Q, v)
15:             fin si
16:         fin para
17:     fin mientras
18:     si |L| < n entonces
19:         error: los vértices que no están en L están en un ciclo o debajo de uno
20:     fin si
21:     devolver L
22: fin procedimiento
```

Proposición 15.5 (Correctitud de Kahn)**.** *Si G es un DAG,* Kahn *devuelve un orden topológico de sus n vértices. Si G tiene un ciclo, devuelve menos de n vértices, y ninguno de los vértices del ciclo está entre ellos.*

Idea de la prueba. faltan$[v]$ cuenta los predecesores de v fuera de L: v entra a la cola justo cuando todos salieron. Si quedan vértices fuera de L, cada uno tiene un predecesor fuera de L, y retrocediendo se cierra un ciclo. □

Todo DAG no vacío tiene entonces una **fuente**, un vértice con grado de entrada cero, y un **sumidero**, con grado de salida cero. **Costo:** grados, cola y listas se recorren una vez cada uno, $\Theta(n+m)$.

Ejemplo 15.6 (La ropa del profesor, con Kahn)**.** Con cola FIFO y empates en orden alfabético. Grados de entrada iniciales: zapatos 3; cinturón y saco 2; pantalón y corbata 1; los otros cuatro, 0.

Paso	Sale	Contadores que bajan	Cola después
0			calcetines, calzoncillos, camisa, reloj
1	calcetines	zapatos $3 \to 2$	calzoncillos, camisa, reloj
2	calzoncillos	pantalón $1 \to \mathbf{0}$, zapatos $2 \to 1$	camisa, reloj, pantalón
3	camisa	cinturón $2 \to 1$, corbata $1 \to \mathbf{0}$	reloj, pantalón, corbata
4	reloj		pantalón, corbata
5	pantalón	cinturón $1 \to \mathbf{0}$, zapatos $1 \to \mathbf{0}$	corbata, cinturón, zapatos
6	corbata	saco $2 \to 1$	cinturón, zapatos
7	cinturón	saco $1 \to \mathbf{0}$	zapatos, saco
8	zapatos		saco
9	saco		

El orden de Kahn es calcetines, calzoncillos, camisa, reloj, pantalón, corbata, cinturón, zapatos, saco, distinto del de DFS y también válido. Con la arista saco → camisa agregada, Kahn saca cinco prendas y se detiene: camisa, corbata, cinturón y saco nunca llegan a cero.

Ojito...

Kahn deja fuera los vértices que están en un ciclo *o debajo de uno*: con $a \to b$, $b \to a$ y $b \to c$ la cola nace vacía y c se queda fuera sin estar en ningún ciclo. Si se quiere el ciclo mismo, Kahn no lo da directo; DFS sí, con la arista de retroceso y los padres.

Componentes fuertemente conexas

En un grafo dirigido, que de u se llegue a v no dice nada sobre si de v se llega a u. Al contraer cada componente fuertemente conexa a un punto queda un DAG, el esqueleto del grafo.

Definición 15.7 (Componente fuertemente conexa)**.** Una **componente fuertemente conexa** (*strongly connected component*) de un grafo dirigido $G = (V, E)$ es un conjunto maximal $C \subseteq V$ tal que, para todo $u, v \in C$, se tiene $u \rightsquigarrow v$ y $v \rightsquigarrow u$.

Alcanzarse mutuamente es una relación de equivalencia, así que las componentes parten a V. El grafo es **fuertemente conexo** si tiene una sola componente.

Definición 15.8 (Grafo transpuesto y grafo de componentes)**.** El **transpuesto** de $G = (V, E)$ es $G^T = (V, E^T)$, con $E^T = \{(u, v) : (v, u) \in E\}$: las mismas aristas, volteadas. Si $C_1, \ldots, C_k$ son las componentes fuertemente conexas de G, el **grafo de componentes** G^{CFC} tiene un vértice c_i por cada C_i, y la arista (c_i, c_j), con $i \neq j$, si hay alguna arista $(x, y) \in E$ con $x \in C_i$ y $y \in C_j$.

El algoritmo de dos DFS

El algoritmo, de Kosaraju y Sharir, corre dos DFS. El primero, sobre G, solo sirve para calcular los tiempos de salida. El segundo corre sobre G^T y toma las raíces en orden decreciente de esos tiempos. Cada árbol del segundo bosque es exactamente una componente.

Algoritmo 15.3 Componentes-Fuertes(G), dos DFS (Kosaraju)

```
1:  procedimiento Componentes-Fuertes(G)
2:      DFS sobre G (algoritmo 14.4);  L ← vértices en sale decreciente
3:      construir G^T;  comp[u] ← 0 para todo u ∈ V;  k ← 0        // 0: sin componente
4:      para todo u en el orden de L hacer                          // sale decreciente
5:          si comp[u] = 0 entonces
6:              k ← k + 1;  Marcar(u, k)
7:          fin si
8:      fin para
9:      devolver comp
10: fin procedimiento

11: procedimiento Marcar(u, k)                                      // DFS sobre G^T
12:     comp[u] ← k
13:     para todo v ∈ Adj^T[u] hacer
14:         si comp[v] = 0, Marcar(v, k)
15:     fin para
16: fin procedimiento
```

La línea 2 es el orden por sale del algoritmo 15.1 sin la prueba de ciclo, y G^T se construye agregando u a $\text{Adj}^T[v]$ por cada arista (u, v). Cada llamada a Marcar desde el ciclo de la línea 4 es la raíz de una componente nueva; $\text{comp}[v] = 0$ hace de blanco. **Costo:** dos DFS y un transpuesto, todos de $\Theta(n + m)$; L se arma en $\mathcal{O}(1)$ por vértice, sin ordenar.

Por qué funciona

G y G^T tienen exactamente las mismas componentes, porque voltear todas las aristas voltea todos los caminos. Y entre dos componentes distintas los caminos van en un solo sentido:

Lema 15.9 (Cormen, lema 22.13)**.** *Sean $C \neq C'$ componentes, $u, v \in C$ y $u', v' \in C'$. Si hay un camino $u \rightsquigarrow u'$, no hay camino $v' \rightsquigarrow v$. En consecuencia, G^{CFC} es un DAG.*

Idea de la prueba. Con un camino $v' \rightsquigarrow v$ se tendría $u \rightsquigarrow u' \rightsquigarrow v'$ y $v' \rightsquigarrow v \rightsquigarrow u$: u y v' estarían en la misma componente. □

Todo lo demás descansa en un lema sobre los tiempos de salida de las componentes.

Lema 15.10 (Cormen, lema 22.14)**.** *Sean $C \neq C'$ componentes y $(u, v) \in E$ con $u \in C$ y $v \in C'$. Entonces sale$(C) >$ sale(C').*

Idea de la prueba. Si el DFS toca primero a C, en su primer vértice x hay caminos blancos a todo $C \cup C'$ (pasando por (u, v)), así que todo queda debajo de x y $\text{sale}[x] = \text{sale}(C) > \text{sale}(C')$. Si toca primero a C', todo C' termina antes de que se descubra algún vértice de C, porque por el lema 15.9 desde C' no se llega a C. □

Corolario 15.11 (Cormen, corolario 22.15)**.** *Si $(u, v) \in E^T$ con $u \in C$, $v \in C'$ y $C \neq C'$, entonces sale$(C) <$ sale(C'). En palabras: en G^T, toda arista entre componentes va de la que terminó antes a la que terminó después.*

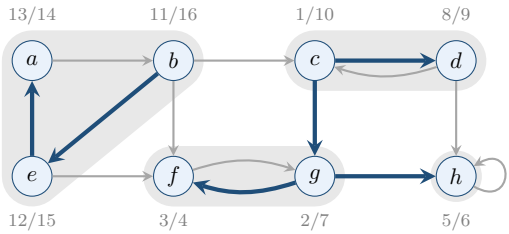
Teorema 15.12 (Cormen, teorema 22.16)**.** Componentes-Fuertes *calcula correctamente las componentes fuertemente conexas de G.*

Idea de la prueba. La componente C con sale(C) máximo es una fuente de G^{CFC} por el lema 15.10; en G^T se vuelve un sumidero, y el DFS que arranca en ella la recorre toda sin poder salir. Por inducción, la raíz de cada árbol siguiente tiene el mayor sale entre los vértices sin marcar, y por el corolario 15.11 las aristas de G^T que salen de su componente solo llevan a componentes ya marcadas. Por eso el orden es *decreciente*. □

Ejemplo 15.13 (La figura 22.9 de Cormen)**.** Se usan las letras minúsculas de Cormen para no confundir con el grafo G de la libreta. $V = \{a, b, c, d, e, f, g, h\}$, $n = 8$ y $m = 14$:

$$E = \{(a,b), (b,c), (b,e), (b,f), (c,d), (c,g), (d,c), (d,h),\\ (e,a), (e,f), (f,g), (g,f), (g,h), (h,h)\}.$$

Primer DFS. Para reproducir los tiempos de la figura, el ciclo exterior empieza en c y el siguiente blanco es b; en la lista de c va g antes que d, y en la de g, f antes que h. En sale decreciente, $L = b\,(16), e\,(15), a\,(14), c\,(10), d\,(9), g\,(7), h\,(6), f\,(4)$.



G y el primer DFS

Las aristas de árbol del primer DFS van en azul grueso, las componentes sombreadas y entra/sale junto a cada vértice.

Segundo DFS, sobre G^T y en el orden de L. Las listas de G^T son a: e; b: a; c: b, d; d: c; e: b; f: b, e, g; g: c, f; h: d, g, h.

Árbol	Raíz (sale)	Vértices	Aristas de G^T que topan con árboles anteriores
1	b (16)	b, a, e	ninguna: en G^T nada sale de $\{a, b, e\}$
2	c (10)	c, d	$c \to b$
3	g (7)	g, f	$g \to c$, $f \to b$, $f \to e$
4	h (6)	h	$h \to d$, $h \to g$

e, a, d y f aparecen en L, pero cuando les toca ya están marcados. Las componentes son $\{a, b, e\}$, $\{c, d\}$, $\{f, g\}$ y $\{h\}$, y salieron en orden topológico de G^{CFC}. La última columna es el corolario 15.11 en acción: cada arista de G^T que sale de un árbol choca con un árbol anterior, nunca con uno futuro.

Ojito...

El primer DFS *no* respeta las componentes: en la figura, el árbol que empieza en c se lleva $\{c, d\}$, $\{f, g\}$ y $\{h\}$ juntos. Del primer DFS solo sirven los tiempos de salida. Las dos piezas del segundo DFS son necesarias: sobre G sin transponer, en sale decreciente, desde b se alcanza todo y sale un solo árbol; sobre G^T en sale creciente, desde f salen $\{f, b, a, e, g, c, d\}$ y luego $\{h\}$.

Puntos de articulación, puentes y componentes biconexas

Esta parte es sobre grafos *no dirigidos*. Un **punto de articulación** (vértice de corte en la sección 12) es un vértice cuya eliminación desconecta el grafo; un **puente** es una arista cuya eliminación lo desconecta, y lo es si y solo si no está en ningún ciclo. Con la definición cuesta $\mathcal{O}(n(n+m))$; aquí se hace todo en un solo DFS (Cormen, problema 22-2).

Definición 15.14 (Componente biconexa)**.** Una **componente biconexa** (*biconnected component*) de un grafo no dirigido es un conjunto maximal de aristas tal que dos aristas cualesquiera del conjunto están en un ciclo común.

"$e = e'$, o e y e' están en un ciclo común" es una relación de equivalencia sobre las aristas, y sus clases son las componentes biconexas. Un puente forma él solo una **componente biconexa de una sola arista**; con esa convención, que es la del algoritmo, las componentes parten a todas las aristas y los puntos de articulación son los vértices que tocan dos o más. Cormen deja los puentes fuera.

Criterios con el árbol DFS

Sea G no dirigido y conexo, y sea T el árbol de un DFS desde una raíz r. Todo lo que sigue depende de un hecho de la sección 14: en un no dirigido no hay aristas de avance ni de cruce, así que toda arista que no es del árbol une a un vértice con un antecesor suyo. El **subárbol** de un vértice v son v y sus descendientes en T.

Proposición 15.15 (Regla de la raíz)**.** *La raíz r es punto de articulación si y solo si tiene al menos dos hijos en T.*

Idea de la prueba. Una arista que sale del subárbol de un hijo llega a un antecesor, y el único fuera del subárbol es r: sin r, dos subárboles hijos quedan separados. Con un solo hijo, su subárbol contiene a todos los demás vértices y sigue conexo. □

Proposición 15.16 (Vértices que no son la raíz)**.** *Un vértice $v \neq r$ es punto de articulación si y solo si tiene un hijo s tal que ninguna arista de retroceso sale del subárbol de s hacia un antecesor propio de v.*

Idea de la prueba. Si existe ese hijo s, toda arista que sale del subárbol de s llega a v, y quitar v lo separa de r. Si cada hijo tiene una arista de retroceso por encima de v, cada subárbol queda colgado, sin v, de la parte de T que contiene a r. □

Definición 15.17 (El valor low)**.** Para cada vértice v, low$[v]$ es el mínimo entre entra$[v]$ y los entra$[w]$ de todas las aristas de retroceso $\{x, w\}$ con x en el subárbol de v y w antecesor de x. Es el descubrimiento más antiguo al que se puede saltar desde el subárbol de v con una sola arista de retroceso. Cormen lo escribe $v.low$.

Como el subárbol de v es v junto con los subárboles de sus hijos,

$$\text{low}[v] = \text{mín}\Big(\text{entra}[v],\ \min_{\{v,w\} \text{ de retroceso}} \text{entra}[w],\ \min_{s \text{ hijo de } v} \text{low}[s]\Big),$$

y se calcula en postorden dentro del mismo DFS: al regresar de cada hijo, su low ya es definitivo.

Corolario 15.18 (Criterio de articulación con low)**.** *Un vértice $v \neq r$ es punto de articulación si y solo si tiene un hijo s con low$[s] \geq$ entra$[v]$.*

Demostración. Los antecesores propios de v tienen entra menor que entra$[v]$. Una arista de retroceso desde el subárbol de s llega a un antecesor de su extremo de abajo, y ese antecesor está dentro del subárbol de s (con entra mayor que entra$[v]$), es v, o es un antecesor propio de v. Por lo tanto, alguna llega a un antecesor propio de v si y solo si low$[s] <$ entra$[v]$. Se aplica la proposición 15.16. □

Proposición 15.19 (Criterio de puente)**.** *Una arista de retroceso nunca es puente. Una arista de árbol $\{u, v\}$, con $u = \pi[v]$, es puente si y solo si low$[v] >$ entra$[u]$, lo que equivale a low$[v] =$ entra$[v]$.*

Idea de la prueba. Una arista de retroceso cierra un ciclo con el camino del árbol. Para la de árbol $\{u, v\}$: si una arista de retroceso sale del subárbol de v hacia u o más arriba, cierra un ciclo que pasa por $\{u, v\}$; si ninguna sale, $\{u, v\}$ es lo único que une el subárbol con el resto. "Alguna sale" es low$[v] \leq$ entra$[u]$, y low$[v]$ no puede caer estrictamente entre entra$[u]$ y entra$[v]$. □

Las componentes biconexas salen en el mismo recorrido con una pila de aristas: cada arista se apila la primera vez que se explora, y cuando al regresar del hijo v a u se cumple low$[v] \geq$ entra$[u]$, las aristas apiladas desde $\{u, v\}$ hasta el tope forman una componente completa y se desapilan juntas.

El algoritmo 15.4 junta los tres criterios en un solo DFS; no necesita colores, porque entra$[v] = 0$ indica que v no se ha visto.

Algoritmo 15.4 Articulaciones(G): puntos de articulación, puentes y componentes biconexas

```
1:  procedimiento Articulaciones(G)                                  // G no dirigido
2:      entra[u] ← 0,  hijos[u] ← 0,  π[u] ← nulo, para todo u ∈ V
3:      t ← 0;  P ← pila de aristas vacía
4:      para todo r ∈ V hacer
5:          si entra[r] = 0 entonces                                 // r es raíz de un árbol nuevo
6:              Visitar-Low(r)
7:              si hijos[r] ≥ 2, marcar r como punto de articulación  // regla de la raíz
8:          fin si
9:      fin para
10: fin procedimiento

11: procedimiento Visitar-Low(u)
12:     t ← t + 1,    entra[u] ← t,    low[u] ← t
13:     para todo v vecino de u hacer
14:         si entra[v] = 0 entonces                                 // arista de árbol
15:             π[v] ← u,    hijos[u] ← hijos[u] + 1;  apilar {u, v} en P
16:             Visitar-Low(v)
17:             low[u] ← mín(low[u], low[v])
18:             si low[v] ≥ entra[u] entonces                        // nada debajo de v salta arriba de u
19:                 si π[u] ≠ nulo, marcar u como punto de articulación
20:                 desapilar de P hasta {u, v} inclusive: es una componente biconexa
21:             fin si
22:             si low[v] > entra[u], {u, v} es puente
23:         si no si v ≠ π[u] y entra[v] < entra[u] entonces          // retroceso
24:             apilar {u, v} en P
25:             low[u] ← mín(low[u], entra[v])
26:         fin si
27:     fin para
28:     t ← t + 1,    sale[u] ← t
29: fin procedimiento
```

- **Línea 18.** low$[v] \geq$ entra$[u]$: nada del subárbol de v salta por encima de u, y si u no es raíz eso lo hace articulación (corolario 15.18). Las componentes biconexas se desapilan también en la raíz.
- **Línea 22.** Desigualdad estricta: con igualdad, el subárbol de v regresa justo a u por otra arista.
- **Línea 23.** entra$[v] <$ entra$[u]$ hace que cada arista de retroceso se apile una sola vez, desde abajo.

Costo. Es un DFS sobre un no dirigido, y cada arista se apila y se desapila una vez: $\Theta(n + m)$.

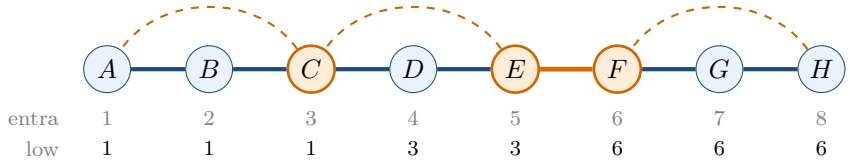
Ejemplo 15.20 (Un moño con cola)**.** El grafo H está hecho para que aparezcan todos los casos. $V = \{A, \ldots, H\}$, $n = 8$, $m = 10$:

$$E = \{\{A, B\}, \{A, C\}, \{B, C\}, \{C, D\}, \{C, E\}, \{D, E\}, \{E, F\}, \{F, G\}, \{F, H\}, \{G, H\}\}.$$

Son dos triángulos que comparten el vértice C, un moño, más un puente $\{E, F\}$ y un tercer triángulo colgado de F.

El DFS empieza en A, con los vecinos en orden alfabético. El árbol resulta ser un camino, $A, B, C, \ldots, H$, así que entra va de 1 a 8, y las aristas de retroceso son $\{C, A\}$, $\{E, C\}$ y $\{H, F\}$. Los low se llenan en

postorden, de H hacia arriba, y la figura los muestra. Las pruebas con el hijo dan: en C, $\text{low}[D] = 3 \geq 3$; en E, $\text{low}[F] = 6 \geq 5$, y además $6 > 5$, así que $\{E, F\}$ es puente; en F, $\text{low}[G] = 6 \geq 6$. En los demás falla: en B, $\text{low}[C] = 1 < 2$; en D, $\text{low}[E] = 3 < 4$; en G, $\text{low}[H] = 6 < 7$; y la raíz A tiene un solo hijo.



El árbol DFS tendido en línea, con las aristas de retroceso punteadas y las filas de entra y low. Resultado: puntos de articulación C, E y F, resaltados, y un solo puente, $\{E, F\}$, en naranja. C es el caso fino: $\text{low}[D] = \text{entra}[C] = 3$, así que C es articulación pero $\{C, D\}$ no es puente, porque la arista $\{E, C\}$ regresa justo a C.

Las componentes biconexas salen de la pila en este orden: $\{FG, GH, HF\}$ al regresar de G a F; $\{EF\}$, el puente, al regresar de F a E; $\{CD, DE, EC\}$ al regresar de D a C; y $\{AB, BC, CA\}$ al regresar de B a A. Las articulaciones son justo los vértices que tocan dos componentes, y todo coincide con la fuerza bruta: quitar cada vértice y cada arista y revisar si lo que queda sigue conexo.

Ojito...

En la línea 25, la arista de retroceso aporta $\text{entra}[v]$, no $\text{low}[v]$. Es un error muy común, y en H se nota: cuando E ve su arista de retroceso hacia C, $\text{low}[C]$ ya vale 1 por la arista $\{C, A\}$, aunque C todavía no termina. Con $\text{low}[C]$, los valores de E y de D bajan a 1, la prueba en C da $1 < 3$ y el algoritmo *pierde la articulación* C. Para puentes el error no se nota; para articulaciones sí.

Ojito...

Dos detalles más rompen el algoritmo si se olvidan. Si no se excluye la arista por la que se llegó a u, siempre $\text{low}[u] \leq \text{entra}[\pi[u]]$ y no se encuentra ningún puente; en un multigrafo hay que excluir *esa arista*, por identificador, no al vértice padre. Y la prueba $\text{low}[s] \geq \text{entra}[r]$ se cumple siempre en la raíz: aplicarla ahí declara articulación a toda raíz.

Recorrido de Euler en grafos dirigidos

La sección 12 trata el caso no dirigido y el algoritmo de Hierholzer. Un **recorrido de Euler** de un grafo dirigido es un circuito que usa cada arista exactamente una vez; puede repetir vértices, así que no es un ciclo en el sentido de esta libreta, aunque Cormen lo llame así.

Teorema 15.21 (Cormen, problema 22-3)**.** *Un grafo dirigido fuertemente conexo tiene recorrido de Euler si y solo si* $gr^-(v) = gr^+(v)$ *para todo vértice* v.

Es el teorema 12.23 de la sección 12 en la forma de Cormen. Allá basta la conexidad débil, porque con los grados balanceados un grafo débilmente conexo resulta fuertemente conexo.

Idea de la prueba. Cada paso del recorrido por v entra por una arista y sale por otra: los grados quedan balanceados. Al revés, una caminata desde s por aristas no usadas solo puede atorarse en s, porque a cualquier otro vértice le queda una salida libre, y al atorarse cierra un circuito. Si sobran aristas, alguna sale de un vértice x del circuito, y desde x sale otro circuito que se empalma ahí. □

Hierholzer funciona igual siguiendo solo las aristas de salida: una pila guarda el camino actual, se

avanza mientras el tope tenga aristas sin usar, y cuando se atora se desapila e inserta al frente del recorrido. Cada arista se usa una vez: $\Theta(n+m)$.

Ojito...

Sin conexidad fuerte el balance no basta: dos ciclos ajenos están balanceados y no tienen recorrido de Euler. La condición general es el balance más que todas las aristas estén en una misma componente fuertemente conexa.

Alcanzabilidad mínima

En el problema 22-4 de Cormen cada vértice u de un grafo dirigido tiene una etiqueta distinta $L(u) \in \{1, \ldots, n\}$, y se quiere el vértice mín$[u]$ de etiqueta mínima entre los alcanzables desde u, incluido u. Como $u \rightsquigarrow v$ en G si y solo si $v \rightsquigarrow u$ en G^T, se procesan los vértices en orden *creciente* de etiqueta, y desde cada v todavía sin asignar un DFS en G^T asigna mín$[u] = v$ a todo vértice sin asignar que alcance.

Algoritmo 15.5 Mínimos-Alcanzables(G, L), problema 22-4

```
1:  procedimiento Mínimos-Alcanzables(G, L)
2:      construir G^T
3:      para todo u ∈ V hacer
4:          vert[L(u)] ← u,    mín[u] ← nulo          // ordena por L en Θ(n)
5:      fin para
6:      para i ← 1 hasta n hacer
7:          v ← vert[i];  si mín[v] = nulo, Asignar(v, v)
8:      fin para
9:      devolver mín
10: fin procedimiento

11: procedimiento Asignar(u, v)                        // DFS sobre G^T
12:     mín[u] ← v
13:     para todo w ∈ Adj^T[u] hacer
14:         si mín[w] = nulo, Asignar(w, v)
15:     fin para
16: fin procedimiento
```

Por qué funciona. Si u sigue sin asignar cuando le toca a v, ningún vértice de etiqueta menor es alcanzable desde u; si además $u \rightsquigarrow v$, v es su mínimo. Detener el DFS en los vértices ya asignados no pierde nada: todo lo que alcanza a un vértice asignado por v' alcanza a v', de etiqueta menor. El costo es el de transponer más un DFS, $\Theta(n+m)$; invertir L cuesta $\Theta(n)$, porque es una biyección con $\{1, \ldots, n\}$.

Ojito...

Aquí importa la dirección dos veces. El DFS va sobre G^T, porque se busca quién *alcanza* a v, y en orden *creciente* de etiqueta. Sobre G se calcularía otra cosa: el mínimo de los vértices que alcanzan a cada uno.

16. Árboles de expansión mínima

Complemento: Cormen, cap. 23

Resumen rápido. La clase del 22 de octubre presentó Kruskal con un ejemplo y dejó a Prim en una observación. Esta sección da la teoría que los sostiene y las dos implementaciones completas. Todo descansa en la propiedad del corte: si se parten los vértices en dos lados sin separar ninguna arista ya elegida, la arista más ligera que cruza se puede agregar sin perder la optimalidad. De ahí salen el método genérico, la propiedad del ciclo y la unicidad con pesos distintos. Kruskal usa conjuntos disjuntos y cuesta $\mathcal{O}(m \log n)$; Prim usa una cola de prioridad y cuesta $\mathcal{O}(m \log n)$ con montículo binario, $\mathcal{O}(m + n \log n)$ con Fibonacci y $\Theta(n^2)$ con un arreglo, y su prueba $v \in Q$, a diferencia de la de Dijkstra, no se puede quitar.

El problema y el método genérico

La clase del 22 de octubre (sección 11) definió el problema: en un grafo $G = (V, E)$ no dirigido y conexo, con un peso $w : E \to \mathbb{R}$ en cada arista, se busca un conjunto $T \subseteq E$ acíclico que conecte todos los vértices y minimice

$$w(T) = \sum_{\{u,v\} \in T} w(u, v).$$

Un conjunto así es un árbol de $n - 1$ aristas, el **árbol de expansión de peso mínimo** o MST (*minimum spanning tree*). Los pesos son reales cualesquiera, negativos incluidos. Las letras e y f denotan aristas, y $T - f + e$ abrevia $(T \setminus \{f\}) \cup \{e\}$.

Aristas seguras

Kruskal y Prim son voraces y agregan una arista a la vez. Lo que tienen en común es un invariante sobre el conjunto A de aristas elegidas hasta el momento: *antes de cada iteración, A es subconjunto de algún árbol de expansión mínima.*

Definición 16.1 (Arista segura)**.** Sea $A \subseteq E$ un subconjunto de algún MST de G. Una arista $e \notin A$ es **segura** para A (*safe edge*) si $A \cup \{e\}$ también es subconjunto de algún MST.

Con esa definición el algoritmo cabe en cuatro líneas, aunque todavía no se pueda ejecutar:

Algoritmo 16.1 MST-Genérico(G, w)

```
1: procedimiento MST-Genérico(G, w)
2:     A ← ∅
3:     mientras A no forma un árbol de expansión hacer
4:         encontrar una arista e segura para A
5:         A ← A ∪ {e}
6:     fin mientras
7:     devolver A
8: fin procedimiento
```

La línea 4 no dice cómo encontrar la arista, y es justo donde Kruskal y Prim se separan. La corrección se prueba una sola vez, aquí: al inicio $A = \emptyset$ está en cualquier MST, cada arista segura conserva el invariante, y al salir A es un árbol de expansión contenido en un MST T; los dos tienen $n - 1$ aristas, así que $A = T$. A cada algoritmo concreto solo le queda mostrar que la arista que escoge es segura, y reconocerla sin conocer el árbol es el trabajo del teorema central de la sección.

Cortes y la propiedad del corte

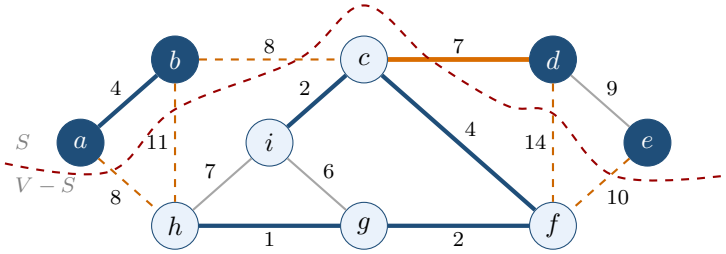
Definición 16.2 (Corte, cruce, arista ligera)**.** Un **corte** $(S, V-S)$ de G (*cut*) es una partición de V en dos conjuntos. La arista $\{u,v\}$ **cruza** el corte (*crosses*) si uno de sus extremos está en S y el otro en $V-S$. El corte **respeta** un conjunto de aristas A (*respects*) si ninguna arista de A lo cruza. Una **arista ligera** que cruza el corte (*light edge*) es una de peso mínimo entre las que lo cruzan.

Con empates puede haber varias aristas ligeras para el mismo corte, y el teorema vale para cualquiera. "Ligera" siempre se dice respecto a un corte, no respecto al grafo.

Ejemplo 16.3 (Un corte del grafo de Cormen)**.** Este ejemplo y casi todos los que siguen usan el grafo de la figura 23.1 de Cormen, con $n=9$ vértices y $m=14$ aristas. Los vértices van en minúscula para no confundirlos con los del grafo G de la libreta:

$$\{a,b\}\ 4,\ \{a,h\}\ 8,\ \{b,c\}\ 8,\ \{b,h\}\ 11,\ \{c,d\}\ 7,\ \{c,f\}\ 4,\ \{c,i\}\ 2,$$
$$\{d,e\}\ 9,\ \{d,f\}\ 14,\ \{e,f\}\ 10,\ \{f,g\}\ 2,\ \{g,h\}\ 1,\ \{g,i\}\ 6,\ \{h,i\}\ 7.$$

Se toma $S=\{a,b,d,e\}$, en relleno oscuro, y $A=\{\{a,b\},\{c,i\},\{c,f\},\{g,h\},\{f,g\}\}$, en azul grueso. Ninguna arista de A tiene un extremo de cada lado, así que el corte $(S, V-S)$ respeta A. Cruzan seis aristas, punteadas: $\{c,d\}$ con 7, $\{a,h\}$ y $\{b,c\}$ con 8, $\{e,f\}$ con 10, $\{b,h\}$ con 11 y $\{d,f\}$ con 14. La arista ligera es $\{c,d\}$, y es la única. Es ligera aunque el grafo tenga seis aristas más baratas que ella: ninguna de esas seis cruza este corte.



Las pruebas que siguen quitan una arista de un árbol, agregan otra y comprueban que lo que queda sigue siendo árbol.

Lema 16.4 (Tres hechos sobre árboles de expansión)**.** *Sea T un árbol de expansión de G.*

(a) Entre cada par de vértices hay un único camino dentro de T.

(b) Quitar una arista $f=\{x,y\}$ de T deja exactamente dos componentes, una con x y otra con y. Ellas definen un corte, y f es la única arista de T que lo cruza.

(c) Si $e=\{u,v\}\notin T$ y f es cualquier arista del camino de u a v en T, entonces $T-f+e$ es otra vez un árbol de expansión.

Idea de la prueba. (a) y (b) son caracterizaciones de árbol de la sección 13; (c) sale de (b), porque e une las dos componentes de $T-f$. □

Teorema 16.5 (Propiedad del corte; Cormen, teorema 23.1)**.** *Sea $G=(V,E)$ un grafo no dirigido y conexo con pesos $w: E\to\mathbb{R}$. Sea $A\subseteq E$ un subconjunto de algún árbol de expansión mínima de G, sea $(S, V-S)$ un corte que respeta A y sea $e=\{u,v\}$ una arista ligera que cruza $(S, V-S)$. Entonces e es segura para A.*

Demostración. Sea T un MST con $A\subseteq T$. Si $e\in T$, entonces $A\cup\{e\}\subseteq T$ y no hay nada que probar. Supongamos que $e\notin T$; vamos a fabricar otro MST que sí la contenga.

Una arista de T que cruza. Sin perder generalidad $u \in S$ y $v \in V - S$. Sea p el camino de u a v dentro de T. Empieza en S y termina fuera de S, así que al recorrerlo desde u hay una primera arista $f = \{x, y\}$ con $x \in S$ y $y \in V - S$. Esa arista cruza el corte. No está en A, porque el corte respeta A. Y es distinta de e, porque $f \in T$ y $e \notin T$.

El intercambio. Por el lema 16.4(c), $T' = T - f + e$ es un árbol de expansión, y pesa

$$w(T') = w(T) - w(f) + w(e).$$

Las dos aristas e y f cruzan el corte y e es ligera, así que $w(e) \leq w(f)$ y por lo tanto $w(T') \leq w(T)$.

Los dos casos. Si $w(e) < w(f)$, entonces $w(T') < w(T)$, lo que es imposible porque T es mínimo: este caso no ocurre. Queda el empate, $w(e) = w(f)$. Entonces $w(T') = w(T)$ y T' es otro MST: el intercambio no mejora, empata.

T' contiene lo que hace falta. Como $A \subseteq T$ y $f \notin A$, se tiene $A \subseteq T - f \subseteq T'$; además $e \in T'$. Entonces $A \cup \{e\} \subseteq T'$, con T' un MST, y e es segura. □

El teorema deja escoger el corte con total libertad, siempre que respete A. Kruskal y Prim usan el mismo caso particular: el corte que separa uno de los árboles ya construidos del resto del grafo.

Corolario 16.6 (Cormen, corolario 23.2)**.** *Sean G y A como en el teorema 16.5 y sea $C = (V_C, E_C)$ un componente del bosque $G_A = (V, A)$. Si e es una arista ligera entre las que unen C con otro componente de G_A, entonces e es segura para A.*

Idea de la prueba. El corte $(V_C, V - V_C)$ respeta A, y las aristas que lo cruzan son exactamente las que unen C con otro componente. □

Del teorema sale también la forma del método genérico: $G_A = (V, A)$ es siempre un bosque, que empieza con n árboles de un vértice; toda arista segura une dos componentes distintos de G_A, y el ciclo da exactamente $n - 1$ vueltas.

Ojito...

Segura no es lo mismo que "está en algún MST". El invariante pide que $A \cup \{e\}$ quepa entero en *un mismo* MST. El grafo de Cormen tiene dos MST, que difieren en $\{a, h\}$ contra $\{b, c\}$. Si A son las siete aristas comunes más $\{a, h\}$, la arista $\{b, c\}$ ya no es segura, aunque pertenezca a un MST: con A cierra el ciclo a, b, c, f, g, h, a. Por eso el teorema exige que el corte respete A.

La propiedad del ciclo y la unicidad

El teorema del corte dice qué aristas se pueden meter. Su gemelo dice cuáles se pueden dejar fuera.

Lema 16.7 (Propiedad del ciclo; Cormen, ejercicio 23.1-5)**.** *Sea e una arista de peso máximo en algún ciclo K de G. Entonces hay un MST de G que no contiene a e. Si además e es la única de peso máximo en K, ningún MST contiene a e.*

Idea de la prueba. Si un MST T contiene a e, quitarla deja un corte que el resto de K cruza por alguna arista $f \notin T$. $T - e + f$ es un árbol de peso $w(T) - w(e) + w(f) \leq w(T)$; si e es la única máxima pesa estrictamente menos, y T no era mínimo. □

Ojito...

La versión que circula, "la arista más pesada de un ciclo no está en ningún MST", es falsa con empates: en el ciclo a, b, c, i, h, a, de pesos $4, 8, 2, 7, 8$, las dos de peso máximo están cada una en uno de los dos MST. Con un máximo estricto sí vale: $\{d, f\}$, única máxima del ciclo c, d, f, c, no está en ningún MST.

Teorema 16.8 (Unicidad; Cormen, ejercicio 23.1-6 y problema 23-1a)**.** *Si todo corte de G tiene una sola arista ligera, en particular si todos los pesos son distintos, el MST de G es único.*

Idea de la prueba. Si $T \neq T'$ son MST, sea $e \in T \setminus T'$. Quitar e de T define un corte del que e es la única arista ligera. El camino entre sus extremos en T' cruza ese corte por alguna $f \neq e$, con $w(f) > w(e)$, y $T' - f + e$ pesaría menos que T'. □

Observación. Kruskal y Prim solo comparan pesos: los negativos no estorban, sumar una constante a todos no cambia cuáles árboles son mínimos, y negándolos sale el de peso máximo.

Conjuntos disjuntos

Kruskal pregunta una y otra vez si dos vértices están en el mismo árbol de G_A, mientras los árboles se fusionan. Necesita tres operaciones (Cormen, cap. 21): Crear-Conjunto(x) forma $\{x\}$, Encontrar(x) devuelve un representante del conjunto de x y Unir(x, y) junta los de x y y. Cada conjunto se guarda como un árbol en el que cada nodo apunta a su padre $p[x]$ y la raíz, el representante, se apunta a sí misma. Para evitar cadenas de largo n, con la **unión por rango** cada raíz guarda un rango, cota superior de la altura de su árbol, y al unir la de rango menor cuelga de la de mayor (si empatan, la que queda arriba sube su rango); con la **compresión de caminos**, Encontrar deja a cada nodo del camino apuntando a la raíz.

Algoritmo 16.2 Conjuntos disjuntos con unión por rango y compresión de caminos

```
1:  procedimiento Crear-Conjunto(x)
2:      p[x] ← x,    rango[x] ← 0
3:  fin procedimiento
4:  procedimiento Encontrar(x)
5:      si x ≠ p[x] entonces
6:          p[x] ← Encontrar(p[x])                    // compresión de caminos
7:      fin si
8:      devolver p[x]
9:  fin procedimiento
10: procedimiento Unir(x, y)
11:     a ← Encontrar(x),    b ← Encontrar(y)
12:     si rango[a] < rango[b] entonces
13:         p[a] ← b                                  // el árbol bajo cuelga del alto
14:     si no
15:         p[b] ← a
16:         si rango[a] = rango[b] entonces
17:             rango[a] ← rango[a] + 1
18:         fin si
19:     fin si
20: fin procedimiento
```

Unir recibe vértices cualesquiera y busca las raíces por su cuenta.

Lema 16.9 (Unión por rango)**.** *Con unión por rango y sin compresión de caminos, un árbol cuya raíz tiene rango r tiene altura r y al menos 2^r nodos. En consecuencia la altura nunca pasa de $\log_2 n$ y* Encontrar *cuesta $\mathcal{O}(\log n)$.*

Idea de la prueba. Inducción sobre las uniones. Si los rangos difieren, el árbol bajo cuelga del alto sin cambiar su altura; si empatan en r, el nuevo tiene altura $r+1$ y al menos $2^r + 2^r = 2^{r+1}$ nodos. □

Con las dos heurísticas juntas la cota baja a casi constante por operación; la prueba ocupa la sección 21.4 de Cormen.

Teorema 16.10 (Cormen, teorema 21.14)**.** *Con unión por rango y compresión de caminos, una sucesión de k operaciones, de las cuales n son* Crear-Conjunto*, cuesta $\mathcal{O}(k\,\alpha(n))$ en el peor caso.*

Cormen llama m al número de operaciones; aquí es k, porque m ya es el número de aristas. La función α es la inversa de la función de Ackermann y $\alpha(n) \le 4$ para todo $n \le 10^{80}$: en la práctica es una constante, aunque en teoría $\alpha(n) \to \infty$.

Kruskal

Kruskal toma, entre todas las aristas que unen dos árboles distintos del bosque G_A, la de menor peso, y para eso basta recorrerlas en orden de peso. Cada conjunto disjunto guarda los vértices de un árbol del bosque.

Algoritmo 16.3 MST-Kruskal(G, w)

```
1:  procedimiento MST-Kruskal(G, w)
2:      A ← ∅
3:      para todo v ∈ V hacer
4:          Crear-Conjunto(v)
5:      fin para
6:      ordenar E por peso, de menor a mayor
7:      para todo {u, v} ∈ E, en ese orden hacer
8:          si Encontrar(u) ≠ Encontrar(v) entonces
9:              A ← A ∪ {{u, v}}                    // une dos árboles distintos
10:             Unir(u, v)
11:         fin si
12:     fin para
13:     devolver A
14: fin procedimiento
```

En la línea 6 el orden es no decreciente y los empates se rompen como sea; esa decisión cambia qué árbol sale, no su peso. La línea 8 es la que decide, y su justificación es el corolario.

Proposición 16.11 (Corrección de Kruskal)**.** *Si G es conexo, MST-Kruskal devuelve un árbol de expansión mínima.*

Idea de la prueba. Los conjuntos son siempre los vértices de los árboles de G_A. Si u y v están en el mismo, e cerraría un ciclo. Si no, sea C el árbol de u: las aristas que unen C con otro árbol y vienen después de e pesan al menos $w(e)$, y las que vinieron antes ya tienen sus extremos en un mismo árbol.

Así e es ligera y el corolario 16.6 la hace segura. Al final A es un árbol, porque G es conexo y una arista entre dos árboles restantes se habría aceptado. □

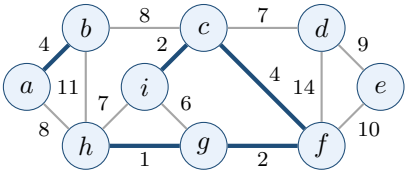
Costo. Con la estructura del algoritmo 16.2, las líneas 2 a 5 cuestan $\mathcal{O}(n)$ y el ordenamiento de la línea 6, $\mathcal{O}(m \log m)$. Las líneas 7 a 12 hacen $2m$ llamadas a Encontrar y $n-1$ a Unir, que por el teorema 16.10 cuestan $\mathcal{O}((n+m)\,\alpha(n)) = \mathcal{O}(m\,\alpha(n))$, porque $m \geq n-1$. El total es $\mathcal{O}(m \log m)$, y como $m < n^2$, $\log m < 2 \log n$: Kruskal cuesta $\mathcal{O}(m \log n)$.

El cuello de botella es el ordenamiento, no la estructura: para la cota $\mathcal{O}(m \log n)$ basta la unión por rango sola. La compresión de caminos importa cuando ordenar es barato; si los pesos son enteros en un rango chico, counting sort los ordena en $\mathcal{O}(n+m)$ y Kruskal baja a $\mathcal{O}(m\,\alpha(n))$. Sin conjuntos disjuntos, revisando con BFS si u y v ya están conectados, el total sube a $\mathcal{O}(mn)$.

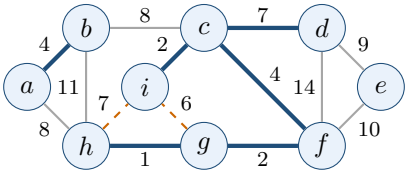
Ejemplo 16.12 (Kruskal sobre el grafo de Cormen)**.** Las 14 aristas en orden de peso, con los empates en el orden de la figura 23.4 de Cormen. La última columna es el bosque G_A después del paso; en los rechazos no cambia.

Paso	Arista	Peso	¿Entra?	Árboles del bosque después
1	$\{h, g\}$	1	sí	$\{a\}$ $\{b\}$ $\{c\}$ $\{d\}$ $\{e\}$ $\{f\}$ $\{g, h\}$ $\{i\}$
2	$\{c, i\}$	2	sí	$\{a\}$ $\{b\}$ $\{c, i\}$ $\{d\}$ $\{e\}$ $\{f\}$ $\{g, h\}$
3	$\{g, f\}$	2	sí	$\{a\}$ $\{b\}$ $\{c, i\}$ $\{d\}$ $\{e\}$ $\{f, g, h\}$
4	$\{a, b\}$	4	sí	$\{a, b\}$ $\{c, i\}$ $\{d\}$ $\{e\}$ $\{f, g, h\}$
5	$\{c, f\}$	4	sí	$\{a, b\}$ $\{c, f, g, h, i\}$ $\{d\}$ $\{e\}$
6	$\{i, g\}$	6	no	cerraría el ciclo i, c, f, g, i
7	$\{c, d\}$	7	sí	$\{a, b\}$ $\{c, d, f, g, h, i\}$ $\{e\}$
8	$\{i, h\}$	7	no	cerraría el ciclo i, c, f, g, h, i
9	$\{a, h\}$	8	sí	$\{a, b, c, d, f, g, h, i\}$ $\{e\}$
10	$\{b, c\}$	8	no	cerraría el ciclo b, a, h, g, f, c, b
11	$\{d, e\}$	9	sí	$\{a, b, c, d, e, f, g, h, i\}$
12–14	$\{e, f\}$, $\{b, h\}$, $\{d, f\}$	no		ya hay $n - 1 = 8$ aristas

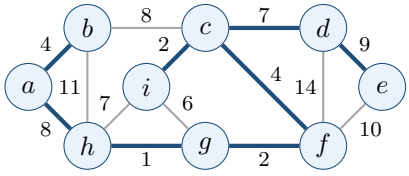
Peso total $1 + 2 + 2 + 4 + 4 + 7 + 8 + 9 = 37$. En el paso 9 el empate entre $\{a, h\}$ y $\{b, c\}$ se rompió a favor de $\{a, h\}$ solo porque aparece antes en la lista ordenada. Las instantáneas: después del paso 5, del 8 (en naranja los dos rechazos) y al final.



pasos 1 a 5



pasos 6 a 8



final: peso 37

Prim

Prim hace crecer un solo árbol desde una raíz r hasta cubrir V. En cada paso agrega la arista más ligera que une el árbol con un vértice de afuera: es el corolario 16.6 con C igual al árbol que se está construyendo. Para escoger esa arista rápido, los vértices de afuera viven en una cola de prioridad Q con la llave llave[v]: el peso de la arista más barata que une a v con el árbol, o ∞ si no hay ninguna (Cormen escribe $v.key$). El padre $\pi[v]$ es el extremo de esa arista dentro del árbol. Es la misma organización que Dijkstra, con otra llave.

Algoritmo 16.4 MST-Prim(G, w, r)

```
1:  procedimiento MST-Prim(G, w, r)
2:      para todo u ∈ V hacer
3:          llave[u] ← ∞,    π[u] ← nulo
4:      fin para
5:      llave[r] ← 0
6:      Q ← montículo de mínimos con todo V, ordenado por llave
7:      mientras Q ≠ ∅ hacer
8:          u ← Extraer-Min(Q)                          // entra u con la arista {u, π[u]}
9:          para todo v adyacente a u hacer
10:             si v ∈ Q y w(u, v) < llave[v] entonces
11:                 π[v] ← u
12:                 llave[v] ← w(u, v)                  // Decrementar-Llave
13:             fin si
14:         fin para
15:     fin mientras
16:     devolver {{v, π[v]} : v ∈ V − {r}}
17: fin procedimiento
```

La raíz sale primero porque su llave es 0, y es el único vértice con π nulo al final. Los vértices del árbol son los de $V - Q$; el que sale en la línea 8 tiene la arista más barata hacia el árbol, así que $\{u, \pi[u]\}$ es una arista ligera que cruza el corte $(V - Q, Q)$. La prueba $v \in Q$ de la línea 10, un bit por vértice, no es decorativa: sin ella el resultado deja de ser un árbol, como se ve más abajo.

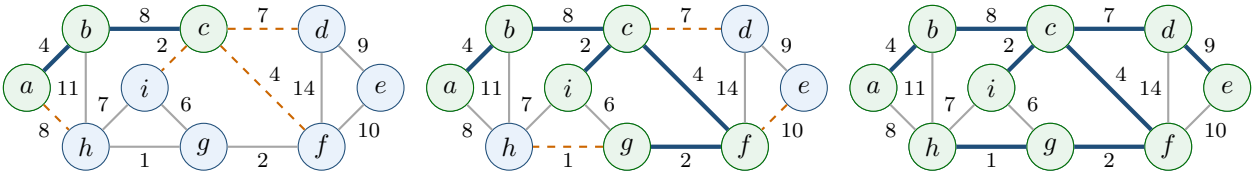
Proposición 16.13 (Corrección de Prim)**.** *Si G es conexo, MST-Prim devuelve un árbol de expansión mínima.*

Idea de la prueba. Antes de cada vuelta, salvo la primera, las aristas $\{v, \pi[v]\}$ de los vértices que ya salieron forman un conjunto A contenido en un MST, y para cada $v \in Q$, llave$[v]$ es el peso mínimo de una arista entre v y $V - Q$. El u que sale trae entonces la arista ligera del corte $(V - Q, Q)$, que respeta A, y el teorema 16.5 la hace segura. La prueba $v \in Q$ sostiene las dos partes: sin ella, las llaves y los padres de los vértices que ya salieron se siguen modificando. □

Ejemplo 16.14 (Prim sobre el grafo de Cormen, desde a)**.** Cada renglón muestra las llaves después de procesar el vértice que sale; en negritas, los que ya están en el árbol, cuya llave ya no cambia y es el peso de la arista con la que entraron. Los empates en la llave mínima se rompen por orden alfabético, como en la figura 23.5 de Cormen.

Paso	**Sale**	a	b	c	d	e	f	g	h	i	**Cambios**
0		0	∞	∞	∞	∞	∞	∞	∞	∞	
1	a	**0**	4	∞	∞	∞	∞	∞	8	∞	b, h desde a
2	b	**0**	**4**	8	∞	∞	∞	∞	8	∞	c desde b; h: $11 \not< 8$
3	c	**0**	**4**	**8**	7	∞	4	∞	8	2	empate c, h en 8; d, f, i desde c
4	i	**0**	**4**	**8**	7	∞	4	6	7	**2**	g desde i; h: $7 < 8$
5	f	**0**	**4**	**8**	7	10	**4**	2	7	**2**	e desde f; g: $2 < 6$
6	g	**0**	**4**	**8**	7	10	**4**	**2**	1	**2**	h: $1 < 7$
7	h	**0**	**4**	**8**	7	10	**4**	**2**	**1**	**2**	nada
8	d	**0**	**4**	**8**	**7**	9	**4**	**2**	**1**	**2**	e: $9 < 10$
9	e	**0**	**4**	**8**	**7**	**9**	**4**	**2**	**1**	**2**	

Orden de entrada: $a, b, c, i, f, g, h, d, e$, y el peso es la suma de las llaves finales, $0 + 4 + 8 + 7 + 9 + 4 + 2 + 1 + 2 = 37$. La llave de h cambió tres veces, $\infty \to 8 \to 7 \to 1$. En las instantáneas, en verde los vértices del árbol y en naranja, para cada vértice de afuera con llave finita, la arista $\{v, \pi[v]\}$ que le da esa llave: la más corta es la que entra en el paso siguiente.



sale c sale g final: peso 37

Kruskal y Prim llegan a 37 por caminos distintos y difieren en una arista: Kruskal se quedó con $\{a, h\}$ porque aparecía antes en la lista ordenada; Prim con $\{b, c\}$ porque en el paso 3 la llave de c empató con la de h en 8 y el desempate alfabético sacó a c.

Ojito...

La prueba $v \in Q$ de Prim es indispensable. Sin ella, en el grafo de Cormen desde a, al salir i en el paso 4 su vecino c, ya en el árbol con llave 8, pasa a $\pi[c] = i$ porque $w(i, c) = 2 < 8$, mientras $\pi[i] = c$; lo mismo pasa con f en el paso 6 y con g en el 7. El orden de extracción no cambia, pero la línea 16 devuelve ocho pares con solo seis aristas distintas, en tres componentes, y la suma de las llaves da $28 < 37$: no es un árbol. En Dijkstra la prueba se puede omitir, porque con pesos no negativos todo v que ya salió tiene $d[v] \leq d[u]$ y la relajación nunca dispara sobre él; en Prim, $w(u, v) <$ llave$[v]$ no guarda relación con el orden de salida. Por eso, aunque el código se parezca, en el grafo pesado de la libreta Prim desde A termina con el MST de peso 13 y Dijkstra con un árbol de caminos más cortos de peso 15.

Cuánto cuesta Prim

Con un **montículo binario de mínimos** se construye en $\mathcal{O}(n)$, se hacen n extracciones de $\mathcal{O}(\log n)$ y a lo más m decrementos de $\mathcal{O}(\log n)$, porque una arista solo puede bajar una llave cuando sale el primero de sus extremos: $\mathcal{O}(m \log n)$ con G conexo, igual que Kruskal. El decremento en $\mathcal{O}(\log n)$ exige que el montículo guarde la posición de cada vértice. Con un **montículo de Fibonacci** (Cormen, cap. 19), Decrementar-Llave cuesta $\mathcal{O}(1)$ amortizado y el total baja a $\mathcal{O}(m + n \log n)$. Con un **arreglo** de llaves, conveniente cuando el grafo viene como matriz, buscar el mínimo y revisar los vecinos son barridos de $\Theta(n)$: $\Theta(n^2)$ sin importar m. La tabla 7 reúne las cuatro opciones.

Implementación	**Costo**	**Disperso,** $m = \Theta(n)$	**Denso,** $m = \Theta(n^2)$
Kruskal, conjuntos disjuntos	$\mathcal{O}(m \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2 \log n)$
Prim, montículo binario	$\mathcal{O}(m \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2 \log n)$
Prim, montículo de Fibonacci	$\mathcal{O}(m + n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$
Prim, arreglo o matriz	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$

Tabla 7: Costo de los algoritmos de árbol de expansión mínima según la densidad del grafo.

En grafos densos el arreglo empata con Fibonacci y tiene constantes mucho menores; en dispersos todos dan $\mathcal{O}(n \log n)$ y deciden las constantes.

Ojito...

El montículo de la biblioteca no trae Decrementar-Llave. `heapq` de Python y `priority_queue` de C++ no decrementan. La salida usual es insertar otra copia (llave, v) y, al extraer, descartar los v que ya están en el árbol; ese descarte es la prueba $v \in Q$ con otro disfraz. El costo es $\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$.

Ojito...

Si G no es conexo, Kruskal devuelve sin quejarse un bosque de expansión mínima, y Prim desde r solo cubre el componente de r: los demás vértices salen de Q con llave ∞. Con grafos dirigidos el capítulo no aplica, porque el corte y el ciclo usan que la arista sirve en los dos sentidos.

17. Caminos más cortos desde una fuente *Complemento: Cormen, cap. 24.1 a 24.4*

Resumen rápido. El problema: un grafo dirigido con pesos reales, que pueden ser negativos, y una fuente s; se quiere el peso $\delta(s, v)$ de un camino más corto a cada vértice, y el camino. La sección fija qué vale δ con ciclos negativos, por qué basta buscar entre caminos y cómo se guardan en un árbol de predecesores. Luego viene la relajación con sus seis propiedades, y sobre ella tres algoritmos que solo difieren en el orden en que relajan: Bellman-Ford, que admite pesos cualesquiera y detecta ciclos negativos en $\Theta(nm)$; el de los grafos acíclicos, en $\Theta(n + m)$, con su aplicación a rutas críticas; y Dijkstra, con su prueba completa y su costo según la cola de prioridad. Cierra con los sistemas de restricciones de diferencia, que Bellman-Ford resuelve.

El problema

La clase del 22 de octubre resolvió dos casos particulares: BFS, cuando todas las aristas valen lo mismo, y Dijkstra, cuando ningún peso es negativo. Aquí va el caso general. En toda la sección $G = (V, E)$ es un grafo dirigido guardado en listas de adyacencia, y cada entrada trae el peso de su arista, así que leer $w(u, v)$ cuesta $\mathcal{O}(1)$.

Definición 17.1 (Peso de una caminata y peso de camino más corto)**.** El **peso** de una caminata $p = \langle v_0, v_1, \ldots, v_k \rangle$ es la suma de los pesos de sus aristas,

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i).$$

El **peso de camino más corto** (*shortest-path weight*) de u a v es

$$\delta(u, v) = \begin{cases} \text{ínf}\,\{w(p) : p \text{ es una caminata de } u \text{ a } v\} & \text{si hay alguna,} \\ \infty & \text{si no.} \end{cases}$$

Un **camino más corto** de u a v es un camino p de u a v con $w(p) = \delta(u, v)$. No tiene por qué ser único.

Observación. Cormen llama *path* a lo que aquí es una caminata y *simple path* a lo que aquí es un camino. El ínfimo se toma sobre caminatas a propósito: si una caminata puede pasar por un ciclo de peso negativo, las que le dan más vueltas pesan cada vez menos, y eso tiene que reflejarse en δ.

Con la fuente única se resuelven las otras variantes: destino único, invirtiendo las aristas; par único, desde u, porque saber el destino no mejora el peor caso; y todos los pares, desde cada vértice, aunque hay métodos mejores (sección 19).

Lema 17.2 (Los subcaminos de un camino más corto son caminos más cortos; Cormen, lema 24.1)**.** *Sea $p = \langle v_0, v_1, \ldots, v_k \rangle$ un camino más corto de v_0 a v_k. Para todo $0 \le i \le j \le k$, el subcamino $p_{ij} = \langle v_i, v_{i+1}, \ldots, v_j \rangle$ es un camino más corto de v_i a v_j.*

Idea de la prueba. Si una caminata p' de v_i a v_j pesara menos que p_{ij}, cambiarla dentro de p daría una caminata de v_0 a v_k de peso menor que $\delta(v_0, v_k)$, imposible. □

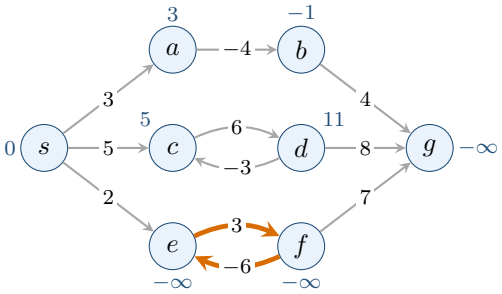
Es la **subestructura óptima** de la programación dinámica, y Cormen apoya en ella tanto a Dijkstra como a Floyd-Warshall.

Pesos negativos y ciclos negativos

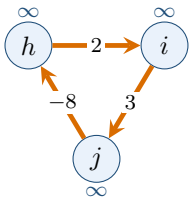
Una arista de peso negativo no rompe nada por sí sola. Lo que rompe la definición es un **ciclo de peso negativo** en alguna caminata de s a v: cada vuelta baja el peso, $\delta(s,v) = -\infty$ y ningún camino es el más corto. Si v no es alcanzable desde s, $\delta(s,v) = \infty$, aunque v esté en un ciclo negativo. Para las cuentas con infinitos se fija

$$a + \infty = \infty \quad (a \neq -\infty), \qquad a + (-\infty) = -\infty \quad (a \neq \infty).$$

Ejemplo 17.3 (Los tres comportamientos; Cormen, figura 24.1)**.** En la figura, junto a cada vértice está $\delta(s,\cdot)$, y en naranja los dos ciclos de peso negativo.



ciclo negativo alcanzable: $3 - 6 = -3$



ciclo negativo no alcanzable

La arista negativa (a,b) sola no estorba, y el ciclo $\langle c,d,c\rangle$ pesa $6-3=3>0$, así que cada vuelta empeora. El ciclo $\langle e,f,e\rangle$ pesa -3 y es alcanzable: e, f y también g quedan en $-\infty$, aunque el camino s,a,b,g pese 3. El ciclo h,i,j también es negativo, pero desde s no se llega. Un ciclo negativo solo contamina lo que está a la vez al alcance de la fuente y río abajo de él.

Proposición 17.4. *Si desde s no se alcanza ningún ciclo de peso negativo, para todo v alcanzable desde s el valor $\delta(s,v)$ es un número real y lo alcanza un camino, que tiene a lo más $n-1$ aristas.*

Idea de la prueba. Una caminata que repite un vértice contiene un ciclo alcanzable, de peso ≥ 0; quitarlo no sube el peso. Repitiendo se llega a un camino, así que el ínfimo es un mínimo sobre la cantidad finita de caminos, y un camino tiene a lo más $n-1$ aristas. □

Ese $n-1$ es exactamente el número de pasadas de Bellman-Ford.

Cómo se guardan los caminos

Igual que BFS, los algoritmos de esta sección guardan en cada vértice un **predecesor** $\pi[v]$, otro vértice o nulo. Al terminar, la cadena $v, \pi[v], \pi[\pi[v]], \ldots$ recorre hacia atrás un camino más corto de s a v y se imprime como en BFS (sección 14). A media ejecución los π no tienen por qué describir caminos más cortos, y con un ciclo negativo ni siquiera forman un árbol (ejemplo 17.16).

Definición 17.5 (Subgrafo predecesor)**.** El **subgrafo predecesor** (*predecessor subgraph*) es $G_\pi = (V_\pi, E_\pi)$, con

$$V_\pi = \{v \in V : \pi[v] \neq \text{nulo}\} \cup \{s\}\,, \qquad E_\pi = \{(\pi[v], v) : v \in V_\pi - \{s\}\}\,.$$

Definición 17.6 (Árbol de caminos más cortos)**.** Si desde s no se alcanza ningún ciclo negativo, un **árbol de caminos más cortos** (*shortest-paths tree*) con raíz s es un subgrafo $G' = (V', E')$, con $V' \subseteq V$ y $E' \subseteq E$, tal que

1. V' es el conjunto de vértices alcanzables desde s;
2. G' es un árbol con raíz s;
3. para todo $v \in V'$, el único camino de s a v en G' es un camino más corto de s a v en G.

Es el árbol BFS con pesos en lugar de número de aristas. No es único: cuando dos caminos empatan, cualquiera de los dos sirve.

Relajación

Todos los algoritmos de la sección comparten el mismo esqueleto. Cada vértice guarda una **estimación** $d[v]$ del peso de camino más corto, que siempre es una cota superior de $\delta(s, v)$, y un predecesor $\pi[v]$. Se arranca con $d[s] = 0$ e ∞ en los demás (Initialize-Single-Source en Cormen):

Algoritmo 17.1 Inicializar-Fuente(G, s)

```
1: procedimiento Inicializar-Fuente(G, s)
2:     para todo v ∈ V hacer
3:         d[v] ← ∞,    π[v] ← nulo
4:     fin para
5:     d[s] ← 0
6: fin procedimiento
```

Después, la única operación que cambia d y π es **relajar** una arista (u, v): si llegar a v pasando por u es más barato que lo que se tiene, se actualizan $d[v]$ y $\pi[v]$ (Relax en Cormen).

Algoritmo 17.2 Relajar(u, v, w)

```
1: procedimiento Relajar(u, v, w)
2:     si d[v] > d[u] + w(u, v) entonces
3:         d[v] ← d[u] + w(u, v),    π[v] ← u
4:     fin si
5: fin procedimiento
```

Inicializar-Fuente cuesta $\Theta(n)$ y Relajar $\mathcal{O}(1)$. Los algoritmos difieren solo en cuántas veces relajan cada arista y en qué orden: Dijkstra y el de grafos acíclicos, una vez cada una; Bellman-Ford, $n - 1$ veces. Que eso baste lo explican seis propiedades.

Las seis propiedades de la relajación

La primera es un hecho sobre δ. Las otras cinco suponen que el grafo se inicializó con Inicializar-Fuente(G, s) y que desde entonces d y π solo cambian por llamadas a Relajar, en cualquier orden. Aquí van los enunciados con su intuición; las pruebas están en la sección 18.

Proposición 17.7 (Desigualdad del triángulo; Cormen, lema 24.10)**.** *Para toda arista $(u, v) \in E$, $\delta(s, v) \leq \delta(s, u) + w(u, v)$.*

Proposición 17.8 (Cota superior; Cormen, lema 24.11)**.** *En todo momento $d[v] \geq \delta(s, v)$ para todo $v \in V$, y en cuanto $d[v]$ alcanza $\delta(s, v)$ ya no cambia.*

Intuición. Cada valor finito de $d[v]$ es el peso de una caminata de verdad de s a v, y relajar solo baja valores.

Proposición 17.9 (Sin camino; Cormen, corolario 24.12)**.** *Si no hay camino de s a v, en todo momento $d[v] = \delta(s, v) = \infty$.*

Proposición 17.10 (Convergencia; Cormen, lema 24.14)**.** *Si $s \rightsquigarrow u \to v$ es un camino más corto y $d[u] = \delta(s, u)$ en algún momento antes de relajar (u, v), entonces $d[v] = \delta(s, v)$ en todo momento después de esa relajación.*

Intuición. Al relajar queda $d[v] \leq d[u] + w(u, v) = \delta(s, u) + w(u, v) = \delta(s, v)$, por el lema 17.2, y la cota superior impide bajar más.

Proposición 17.11 (Relajación de caminos; Cormen, lema 24.15)**.** *Sea $p = \langle v_0, v_1, \ldots, v_k \rangle$ un camino más corto de $s = v_0$ a v_k. Si sus aristas se relajan en el orden $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$, entonces $d[v_k] = \delta(s, v_k)$ desde la última de esas relajaciones en adelante. Vale sin importar qué otras relajaciones se intercalen entre ellas.*

Intuición. Inducción con la convergencia: al relajar (v_{i-1}, v_i), v_{i-1} ya es correcto y v_i queda correcto, y lo que se haga en medio no lo estropea.

Proposición 17.12 (Subgrafo predecesor; Cormen, lema 24.17)**.** *Si desde s no se alcanza ningún ciclo negativo y en algún momento $d[v] = \delta(s, v)$ para todo $v \in V$, entonces G_π es un árbol de caminos más cortos con raíz s.*

Intuición. Sin ciclos negativos una relajación no puede cerrar un ciclo de predecesores, así que G_π es siempre un árbol. Al final cada arista del árbol queda **tensa**, $d[v] = d[\pi[v]] + w(\pi[v], v)$, y sumando a lo largo del camino del árbol los d se cancelan en telescopio: el camino pesa $d[v] - d[s] = \delta(s, v)$.

Idea clave

Un algoritmo de caminos más cortos desde una fuente es una política para escoger qué arista relajar. No necesita conocer los caminos más cortos: si garantiza que las aristas de cada uno se relajen en orden, la relajación de caminos da las distancias y la propiedad del subgrafo predecesor da el árbol. Bellman-Ford lo garantiza a fuerza de repetir, el algoritmo para grafos acíclicos con el orden topológico, y Dijkstra con la cola de prioridad.

Bellman-Ford

Bellman-Ford resuelve el caso general, con pesos cualesquiera y posibles ciclos negativos. Devuelve verdadero si desde s no se alcanza ningún ciclo negativo, y entonces $d[v] = \delta(s, v)$ para todo v y G_π es un árbol de caminos más cortos; si se alcanza alguno devuelve falso. La idea: relajar todas las aristas, $n - 1$ veces, y luego revisar.

Algoritmo 17.3 Bellman-Ford(G, w, s)

```
1:  procedimiento Bellman-Ford(G, w, s)
2:      Inicializar-Fuente(G, s)
3:      para i ← 1 hasta n − 1 hacer                    // n − 1 pasadas
4:          para todo (u, v) ∈ E hacer
5:              Relajar(u, v, w)
6:          fin para
7:      fin para
8:      para todo (u, v) ∈ E hacer                      // revisión
9:          si d[v] > d[u] + w(u, v) entonces
10:             devolver falso                          // hay un ciclo negativo alcanzable
11:         fin si
12:     fin para
13:     devolver verdadero
14: fin procedimiento
```

El orden de las aristas en la línea 4 es arbitrario: no afecta el resultado, solo cuántas pasadas hacen algo de verdad. El $n-1$ es una cota de peor caso, porque cada relajación usa los valores que ya bajaron en la misma pasada.

Lema 17.13 (Cormen, lema 24.2)**.** *Si desde s no se alcanza ningún ciclo negativo, al terminar las $n-1$ pasadas de las líneas 3 a 7 se tiene $d[v] = \delta(s, v)$ para todo v alcanzable desde s.*

Demostración. Sea v alcanzable y sea $p = \langle v_0, v_1, \ldots, v_k \rangle$ un camino más corto de $s = v_0$ a $v = v_k$; existe y tiene $k \leq n-1$ aristas por la proposición 17.4. La pasada i relaja, entre otras, a (v_{i-1}, v_i), así que las aristas de p quedan relajadas en orden, una en cada una de las pasadas $1, \ldots, k$, y la relajación de caminos (proposición 17.11) da $d[v] = \delta(s, v)$. □

Teorema 17.14 (Corrección de Bellman-Ford; Cormen, teorema 24.4)**.** *Si desde s no se alcanza ningún ciclo negativo, Bellman-Ford(G, w, s) devuelve verdadero, $d[v] = \delta(s, v)$ para todo $v \in V$ y G_π es un árbol de caminos más cortos con raíz s. Si desde s se alcanza un ciclo negativo, devuelve falso.*

Demostración. Sin ciclos negativos alcanzables. Para los vértices alcanzables, $d[v] = \delta(s, v)$ es el lema 17.13; para los demás, la propiedad sin camino. Con todas las distancias correctas, la propiedad del subgrafo predecesor da el árbol. Falta ver que la revisión no falla: para toda arista (u, v), por la desigualdad del triángulo,

$$d[v] = \delta(s, v) \leq \delta(s, u) + w(u, v) = d[u] + w(u, v),$$

así que la condición de la línea 9 nunca se cumple y el algoritmo devuelve verdadero.

Con un ciclo negativo alcanzable. Sea $c = \langle v_0, v_1, \ldots, v_k \rangle$, con $v_0 = v_k$, un ciclo alcanzable desde s con

$$\sum_{i=1}^{k} w(v_{i-1}, v_i) < 0.$$

Supongamos que el algoritmo devuelve verdadero. Entonces ninguna arista del ciclo cumplió la condición de la línea 9: $d[v_i] \le d[v_{i-1}] + w(v_{i-1}, v_i)$ para $i = 1, \ldots, k$. Sumando las k desigualdades,

$$\sum_{i=1}^{k} d[v_i] \le \sum_{i=1}^{k} d[v_{i-1}] + \sum_{i=1}^{k} w(v_{i-1}, v_i).$$

Como $v_0 = v_k$, cada vértice del ciclo aparece exactamente una vez en cada una de las dos sumas de d, así que son iguales. Además son finitas. Cada v_i es alcanzable por un camino de a lo más $n-1$ aristas, la pasada j relaja la j-ésima arista de ese camino y d solo baja, así que tras las $n-1$ pasadas $d[v_i] < \infty$; y ningún d vale $-\infty$, porque solo toma valores que son pesos de caminatas. Restando las dos sumas iguales queda $0 \le \sum_{i=1}^{k} w(v_{i-1}, v_i)$, que contradice que el ciclo es negativo. □

Un ciclo negativo no puede tener satisfechas las desigualdades de todas sus aristas: al sumarlas alrededor del ciclo los d se cancelan y queda $0 \le w(c)$. Alguna tiene que fallar, y esa es la que la revisión encuentra.

Costo. Las $n-1$ pasadas recorren las m aristas con relajaciones de $\mathcal{O}(1)$: $\Theta(nm)$, con $\Theta(n)$ de memoria extra. Con listas cada pasada cuesta en rigor $\Theta(n+m)$, que es $\Theta(m)$ cuando todo vértice es alcanzable. En un grafo denso el total es $\Theta(n^3)$: lo mismo que Floyd para *todos* los pares.

Ejemplo 17.15 (Bellman-Ford sobre la figura 24.4 de Cormen)**.** El grafo tiene $V = \{s, t, x, y, z\}$, fuente s y diez aristas:

$$\begin{array}{lllll} (s,t):6 & (s,y):7 & (t,x):5 & (t,y):8 & (t,z):-4 \\ (x,t):-2 & (y,x):-3 & (y,z):9 & (z,s):2 & (z,x):7 \end{array}$$

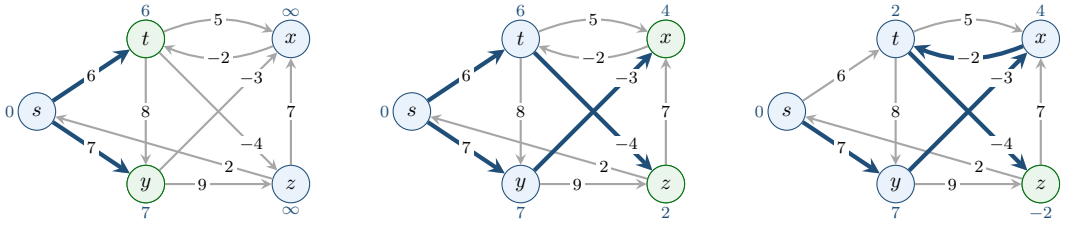
Cada pasada relaja las aristas en el orden de Cormen,

$$(t,x), (t,y), (t,z), (x,t), (y,x), (y,z), (z,x), (z,s), (s,t), (s,y).$$

Las aristas de s van al final a propósito: es un orden malo, que obliga a usar las $n-1=4$ pasadas. Cada celda es $d[v]$ con $\pi[v]$ entre paréntesis, al terminar la pasada.

Pasada	t	x	y	z	**Relajaciones que cambian algo**
inicial	∞	∞	∞	∞	
1	6 (s)	∞	7 (s)	∞	(s,t): $\infty \to 6$; (s,y): $\infty \to 7$
2	6 (s)	4 (y)	7 (s)	2 (t)	(t,x): $\infty \to 11$; (t,z): $\infty \to 2$; (y,x): $11 \to 4$
3	2 (x)	4 (y)	7 (s)	2 (t)	(x,t): $6 \to 2$
4	2 (x)	4 (y)	7 (s)	-2 (t)	(t,z): $2 \to -2$

Tres de las instantáneas. En verde, los vértices cuya d bajó en esa pasada; en azul, las aristas $(\pi[v], v)$.



(a) tras la pasada 1 (b) tras la pasada 2 (c) tras la pasada 4

En la pasada 1 las ocho primeras aristas salen de vértices con $d = \infty$ y no mueven nada. En la pasada 2, (t,x) baja x a 11 y, más adelante en la misma pasada, (y,x) lo baja a 4. En la 3, t mejora por s, y, x, t, y en la 4 z se corrige por segunda vez. La revisión no encuentra nada y devuelve verdadero.

El árbol final es un solo camino, $s \to y \to x \to t \to z$, con $n-1=4$ aristas: el peor caso de la cota. Con las aristas de s al principio converge en tres pasadas; el orden cambia cuántas pasadas sirven, no el resultado: $\delta(s,\cdot) = 0, 2, 4, 7, -2$ para s, t, x, y, z.

Ejemplo 17.16 (El mismo grafo con un ciclo negativo)**.** Se cambia $w(z,x)$ de 7 a 4 (Cormen, ejercicio 24.1-1). Aparece el ciclo $z \to x \to t \to z$ de peso $4-2-4=-2$, alcanzable desde s, y como hay arista (z,s), desde el ciclo se llega a todo: $\delta(s,v) = -\infty$ para todo v, también para $v=s$. Las pasadas 1 a 3 son idénticas a las del ejemplo anterior, porque mientras $d[z] \geq 0$ la arista (z,x) no mejora a x. En la pasada 4, (t,z) baja z a -2 y luego (z,x) da $-2+4=2<4$: queda $d = 0, 2, 2, 7, -2$ para s, t, x, y, z. En la revisión, de las tres desigualdades del ciclo se cumplen las de (z,x), $2 \leq -2+4$, y (t,z), $-2 \leq 2-4$, pero **falla** la de (x,t), $2 \leq 2-2$, y devuelve falso: si las tres se cumplieran, sumándolas quedaría $2 \leq 2+(-2)$. Además, $\pi[t]=x$, $\pi[z]=t$ y $\pi[x]=z$ forman el ciclo $x \to t \to z \to x$: G_π ya no es un árbol. Si se siguiera relajando, los valores bajarían sin fin, 2 cada dos pasadas, y hasta $d[s]$ dejaría de valer 0; por eso el algoritmo se detiene en $n-1$ pasadas y pregunta.

Ojito...

El infinito no es un entero grande. En código es tentador representar ∞ con un centinela como 10^9. Con pesos negativos, $10^9 + (-8) < 10^9$, y un vértice no alcanzable "mejora" a otro no alcanzable: con s aislado y el ciclo h, i, j de la figura 24.1, Bellman-Ford sin protección devuelve falso. La protección es relajar solo si $d[u] \neq \infty$. Si el centinela es el entero más grande del lenguaje, además, la suma se desborda.

Qué detecta. Solo ciclos negativos alcanzables desde s. Para saber si el grafo tiene *alguno* se agrega un vértice nuevo con aristas de peso 0 hacia todos y se corre desde él. Y cuando devuelve falso, las d no significan nada: ni siquiera $d[s]=0$ es confiable.

Grafos no dirigidos. Una arista $\{u,v\}$ de peso negativo son dos arcos que forman el ciclo u, v, u de peso $2w(u,v) < 0$, y Bellman-Ford devuelve falso.

Caminos más cortos en un grafo acíclico

En un grafo dirigido acíclico no hace falta repetir pasadas. Si los vértices se procesan en orden topológico, las aristas de cualquier camino aparecen en orden, y una sola pasada cumple la hipótesis de la relajación de caminos. Los pesos negativos están permitidos: sin ciclos no hay ciclos negativos. El orden topológico se obtiene con DFS, por sale[u] decreciente (sección 15). En Cormen el procedimiento se llama DAG-Shortest-Paths.

Algoritmo 17.4 Caminos-DAG(G, w, s)

```
1: procedimiento Caminos-DAG(G, w, s)
2:     ordenar topológicamente los vértices de G
3:     Inicializar-Fuente(G, s)
4:     para todo u ∈ V, en orden topológico hacer
5:         para todo v en la lista de adyacencia de u hacer
6:             Relajar(u, v, w)
7:         fin para
8:     fin para
9: fin procedimiento
```

Cuando le toca a u, todos los vértices con aristas hacia u ya se procesaron, así que $d[u]$ ya es definitiva: es Dijkstra con el orden conocido de antemano y sin cola de prioridad.

Teorema 17.17 (Cormen, teorema 24.5)**.** *Si G es dirigido, pesado y acíclico, al terminar* Caminos-DAG(G, w, s) *se tiene $d[v] = \delta(s, v)$ para todo $v \in V$, y G_π es un árbol de caminos más cortos con raíz s.*

Idea de la prueba. Los no alcanzables se quedan en ∞ por la propiedad sin camino. Para un v alcanzable, en el orden topológico cada vértice de un camino más corto va antes que el siguiente, así que sus aristas se relajan en orden y la relajación de caminos da $d[v] = \delta(s, v)$. El árbol sale del subgrafo predecesor. □

Costo. El orden topológico cuesta $\Theta(n + m)$ e inicializar $\Theta(n)$. Por análisis agregado, como en BFS, las listas suman m entradas: exactamente m relajaciones de $\mathcal{O}(1)$. Total $\Theta(n + m)$.

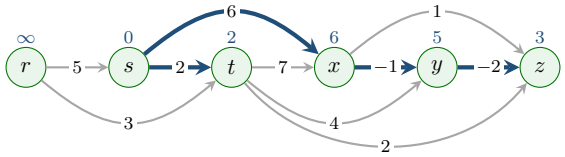
Ejemplo 17.18 (Caminos en el DAG de la figura 24.5 de Cormen)**.** El grafo tiene $V = \{r, s, t, x, y, z\}$, ya en orden topológico (todas las aristas van de izquierda a derecha), fuente s y diez aristas:

$$\begin{array}{lllll} (r,s):5 & (r,t):3 & (s,t):2 & (s,x):6 & (t,x):7 \\ (t,y):4 & (t,z):2 & (x,y):-1 & (x,z):1 & (y,z):-2 \end{array}$$

Una fila por vértice procesado, con las d al terminar de relajar sus aristas:

u	r	s	t	x	y	z	**Relajaciones**
inicial	∞	0	∞	∞	∞	∞	
r	∞	0	∞	∞	∞	∞	$d[r] = \infty$: (r,s) y (r,t) no cambian nada
s	∞	0	2	6	∞	∞	t: $\infty \to 2$; x: $\infty \to 6$
t	∞	0	2	6	6	4	x: $2 + 7 = 9$, no; y: $\infty \to 6$; z: $\infty \to 4$
x	∞	0	2	6	5	4	y: $6 - 1 = 5 < 6$; z: $6 + 1 = 7$, no
y	∞	0	2	6	5	3	z: $5 - 2 = 3 < 4$
z	∞	0	2	6	5	3	sin aristas de salida

El estado final, con las aristas $(\pi[v], v)$ en azul:



r va antes que s y no es alcanzable: sus aristas se relajan sin efecto. y baja a 6 por t y luego a 5 por x; z baja a 4 y luego a 3. Los pesos negativos no estorban: cuando le toca a cada vértice, ya le llegó todo lo que le podía llegar. Se hicieron exactamente $10 = m$ relajaciones.

Ruta crítica

En un **diagrama PERT** las aristas son trabajos con su duración y los vértices, eventos: el trabajo (u, v) tiene que terminar antes de que empiece cualquier (v, x). Una **ruta crítica** (*critical path*) es un camino *más largo* del DAG, una cota inferior de la duración del proyecto aunque todo lo demás se haga en paralelo. Se calcula en $\Theta(n + m)$ negando todos los pesos y corriendo Caminos-DAG, o cambiando ∞ por $-\infty$ en Inicializar-Fuente y $>$ por $<$ en Relajar. Funciona porque en un DAG negar los pesos no crea ciclos negativos; en un grafo con ciclos el camino más largo es NP-difícil.

Ojito...

El DAG tiene que ser DAG. Si el grafo tiene un ciclo no hay orden topológico, pero el orden por sale[u] decreciente se calcula igual sin quejarse, a menos que se revisen las aristas de retroceso, y Caminos-DAG devuelve basura sin avisar. A diferencia de Bellman-Ford, no tiene revisión final.

Dijkstra, versión de Cormen

La clase del 22 de octubre presentó Dijkstra con un ejemplo completo (sección 11). Aquí se reescribe con Inicializar-Fuente y Relajar, como Cormen, para probarlo con las seis propiedades. La hipótesis es $w(u, v) \geq 0$.

Algoritmo 17.5 Dijkstra(G, w, s), versión de Cormen

```
1:  procedimiento DIJKSTRA(G, w, s)
2:      INICIALIZAR-FUENTE(G, s)
3:      S ← ∅
4:      Q ← V                                   // cola de mínimos con llave d
5:      mientras Q ≠ ∅ hacer
6:          u ← EXTRAER-MIN(Q)
7:          S ← S ∪ {u}                         // aquí d[u] = δ(s, u)
8:          para todo v en la lista de adyacencia de u hacer
9:              RELAJAR(u, v, w)                // si baja d[v]: DECREMENTAR-LLAVE
10:         fin para
11:     fin mientras
12: fin procedimiento
```

El **mientras** da exactamente n vueltas, y S, que es $V - Q$, solo está para la prueba. En la línea 9 cada relajación exitosa es un Decrementar-Llave, y con $w \geq 0$ nunca le pega a un vértice de S: si $v \in S$, por el teorema de abajo y la desigualdad del triángulo, $d[v] = \delta(s, v) \leq d[u] + w(u, v)$. Con pesos negativos esto deja de ser cierto (ejemplo 17.21).

Teorema 17.19 (Corrección de Dijkstra; Cormen, teorema 24.6)**.** *Si $G = (V, E)$ es dirigido, $w(u, v) \geq 0$ para toda arista y $s \in V$, Dijkstra(G, w, s) termina con $d[u] = \delta(s, u)$ para todo $u \in V$.*

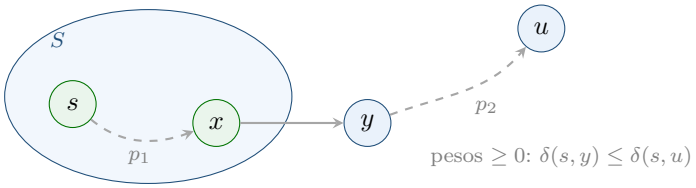
Demostración. Se prueba el invariante: *al inicio de cada vuelta del **mientras**, $d[v] = \delta(s, v)$ para todo $v \in S$.*

Inicio. $S = \emptyset$ y no hay nada que probar.

Mantenimiento. Basta ver que el vértice u que entra a S en la línea 7 ya trae $d[u] = \delta(s, u)$; después no cambia, por la cota superior. Supongamos lo contrario, y sea u el primer vértice que entra a S con $d[u] \neq \delta(s, u)$.

- $u \neq s$. El primero en salir es s, con $d[s] = 0 = \delta(s, s)$; como no hay pesos negativos, ninguna caminata cerrada pesa menos que 0. En particular, justo antes de agregar u, $s \in S$.
- Hay un camino de s a u. Si no, la propiedad sin camino daría $d[u] = \infty = \delta(s, u)$.
- Sea p un camino más corto de s a u; existe por la proposición 17.4, porque sin pesos negativos no hay ciclos negativos. p empieza en $s \in S$ y termina en $u \notin S$. Sea y el primer vértice de p

que no está en S y $x \in S$ el anterior a él en p. Puede ser $x = s$ o $y = u$. El camino se parte en $s \overset{p_1}{\rightsquigarrow} x \to y \overset{p_2}{\rightsquigarrow} u$.



Primero, $d[y] = \delta(s, y)$ en el momento en que u entra a S. En efecto, x entró a S antes que u, así que por la elección de u entró con $d[x] = \delta(s, x)$, y en esa vuelta se relajó (x, y). Como $s \overset{p_1}{\rightsquigarrow} x \to y$ es un camino más corto (lema 17.2), la convergencia da $d[y] = \delta(s, y)$ desde entonces.

Segundo, y va antes que u en p y p_2 no tiene pesos negativos, así que $\delta(s, y) = w(p_1) + w(x, y) \leq w(p) = \delta(s, u)$. Con la cota superior,

$$d[y] = \delta(s, y) \leq \delta(s, u) \leq d[u].$$

Pero cuando se eligió u en la línea 6, y también estaba en Q, así que $d[u] \leq d[y]$. Todas las desigualdades son igualdades, y $d[u] = \delta(s, u)$, lo que contradice la elección de u.

Terminación. El ciclo acaba con $Q = \emptyset$, y como $Q = V - S$ al inicio de cada vuelta, $S = V$: todos los vértices tienen $d[v] = \delta(s, v)$. □

Corolario 17.20 (Cormen, corolario 24.7)**.** *Con las mismas hipótesis, al terminar G_π es un árbol de caminos más cortos con raíz s, por la propiedad del subgrafo predecesor.*

La prueba usa $w \geq 0$ en un solo lugar, la desigualdad $\delta(s, y) \leq \delta(s, u)$. Si el tramo p_2 resta, y puede estar más lejos que u aunque vaya antes en el camino, y el argumento se cae justo ahí.

Costo según la cola de prioridad

Fuera de la cola, todo cuesta $\mathcal{O}(1)$ por vértice o por arista. Hay n inserciones y n extracciones. Los decrementos se cuentan por análisis agregado: la lista de cada u se recorre una sola vez, en la vuelta en que sale u, así que hay exactamente m relajaciones y a lo más m decrementos. Contar por vuelta daría $\mathcal{O}(n^2)$, mucho peor en un grafo disperso. El total es $T = T_{\text{construir}} + n \cdot T_{\text{extraer}} + m \cdot T_{\text{decrementar}}$:

Cola	**Insertar** (n veces)	**Extraer-Min** (n veces)	**Decrementar** ($\leq m$ veces)	**Total**
arreglo por vértice	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$	$\mathcal{O}(n^2 + m) = \mathcal{O}(n^2)$
montículo binario	$\mathcal{O}(n)$ en total	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}((n + m) \log n)$
montículo de Fibonacci	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$ amort.	$\mathcal{O}(1)$ amort.	$\mathcal{O}(n \log n + m)$

Con el montículo binario, para decrementar cada vértice guarda su posición y el montículo la actualiza en cada intercambio. El binario le gana al arreglo cuando $m = o(n^2/\log n)$; en un grafo denso el arreglo gana. Fibonacci nunca pierde asintóticamente, pero sus constantes son grandes, y en la práctica casi siempre se usa el binario. Prim tiene la misma forma, solo cambia la llave, $w(u, v)$ en lugar de $d[u] + w(u, v)$ (sección 16), y por eso cuesta lo mismo con cada cola.

Ejemplo 17.21 (Dijkstra falla con una arista negativa)**.** Grafo dirigido con $A \to B$ de peso 2, $A \to C$ de peso 3, $C \to B$ de peso -2 y $B \to D$ de peso 2, fuente A. Las distancias reales son $\delta(A, B) = 1$ por A, C, B y $\delta(A, D) = 3$ por A, C, B, D. Así corre el pseudocódigo de arriba; en negritas, los vértices de S.

Iter.	Sale	A	B	C	D
0		0	∞	∞	∞
1	A	**0**	2	3	∞
2	B	**0**	**2**	3	4
3	C	**0**	**1**	**3**	4
4	D	**0**	**1**	**3**	**4**
real		0	1	3	3

B sale en la iteración 2 con $d = 2$ y relaja $B \to D$: D queda en 4. En la iteración 3 sale C y la relajación $C \to B$ baja $d[B]$ a $3 - 2 = 1$, pero B ya está en S y nadie vuelve a relajar $B \to D$. D sale con 4 y la respuesta es incorrecta. En la prueba del teorema, cuando sale B el camino más corto hacia él es A, C, B; ahí $y = C$, con $d[C] = \delta(A, C) = 3$, pero $\delta(A, B) = 1 < 3$ porque $p_2 = (C, B)$ pesa -2.

Ojito...

Sumar una constante no arregla nada. Sumar a cada peso lo suficiente para que ninguno sea negativo castiga más a los caminos con más aristas. En el ejemplo, sumando 2, $A \to B$ pesa 4 y A, C, B pesa $5 + 0 = 5$, y el camino más corto a B cambia. La forma correcta es la repesada de Johnson (sección 19).

Montículo sin decremento. Bibliotecas como `heapq` de Python no tienen Decrementar-Llave. Lo usual es meter un par $(d[v], v)$ nuevo cada vez que baja $d[v]$ e ignorarlo al sacarlo si v ya está en S. El costo es $\mathcal{O}(m \log m) = \mathcal{O}(m \log n)$.

Restricciones de diferencia

De un programa lineal $Ax \leq b$ a veces solo interesa la **factibilidad**: encontrar algún x o decidir que no existe. Este caso particular se resuelve con un algoritmo de grafos.

Definición 17.22 (Sistema de restricciones de diferencia)**.** Un **sistema de restricciones de diferencia** (*system of difference constraints*) es un programa lineal $Ax \leq b$ en el que cada renglón de A tiene un 1, un -1 y ceros. Con incógnitas $x_1, \ldots, x_p$ y r restricciones, el renglón k dice

$$x_j - x_i \leq b_k, \qquad 1 \leq i, j \leq p,\ i \neq j,\ 1 \leq k \leq r.$$

Cormen llama n y m a las incógnitas y las restricciones; aquí son p y r, porque el grafo que se va a construir tendrá $n = p + 1$ vértices y $m = p + r$ aristas. En calendarización, con x_i el inicio de la tarea i, "la tarea j empieza al menos c unidades después de la i" es $x_i - x_j \leq -c$: lado derecho negativo.

Lema 17.23 (Traslación; Cormen, lema 24.8)**.** *Si $x = (x_1, \ldots, x_p)$ es solución de un sistema de restricciones de diferencia y a es cualquier constante, $x + a = (x_1 + a, \ldots, x_p + a)$ también lo es.*

Demostración. $(x_j + a) - (x_i + a) = x_j - x_i$. □

La clave es de forma: $x_j - x_i \leq b_k$ se lee $x_j \leq x_i + b_k$, la desigualdad del triángulo con $w(v_i, v_j) = b_k$.

Definición 17.24 (Grafo de restricciones)**.** El **grafo de restricciones** (*constraint graph*) del sistema es el grafo dirigido y pesado con vértices $V = \{v_0, v_1, \ldots, v_p\}$, uno por incógnita y uno extra v_0, y aristas

- (v_i, v_j) con peso b_k por cada restricción $x_j - x_i \leq b_k$;
- (v_0, v_i) con peso 0, para $i = 1, \ldots, p$.

La arista va de la variable que resta a la que suma. v_0 llega a todos, así que cualquier ciclo negativo del grafo es alcanzable desde él.

Teorema 17.25 (Cormen, teorema 24.9)**.** *Sea G el grafo de restricciones de un sistema de restricciones de diferencia. Si G no tiene ciclos de peso negativo,*

$$x = (\delta(v_0, v_1), \delta(v_0, v_2), \ldots, \delta(v_0, v_p))$$

es una solución. Si G tiene un ciclo de peso negativo, el sistema no tiene solución.

Idea de la prueba. Sin ciclos negativos, cada $\delta(v_0, v_i)$ es finito y la desigualdad del triángulo sobre la arista (v_i, v_j) es exactamente $x_j - x_i \leq b_k$. Con un ciclo negativo, que no pasa por v_0, sumar sus restricciones cancela las incógnitas y deja $0 \leq w(c) < 0$. □

Para resolver el sistema basta correr Bellman-Ford desde v_0; Dijkstra no sirve, porque los b_k pueden ser negativos. Por el teorema 17.14, devuelve falso exactamente cuando hay un ciclo negativo. Cada restricción se da como una terna (i, j, b), que significa $x_j - x_i \leq b$.

Algoritmo 17.6 Resolver-Restricciones(p, R)

```
1:  procedimiento RESOLVER-RESTRICCIONES(p, R)
2:      V ← {v_0, v_1, ..., v_p},    E ← ∅
3:      para i ← 1 hasta p hacer
4:          agregar (v_0, v_i) a E con peso 0
5:      fin para
6:      para todo (i, j, b) ∈ R hacer
7:          agregar (v_i, v_j) a E con peso b
8:      fin para
9:      si BELLMAN-FORD(G, w, v_0) entonces
10:         devolver (d[v_1], d[v_2], ..., d[v_p])
11:     si no
12:         devolver no hay solución          // hay un ciclo negativo
13:     fin si
14: fin procedimiento
```

Costo. Construir el grafo cuesta $\Theta(p + r)$ y Bellman-Ford $\Theta(nm) = \Theta((p + 1)(p + r)) = \Theta(p^2 + pr)$. El término p^2 lo ponen las aristas de v_0, que no aportan nada después de la primera pasada; una variante que no construye v_0 cuesta $\mathcal{O}(p + pr)$.

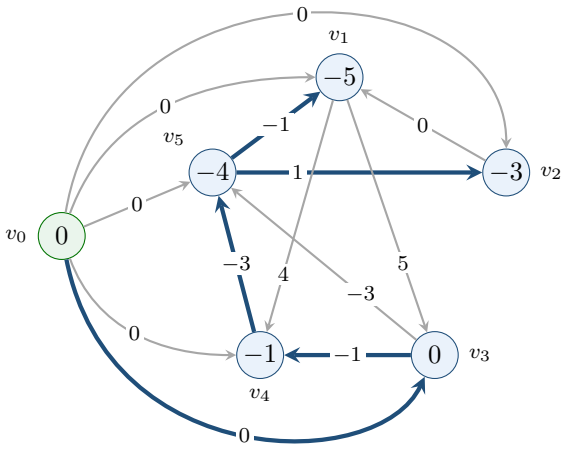
Ejemplo 17.26 (El sistema de la figura 24.8 de Cormen)**.** Cinco incógnitas y ocho restricciones:

$$\begin{array}{llll} x_1 - x_2 \leq 0 & x_1 - x_5 \leq -1 & x_2 - x_5 \leq 1 & x_3 - x_1 \leq 5 \\ x_4 - x_1 \leq 4 & x_4 - x_3 \leq -1 & x_5 - x_3 \leq -3 & x_5 - x_4 \leq -3 \end{array}$$

Bellman-Ford desde v_0, relajando primero las cinco aristas de v_0 y luego las ocho restricciones en el orden de arriba:

	$d[v_1]$	$d[v_2]$	$d[v_3]$	$d[v_4]$	$d[v_5]$
inicio	∞	∞	∞	∞	∞
pasada 1	-1	0	0	-1	-4
pasada 2	-5	-3	0	-1	-4
pasadas 3 a 5	sin cambios				

Al final ninguna arista se puede relajar, Bellman-Ford devuelve verdadero y la solución es $x = (-5, -3, 0, -1, -4)$.



Dentro de cada vértice, $\delta(v_0, v_i)$; en azul, el árbol de caminos más cortos, cuyas aristas son restricciones que se cumplen con igualdad. Sumando 5 a todo sale $x' = (0, 2, 5, 4, 1)$, que también es solución y tiene tiempos no negativos.

Ojito...

La dirección de la arista. $x_j - x_i \leq b$ da la arista (v_i, v_j), no (v_j, v_i). Si se invierten todas, lo que sale es $-x$ para una solución x; mezclar las dos convenciones da basura.

Restricciones de una sola variable, como $x_i \leq b$ (Cormen, ejercicio 24.4-10). Usar v_0 como el cero agregaría a escondidas $x_i \leq 0$, por sus aristas de peso 0. Lo correcto es una incógnita nueva z que haga de cero, con $x_i - z \leq b$, resolver y devolver $x_i - z$, que es solución por el lema 17.23.

18. Demostraciones y problemas de caminos más cortos

Complemento: Cormen, cap. 24.5 y problemas

Resumen rápido. Bellman-Ford, el algoritmo para DAG y Dijkstra solo inicializan y relajan; cambian cuántas veces relajan cada arista y en qué orden. Su corrección se parte en dos capas: seis propiedades de la relajación que valen para *cualquier* secuencia de relajaciones, y un argumento corto por algoritmo de que su orden cumple la hipótesis de alguna. Aquí se demuestran las seis propiedades enunciadas en la sección 17, en orden de dependencia, con el lema auxiliar de la convergencia y la prueba de que el subgrafo predecesor es siempre un árbol, más un ejemplo de relajaciones intercaladas y una tabla de qué propiedad usa cada algoritmo. La segunda mitad aplica las mismas herramientas a los problemas del capítulo 24 de Cormen: la mejora de Yen, las cajas anidadas, el arbitraje, el ciclo de peso medio mínimo de Karp y los caminos bitónicos.

Las reglas del juego

Todo ocurre en un grafo dirigido $G = (V, E)$ con pesos $w : E \to \mathbb{R}$ y una fuente s. Inicializar-Fuente(G, s) (Algoritmo 17.1) pone $d[v] = \infty$ y $\pi[v] =$ nulo en todos los vértices y luego $d[s] = 0$. Relajar(u, v, w) (Algoritmo 17.2) pregunta si $d[v] > d[u] + w(u, v)$ y, si es así, asigna $d[v] \leftarrow d[u] + w(u, v)$ y $\pi[v] \leftarrow u$ en el mismo renglón. De ese pseudocódigo salen tres hechos que las pruebas usan sin parar:

(i) Relajar solo baja estimaciones: $d[v]$ nunca sube.

(ii) $d[v]$ y $\pi[v]$ cambian juntos. Por eso, si $\pi[v] = u$, la última relajación que cambió $d[v]$ fue Relajar(u, v, w).

(iii) Con $d[u] = \infty$ no pasa nada, porque $\infty + w(u, v) = \infty$. Por eso $\pi[v]$ solo recibe vértices con d finita, y un vértice $v \neq s$ tiene $d[v]$ finita si y solo si $\pi[v]$ no es nulo.

Salvo la desigualdad del triángulo, que habla solo de δ, todos los resultados suponen lo mismo: *el grafo se inicializó con Inicializar-Fuente(G, s) y desde entonces d y π solo cambiaron por llamadas a Relajar sobre aristas de G*, en cualquier orden y cuantas veces se quiera. Lo que se pruebe vale para cualquier algoritmo que solo relaje.

De la sección 17 se usan las definiciones de δ, con la aritmética de los infinitos, de camino más corto, de G_π y de árbol de caminos más cortos; que cuando $\delta(s, v)$ es finita la alcanza un camino de a lo más $n - 1$ aristas; y el lema de subcaminos (Cormen, lema 24.1). La figura 8 es el mapa de lo que sigue: cada flecha se lee "se usa para probar"; en azul, las seis propiedades, y en blanco, los auxiliares.

Triángulo, cota superior y sin camino

Lema 18.1 (Desigualdad del triángulo; Proposición 17.7)**.** *Para toda arista $(u, v) \in E$,*

$$\delta(s, v) \leq \delta(s, u) + w(u, v).$$

Demostración. Ir a u por el mejor camino y luego tomar (u, v) es una manera de llegar a v. Los infinitos obligan a separar tres casos.

$\delta(s, u) = \infty$. El lado derecho vale ∞.

$\delta(s, u)$ finito. A un camino más corto de s a u se le pega (u, v): queda una caminata de s a v que pesa $\delta(s, u) + w(u, v)$, y $\delta(s, v)$ es el mínimo sobre todas las caminatas.

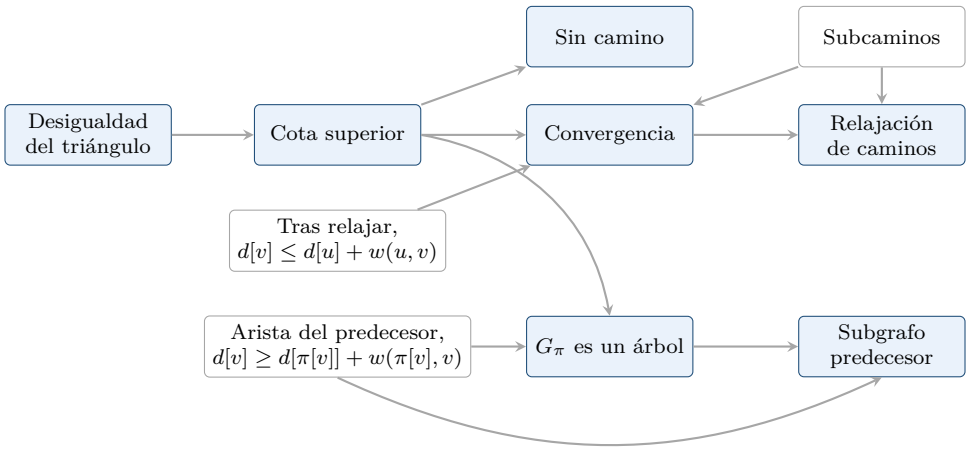


Figura 8: De qué depende cada propiedad de la relajación.

$\delta(s,u) = -\infty$**.** Hay caminatas a u tan negativas como se quiera; pegándoles (u,v) salen caminatas a v igual de negativas, salvo la constante $w(u,v)$. Entonces $\delta(s,v) = -\infty$. □

La desigualdad es sobre las distancias, no sobre los pesos: las aristas no tienen que cumplir ninguna desigualdad del triángulo.

Lema 18.2 (Cota superior; Proposición 17.8)**.** *Después de* Inicializar-Fuente(G,s) *se cumple* $d[v] \ge \delta(s,v)$ *para todo* $v \in V$*, y la desigualdad se mantiene tras cualquier secuencia de relajaciones. Además, en cuanto* $d[v]$ *alcanza* $\delta(s,v)$*, ya no cambia.*

Demostración. La primera parte es un invariante, por inducción sobre el número de llamadas a Relajar, una por una.

Base. Recién inicializado, $d[v] = \infty \ge \delta(s,v)$ para todo $v \ne s$, y $d[s] = 0 \ge \delta(s,s)$, porque la caminata vacía de s a s pesa 0.

Paso inductivo. Se llama Relajar(u,v,w) y, por hipótesis de inducción, $d[x] \ge \delta(s,x)$ para todo x antes de la llamada. La única estimación que puede cambiar es $d[v]$, y si cambia,

$$\begin{aligned} d[v] &= d[u] + w(u,v) \\ &\ge \delta(s,u) + w(u,v) \quad \textit{(hipótesis de inducción)} \\ &\ge \delta(s,v) \quad \textit{(Lema 18.1).} \end{aligned}$$

Una vez alcanzada, no cambia. Si $d[v] = \delta(s,v)$, no puede bajar, por el invariante, y no puede subir, por el hecho (i). □

La segunda mitad es la pieza que más se usa: vuelve inofensivas las relajaciones de más, porque una vez que un vértice tiene su distancia, nada de lo que pase después se la quita.

Corolario 18.3 (Sin camino; Proposición 17.9)**.** *Si no hay camino de* s *a* v*, entonces* $d[v] = \delta(s,v) = \infty$ *después de inicializar y después de cualquier secuencia de relajaciones.*

Demostración. Por la cota superior, $\infty = \delta(s,v) \le d[v]$. □

Convergencia y relajación de caminos

La convergencia necesita saber qué deja una relajación justo después de hacerse. Es el lema auxiliar de Cormen (lema 24.13).

Lema 18.4 (Lo que deja una relajación)**.** *Para una arista (u, v) con $u \neq v$, inmediatamente después de* Relajar(u, v, w) *se cumple $d[v] \leq d[u] + w(u, v)$.*

Demostración. Si antes de la llamada $d[v] > d[u]+w(u, v)$, la llamada asigna justo $d[v] = d[u]+w(u, v)$. Si no, ya se cumplía y la llamada no toca nada. La condición $u \neq v$ excluye lazos, que un camino nunca usa: con un lazo negativo la asignación bajaría $d[u]$ junto con $d[v]$. □

Lema 18.5 (Convergencia; Proposición 17.10)**.** *Sea $s \rightsquigarrow u \to v$ un camino más corto de s a v. Si $d[u] = \delta(s, u)$ en algún momento antes de llamar a* Relajar(u, v, w)*, entonces $d[v] = \delta(s, v)$ en todo momento después de esa llamada.*

Demostración. Por la cota superior, $d[u]$ sigue valiendo $\delta(s, u)$ cuando se relaja (u, v). Justo después de esa llamada,

$$\begin{aligned} d[v] &\leq d[u] + w(u, v) \quad \textit{(Lema 18.4)} \\ &= \delta(s, u) + w(u, v) \\ &= \delta(s, v) \quad \textit{(subcaminos)}, \end{aligned}$$

porque el tramo $s \rightsquigarrow u$ es un subcamino del camino más corto y pesa $\delta(s, u)$. La cota superior da $d[v] \geq \delta(s, v)$, así que $d[v] = \delta(s, v)$, y por su segunda mitad se queda así. □

Lema 18.6 (Relajación de caminos; Proposición 17.11)**.** *Sea $p = \langle v_0, v_1, \ldots, v_k \rangle$ un camino más corto de $s = v_0$ a v_k. Si después de* Inicializar-Fuente(G, s) *ocurre una secuencia de relajaciones que incluye, en este orden, a $(v_0, v_1), (v_1, v_2), \ldots, (v_{k-1}, v_k)$, entonces $d[v_k] = \delta(s, v_k)$ al terminar esas relajaciones y en todo momento posterior. No importa qué otras relajaciones haya ni cómo se intercalen con las de p.*

Demostración. Por inducción sobre i: a partir de la relajación de (v_{i-1}, v_i) se cumple $d[v_i] = \delta(s, v_i)$ (para $i = 0$, desde el principio).

Base, $i = 0$. $d[s] = 0$. Falta ver que $\delta(s, s) = 0$, un detalle que Cormen da por hecho: si un ciclo negativo pasara por s, darle vueltas antes de seguir p abarataría sin límite las caminatas a v_k, y no habría camino más corto a v_k. Por la cota superior, $d[s]$ ya no cambia.

Paso inductivo. $d[v_{i-1}] = \delta(s, v_{i-1})$ desde que se relajó (v_{i-2}, v_{i-1}), y las relajaciones intercaladas no lo rompen, por la cota superior. Cuando le toca a (v_{i-1}, v_i), el tramo $\langle v_0, \ldots, v_i \rangle$ es un subcamino de p y, por lo tanto, un camino más corto; la convergencia da $d[v_i] = \delta(s, v_i)$ desde ese momento. Con $i = k$ queda el enunciado. □

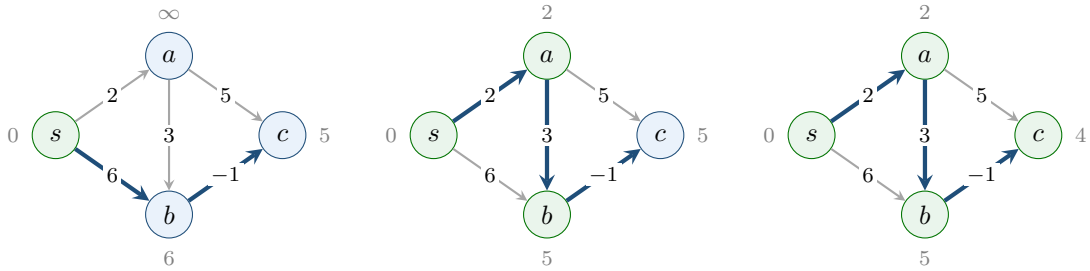
Idea clave

Lo único que importa es una *subsecuencia*: que las aristas de un camino más corto aparezcan en orden dentro de la secuencia de relajaciones. Lo que haya entre ellas no estorba, por la cota superior, y lo que se relaje antes de tiempo no cuenta.

Ejemplo 18.7 (Relajación de caminos con relajaciones intercaladas)**.** Un grafo chico, con una arista negativa a propósito para que se vea que la propiedad no depende de Dijkstra. $V = \{s, a, b, c\}$ y $w(s,a) = 2$, $w(s,b) = 6$, $w(a,b) = 3$, $w(a,c) = 5$, $w(b,c) = -1$. Las distancias son $\delta(s,a) = 2$, $\delta(s,b) = 5$ y $\delta(s,c) = 4$, y el camino más corto a c es $p = \langle s, a, b, c\rangle$. Se relajan siete aristas en un orden cualquiera que contiene a las de p en orden (pasos 4, 6 y 7), mezcladas con otras:

Paso	**Arista**	$d[s]$	$d[a]$	$d[b]$	$d[c]$	**¿Cambia?**	**Comentario**
0		0	∞	∞	∞		inicialización
1	(b,c)	0	∞	∞	∞	no	$\infty + (-1) = \infty$
2	(s,b)	0	∞	6	∞	sí	$\pi[b] = s$
3	(b,c)	0	∞	6	5	sí	$\pi[c] = b$; $5 > \delta(s,c)$
4	(s,a)	0	**2**	6	5	sí	primera de p: a converge
5	(a,c)	0	2	6	5	no	$2 + 5 = 7 > 5$
6	(a,b)	0	2	**5**	5	sí	segunda de p: b converge, $\pi[b] = a$
7	(b,c)	0	2	5	**4**	sí	tercera de p: c converge

En verde, los vértices con $d[v] = \delta(s,v)$; las flechas azules son las aristas $(\pi[v], v)$ de G_π, y el número junto a cada vértice es $d[v]$.



Tras el paso 3, $d[c] = 5$ es el peso de un camino real, $s \to b \to c$, pero no del más corto: la cota superior se cumple y nada más. Tras el paso 6, b ya convergió, pero c no se entera hasta que se vuelve a relajar (b,c); la relajación del paso 3 se hizo cuando $d[b]$ todavía no era correcta y no cuenta. En el paso 7 se completa la subsecuencia y c queda en 4 para siempre. Los pasos 1, 3 y 5 sobran, pero tampoco estorban.

El mismo grafo explica por qué Bellman-Ford hace $n - 1$ pasadas y al DAG le basta una. Si cada pasada relaja las aristas en el orden (b,c), (a,b), (s,a), (a,c), (s,b), que recorre p al revés, cada pasada avanza una sola arista de p: a converge en la pasada 1, b en la 2 y c en la 3, el peor caso de $n-1$. Como s, a, b, c es un orden topológico, Caminos-DAG relaja (s,a), (s,b), (a,b), (a,c), (b,c) y termina en una pasada con $d = (0, 2, 5, 4)$: en un orden topológico las aristas de *todo* camino se relajan en orden.

El subgrafo predecesor

Las dos pruebas que siguen se apoyan en una desigualdad que vale en todo momento para las aristas de G_π; en Cormen aparece dentro de la prueba del lema 24.16.

Lema 18.8 (Arista del predecesor)**.** *En todo momento, para cada vértice v con $\pi[v]$ no nulo, $d[\pi[v]]$ es finita y*

$$d[v] \geq d[\pi[v]] + w(\pi[v], v).$$

Demostración. Sea $u = \pi[v]$. Por el hecho (ii), la última relajación que cambió $d[v]$ fue Relajar(u, v, w), y dejó $d[v] = d[u] + w(u,v)$ con el valor que $d[u]$ tenía entonces, que era finito por el hecho (iii). Desde

entonces $d[v]$ no ha cambiado y $d[u]$ solo pudo bajar, por el hecho (i). □

Lema 18.9 (G_π es un árbol)**.** *Supóngase que G no tiene ciclos de peso negativo alcanzables desde s. Después de* Inicializar-Fuente(G, s)*, el subgrafo predecesor G_π es un árbol con raíz s, y cualquier secuencia de relajaciones mantiene esa propiedad.*

Demostración. Al inicio $V_\pi = \{s\}$ y $E_\pi = \emptyset$. Hay que ver dos cosas después de cada relajación: que G_π no tiene ciclos y que de s hay un único camino a cada vértice de V_π.

G_π no tiene ciclos. Supóngase que alguna relajación cierra un ciclo $c = \langle v_0, v_1, \ldots, v_k\rangle$ en G_π, con $v_k = v_0$ y $\pi[v_i] = v_{i-1}$, y que la llamada que lo cierra es Relajar(v_{k-1}, v_k, w). Justo antes de esa llamada, para $i = 1, \ldots, k-1$ el apuntador $\pi[v_i] = v_{i-1}$ ya estaba puesto, y el lema 18.8 da

$$d[v_i] \geq d[v_{i-1}] + w(v_{i-1}, v_i), \qquad i = 1, \ldots, k-1.$$

En la arista que cierra el ciclo la desigualdad es estricta, porque la llamada sí cambia $\pi[v_k]$: $d[v_k] > d[v_{k-1}] + w(v_{k-1}, v_k)$. Las k estimaciones son finitas. Sumando las k desigualdades,

$$\sum_{i=1}^{k} d[v_i] > \sum_{i=1}^{k} d[v_{i-1}] + \sum_{i=1}^{k} w(v_{i-1}, v_i).$$

Como $v_0 = v_k$, las dos sumas de estimaciones recorren los mismos vértices y se cancelan:

$$0 > \sum_{i=1}^{k} w(v_{i-1}, v_i) = w(c).$$

El ciclo c es negativo, y es alcanzable desde s, porque por la cota superior $\delta(s, v_i) \leq d[v_i] < \infty$. Eso contradice la hipótesis.

Hay un único camino de s a cada vértice de V_π. $\pi[s]$ sigue siendo nulo: cambiarlo exigiría bajar $d[s]$ de 0, y la cota superior lo impide, porque sin ciclos negativos alcanzables $\delta(s, s) = 0$. Para $v \in V_\pi$, la cadena $v, \pi[v], \pi[\pi[v]], \ldots$ no sale de V_π, porque cada eslabón tiene d finita (lema 18.8 y hecho (iii)), y no repite vértices, porque G_π no tiene ciclos. Entonces se detiene en el único vértice de V_π con π nulo, s, y leída al revés es un camino de s a v. Es el único: a cada $z \neq s$ de V_π le entra exactamente una arista, $(\pi[z], z)$, y a s ninguna, así que todo camino de s a v está forzado a ser la cadena. □

Lema 18.10 (Subgrafo predecesor; Proposición 17.12)**.** *Supóngase que G no tiene ciclos negativos alcanzables desde s. Si después de* Inicializar-Fuente(G, s) *una secuencia de relajaciones deja $d[v] = \delta(s, v)$ para todo $v \in V$, entonces G_π es un árbol de caminos más cortos con raíz s.*

Demostración. Hay que comprobar las tres condiciones de la definición.

(1) V_π son los alcanzables. Sin ciclos negativos alcanzables, $\delta(s, v)$ es finita si y solo si v es alcanzable. Por el hecho (iii), V_π es el conjunto de vértices con d finita, y como $d = \delta$, son los alcanzables.

(2) Es un árbol con raíz s. Es el lema 18.9.

(3) Sus caminos son caminos más cortos. Sea $p = \langle v_0, \ldots, v_k\rangle$ el camino de $s = v_0$ a $v = v_k$ en G_π. Para cada i, $\pi[v_i] = v_{i-1}$, y el lema 18.8 con $d = \delta$ dice $w(v_{i-1}, v_i) \leq \delta(s, v_i) - \delta(s, v_{i-1})$. Al sumar,

el lado derecho se telescopa:

$$w(p) = \sum_{i=1}^{k} w(v_{i-1}, v_i) \leq \sum_{i=1}^{k} \big(\delta(s, v_i) - \delta(s, v_{i-1})\big)$$
$$= \delta(s, v_k) - \delta(s, v_0) = \delta(s, v) \quad (\delta(s, s) = 0).$$

Ningún camino de s a v pesa menos que $\delta(s, v)$, así que $w(p) = \delta(s, v)$. De paso, sobre las aristas del árbol el triángulo se cumple justo. □

Ojito...

Sin ciclos negativos, G_π es árbol en todo momento, pero a media corrida no tiene por qué ser de caminos más cortos: tras el paso 3 del ejemplo 18.7 contiene $s \to b \to c$. Y con un ciclo negativo alcanzable puede dejar de ser árbol, como en el ejemplo 18.11. La prueba de aciclicidad solo usó la hipótesis al final; leída sin ella dice algo más fuerte: *todo ciclo que aparezca en G_π es negativo*. Eso es lo que permite recuperar un ciclo negativo siguiendo apuntadores.

Quién usa qué

Con las seis propiedades probadas, la corrección de cada algoritmo se reduce a decir cuál aplica y por qué su orden de relajación cumple la hipótesis. La tabla 8 lo resume para los tres algoritmos de la sección 17 y dos de los problemas que siguen.

Algoritmo	**Orden de relajación**	**Propiedades**	**Dónde entran**
Bellman-Ford (Alg. 17.3)	$n-1$ pasadas sobre todas las aristas	relajación de caminos, sin camino, triángulo, subgrafo predecesor	la pasada i relaja la i-ésima arista de todo camino más corto; la revisión final pasa por el triángulo y falla con un ciclo negativo por la suma alrededor del ciclo
Caminos-DAG (Alg. 17.4)	una pasada, orígenes en orden topológico	relajación de caminos, sin camino, subgrafo predecesor	el orden topológico pone en orden las aristas de cualquier camino
Dijkstra (Alg. 17.5)	cada arista una vez, orígenes en orden de d creciente	cota superior, sin camino, convergencia, subgrafo predecesor	el primer vértice no fijado de un camino más corto ya tiene su distancia, por convergencia
Yen	$\lceil n/2 \rceil$ pasadas de ida y regreso	relajación de caminos	cada pasada despacha dos corridas del camino
Bitónicos	cuatro pasadas en orden de peso	relajación de caminos	cada pasada despacha una corrida monótona

Tabla 8: Qué propiedad de la relajación sostiene a cada algoritmo.

Problema: la mejora de Yen a Bellman-Ford

Planteamiento (Cormen, problema 24-1). Se fija un orden lineal arbitrario $v_1, \ldots, v_n$ de los vértices y se parten las aristas, sin lazos, en las que van hacia adelante y las que van hacia atrás:

$$E_f = \{(v_i, v_j) \in E : i < j\}, \qquad E_b = \{(v_i, v_j) \in E : i > j\}.$$

Cada pasada recorre los vértices en el orden $v_1, \ldots, v_n$ relajando sus aristas de E_f, y luego en el orden $v_n, \ldots, v_1$ relajando sus aristas de E_b. Yen mostró que así bastan $\lceil n/2 \rceil$ pasadas en lugar de $n-1$.

Idea. $G_f = (V, E_f)$ es acíclico con orden topológico $v_1, \ldots, v_n$, y $G_b = (V, E_b)$ lo es con $v_n, \ldots, v_1$: cada mitad de pasada es Caminos-DAG sobre uno de los dos. Un camino más corto p de s a v, con $k \leq n-1$ aristas, se parte en *corridas*, bloques máximos de aristas consecutivas del mismo tipo, que alternan entre E_f y E_b. Una corrida de E_f se relaja entera y en orden dentro de una mitad de ida, y lo mismo con E_b en la de regreso, así que cada pasada despacha dos corridas. Con $r \leq n-1$ corridas, en el peor caso hacen falta $\lceil (r+1)/2 \rceil \leq \lceil n/2 \rceil$ pasadas, y la relajación de caminos da $d[v] = \delta(s, v)$. La revisión final de Bellman-Ford sigue siendo válida.

Costo. Con listas separadas para E_f y E_b, armadas una vez en $\Theta(n+m)$, cada pasada cuesta $\Theta(n+m)$: en total $\Theta(nm)$ cuando todos son alcanzables. El orden de crecimiento no mejora; se gana la mitad de las pasadas. Un lazo negativo no está en E_f ni en E_b y el esquema no lo vería nunca, así que los lazos se revisan aparte.

Problema: cajas anidadas

Planteamiento (Cormen, problema 24-2). Una caja de k dimensiones $(x_1, \ldots, x_k)$ *anida* en otra $(y_1, \ldots, y_k)$ si existe una permutación σ con $x_{\sigma(i)} < y_i$ para todo i. Cormen llama d a la dimensión; aquí es k, porque d ya es la estimación. Dadas n cajas, se busca la sucesión más larga en la que cada caja anida en la siguiente.

Anidar es transitivo: si $x_{\sigma(i)} < y_i$ y $y_{\tau(j)} < z_j$, la permutación compuesta sirve de testigo, $x_{\sigma(\tau(j))} < y_{\tau(j)} < z_j$.

Prueba sin recorrer permutaciones. Con $x_{(i)}$ la i-ésima dimensión más chica,

$$x \text{ anida en } y \iff x_{(i)} < y_{(i)} \text{ para todo } i.$$

De derecha a izquierda, la permutación que ordena es el testigo. De izquierda a derecha, las i dimensiones más chicas de y están emparejadas con i dimensiones distintas de x, todas menores que $y_{(i)}$, y entonces $x_{(i)} < y_{(i)}$. Cuesta $\mathcal{O}(k \log k)$.

Idea. Un vértice por caja y una arista $B_i \to B_j$ si B_i anida en B_j. Por transitividad y desigualdad estricta el grafo es acíclico, y la sucesión pedida es un camino con el mayor número de vértices: Caminos-DAG con peso -1 en cada arista, o la recurrencia $L(j) = 1 + \max\{L(i) : B_i \to B_j\}$ (con $L(j) = 1$ si no entra ninguna arista) en orden topológico. Ordenar por la suma de las dimensiones da ese orden gratis. La desigualdad tiene que ser estricta: con $\leq$, dos cajas iguales formarían un ciclo.

Costo. $\mathcal{O}(nk \log k)$ para ordenar las dimensiones, $\mathcal{O}(n^2 k)$ para comparar los pares y $\mathcal{O}(n^2)$ para el camino más largo: $\mathcal{O}(nk \log k + n^2 k)$.

Problema: arbitraje

Planteamiento (Cormen, problema 24-3). *Arbitraje* es aprovechar tipos de cambio inconsistentes para convertir una unidad de una moneda en más de una unidad de la misma. Hay n monedas $c_1, \ldots, c_n$ y una tabla R en la que una unidad de c_i compra $R[i, j]$ unidades de c_j. Se pide decidir si existe un ciclo de monedas con

$$R[i_1, i_2] \cdot R[i_2, i_3] \cdots R[i_{k-1}, i_k] \cdot R[i_k, i_1] > 1$$

y, si existe, imprimirlo.

De productos a sumas. El logaritmo es creciente y convierte productos en sumas. Con $i_{k+1} = i_1$,

$$\prod_{j=1}^{k} R[i_j, i_{j+1}] > 1 \iff \sum_{j=1}^{k} (-\ln R[i_j, i_{j+1}]) < 0.$$

Con pesos $w(c_i, c_j) = -\ln R[i, j]$, hay arbitraje si y solo si hay un ciclo negativo, y Bellman-Ford lo detecta; Dijkstra no sirve, porque los tipos mayores que 1 dan pesos negativos. Si $R[i, j] = 0$ no hay arista. Para que todo ciclo sea alcanzable se agrega una fuente virtual s con aristas de peso 0 a todas las monedas: el grafo queda con $n + 1$ vértices.

Algoritmo 18.1 Arbitraje(R, n)

```
1:  procedimiento ARBITRAJE(R, n)
2:      V ← {s, c_1, ..., c_n}; aristas (s, c_i) de peso 0
3:      aristas (c_i, c_j) de peso −ln R[i, j] para cada i ≠ j con R[i, j] > 0
4:      INICIALIZAR-FUENTE(G, s)
5:      para p ← 1 hasta n hacer                          // |V| − 1 = n pasadas
6:          para todo (u, v) ∈ E hacer
7:              RELAJAR(u, v, w)
8:          fin para
9:      fin para
10:     para todo (u, v) ∈ E hacer
11:         si d[v] > d[u] + w(u, v) + ε entonces
12:             RELAJAR(u, v, w)                          // ahora G_π tiene un ciclo
13:             x ← v
14:             para i ← 1 hasta n + 1 hacer
15:                 x ← π[x]                              // al final, x está en el ciclo
16:             fin para
17:             C ← ⟨x⟩,    y ← π[x]
18:             mientras y ≠ x hacer
19:                 anteponer y a C,    y ← π[y]
20:             fin mientras
21:             anteponer x a C
22:             devolver C                                // c_{i_1}, ..., c_{i_k}, c_{i_1}
23:         fin si
24:     fin para
25:     devolver no hay arbitraje
26: fin procedimiento
```

Cómo recuperar el ciclo. Si tras las n pasadas la arista (u, v) todavía se puede relajar, se relaja y se siguen los apuntadores desde v durante $n + 1$ pasos. El vértice al que se llega está en un ciclo de G_π, y ese ciclo es negativo:

- Tras i pasadas, cada $d[x]$ es a lo más el peso de cualquier caminata de s a x con a lo más i aristas (inducción sobre i con el lema 18.4). Entonces la relajación extra bajó $d[v]$ por debajo del peso de toda caminata de s a v con a lo más n aristas.
- Si la cadena $v, \pi[v], \ldots$ llegara a s sin repetir vértices, sería un camino de a lo más n aristas, y sumando el lema 18.8 a lo largo de ella saldría $d[v] \geq w(\text{cadena})$, porque $d[s]$ se queda en 0. Contradice el punto anterior. Así que la cadena cae en un ciclo, y como hay $n + 1$ vértices, tras $n + 1$ pasos ya está dentro.

- Ese ciclo es negativo por la versión fuerte del lema 18.9. Leído al derecho, es la sucesión de monedas; por eso las líneas 17 a 21 anteponen.

Costo. Con la tabla llena el grafo tiene $n+1$ vértices y n^2 aristas: $\Theta(n^2)$ para construirlo, $\Theta(n^3)$ las n pasadas, $\Theta(n^2)$ la revisión y $\Theta(n)$ la recuperación. Total $\Theta(n^3)$, lo mismo que Floyd, que delata el ciclo en la diagonal.

Ejemplo 18.11 (Arbitraje entre dólar, rupia y yen)**.** Los tres tipos de cambio de Cormen (1 dólar compra 49 rupias, 1 rupia compra 2 yenes y 1 yen compra 0.0107 dólares), más los de regreso, con el margen que cobra una casa de cambio:

$R =$

	D	I	Y
D	1	49	92
I	0.0200	1	2
Y	0.0107	0.48	1

$w = -\ln R =$

	D	I	Y
D	0	-3.892	-4.522
I	3.912	0	-0.693
Y	4.538	0.734	0

De los cinco ciclos solo $D \to I \to Y \to D$ gana: $49 \cdot 2 \cdot 0.0107 = 1.0486$, un 4.86 %, y su peso es $-3.892 - 0.693 + 4.538 = -0.0475 = -\ln 1.0486$. Para seguirlo a mano se corre Bellman-Ford desde D en lugar de la fuente virtual: la tabla está llena y D alcanza a las otras dos. Son $|V| - 1 = 2$ pasadas, con las aristas en el orden (D,I), (D,Y), (I,D), (I,Y), (Y,D), (Y,I). La columna *candidato* es $d[u] + w(u,v)$:

Pasada	**Arista**	**Candidato**	**¿Cambia?**	$d[D]$	$d[I]$	$d[Y]$	
1	(D,I)	-3.8918	sí	0	-3.8918	∞	$\pi[I] = D$
	(D,Y)	-4.5218	sí	0	-3.8918	-4.5218	$\pi[Y] = D$
	(I,D)	0.0202	no	0	-3.8918	-4.5218	
	(I,Y)	-4.5850	sí	0	-3.8918	-4.5850	$\pi[Y] = I$
	(Y,D)	-0.0475	sí	-0.0475	-3.8918	-4.5850	$\pi[D] = Y$
	(Y,I)	-3.8510	no	-0.0475	-3.8918	-4.5850	
2	las seis		3 de 6	-0.0949	-3.9393	-4.6324	bajan D, I y Y
revisión	(D,I)	-3.9867	falla				$-3.9393 > -3.9867$

Ya en la pasada 1, al relajar (Y,D), la fuente recibe $\pi[D] = Y$ y G_π queda con el ciclo $D \to I \to Y \to D$. Cada vuelta baja las tres estimaciones en 0.0475. La revisión encuentra que (D,I) todavía se puede relajar; se relaja y se retrocede $|V| = 3$ veces desde I: D, Y, I. Desde I, $\pi[I] = D$, $\pi[D] = Y$ y $\pi[Y] = I$: al derecho, $I \to Y \to D \to I$, el mismo ciclo empezando en otra moneda.

Ojito…

Los logaritmos se redondean. Un ciclo con producto exactamente 1 puede sumar -10^{-16} y parecer arbitraje; por eso la línea 11 compara con una tolerancia ε. Y para recuperar el ciclo hay que relajar primero la arista que falla y luego retroceder $|V|$ veces: retroceder menos puede dejar en un vértice que cuelga del ciclo sin estar en él.

Problema: ciclo de peso medio mínimo (Karp)

Planteamiento (Cormen, problema 24-5). El *peso medio* de un ciclo c con ℓ aristas es $\mu(c) = w(c)/\ell$, y $\mu^* = \min_c \mu(c)$ sobre todos los ciclos dirigidos de G. Se pide calcular μ^* en $\mathcal{O}(nm)$. Se supone que todo vértice es alcanzable desde una fuente s, y $\delta_k(s,v)$ es el peso mínimo de una caminata de s a v con *exactamente* k aristas, o ∞ si no hay ninguna. La tabla se llena por niveles, $\delta_k(s,v) = \min_{(u,v)\in E}(\delta_{k-1}(s,u) + w(u,v))$, con $\delta_0(s,s) = 0$ y $\delta_0(s,v) = \infty$ para $v \neq s$.

Teorema 18.12 (Karp). *Si G tiene al menos un ciclo y todo vértice es alcanzable desde s, entonces*

$$\mu^* = \min_{v \in V} \max_{0 \le k \le n-1} \frac{\delta_n(s,v) - \delta_k(s,v)}{n-k},$$

donde el mínimo omite los v con $\delta_n(s,v) = \infty$ y el máximo omite los k con $\delta_k(s,v) = \infty$.

Idea de la prueba. Sumar una constante t a cada peso sube μ^* y cada cociente en t, así que basta el caso $\mu^* = 0$. Entonces no hay ciclos negativos y $\delta(s,v) = \min_{k \le n-1} \delta_k(s,v)$; como $\delta_n(s,v) \ge \delta(s,v)$, cada máximo es ≥ 0. Para un vértice u de un ciclo de peso 0, un camino más corto a u alargado con vueltas al ciclo hasta tener n aristas termina en un v del ciclo con $\delta_n(s,v) = \delta(s,v)$, y ahí el máximo vale 0. □

Algoritmo y costo. Se llena la tabla $D[k][v]$ para $0 \le k \le n$ leyendo el nivel $k-1$ y escribiendo el k: n niveles, cada uno recorre las m aristas, $\Theta(nm)$. Luego se evalúan los n cocientes de cada vértice, $\Theta(n^2)$, y como todos son alcanzables, $n^2 = \mathcal{O}(nm)$: total $\Theta(nm)$, con $\Theta(n^2)$ de memoria. Si ninguna fuente alcanza todo, se inicializa $D[0][v] = 0$ para todo v, que equivale a una fuente virtual.

Ojito...

Tres trampas al programarlo. La tabla no se actualiza en el lugar: eso sería Bellman-Ford y se perdería el "exactamente k aristas". Los ∞ se saltan, el vértice entero si $\delta_n(s,v) = \infty$ y el cociente si $\delta_k(s,v) = \infty$. Y δ_k es sobre caminatas: restringida a caminos, δ_n sería ∞ en todos lados.

Problema: caminos bitónicos

Planteamiento (Cormen, problema 24-6). Una sucesión es *bitónica* si crece y luego decrece, o si se vuelve así con un corrimiento circular: $\langle 1,4,6,8,3,-2\rangle$ lo es y $\langle 1,3,12,4,2,10\rangle$ no. Se da un grafo con pesos todos distintos y la promesa de que, para cada v, los pesos a lo largo de *cualquier* camino más corto de s a v forman una sucesión bitónica. Se piden los caminos más cortos lo más rápido posible.

Lema 18.13. *Una sucesión bitónica se parte en a lo más tres bloques consecutivos, cada uno monótono, con direcciones alternadas: sube, baja, sube, o baja, sube, baja (alguno puede faltar).*

Idea de la prueba. Algún corrimiento sube hasta un máximo y luego baja. La sucesión original es ese corrimiento cortado en un punto, con las dos partes intercambiadas; según el corte caiga en la subida o en la bajada, queda sube-baja-sube o baja-sube-baja. □

Idea. Se ordenan las aristas por peso y se hacen cuatro pasadas de relajación: en orden creciente, decreciente, creciente y decreciente. Un bloque creciente de un camino más corto p tiene pesos estrictamente crecientes, y una pasada creciente lo relaja en el orden del camino; lo mismo con los decrecientes. Por el lema 18.13, sube-baja-sube lo cubren las pasadas 1, 2 y 3, y baja-sube-baja las 2, 3 y 4. La relajación de caminos da $d[v] = \delta(s,v)$.

Costo. Ordenar cuesta $\mathcal{O}(m \log n)$; inicializar, $\Theta(n)$, y las cuatro pasadas, $\Theta(m)$. Total $\mathcal{O}(n + m \log n)$, o $\Theta(n+m)$ si los pesos se ordenan en tiempo lineal.

Ojito...

La solución que suele circular hace dos pasadas, creciente y luego decreciente, y solo cubre la forma sube-baja sin corrimiento. Con corrimientos dos pasadas no bastan: el camino de pesos $\langle 3, 1, 2, 5, 4\rangle$ es baja-sube-baja y su último vértice sigue en ∞ hasta la cuarta pasada. Tres tampoco: ningún orden de tres cubre a la vez los dos patrones. Los pesos distintos también son necesarios: con empates, dos aristas consecutivas del mismo peso pueden relajarse en el orden equivocado.

19. Caminos más cortos entre todos los pares *Complemento: Cormen, cap. 25*

Resumen rápido. Calcular $\delta(u, v)$ para todo par de un grafo dirigido con pesos posiblemente negativos, sin ciclos negativos, y entregar D con los pesos y Π con los predecesores. La vara de medir es correr n veces un algoritmo de fuente única. La programación dinámica por número de aristas es un producto de matrices con mín y $+$, en $\Theta(n^3 \log n)$ elevando al cuadrado. Floyd-Warshall baja a $\Theta(n^3)$, con Π, la versión en el lugar, la detección de ciclos negativos en la diagonal y la cerradura transitiva. Johnson repondera con un potencial de Bellman-Ford y corre Dijkstra desde cada vértice; en grafos dispersos gana. Una tabla final dice cuál conviene según la densidad.

El problema y su representación

Sea $G = (V, E)$ un grafo dirigido con pesos $w : E \to \mathbb{R}$. Se quiere $\delta(u, v)$ entre **todo** par de vértices, y algo que permita reconstruir los caminos. Los pesos pueden ser negativos, pero se supone que no hay ciclos de peso negativo; Floyd-Warshall y Johnson, además, los detectan en lugar de devolver basura.

La clase del 22 de octubre (sección 11) presentó Floyd con un ejemplo. Aquí los vértices se numeran de 1 a n; si un grafo tiene letras, $A = 1$, $B = 2$, y así.

Definición 19.1 (Matriz de pesos)**.** La matriz de pesos de G es $W = (w_{ij})$, de $n \times n$, con

$$w_{ij} = \begin{cases} 0 & \text{si } i = j, \\ w(i, j) & \text{si } i \neq j \text{ y } (i, j) \in E, \\ \infty & \text{si } i \neq j \text{ y } (i, j) \notin E. \end{cases}$$

Las matrices se escriben con mayúscula y sus casillas con minúscula, $D = (d_{ij})$, y un superíndice entre paréntesis marca la iteración, como en $D^{(k)}$.

Definición 19.2 (Distancias, predecesores y subgrafo de predecesores)**.** Al terminar, un algoritmo de todos los pares entrega $D = (d_{ij})$ con $d_{ij} = \delta(i, j)$ y una **matriz de predecesores** $\Pi = (\pi_{ij})$: π_{ij} es nulo si $i = j$ o si no hay camino de i a j, y en otro caso es el predecesor de j en algún camino más corto desde i. El **subgrafo de predecesores** de i es $G_{\pi,i} = (V_{\pi,i}, E_{\pi,i})$, con

$$V_{\pi,i} = \{j \in V : \pi_{ij} \neq \text{nulo}\} \cup \{i\}, \qquad E_{\pi,i} = \{(\pi_{ij}, j) : j \in V_{\pi,i} - \{i\}\}.$$

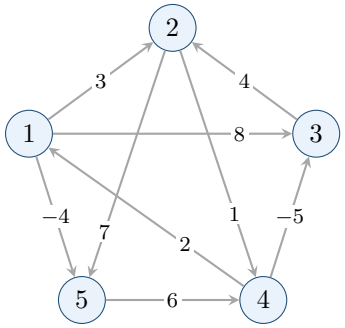
Se pide que cada $G_{\pi,i}$ sea un árbol de caminos más cortos con raíz i: el renglón i de Π es el arreglo $\pi[\cdot]$ que dejaría una corrida de fuente única desde i (sección 17).

Algoritmo 19.1 Imprimir-Camino-Pares(Π, i, j)

```
1:  procedimiento Imprimir-Camino-Pares(Π, i, j)
2:      si i = j entonces
3:          imprimir i
4:      si no si π_ij = NULO entonces
5:          imprimir que no hay camino de i a j
6:      si no
7:          Imprimir-Camino-Pares(Π, i, π_ij)        // de i al penúltimo
8:          imprimir j
9:      fin si
10: fin procedimiento
```

El procedimiento solo lee el renglón i y cuesta $\mathcal{O}(n)$, porque la recursión baja tantas veces como aristas tiene el camino. Supone que $G_{\pi,i}$ es un árbol; con una Π mal construida puede ciclarse para siempre.

Ejemplo 19.3 (El grafo del capítulo)**.** Cormen usa un solo grafo en todo el capítulo (su figura 25.1), y esta sección también: cinco vértices, nueve aristas, dos de ellas negativas, y ningún ciclo negativo.



$$W = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix}$$

El ciclo $1 \to 5 \to 4 \to 1$ pesa $-4 + 6 + 2 = 4$, y $2 \to 4 \to 3 \to 2$ pesa $1 - 5 + 4 = 0$.

Correr n veces un algoritmo de fuente única

Con pesos no negativos, n corridas de Dijkstra (algoritmo 17.5) cuestan $\Theta(n^3)$ con la cola en un arreglo, $\mathcal{O}(n(n+m)\log n)$ con montículo binario, que Cormen escribe $\mathcal{O}(nm \log n)$, y $\mathcal{O}(n^2 \log n + nm)$ con montículo de Fibonacci. Con pesos de cualquier signo, n corridas de Bellman-Ford (algoritmo 17.3) cuestan $\mathcal{O}(n^2 m)$, que en un grafo denso es $\mathcal{O}(n^4)$. Al comparar con Floyd hay que comparar contra las n corridas, no contra el costo de una sola fuente. El resto de la sección persigue dos metas: bajar a $\Theta(n^3)$ sin renunciar a los pesos negativos, que logra Floyd-Warshall, y alcanzar el costo de n veces Dijkstra también con pesos negativos, que logra Johnson.

Programación dinámica por número de aristas

Sea $l_{ij}^{(r)}$ el peso mínimo de una caminata de i a j con a lo más r aristas, y $L^{(r)} = (l_{ij}^{(r)})$; Cormen usa m para este contador. Por el lema de subcaminos (Cormen, lema 24.1), una caminata de a lo más r aristas o tiene a lo más $r-1$, o termina con una arista (k, j) precedida de una caminata con a lo más $r-1$:

$$l_{ij}^{(0)} = \begin{cases} 0 & i = j \\ \infty & i \neq j \end{cases} \qquad l_{ij}^{(r)} = \min_{1 \le k \le n} \left\{ l_{ik}^{(r-1)} + w_{kj} \right\},$$

donde el término $k = j$, con $w_{jj} = 0$, es no agregar arista. Sin ciclos negativos un camino más corto tiene a lo más $n-1$ aristas, así que $l_{ij}^{(r)} = \delta(i,j)$ para todo $r \ge n-1$.

Pasar de $L^{(r-1)}$ a $L^{(r)}$ (Extender-Caminos) son tres ciclos anidados, $\Theta(n^3)$, y es el producto de matrices $c_{ij} = \sum_k a_{ik} \cdot b_{kj}$ con la suma cambiada por mín y el producto por $+$. Ese producto $(\text{mín}, +)$ es asociativo, porque $+$ se distribuye sobre mín, y $L^{(r)} = W^r$. La versión lenta calcula $W^2, W^3, \ldots, W^{n-1}$: $n-2$ productos, $\Theta(n^4)$, lo mismo que n veces Bellman-Ford en un grafo denso. La rápida eleva al cuadrado, $W, W^2, W^4, \ldots$, hasta pasar de $n-1$, que no estropea nada: $\lceil \log(n-1) \rceil$ productos, $\Theta(n^3 \log n)$, con dos matrices de memoria. Sobre el grafo del ejemplo 19.3 las dos llegan a la misma $L^{(4)}$, que es la D final de Floyd-Warshall. Strassen no sirve aquí, porque sus fórmulas restan y el mínimo no tiene inverso. Y pasarse de $n-1$ solo es inofensivo sin ciclos negativos: con uno, la diagonal de una $L^{(r)}$ con $r \ge n$ tiene una casilla negativa.

Floyd-Warshall

La clase del 22 de octubre trae la recurrencia, el pseudocódigo en el lugar y un ejemplo. Aquí se justifica cada pieza: por qué la recurrencia es correcta, cómo se construye Π, por qué basta una matriz y cómo se detectan los ciclos negativos. La idea es cambiar el estado de la programación dinámica: en lugar de cuántas aristas usa el camino, qué vértices puede usar como intermedios.

Definición 19.4 (Vértice intermedio)**.** Los **intermedios** de una caminata $p = \langle v_1, v_2, \ldots, v_l \rangle$ son los vértices que aparecen en las posiciones 2 a $l-1$, es decir, todos menos los extremos. Si G no tiene ciclos negativos, $d_{ij}^{(k)}$ es el peso mínimo de una caminata de i a j cuyos intermedios están todos en $\{1, \ldots, k\}$, o ∞ si no hay ninguna, y $D^{(k)} = (d_{ij}^{(k)})$.

Con $k = 0$ no se permite ningún intermedio, de ahí $d_{ij}^{(0)} = w_{ij}$. Con $k = n$ se permite todo, así que $d_{ij}^{(n)} = \delta(i, j)$.

Lema 19.5 (Estructura de Floyd-Warshall)**.** *Si G no tiene ciclos negativos, para $k \geq 1$*

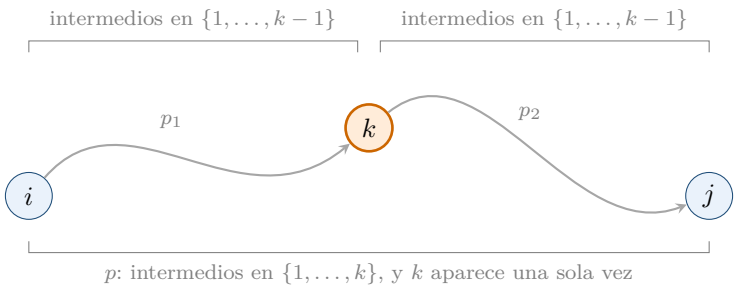
$$d_{ij}^{(k)} = \min(d_{ij}^{(k-1)},\ d_{ik}^{(k-1)} + d_{kj}^{(k-1)}).$$

Demostración. Primero $\leq$. Una caminata con intermedios en $\{1, \ldots, k-1\}$ también los tiene en $\{1, \ldots, k\}$, así que $d_{ij}^{(k)} \leq d_{ij}^{(k-1)}$. Y pegar una caminata óptima de i a k con una de k a j, las dos con intermedios en $\{1, \ldots, k-1\}$, da una caminata de i a j cuyos intermedios son los de ambas más el propio k: $d_{ij}^{(k)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$.

Ahora $\geq$, y aquí entra la hipótesis. Sea p una caminata óptima de i a j con intermedios en $\{1, \ldots, k\}$. Si p repite un vértice, contiene un ciclo; ese ciclo no es negativo, y quitarlo deja una caminata con los mismos extremos, intermedios en el mismo conjunto y peso no mayor. Repitiendo, p se puede tomar sin vértices repetidos. Hay dos casos.

- k no es intermedio de p. Entonces todos sus intermedios están en $\{1, \ldots, k-1\}$, así que $d_{ij}^{(k)} = w(p) \geq d_{ij}^{(k-1)}$.
- k es intermedio de p. Aparece exactamente una vez, porque p no repite vértices. Partiendo p en k quedan p_1 de i a k y p_2 de k a j, y en ninguno de los dos k es intermedio: es un extremo y no aparece en otra parte. Los dos usan solo $\{1, \ldots, k-1\}$, así que $d_{ij}^{(k)} = w(p_1) + w(p_2) \geq d_{ik}^{(k-1)} + d_{kj}^{(k-1)}$. □

La figura es la 25.3 de Cormen: el camino óptimo que pasa por k se parte en dos tramos que solo usan intermedios menores que k.



Idea clave

La hipótesis de no haber ciclos negativos sirve para una sola cosa: tomar el óptimo sin vértices repetidos, para que k aparezca a lo más una vez y la caminata se parta en solo dos tramos. Ganada la hipótesis, cada casilla elige entre dos opciones, usar k o no, en lugar de entre n penúltimos vértices. Las dos programaciones dinámicas tienen n^3 estados; la diferencia entre $\Theta(n^4)$ y $\Theta(n^3)$ es lo que cuesta resolver cada uno.

Los predecesores

Al principio el predecesor de j desde i es i si hay arista, y nulo si no:

$$\pi_{ij}^{(0)} = \begin{cases} \text{nulo} & \text{si } i = j \text{ o } w_{ij} = \infty, \\ i & \text{si } i \neq j \text{ y } w_{ij} < \infty. \end{cases}$$

En la vuelta k, el predecesor solo cambia si pasar por k mejora estrictamente:

$$\pi_{ij}^{(k)} = \begin{cases} \pi_{ij}^{(k-1)} & \text{si } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)}, \\ \pi_{kj}^{(k-1)} & \text{si } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)}. \end{cases}$$

Si el mejor camino pasa por k, su último tramo es un camino óptimo de k a j, así que el predecesor de j es el que ya se tenía para el par (k, j). En un empate se conserva el predecesor viejo, y eso no es un detalle de estilo: la otra elección rompe Π (Cormen, ejercicio 25.2-5). Que cada $G_{\pi,i}$ salga un árbol de caminos más cortos es el ejercicio 25.2-3, y se adapta del subgrafo predecesor de fuente única (proposición 17.12).

El algoritmo completo

Proposición 19.6 (Basta una matriz)**.** *Si G no tiene ciclos negativos, durante la vuelta k el renglón k y la columna k de D y de Π no cambian. Por eso la versión sin superíndices calcula exactamente $D^{(n)}$ y $\Pi^{(n)}$, con $\Theta(n^2)$ de memoria.*

Idea de la prueba. Sin ciclos negativos $d_{kk}^{(k-1)} = 0$, así que $d_{ik}^{(k)} = \min(d_{ik}^{(k-1)}, d_{ik}^{(k-1)} + 0)$ no cambia, y lo mismo d_{kj} y π_{kj}. Las únicas casillas que se leen además de d_{ij} valen lo mismo antes y después de que el ciclo pase por ellas. □

Teorema 19.7 (La diagonal delata los ciclos negativos)**.** *Sin suponer nada sobre los ciclos, sea D la matriz que deja Floyd-Warshall, por capas o en el lugar. G tiene un ciclo de peso negativo si y solo si $d_{ii} < 0$ para algún i.*

Idea de la prueba. Cada valor finito es el peso de alguna caminata, así que $d_{ii} < 0$ da una caminata cerrada negativa, y de ella sale un ciclo negativo. Al revés, si q es el vértice de número más alto de un ciclo negativo C e i otro vértice de C, en la vuelta q queda $d_{ii} \leq d_{iq} + d_{qi} \leq w(C) < 0$, y los valores nunca suben. □

La versión de abajo junta la recurrencia sin superíndices, que es la de la clase, con el cálculo de Π y la prueba de la diagonal.

Algoritmo 19.2 Floyd-Warshall(W), con Π y en el lugar

```
1:  procedimiento Floyd-Warshall(W)
2:      n ← número de renglones de W;    D ← W
3:      para i ← 1 hasta n hacer
4:          para j ← 1 hasta n hacer
5:              π_ij ← i si i ≠ j y w_ij < ∞; si no, π_ij ← NULO
6:          fin para
7:      fin para
8:      para k ← 1 hasta n hacer                          // k va afuera, siempre
9:          para i ← 1 hasta n hacer
10:             para j ← 1 hasta n hacer
11:                 si d_ik + d_kj < d_ij entonces
12:                     d_ij ← d_ik + d_kj
13:                     π_ij ← π_kj
14:                 fin si
15:             fin para
16:         fin para
17:     fin para
18:     para i ← 1 hasta n hacer
19:         si d_ii < 0 entonces
20:             devolver "hay un ciclo de peso negativo"
21:         fin si
22:     fin para
23:     devolver D, Π
24: fin procedimiento
```

La línea 8 pone k afuera porque la recurrencia necesita $D^{(k-1)}$ completa antes de empezar con k. En la línea 11 la desigualdad es estricta, que es el $\leq$ de la regla de Π. La línea 13 lee π_{kj} de la misma matriz que se modifica; la proposición 19.6 dice por qué está bien. Las líneas 18 a 22 son el teorema 19.7: si hay un ciclo negativo, D y Π no deben usarse.

Costo. La inicialización es $\Theta(n^2)$ y el triple ciclo ejecuta su cuerpo de $\mathcal{O}(1)$ exactamente n^3 veces: $\Theta(n^3)$ en tiempo y $\Theta(n^2)$ en memoria. El costo no depende de m ni de los pesos, una virtud en grafos densos y un defecto en los dispersos.

Ejemplo 19.8 (Floyd-Warshall sobre el grafo del capítulo)**.** Tres de las seis parejas $D^{(k)}$, $\Pi^{(k)}$ sobre el grafo del ejemplo 19.3; en azul, las casillas que cambian en la vuelta k. Coinciden con la figura 25.4 de Cormen.

$$D^{(0)} = \begin{pmatrix} 0 & 3 & 8 & \infty & -4 \\ \infty & 0 & \infty & 1 & 7 \\ \infty & 4 & 0 & \infty & \infty \\ 2 & \infty & -5 & 0 & \infty \\ \infty & \infty & \infty & 6 & 0 \end{pmatrix} \qquad \Pi^{(0)} = \begin{pmatrix} \text{NULO} & 1 & 1 & \text{NULO} & 1 \\ \text{NULO} & \text{NULO} & \text{NULO} & 2 & 2 \\ \text{NULO} & 3 & \text{NULO} & \text{NULO} & \text{NULO} \\ 4 & \text{NULO} & 4 & \text{NULO} & \text{NULO} \\ \text{NULO} & \text{NULO} & \text{NULO} & 5 & \text{NULO} \end{pmatrix}$$

$$D^{(4)} = \begin{pmatrix} 0 & 3 & \mathbf{-1} & 4 & -4 \\ \mathbf{3} & 0 & \mathbf{-4} & 1 & \mathbf{-1} \\ \mathbf{7} & 4 & 0 & 5 & \mathbf{3} \\ 2 & -1 & -5 & 0 & -2 \\ \mathbf{8} & \mathbf{5} & \mathbf{1} & 6 & 0 \end{pmatrix} \qquad \Pi^{(4)} = \begin{pmatrix} \text{NULO} & 1 & \mathbf{4} & 2 & 1 \\ \mathbf{4} & \text{NULO} & \mathbf{4} & 2 & \mathbf{1} \\ \mathbf{4} & 3 & \text{NULO} & 2 & \mathbf{1} \\ 4 & 3 & 4 & \text{NULO} & 1 \\ \mathbf{4} & \mathbf{3} & \mathbf{4} & 5 & \text{NULO} \end{pmatrix}$$

$$D^{(5)} = \begin{pmatrix} 0 & \mathbf{1} & \mathbf{-3} & \mathbf{2} & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix} \qquad \Pi^{(5)} = \begin{pmatrix} \text{NULO} & \mathbf{3} & 4 & \mathbf{5} & 1 \\ 4 & \text{NULO} & 4 & 2 & 1 \\ 4 & 3 & \text{NULO} & 2 & 1 \\ 4 & 3 & 4 & \text{NULO} & 1 \\ 4 & 3 & 4 & 5 & \text{NULO} \end{pmatrix}$$

Ninguna casilla del renglón k ni de la columna k cambia en la vuelta k, como dice la proposición 19.6. En $k = 1$ solo el renglón 4 mejora pasando por 1: $d_{42} = 2 + 3 = 5$ y $d_{45} = 2 + (-4) = -2$. En $k = 2$ el renglón 1 gana $1 \to 2 \to 4$, con $d_{14} = 4$, y el 3 gana $3 \to 2 \to 4$ y $3 \to 2 \to 5$, con 5 y 11. En $k = 3$ hay un solo cambio, $d_{42} = -5 + 4 = -1 < 5$ por la arista negativa $4 \to 3$, con $\pi_{42} \leftarrow \pi_{32} = 3$. La vuelta $k = 4$ es la grande, nueve casillas: 4 tiene las salidas $4 \to 1$ y $4 \to 3$ y es la única salida de 5; por ejemplo, $d_{52} = d_{54} + d_{42} = 6 + (-1) = 5$ con $\pi_{52} \leftarrow \pi_{42} = 3$. En $k = 5$ solo mejora el renglón 1, gracias a la arista $1 \to 5$ de peso -4: $d_{12} = -4 + 5 = 1$ y $\pi_{12} \leftarrow \pi_{52} = 3$.

Reconstrucción. Imprimir-Camino-Pares$(\Pi^{(5)}, 1, 2)$ sube por el renglón 1: $\pi_{12} = 3$, $\pi_{13} = 4$, $\pi_{14} = 5$, $\pi_{15} = 1$, e imprime $1, 5, 4, 3, 2$, con peso $-4 + 6 - 5 + 4 = 1 = d_{12}$. A diferencia de Dijkstra, ningún valor es definitivo antes de la última vuelta: d_{12} valió 3 desde el principio y cambió hasta $k = 5$.

Ejemplo 19.9 (Caso borde: un ciclo negativo)**.** En el grafo del ejemplo 19.3 la arista $4 \to 3$ pasa a pesar -8, y el ciclo $2 \to 4 \to 3 \to 2$ pesa $1 - 8 + 4 = -3$. La diagonal de $D^{(k)}$ en cada vuelta de la versión por capas:

k	d_{11}	d_{22}	d_{33}	d_{44}	d_{55}
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	-3	0
4	0	-3	-3	-6	0
5	0	-3	-3	-6	0

La primera casilla negativa aparece en $k = 3$: para $i = 4$ el ciclo, leído como $4 \to 3 \to 2 \to 4$, solo tiene intermedios 3 y 2. En $k = 4$, $d_{44} = -6$: la caminata da dos vueltas al ciclo, y los números ya no son pesos de caminos más cortos. d_{11} y d_{55} se quedan en 0, aunque en realidad los 25 pares tienen $\delta = -\infty$: la diagonal avisa que hay un ciclo, no qué pares están afectados. Para marcarlos se pone $d_{ij} = -\infty$ cuando existe k con $d_{kk} < 0$, $d_{ik} < \infty$ y $d_{kj} < \infty$.

Ojito...

- k **va afuera.** Con el ciclo de k adentro, en el orden i, j, k, el algoritmo es otro y falla: en el grafo del capítulo da $d_{12} = 3$ y $d_{13} = 8$ en lugar de 1 y -3.
- **Aritmética con ∞.** En código, ∞ suele ser un entero grande. Entonces $\infty + (-5) < \infty$ inventa caminos que no existen, y $\infty + \infty$ puede desbordar. Hay que saltarse la actualización si $d_{ik} = \infty$ o $d_{kj} = \infty$.
- **Lazos negativos.** La matriz de pesos pone $w_{ii} = 0$ siempre, así que un lazo (i, i) negativo, que es un ciclo negativo, desaparece y nadie lo detecta. Si el grafo puede tener lazos, hay que poner $w_{ii} = \min(0, w(i, i))$.
- **Los números explotan.** Con un ciclo negativo los valores bajan muy rápido y con pesos grandes desbordan. Revisar $d_{kk} < 0$ al terminar cada vuelta y detenerse en cuanto aparezca cuesta $\Theta(n)$ extra por vuelta.

Cerradura transitiva

A veces no interesa cuánto cuesta ir de i a j, sino si se puede.

Definición 19.10 (Cerradura transitiva)**.** La cerradura transitiva de G es $G^* = (V, E^*)$ con $E^* = \{(i,j) : \text{hay un camino de } i \text{ a } j \text{ en } G\}$. Como el camino vacío cuenta, E^* incluye todos los pares (i,i).

Es el esquema de Floyd-Warshall con un camino que existe o no en lugar de un peso. Sea $t_{ij}^{(k)} \in \{0,1\}$ igual a 1 si hay un camino de i a j con intermedios en $\{1,\ldots,k\}$. Entonces $t_{ij}^{(0)} = 1$ si $i = j$ o $(i,j) \in E$, y 0 si no, y

$$t_{ij}^{(k)} = t_{ij}^{(k-1)} \vee \left(t_{ik}^{(k-1)} \wedge t_{kj}^{(k-1)}\right),$$

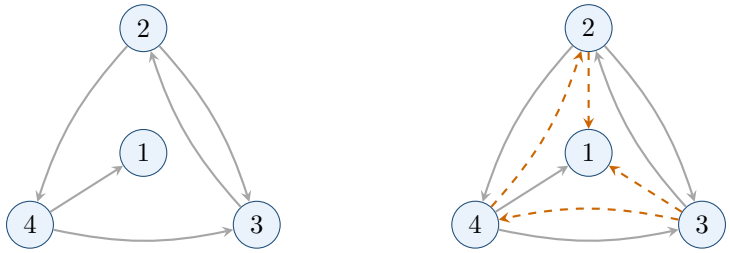
que es el lema 19.5 con $\vee$ en lugar de mín y $\wedge$ en lugar de $+$. Aquí no hace falta ninguna hipótesis sobre ciclos: si hay alguna caminata de i a j, hay un camino.

Algoritmo 19.3 Cerradura-Transitiva(G), en el lugar

```
1:  procedimiento Cerradura-Transitiva(G)
2:      para i ← 1 hasta n hacer
3:          para j ← 1 hasta n hacer
4:              t_ij ← 1 si i = j o (i,j) ∈ E; si no, t_ij ← 0
5:          fin para
6:      fin para
7:      para k ← 1 hasta n hacer
8:          para i ← 1 hasta n hacer
9:              para j ← 1 hasta n hacer
10:                 t_ij ← t_ij ∨ (t_ik ∧ t_kj)
11:             fin para
12:         fin para
13:     fin para
14:     devolver T
15: fin procedimiento
```

Costo. $\Theta(n^3)$, igual que Floyd-Warshall; la versión en el lugar es correcta por el argumento de la proposición 19.6, con $t_{kk} = 1$. La ganancia es práctica: operaciones lógicas sobre bits en lugar de aritmética, y si cada renglón de T es un vector de bits, la línea 10 para i fija se vuelve "si $t_{ik} = 1$, renglón $i \leftarrow$ renglón i OR renglón k": $\Theta(n^3/b)$ operaciones con palabras de b bits. En grafos dispersos conviene un BFS o DFS desde cada vértice (sección 14), en $\Theta(n(n+m))$.

Ejemplo 19.11 (Cerradura transitiva, figura 25.5 de Cormen)**.** Un grafo de cuatro vértices con aristas $2 \to 3$, $2 \to 4$, $3 \to 2$, $4 \to 1$ y $4 \to 3$. En naranja punteado, las aristas que agrega la cerradura, sin dibujar los pares (i,i).



G G^*

$$T^{(0)} = T^{(1)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(2)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & \mathbf{1} \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(3)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & \mathbf{1} & 1 & 1 \end{pmatrix} \quad T^{(4)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ \mathbf{1} & 1 & 1 & 1 \\ \mathbf{1} & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

En $k = 1$ nada cambia, porque 1 no tiene aristas de salida. En $k = 2$, $t_{34} = t_{32} \wedge t_{24} = 1$; en $k = 3$, $t_{42} = t_{43} \wedge t_{32} = 1$; en $k = 4$, $t_{21} = 1$ y $t_{31} = t_{34} \wedge t_{41} = 1$, que usa el t_{34} de $k = 2$. Al final 1 solo se alcanza a sí mismo, y 2, 3 y 4 alcanzan a todos.

Johnson: reponderar para poder usar Dijkstra

Floyd-Warshall cuesta $\Theta(n^3)$ aunque el grafo tenga $m = 2n$ aristas. En un grafo disperso, n corridas de Dijkstra harían del orden de $n^2 \log n$, pero Dijkstra no admite pesos negativos. El algoritmo de Johnson (1977) junta lo mejor de los dos con una idea que vale más que el algoritmo mismo: cambiar los pesos sin cambiar los caminos más cortos.

Definición 19.12 (Reponderación)**.** Dada una función $h : V \to \mathbb{R}$ cualquiera, la **reponderación** de w por h es

$$\hat{w}(u, v) = w(u, v) + h(u) - h(v),$$

y $\hat{\delta}$ denota los pesos de caminos más cortos con $\hat{w}$.

A h se le suele llamar **potencial**, y a $\hat{w}$, **costos reducidos**; Cormen no les da nombre.

Lema 19.13 (La reponderación conserva los caminos más cortos)**.** *Sea $h : V \to \mathbb{R}$ cualquiera y $\hat{w}$ la reponderación por h.*

(a) Para toda caminata $p = \langle v_0, v_1, \ldots, v_k \rangle$, $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$.

(b) p es un camino más corto de v_0 a v_k con w si y solo si lo es con $\hat{w}$.

(c) G tiene un ciclo negativo con w si y solo si lo tiene con $\hat{w}$.

Demostración. La suma es telescópica:

$$\hat{w}(p) = \sum_{i=1}^{k} \big(w(v_{i-1}, v_i) + h(v_{i-1}) - h(v_i)\big) = w(p) + \sum_{i=1}^{k} \big(h(v_{i-1}) - h(v_i)\big) = w(p) + h(v_0) - h(v_k),$$

porque cada $h(v_i)$ intermedio aparece una vez sumando y una vez restando. El término $h(v_0) - h(v_k)$ depende solo de los extremos: a todas las caminatas de v_0 a v_k se les suma la misma cantidad, así que una pesa menos que otra con w si y solo si pesa menos con $\hat{w}$, y eso es (b). Para un ciclo, $v_0 = v_k$ y (a) da $\hat{w}(c) = w(c)$, que es (c). □

Corolario 19.14. *Si G no tiene ciclos negativos, $\delta(u, v) = \hat{\delta}(u, v) + h(v) - h(u)$ para todo par, con ∞ en los dos lados cuando no hay camino.*

Basta aplicar (a) a un camino más corto de u a v, que por (b) también lo es con $\hat{w}$. Lo que el lema no conserva es el orden entre destinos distintos, porque cada uno tiene su propia $h(v)$: Dijkstra con $\hat{w}$ puede extraer los vértices en un orden que no tiene nada que ver con las distancias reales, y aun así las distancias salen bien.

Cómo escoger h

La gracia es encontrar una h que deje todos los pesos no negativos. Se agrega un vértice nuevo s con una arista de peso 0 hacia cada vértice: el **grafo aumentado** es $G' = (V', E')$, con $V' = V \cup \{s\}$, $E' = E \cup \{(s, v) : v \in V\}$ y $w(s, v) = 0$. A s no entra ninguna arista, así que G' tiene un ciclo negativo si y solo si G lo tiene, y ningún camino más corto entre vértices de G pasa por s.

Proposición 19.15 (El potencial de Johnson)**.** *Si G' no tiene ciclos negativos y $h(v) = \delta(s, v)$ para todo $v \in V'$, entonces $\hat{w}(u, v) \geq 0$ para toda $(u, v) \in E'$.*

Demostración. Todo v es alcanzable desde s por la arista (s, v), así que $h(v)$ es finito, y de hecho $h(v) \leq 0$. Por la desigualdad del triángulo (proposición 17.7), $h(v) \leq h(u) + w(u, v)$ para toda arista (u, v), que reescrita es $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$. Las aristas de s quedan con $\hat{w}(s, v) = 0 + 0 - h(v) \geq 0$. □

$\hat{w}(u, v) = 0$ exactamente cuando la arista está sobre algún camino más corto desde s. Y sirve cualquier h con $h(v) - h(u) \leq w(u, v)$ en cada arista: una solución de un sistema de restricciones de diferencia.

Algoritmo 19.4 Johnson(G, w)

```
1:  procedimiento Johnson(G, w)
2:      V' ← V ∪ {s},    E' ← E ∪ {(s, v) : v ∈ V}          // s es un vértice nuevo
3:      w(s, v) ← 0 para todo v ∈ V
4:      si Bellman-Ford(G', w, s) devuelve falso entonces
5:          devolver "G tiene un ciclo de peso negativo"
6:      fin si
7:      h(v) ← d[v] para todo v ∈ V                          // d[v] = δ(s, v)
8:      ŵ(u, v) ← w(u, v) + h(u) − h(v) para toda (u, v) ∈ E   // ≥ 0
9:      para todo u ∈ V hacer
10:         Dijkstra(G, ŵ, u)                               // deja d[v] = δ̂(u, v)
11:         para todo v ∈ V hacer
12:             d_uv ← d[v] + h(v) − h(u)                   // si d[v] = ∞, queda ∞
13:         fin para
14:     fin para
15:     devolver D
16: fin procedimiento
```

Bellman-Ford y Dijkstra son los algoritmos 17.3 y 17.5; $d[v]$ es el arreglo de distancias que dejan, y d_{uv} la casilla de la matriz de salida.

- Línea 4: la única corrida de Bellman-Ford calcula h y detecta ciclos negativos en todo G, porque desde s se alcanza todo. Desde un vértice de G solo vería los ciclos alcanzables desde él y dejaría $h = \infty$ en los que no alcanza (Cormen, ejercicio 25.3-6).
- Línea 7: se copia $d[v]$ en $h(v)$ porque cada Dijkstra de la línea 10 sobrescribe d. Olvidar esa copia es el error de implementación más común.
- Línea 12: si $d[v] = \infty$, la casilla queda en ∞; con un centinela entero, sumarle $h(v) - h(u)$ da un número finito sin sentido.
- Los predecesores salen gratis: por el lema 19.13, el árbol de Dijkstra con $\hat{w}$ desde u es árbol de caminos más cortos con w, y es el renglón u de Π.

Teorema 19.16 (Correctitud de Johnson)**.** *Si G tiene un ciclo negativo, Johnson lo reporta. Si no, devuelve D con $d_{uv} = \delta(u, v)$ para todo par.*

Demostración. Bellman-Ford devuelve falso si y solo si hay un ciclo negativo alcanzable desde la fuente (sección 17; Cormen, teorema 24.4). En G' todo es alcanzable desde s, así que devuelve falso si y solo si G' tiene un ciclo negativo, y eso pasa si y solo si G lo tiene. Si devuelve verdadero, deja $d[v] = \delta(s, v) = h(v)$; por la proposición 19.15, $\hat{w} \geq 0$, así que cada Dijkstra calcula $\hat{\delta}(u, \cdot)$ correctamente, y el corolario 19.14 lo convierte en $\delta(u, \cdot)$. □

Costo. Construir G' cuesta $\Theta(n + m)$; Bellman-Ford en G', $\mathcal{O}((n + 1)(m + n)) = \mathcal{O}(nm + n^2)$; reponderar, $\Theta(m)$, y deshacer la reponderación, $\Theta(n^2)$. Domina lo que cuestan las n corridas de Dijkstra: con montículo de Fibonacci, $\mathcal{O}(n \log n + m)$ por corrida, y el total es

$$\mathcal{O}\Big(nm + n^2\Big) + \mathcal{O}\Big(n^2 \log n + nm\Big) + \Theta\Big(n^2\Big) = \mathcal{O}\Big(n^2 \log n + nm\Big).$$

Con montículo binario, $\mathcal{O}(n(n + m) \log n)$, que Cormen escribe $\mathcal{O}(nm \log n)$; con un arreglo, $\mathcal{O}(n^3)$, y entonces no tiene caso. En los tres casos es el costo de n veces Dijkstra: admitir pesos negativos cuesta una corrida de Bellman-Ford, que no domina.

Ejemplo 19.17 (El contraejemplo de Dijkstra, resuelto)**.** La clase del 22 de octubre muestra que Dijkstra falla con las aristas $A \to B$ de peso 2, $A \to C$ de peso 3 y $C \to B$ de peso -2: desde A fija B con $d = 2$, pero A, C, B cuesta 1.

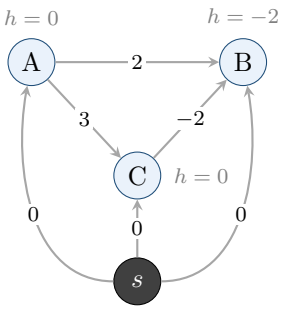
Bellman-Ford en G'. Con las aristas en el orden (s, A), (s, B), (s, C), (A, B), (A, C), (C, B), la primera pasada pone $d = 0$ en A, B y C, y luego (C, B) baja B a -2. Las pasadas 2 y 3 no cambian nada: $h(A) = 0$, $h(B) = -2$ y $h(C) = 0$.

Reponderación. $\hat{w}(A, B) = 2 + 0 - (-2) = 4$, $\hat{w}(A, C) = 3 + 0 - 0 = 3$ y $\hat{w}(C, B) = -2 + 0 - (-2) = 0$.

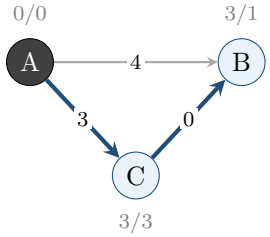
Dijkstra desde A con $\hat{w}$; en negritas, las d ya fijas.

Paso	**Sale**	A	B	C	**Relajaciones**
0		0	∞	∞	
1	A	**0**	4	3	B y C desde A
2	C	**0**	3	**3**	B: $3 + 0 = 3 < 4$
3	B	**0**	**3**	**3**	
$\delta(A, v)$		0	$3 + (-2) - 0 = 1$	$3 + 0 - 0 = 3$	

B empieza en 4 por la arista directa y baja a 3 al pasar por C, como debe. Las dos instantáneas son el grafo aumentado con h y el reponderado con el árbol de Dijkstra desde A, con $\hat{\delta}(A, v)/\delta(A, v)$ junto a cada vértice:



(a) G' y $h = \delta(s, \cdot)$



(b) $\hat{w}$, Dijkstra desde A

Ejemplo 19.18 (La figura 25.6 de Cormen)**.** Es la corrida de Johnson sobre el grafo del ejemplo 19.3.

Paso 1: Bellman-Ford en G', con las cinco aristas (s, v) primero y luego las de E en orden lexicográfico. Converge en dos pasadas, aunque hace las $|V'| - 1 = 5$:

Pasada	s	1	2	3	4	5	**Cambios**
0	0	∞	∞	∞	∞	∞	
1	0	0	0	-5	0	-4	las cinco (s, v); $(1, 5)$: -4; $(4, 3)$: -5
2	0	0	-1	-5	0	-4	$(3, 2)$: $-5 + 4 = -1$
3 a 5	0	0	-1	-5	0	-4	ninguno
h	0	0	-1	-5	0	-4	verificación: verdadero

Paso 2: reponderar, con $\hat{w}(u, v) = w(u, v) + h(u) - h(v)$:

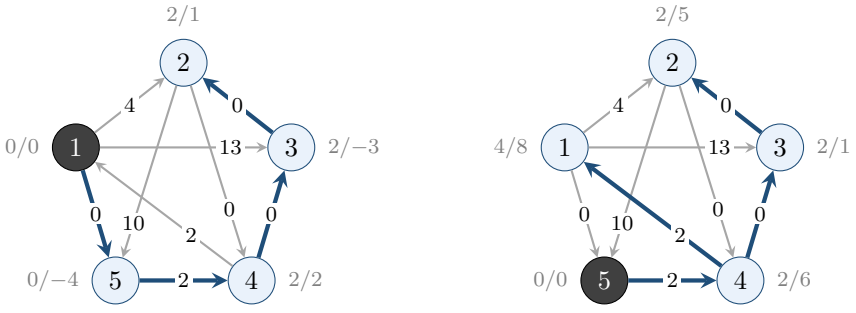
(u, v)	$(1, 2)$	$(1, 3)$	$(1, 5)$	$(2, 4)$	$(2, 5)$	$(3, 2)$	$(4, 1)$	$(4, 3)$	$(5, 4)$
$w(u, v)$	3	8	-4	1	7	4	2	-5	6
$h(u)$	0	0	0	-1	-1	-5	0	0	-4
$h(v)$	-1	-5	-4	0	-4	-1	0	-5	0
$\hat{w}(u, v)$	4	13	0	0	10	0	2	0	2

Las aristas de s quedan con $\hat{w}(s, v) = -h(v)$, o sea $0, 1, 5, 0, 4$.

Paso 3: Dijkstra desde cada vértice con $\hat{w}$**.** Desde el vértice 1:

Paso	**Sale**	1	2	3	4	5	**Relajaciones**
0		0	∞	∞	∞	∞	
1	1	**0**	4	13	∞	0	2, 3, 5 desde 1
2	5	**0**	4	13	2	**0**	4: $0 + 2$
3	4	**0**	4	2	**2**	**0**	3: $2 + 0 = 2 < 13$
4	3	**0**	2	**2**	**2**	**0**	2: $2 + 0 = 2 < 4$
5	2	**0**	**2**	**2**	**2**	**0**	ninguna
$h(v) - h(1)$		0	-1	-5	0	-4	
$\delta(1, v)$		0	1	-3	2	-4	

El vértice 2 baja de 4 a 2 por el camino $1, 5, 4, 3, 2$, que pesa $0+2+0+0 = 2$ con $\hat{w}$ y $-4+6-5+4 = 1$ con w. Dos de las cinco corridas, con el árbol de Dijkstra en azul, la fuente en negro y $\hat{\delta}(u, v)/\delta(u, v)$ junto a cada vértice:



(a) fuente 1 (b) fuente 5

Desde el vértice 5, Dijkstra extrae 4, 3 y 2 empatados en $\hat{\delta} = 2$, pero sus distancias reales son 6, 1 y 5:

el orden de extracción no dice nada sobre w. Juntando los renglones,

$$D = \begin{pmatrix} 0 & 1 & -3 & 2 & -4 \\ 3 & 0 & -4 & 1 & -1 \\ 7 & 4 & 0 & 5 & 3 \\ 2 & -1 & -5 & 0 & -2 \\ 8 & 5 & 1 & 6 & 0 \end{pmatrix},$$

que es la $D^{(5)}$ del ejemplo 19.8.

Ojito...

- **Deshacer la reponderación, y con el signo correcto.** Sin la corrección se devuelve $\hat{\delta}$: en el ejemplo 19.18, $\hat{\delta}(1,3) = 2$ pero $\delta(1,3) = -3$. Se suma $h(v) - h(u)$, no al revés.
- **Pesos reales.** Con punto flotante, $\hat{w}$ puede salir -10^{-16} por redondeo, y Dijkstra supone pesos no negativos. Conviene redondear a 0 los negativos diminutos o trabajar con enteros.
- **Un ciclo negativo en cualquier parte tumba todo.** Johnson rechaza la entrada completa aunque muchos pares no estén afectados. En un grafo no dirigido basta una arista negativa, que son dos arcos que forman un ciclo negativo.

Qué algoritmo usar

"Sí" en la columna de pesos negativos quiere decir "sí, siempre que no haya ciclos negativos", y todos los que admiten negativos también detectan esos ciclos: Floyd-Warshall y los cuadrados repetidos con la diagonal, Bellman-Ford y Johnson con la verificación final.

Algoritmo	**Pesos negativos**	**General**	**Disperso** $m = \Theta(n)$	**Denso** $m = \Theta(n^2)$
Floyd-Warshall	sí	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^3)$
Cuadrados repetidos	sí	$\Theta(n^3 \log n)$	$\Theta(n^3 \log n)$	$\Theta(n^3 \log n)$
n veces Dijkstra, arreglo	no	$\Theta(n^3)$	$\Theta(n^3)$	$\Theta(n^3)$
n veces Dijkstra, binario	no	$\mathcal{O}(n(n+m)\log n)$	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n^3 \log n)$
n veces Dijkstra, Fibonacci	no	$\mathcal{O}(n^2 \log n + nm)$	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n^3)$
n veces Bellman-Ford	sí	$\mathcal{O}(n^2 m)$	$\mathcal{O}(n^3)$	$\mathcal{O}(n^4)$
Johnson, binario	sí	$\mathcal{O}(n(n+m)\log n)$	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n^3 \log n)$
Johnson, Fibonacci	sí	$\mathcal{O}(n^2 \log n + nm)$	$\mathcal{O}(n^2 \log n)$	$\mathcal{O}(n^3)$

Dónde está el cruce. Con montículo binario, Johnson le gana a Floyd-Warshall cuando $m = o(n^2/\log n)$, porque entonces $n(n+m)\log n = o(n^3)$. Con Fibonacci, cuando $m = o(n^2)$; en el caso denso empatan asintóticamente. En la práctica, en grafos densos gana Floyd-Warshall: tres ciclos anidados sobre una matriz contigua tienen constantes mucho menores que un montículo de Fibonacci.

Idea clave

Grafo denso o chico: Floyd-Warshall, por sencillez y constantes, con pesos de cualquier signo. Grafo grande y disperso con pesos no negativos: n veces Dijkstra. Grafo grande y disperso con pesos negativos: Johnson, que cuesta lo mismo que n veces Dijkstra más una corrida de Bellman-Ford. Si solo importa si hay camino: cerradura transitiva con bits, o un BFS por vértice si el grafo es disperso.

20. Flujo máximo: Ford-Fulkerson y Edmonds-Karp *Complemento: Cormen, cap. 26.1 y 26.2*

Resumen rápido. Una red de flujo es un grafo dirigido con una capacidad en cada arista, una fuente s y un sumidero t; el problema es mandar de s a t tanto material como se pueda sin rebasar ninguna capacidad. La sección construye el método de Ford-Fulkerson: empezar con flujo cero y aumentar mientras exista un camino de s a t en la red residual, el grafo que registra cuánto cabe todavía en cada arista y cuánto se puede regresar. Regresar flujo es lo que separa al método de un voraz, que en la red de ejemplo se atora en 19 cuando el máximo es 23. El teorema de flujo máximo y corte mínimo, con prueba completa, dice que cuando ya no hay camino el flujo es óptimo, y entrega un corte que lo certifica. Con capacidades enteras Ford-Fulkerson cuesta $\mathcal{O}(m\,|f^*|)$, y una mala elección de caminos alcanza esa cota: dos millones de aumentos donde bastan dos. Edmonds-Karp escoge siempre el camino con menos aristas, con un BFS, y baja el costo a $\mathcal{O}(nm^2)$ sin importar las capacidades.

Redes de flujo

Una compañía fabrica discos de hockey en Vancouver y los guarda en una bodega en Winnipeg. Renta espacio en camiones que cubren rutas entre ciudades, y cada ruta admite a lo más cierto número de cajas por día. La pregunta es cuántas cajas diarias pueden llegar a Winnipeg: no importa por dónde viajen ni cuánto cueste, importa la tasa. Ese es el problema de flujo máximo. El ejemplo es de Cormen, y su red se usa en toda la sección.

Definición 20.1 (Red de flujo)**.** Una **red de flujo** (*flow network*) es un grafo dirigido $G = (V, E)$ en el que cada arista (u, v) tiene una **capacidad** $c(u, v) \geq 0$, junto con dos vértices distinguidos: la **fuente** s y el **sumidero** t. Por comodidad, $c(u, v) = 0$ si $(u, v) \notin E$. Se pide además que no haya lazos, que no haya **aristas antiparalelas** (si $(u, v) \in E$, entonces $(v, u) \notin E$) y que todo vértice esté en algún camino de s a t.

La última condición tiene una consecuencia que se usa en todos los análisis: todo vértice salvo s tiene al menos una arista de entrada, así que $m \geq n - 1$ y $\mathcal{O}(n + m) = \mathcal{O}(m)$.

Definición 20.2 (Flujo)**.** Un **flujo** en G es una función $f : V \times V \to \mathbb{R}$ que cumple dos propiedades:

- **Restricción de capacidad**: $0 \leq f(u, v) \leq c(u, v)$ para todos $u, v \in V$.
- **Conservación**: para todo $u \in V - \{s, t\}$,

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v).$$

El **valor** del flujo es lo que sale de la fuente menos lo que entra a ella:

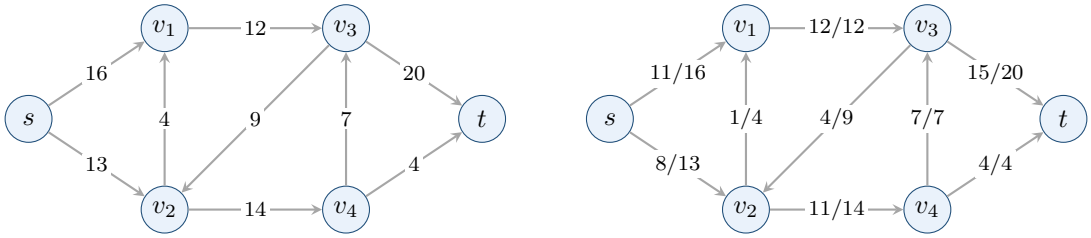
$$|f| = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s).$$

El **problema de flujo máximo** pide, dados G, s y t, un flujo de valor máximo. A un flujo así se le llama f^*.

La restricción de capacidad obliga $f(u, v) = 0$ cuando $(u, v) \notin E$, porque ahí $c(u, v) = 0$. Las barras de $|f|$ no son valor absoluto: denotan el valor del flujo. El término que resta lo que entra a s no sobra, porque la definición se aplica también a flujos en redes residuales, que sí tienen aristas hacia s.

Ejemplo 20.3 (La red de Cormen)**.** La red tiene $V = \{s, v_1, v_2, v_3, v_4, t\}$ y nueve aristas: $n = 6$ y $m = 9$. Las ciudades son Vancouver (s), Edmonton (v_1), Calgary (v_2), Saskatoon (v_3), Regina (v_4) y

Winnipeg (t). A la izquierda van las capacidades; a la derecha, un flujo, con cada arista etiquetada $f(u,v)/c(u,v)$. La diagonal separa los dos números, no los divide.



capacidades $c(u,v)$ flujo $f(u,v)/c(u,v)$, $|f| = 19$

El flujo respeta todas las capacidades. A v_2 le entran $f(s,v_2) + f(v_3,v_2) = 8 + 4 = 12$ unidades y le salen $f(v_2,v_1) + f(v_2,v_4) = 1 + 11 = 12$; en v_1, v_3 y v_4 entran y salen 12, 19 y 11, respectivamente. El valor es $|f| = f(s,v_1) + f(s,v_2) = 11 + 8 = 19$. El máximo de esta red, se verá, es 23.

Aristas antiparalelas y varias fuentes

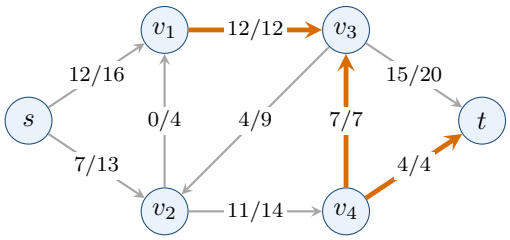
La definición prohíbe los pares (u,v), (v,u), y no por capricho. La red residual, que viene enseguida, le asigna a cada par ordenado una sola capacidad residual, y con un par antiparalelo no sabría si (u,v) es una arista original o el regreso de (v,u). Cuando un modelo trae un par así, se parte una de las dos aristas con un vértice nuevo v': se reemplaza (v_1,v_2) por (v_1,v') y (v',v_2), las dos con la capacidad original. Las dos redes admiten los mismos valores de flujo (Cormen, ejercicio 26.1-1), y cuesta un vértice y una arista por cada par.

Si hay fuentes $s_1,\ldots,s_k$ y sumideros $t_1,\ldots,t_\ell$, se agrega una **superfuente** s con aristas (s,s_i) de capacidad ∞ y un **supersumidero** t con aristas (t_j,t) de capacidad ∞. La superfuente le da a cada s_i todo lo que pida y el supersumidero recibe todo lo que llegue, así que los flujos de la red original corresponden uno a uno con los de la red nueva, con el mismo valor (Cormen, ejercicio 26.1-2). En una implementación, ∞ se reemplaza por un número grande, por ejemplo la suma de todas las capacidades originales.

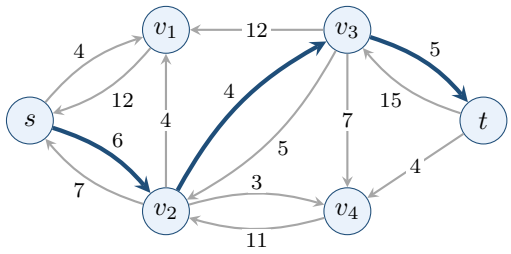
La red residual

La idea más directa es un voraz: mientras haya un camino de s a t cuyas aristas tengan holgura, $f(u,v) < c(u,v)$, mandar por él todo lo que permita su arista más angosta. Toma lo mejor que ve en el momento y nunca revisa una decisión ya tomada. En esta red eso no alcanza.

Ejemplo 20.4 (El voraz se atora en 19)**.** El voraz empieza con el camino s,v_1,v_3,v_2,v_4,t, que admite $\min(16,12,9,14,4) = 4$ unidades. Después busca con BFS entre las aristas con holgura y encuentra s,v_1,v_3,t, con 8, y s,v_2,v_4,v_3,t, con 7. Lleva 19. A la izquierda está ese flujo: (v_1,v_3), (v_4,v_3) y (v_4,t), en naranja, quedaron saturadas, y desde s solo se llega a v_1, v_2 y v_4 por aristas con holgura. No hay más camino y el voraz se detiene.



flujo del voraz, $|f| = 19$



su red residual G_f

Pero 19 no es el máximo. Estorban las 4 unidades que el primer camino mandó de v_3 a v_2: ocupan capacidad de (v_1, v_3) que hace falta para llegar a t por arriba. A la derecha está la red residual de ese flujo, que se define enseguida. En ella sí hay camino, s, v_2, v_3, t, y su arista de en medio, (v_2, v_3), no existe en G: es el regreso de las 4 unidades de (v_3, v_2). Mandar 4 por ese camino deja $f(s, v_2) = 11$, $f(v_3, v_2) = 0$ y $f(v_3, t) = 19$, de valor 23.

El voraz falla cuando una decisión temprana le estorba a una posterior y no tiene forma de deshacerla. La red residual es justo esa forma.

Capacidad residual y aumento

Definición 20.5 (Red residual)**.** Sea f un flujo en G. La **capacidad residual** (*residual capacity*) de un par $u, v \in V$ es

$$c_f(u,v) = \begin{cases} c(u,v) - f(u,v) & \text{si } (u,v) \in E, \\ f(v,u) & \text{si } (v,u) \in E, \\ 0 & \text{en otro caso.} \end{cases}$$

La **red residual** de G inducida por f es $G_f = (V, E_f)$, con

$$E_f = \{(u,v) \in V \times V : c_f(u,v) > 0\}.$$

Sus aristas se llaman **aristas residuales**.

El primer renglón es el espacio que queda en la arista. El segundo es el **regreso de flujo**: si de v a u van $f(v,u)$ unidades, se puede cancelar cualquier parte de ellas, y visto desde el resultado eso equivale a mandar flujo de u a v. Como no hay aristas antiparalelas, a cada par ordenado le toca exactamente un renglón, y $|E_f| \leq 2m$. Si $c(u,v) = 16$ y $f(u,v) = 11$, caben 5 unidades más, $c_f(u,v) = 5$, y se pueden regresar hasta 11, $c_f(v,u) = 11$; una arista saturada solo deja la residual de regreso, y una vacía solo la de ida. G_f no es una red de flujo, porque puede tener pares antiparalelos, pero un flujo en G_f se define igual que en una red, con las capacidades c_f. En las figuras, las aristas residuales de capacidad 0 no se dibujan.

Definición 20.6 (Aumento)**.** Si f es un flujo en G y f' uno en G_f, el **aumento** de f por f' es la función $f \uparrow f'$ dada por

$$(f \uparrow f')(u,v) = \begin{cases} f(u,v) + f'(u,v) - f'(v,u) & \text{si } (u,v) \in E, \\ 0 & \text{en otro caso.} \end{cases}$$

Se suma lo que f' manda por (u,v) y se resta lo que manda por la arista de regreso, (v,u). Empujar por la de regreso se llama **cancelación**: mandar 5 cajas de u a v y 2 de v a u es, visto desde el resultado, mandar 3 de u a v.

Lema 20.7 (Cormen, lema 26.1)**.** *Si f es un flujo en G y f' es un flujo en G_f, entonces $f \uparrow f'$ es un flujo en G y $|f \uparrow f'| = |f| + |f'|$.*

Idea de la prueba. La capacidad sale de $f'(v,u) \leq c_f(v,u) = f(u,v)$ y de $f'(u,v) \leq c_f(u,v) = c(u,v) - f(u,v)$. Para la conservación y el valor se mira el flujo neto $\bar{g}(u,v) = g(u,v) - g(v,u)$: con $h = f \uparrow f'$, en cada par $\bar{h} = \bar{f} + \bar{f'}$, y al sumar sobre v se suman las conservaciones y los valores. □

Caminos aumentantes

Definición 20.8 (Camino aumentante)**.** Un **camino aumentante** (*augmenting path*) es un camino p de s a t en la red residual G_f. Su **capacidad residual** es

$$c_f(p) = \min\{c_f(u,v) : (u,v) \text{ está en } p\},$$

lo más que se puede mandar por todas sus aristas a la vez. El flujo f_p en G_f vale $c_f(p)$ en las aristas de p y 0 en las demás.

Lema 20.9 (Cormen, lema 26.2 y corolario 26.3)**.** *Si p es un camino aumentante, f_p es un flujo en G_f con $|f_p| = c_f(p) > 0$, y $f \uparrow f_p$ es un flujo en G con $|f \uparrow f_p| = |f| + c_f(p) > |f|$.*

Idea de la prueba. f_p no rebasa ninguna $c_f(u,v)$ porque $c_f(p)$ es el mínimo, y como p no repite vértices, a cada vértice intermedio le entra y le sale exactamente una arista de p, las dos con $c_f(p)$. La segunda parte es el lema 20.7 con $f' = f_p$. □

Con este lema el método ya está completo: mientras haya un camino aumentante, aumentar por él, y cada vuelta sube el valor. Falta saber que cuando no queda ningún camino el flujo es máximo, y para eso hacen falta los cortes.

Cortes y el teorema de flujo máximo y corte mínimo

Definición 20.10 (Corte)**.** Un **corte** (*cut*) (S,T) de la red es una partición de V con $s \in S$ y $t \in T = V - S$. El **flujo neto** que lo cruza es

$$f(S,T) = \sum_{u \in S} \sum_{v \in T} f(u,v) - \sum_{u \in S} \sum_{v \in T} f(v,u),$$

y su **capacidad** es

$$c(S,T) = \sum_{u \in S} \sum_{v \in T} c(u,v).$$

Un **corte mínimo** es uno de capacidad mínima entre todos los cortes de la red.

La asimetría es deliberada. El flujo neto cuenta lo que va de S a T y resta lo que regresa de T a S; la capacidad solo cuenta las aristas de S a T, porque una arista de regreso no le agrega espacio al cruce: lo mejor que puede hacer es no llevar nada.

Ejemplo 20.11 (Un corte de la red de Cormen)**.** Con el flujo de valor 19 del ejemplo 20.3, tómese $S = \{s, v_1, v_2\}$ y $T = \{v_3, v_4, t\}$. De S a T cruzan (v_1, v_3) y (v_2, v_4); de T a S, (v_3, v_2). Entonces

$$f(S,T) = 12 + 11 - 4 = 19, \qquad c(S,T) = 12 + 14 = 26.$$

La capacidad de (v_3, v_2), 9, no se suma a $c(S,T)$, pero su flujo, 4, sí se resta en $f(S,T)$. Confundir las dos cuentas rompe la prueba del corolario 20.13.

Que el flujo neto haya salido igual a $|f|$ no es casualidad. Todo lo que sale de s tiene que cruzar el corte para llegar a t, y lo que cruza de regreso se descuenta.

Lema 20.12 (Cormen, lema 26.4)**.** *Para todo flujo f y todo corte (S,T), $f(S,T) = |f|$.*

Idea de la prueba. Se suman a la definición de $|f|$ los ceros que da la conservación en cada $u \in S - \{s\}$, y se parte cada suma interior en $v \in S$ y $v \in T$. Los términos $f(x,y)$ con $x, y \in S$ aparecen una vez sumando y otra restando, y lo que queda es $f(S,T)$. □

Corolario 20.13 (Cormen, corolario 26.5)**.** *Para todo flujo f y todo corte (S,T), $|f| \leq c(S,T)$.*

Idea de la prueba. $|f| = f(S,T)$ por el lema. Tirar los términos no negativos que se restan solo sube la suma, y cada $f(u,v)$ que queda está acotado por $c(u,v)$. □

Cualquier corte es una cota superior para cualquier flujo: el del ejemplo 20.11 dice que ningún flujo pasa de 26. La mejor de esas cotas la da un corte mínimo, y el teorema central dice que esa cota se alcanza.

Teorema 20.14 (Flujo máximo y corte mínimo; Cormen, teorema 26.6)**.** *Si f es un flujo en G, son equivalentes:*

1. *f es un flujo máximo en G.*
2. *La red residual G_f no tiene caminos aumentantes.*
3. *$|f| = c(S,T)$ para algún corte (S,T) de G.*

Demostración. (1) ⇒ (2). Si hubiera un camino aumentante p, por el lema 20.9 el flujo $f \uparrow f_p$ tendría valor mayor que $|f|$, y f no sería máximo.

(2) ⇒ (3). Sea

$$S = \{v \in V : \text{hay un camino de } s \text{ a } v \text{ en } G_f\}$$

y $T = V - S$. Es un corte: $s \in S$, y $t \notin S$ porque en G_f no hay camino de s a t. Sean $u \in S$ y $v \in T$. Si $(u,v) \in E$, entonces $f(u,v) = c(u,v)$: de lo contrario $c_f(u,v) > 0$, y v sería alcanzable desde s pasando por u. Si $(v,u) \in E$, entonces $f(v,u) = 0$: de lo contrario $c_f(u,v) = f(v,u) > 0$, y otra vez v estaría en S. Si ninguna de las dos está en E, las dos valen 0 en f y en c. Así

$$f(S,T) = \sum_{u \in S} \sum_{v \in T} c(u,v) - \sum_{u \in S} \sum_{v \in T} 0 = c(S,T),$$

y por el lema 20.12, $|f| = f(S,T) = c(S,T)$.

(3) ⇒ (1). Por el corolario 20.13, $|f'| \leq c(S,T)$ para todo flujo f'. Si f alcanza esa cota, ningún flujo lo supera. □

Del teorema salen dos cosas. La primera, que el valor de un flujo máximo es igual a la capacidad de un corte mínimo. La segunda, que ese corte se obtiene con una búsqueda desde s sobre la red residual final, en $\mathcal{O}(n+m) = \mathcal{O}(m)$. El corte es un **certificado**: cualquiera puede comprobar que un flujo es máximo sumando unas cuantas capacidades, sin volver a correr nada.

El método de Ford-Fulkerson

El método junta todo lo anterior: empezar con flujo cero y, mientras la red residual tenga un camino aumentante, aumentar por él. Cormen lo llama *método* y no algoritmo porque no dice cómo se encuentra el camino. Cada manera de buscarlo es una implementación distinta, con su propio costo.

Estructura de datos. G_f nunca se construye. Se guarda $G' = (V, E')$, con

$$E' = \{(u, v) : (u, v) \in E \text{ o } (v, u) \in E\},$$

así que la lista de cada vértice trae a sus vecinos de salida y también a los de entrada. Cada entrada sabe si va hacia adelante o de regreso y apunta a la arista original, que guarda c y f. Así $c_f(u, v)$ cuesta $\mathcal{O}(1)$: $c - f$ hacia adelante, f de regreso. Las aristas de G_f son las de G' con $c_f > 0$, y se filtran durante la búsqueda. Sin antiparalelas, $|E'| = 2m$.

Algoritmo 20.1 Ford-Fulkerson(G, s, t)

```
1:  procedimiento Ford-Fulkerson(G, s, t)
2:      para todo (u, v) ∈ E hacer
3:          f(u, v) ← 0
4:      fin para
5:      mientras existe un camino p de s a t en la red residual G_f hacer
6:          c_f(p) ← mín {c_f(u, v) : (u, v) está en p}
7:          para todo (u, v) en p hacer
8:              si (u, v) ∈ E entonces
9:                  f(u, v) ← f(u, v) + c_f(p)          // empuja por una arista de G
10:             si no
11:                 f(v, u) ← f(v, u) − c_f(p)          // cancela flujo de (v, u)
12:             fin si
13:         fin para
14:     fin mientras
15:     devolver f
16: fin procedimiento
```

En la línea 5, "existe un camino" es cualquier búsqueda desde s sobre G_f, BFS o DFS; si no llega a t, el ciclo termina y por el teorema 20.14 el flujo es máximo. En las líneas 8 a 12, cada arista residual de p es una arista de G o la inversa de una, nunca las dos cosas: en el primer caso se empuja flujo y en el segundo se cancela flujo que iba en sentido contrario. Es el aumento $f \uparrow f_p$ escrito arista por arista.

Costo. La inicialización cuesta $\Theta(m)$. Cada iteración hace una búsqueda sobre G', de $\mathcal{O}(n + 2m) = \mathcal{O}(m)$, y recorre p, que tiene a lo más $n - 1$ aristas. Si las capacidades son enteras, por inducción f y todas las c_f son enteras: empiezan en 0 y cada actualización suma o resta el entero $c_f(p)$. Entonces $c_f(p) \geq 1$ y, por el lema 20.9, $|f|$ sube al menos 1 en cada iteración. Como nunca pasa de $|f^*|$, hay a lo más $|f^*|$ iteraciones.

Resultado

Con capacidades enteras, Ford-Fulkerson hace a lo más $|f^*|$ aumentos y cuesta

$$\mathcal{O}(m) + |f^*| \cdot \mathcal{O}(m) = \mathcal{O}(m\,|f^*|)\,.$$

El costo depende del valor del flujo y no solo del tamaño de la red: es pseudopolinomial, porque $|f^*|$ puede ser exponencial en el número de bits de la entrada.

Corolario 20.15 (Integralidad)**.** *Si todas las capacidades son enteras, Ford-Fulkerson devuelve un flujo máximo con $f(u, v)$ entero en cada arista.*

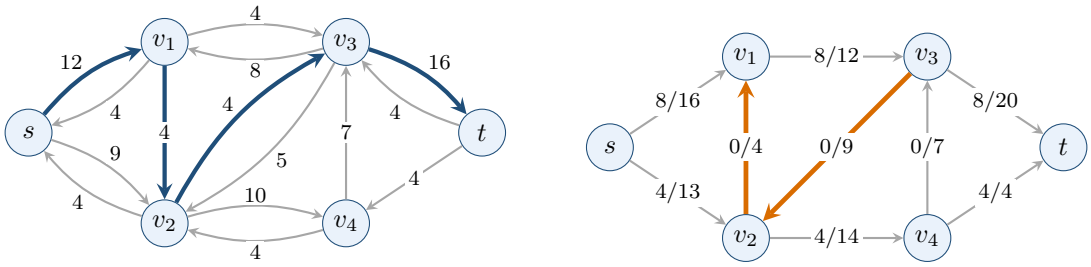
Idea de la prueba. Es la misma inducción del costo: f empieza en 0 y cada actualización suma o resta un entero. Al terminar, el teorema 20.14 dice que el flujo es máximo. □

Es lo que hace funcionar el emparejamiento bipartito de la sección 21. Con capacidades racionales todo se multiplica por el común denominador y el argumento sigue igual. Con irracionales el método puede no terminar, y Cormen advierte que el valor puede ni siquiera converger al máximo.

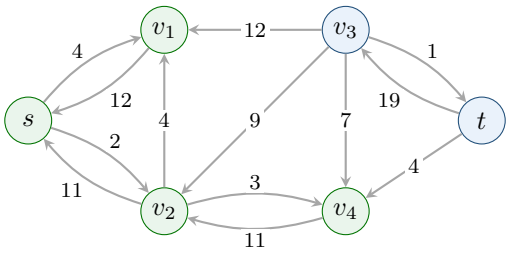
Ejemplo 20.16 (La ejecución de la figura 26.6 de Cormen)**.** Ford-Fulkerson sobre la red de Cormen, desde flujo cero, con los caminos que escoge Cormen. En la última columna, el valor después de cada aumento.

	Camino aumentante p	$c_f(p)$	**Arista crítica**	$\lvert f\rvert$
(a)	s, v_1, v_3, v_2, v_4, t	$\min(16, 12, 9, 14, 4) = 4$	(v_4, t)	4
(b)	s, v_2, v_1, v_3, t	$\min(13, 4, 8, 20) = 4$	(v_2, v_1)	8
(c)	s, v_1, v_2, v_3, t	$\min(12, 4, 4, 16) = 4$	(v_1, v_2) y (v_2, v_3)	12
(d)	s, v_2, v_4, v_3, t	$\min(9, 10, 7, 12) = 7$	(v_4, v_3)	19
(e)	s, v_1, v_3, t	$\min(8, 4, 5) = 4$	(v_1, v_3)	23
(f)	no hay camino			23

La iteración que enseña algo es la (c). Su camino usa dos aristas de regreso: (v_1, v_2), inversa de (v_2, v_1), y (v_2, v_3), inversa de (v_3, v_2). El aumento cancela las 4 unidades de cada una, y $f(v_2, v_1)$ y $f(v_3, v_2)$ vuelven a 0 (en naranja). El flujo de valor 12 se lee ahora como 8 unidades por s, v_1, v_3, t y 4 por s, v_2, v_4, t: el método deshizo el rodeo de (a) y (b). A la izquierda, G_f con el camino aumentante en azul; a la derecha, el flujo que resulta del aumento.



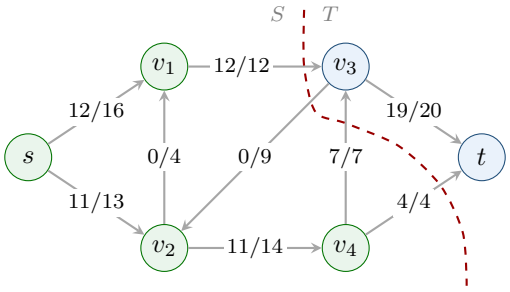
Después de (d) y (e) el flujo vale 23. En la red residual final, desde s se llega a v_1 y a v_2; desde v_2, a v_4; desde v_1 y v_4 solo se regresa. t no es alcanzable y el ciclo termina con $|f^*| = 23$. En verde, los vértices alcanzables desde s.



Ojito...

Los caminos de la figura 26.6 no salen de un BFS: Cormen no dice cómo los eligió, y el primero tiene cinco aristas cuando hay caminos de tres, como s, v_1, v_3, t. Quien intente reproducir la figura con un BFS obtiene otra ejecución, la de Edmonds-Karp. Cualquier elección de caminos es una ejecución válida del método.

Ejemplo 20.17 (El corte mínimo)**.** Los vértices verdes de la red residual final forman $S = \{s, v_1, v_2, v_4\}$, con $T = \{v_3, t\}$. La figura muestra ese corte sobre el flujo máximo, con los vértices de S en verde.



De S a T cruzan (v_1, v_3) con 12/12, (v_4, v_3) con 7/7 y (v_4, t) con 4/4: todas saturadas, como exige la prueba de (2) $\Rightarrow$ (3). La única de T a S es (v_3, v_2), con 0/9: vacía. Entonces

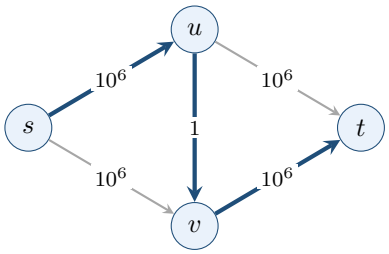
$$f(S,T) = 12 + 7 + 4 - 0 = 23 = c(S,T).$$

La lectura física: a Saskatoon solo pueden entrar $12 + 7 = 19$ cajas diarias, aunque la ruta a Winnipeg aguante 20, y por Regina pasan directo 4 más. No hay manera de enrutar que pase de 23.

Cuando la elección del camino importa

Ford-Fulkerson no dice cómo escoger el camino, y eso decide el costo. La cota $\mathcal{O}(m\,|f^*|)$ no es pesimismo del análisis: hay redes donde se alcanza.

Ejemplo 20.18 (El caso patológico; Cormen, figura 26.7)**.** La red tiene $V = \{s, u, v, t\}$, con $c(s,u) = c(s,v) = c(u,t) = c(v,t) = 10^6$ y $c(u,v) = 1$. El flujo máximo vale $2 \cdot 10^6$: 10^6 por s, u, t y 10^6 por s, v, t, y el corte $(\{s\}, \{u, v, t\})$ tiene capacidad $2 \cdot 10^6$. Supóngase que Ford-Fulkerson elige s, u, v, t en las iteraciones impares y s, v, u, t en las pares. Cada camino pasa por (u, v) o por su inversa, de capacidad residual 1, así que el flujo sube de uno en uno.



i	**Camino**	$c_f(p)$	$f(u,v)$ después	$\lvert f\rvert$
1	s,u,v,t	1	1	1
2	s,v,u,t	1	0	2
3	s,u,v,t	1	1	3
⋮				⋮
$2\cdot 10^6$	s,v,u,t	1	0	$2\cdot 10^6$

A la izquierda, la red original con el primer camino en azul. Tras cada par de aumentos, (u,v) está otra vez disponible con capacidad 1 y las cuatro aristas de los lados perdieron una unidad: el ciclo se repite. Nada en el método impide esa elección; la línea 5 solo pide que exista un camino. Son $2\cdot 10^6$ iteraciones de $\mathcal{O}(m)$ cada una, y la cota $\Theta(m\,|f^*|)$ se alcanza. Mientras tanto, los caminos s,u,t y s,v,t, de dos aristas cada uno, resuelven la red en dos aumentos.

Edmonds-Karp

Edmonds-Karp fija la elección: el camino aumentante es siempre un camino de s a t con el menor número de aristas en G_f, y lo encuentra un BFS. En el ejemplo patológico eso ya basta: los caminos más cortos son s,u,t y s,v,t, y el de tres aristas que pasa por (u,v) nunca se toca.

BFS-Residual(G,f,s) es el BFS de la clase del 22 de octubre (sección 11; a fondo en la sección 14) con dos cambios: recorre las listas de G', que incluyen a los vecinos de entrada, y solo cruza (u,v) si $c_f(u,v)>0$. Recibe f porque con él calcula cada $c_f(u,v)$ en $\mathcal{O}(1)$; la red residual sigue sin construirse. Llena visto$[v]$, $d[v]$ y $\pi[v]$, y su $d[v]$ es la **distancia residual** $\delta_f(s,v)$: el número de aristas de un camino más corto de s a v en G_f, o ∞ si no hay camino.

Algoritmo 20.2 Edmonds-Karp(G, s, t)

```
1:  procedimiento EDMONDS-KARP(G, s, t)
2:      para todo (u, v) ∈ E hacer
3:          f(u, v) ← 0
4:      fin para
5:      ciclo
6:          BFS-RESIDUAL(G, f, s)                          // llena visto, d y π
7:          si no visto[t] entonces
8:              devolver f                                 // no hay camino: f es máximo
9:          fin si
10:         Δ ← ∞,    v ← t
11:         mientras v ≠ s hacer                           // primera pasada: cuello de botella
12:             Δ ← mín(Δ, c_f(π[v], v)),    v ← π[v]
13:         fin mientras
14:         v ← t
15:         mientras v ≠ s hacer                           // segunda pasada: aumento
16:             u ← π[v]
17:             si (u, v) ∈ E entonces
18:                 f(u, v) ← f(u, v) + Δ
19:             si no
20:                 f(v, u) ← f(v, u) − Δ
21:             fin si
22:             v ← u
23:         fin mientras
24:     fin ciclo
25: fin procedimiento
```

Después del BFS, siguiendo π desde t se recorre un camino más corto, al revés. Hacen falta dos pasadas por él porque $c_f(p)$ solo se conoce después de recorrerlo completo; la segunda es la misma actualización que las líneas 7 a 13 de Ford-Fulkerson. El BFS puede detenerse en cuanto descubre t: ahorra trabajo y no cambia la cota.

Ejemplo 20.19 (Edmonds-Karp sobre la red de Cormen; ejercicio 26.2-3)**.** Misma red, pero ahora cada camino lo elige un BFS que revisa los vecinos en el orden s, v_1, v_2, v_3, v_4, t. La tabla da las distancias residuales antes de cada aumento.

	$\delta_f(s,\cdot)$ antes del aumento								
Iter.	v_1	v_2	v_3	v_4	t	**Camino**	$c_f(p)$	**Crítica**	$\lvert f\rvert$
1	1	1	2	2	3	s, v_1, v_3, t	12	(v_1, v_3)	12
2	1	1	3	2	3	s, v_2, v_4, t	4	(v_4, t)	16
3	1	1	3	2	4	s, v_2, v_4, v_3, t	7	(v_4, v_3)	23
fin	1	1	∞	2	∞	no hay camino			

La red residual final es la misma del ejemplo 20.16, y el flujo final también. Son tres aumentos en lugar de cinco, y ninguno usa una arista de regreso. Las distancias ilustran el lema que sigue: $\delta_f(s, v_3)$ pasa de 2 a 3 y luego a ∞, $\delta_f(s, t)$ va de 3 a 3 a 4, y ninguna baja. Cada aumento tiene exactamente una arista crítica, y ninguna se repite.

Por qué Edmonds-Karp es polinomial

Todo el análisis descansa en un hecho sobre las distancias residuales: con caminos más cortos, ningún vértice se acerca a s.

Lema 20.20 (Distancias residuales; Cormen, lema 26.7)**.** *Durante Edmonds-Karp, para todo $v \in V$, la distancia residual $\delta_f(s, v)$ nunca baja de un aumento al siguiente.*

Idea de la prueba. Se toma el primer aumento que baja alguna distancia y el vértice v más cercano a s entre los que bajaron; sea u su antecesor en un camino más corto de $G_{f'}$. Si (u, v) ya era residual, la desigualdad del triángulo (propiedad 17.7) impide que v haya bajado; si apareció con el aumento, el camino usó (v, u) y sale $\delta_f(s, v) \leq \delta_{f'}(s, v) - 2$. Las dos cosas contradicen que la distancia de v bajó. □

Ojito...

El lema dice "nunca baja", no "siempre sube": en el ejemplo 20.19, $\delta_f(s, v_1)$ y $\delta_f(s, v_2)$ se quedan en 1 todo el tiempo. Y la garantía es del BFS: con DFS los caminos no son los más cortos y el lema deja de valer.

Definición 20.21 (Arista crítica)**.** Una arista residual (u, v) de un camino aumentante p es **crítica** si $c_f(u, v) = c_f(p)$: es un cuello de botella.

Todo camino aumentante tiene al menos una arista crítica, y después del aumento esa arista sale de G_f: si era una arista de E, quedó saturada; si era la inversa de una, el flujo de esa arista volvió a 0.

Teorema 20.22 (Cormen, teorema 26.8)**.** *Edmonds-Karp hace a lo más nm aumentos.*

Idea de la prueba. Si (u, v) es crítica con flujo f, sale de G_f y solo vuelve si un aumento posterior, con flujo f', empuja por (v, u). Los dos caminos son de los más cortos y las distancias no bajan, así que $\delta_{f'}(s, u) \geq \delta_f(s, u) + 2$. Como $\delta_f(s, u) \leq n - 2$, cada una de las a lo más $2m$ aristas residuales posibles es crítica a lo más $n/2$ veces. □

Costo. Cada iteración hace un BFS sobre G', de $\mathcal{O}(n + 2m) = \mathcal{O}(m)$ con listas, y dos pasadas por el camino, de $\mathcal{O}(n)$. Son a lo más nm iteraciones, más un último BFS que no encuentra camino.

Resultado

Edmonds-Karp hace $\mathcal{O}(nm)$ aumentos y cuesta

$$\mathcal{O}(nm) \cdot \mathcal{O}(m) + \mathcal{O}(m) = \mathcal{O}\left(nm^2\right),$$

sin importar las capacidades.

La cota no usa que las capacidades sean enteras, así que Edmonds-Karp termina con capacidades reales cualesquiera. Y como sigue siendo una implementación de Ford-Fulkerson, con capacidades enteras también cumple $\mathcal{O}(m\,|f^*|)$; vale la menor de las dos cotas. En el caso patológico son $nm = 20$ contra $2 \cdot 10^6$, y bastan 2. En un grafo denso, $m = \Theta(n^2)$ y $\mathcal{O}(nm^2) = \mathcal{O}(n^5)$. Eso motiva los algoritmos de empujar y reetiquetar de la sección 22, que no buscan caminos aumentantes y llegan a $\mathcal{O}(n^2 m)$ y a $\mathcal{O}(n^3)$.

Errores comunes

Ojito...

- **Antiparalelas en la entrada.** La definición de c_f supone que no hay. Un grafo no dirigido, donde cada arista se modela con dos dirigidas, las tiene siempre. Hay dos salidas: partir cada par con un vértice v', o usar $c_f(u,v) = c(u,v) - f(u,v) + f(v,u)$.
- **El flujo máximo no es único; su valor sí.** Aumentar el flujo de valor 19 del ejemplo 20.3 por s, v_2, v_3, t da un flujo de valor 23 con $f(v_2, v_1) = 1$; el de la figura 26.6 vale 23 con $f(v_2, v_1) = 0$. También puede haber varios cortes mínimos, y la búsqueda final entrega uno.
- **La red es dirigida.** El flujo máximo de s a t no es el de t a s. En la red de Cormen no sale ninguna arista de t, y el flujo de t a s vale 0.

21. Emparejamiento bipartito máximo

Complemento: Cormen, cap. 26.3

Resumen rápido. Un emparejamiento es un conjunto de aristas que no comparten extremos: cada alumno toma a lo más un proyecto. En un grafo bipartito, el emparejamiento más grande se encuentra con flujo máximo: se agregan una fuente y un sumidero, todas las capacidades valen 1, y los flujos enteros de esa red son exactamente los emparejamientos; Ford-Fulkerson lo resuelve en $\mathcal{O}(nm)$. Sobre el grafo original, cada camino aumentante es una cadena de reasignaciones, y el teorema de Berge dice que un emparejamiento es máximo justo cuando no queda ninguna. Cierran los teoremas de Hall y de König, que dan testigos revisables a mano, la idea de Hopcroft-Karp, en $\mathcal{O}(\sqrt{n}\,m)$, y la razón por la que todo esto se queda corto fuera de los bipartitos.

El problema

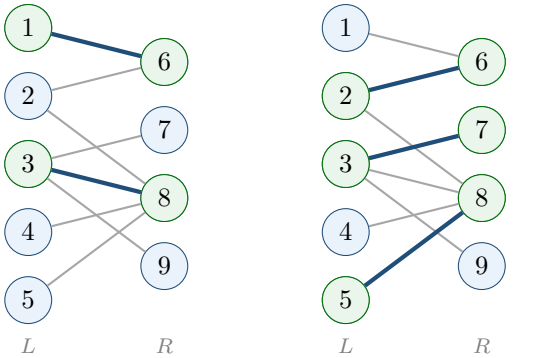
Cormen lo plantea con máquinas y tareas. Hay un conjunto L de máquinas y un conjunto R de tareas; la arista $\{u, v\}$ dice que la máquina u sabe hacer la tarea v. Cada máquina hace a lo más una tarea y cada tarea la hace a lo más una máquina, y se quiere dar trabajo a tantas máquinas como se pueda. El grafo es bipartito: $V = L \cup R$ con $L \cap R = \emptyset$, y toda arista une L con R; son los V_1 y V_2 de la sección 13.

Definición 21.1 (Emparejamiento)**.** En un grafo no dirigido $G = (V, E)$, un **emparejamiento** (*matching*) es un conjunto $M \subseteq E$ tal que cada vértice es incidente en a lo más una arista de M. Un vértice incidente en una arista de M está **emparejado**; si no, está **libre**. Si $\{u, v\} \in M$, v es la **pareja** de u y u la de v.

Definición 21.2 (Máximo, maximal y perfecto)**.** Un emparejamiento M es **máximo** (*maximum*) si $|M| \geq |M'|$ para todo emparejamiento M' de G; **maximal** (*maximal*) si no está contenido propiamente en otro emparejamiento, es decir, si ninguna arista de $E - M$ se le puede agregar; y **perfecto** (*perfect*) si todo vértice de G está emparejado.

Todo máximo es maximal, pero no al revés, y esa diferencia es todo el problema; un perfecto es máximo, y en un bipartito exige $|L| = |R|$. El **problema del emparejamiento bipartito máximo** pide un emparejamiento máximo de un G bipartito. Cormen supone que todo vértice tiene al menos una arista, de donde $m \geq n/2$.

Ejemplo 21.3 (Figura 26.8 de Cormen: maximal contra máximo)**.** Los vértices se numeran como en el ejercicio 26.3-1 de Cormen: $L = \{1, 2, 3, 4, 5\}$ y $R = \{6, 7, 8, 9\}$, de arriba abajo, con $E = \{\{1, 6\}, \{2, 6\}, \{2, 8\}, \{3, 7\}, \{3, 8\}, \{3, 9\}, \{4, 8\}, \{5, 8\}\}$, $n = 9$ y $m = 8$. En azul van las aristas del emparejamiento y en verde los vértices emparejados.



(a) maximal, $|M_a| = 2$ (b) máximo, $|M_b| = 3$

En (a), $M_a = \{\{1, 6\}, \{3, 8\}\}$ es maximal: cada una de las otras seis aristas toca a 1, 3, 6 u 8. En (b),

$M_b = \{\{2,6\},\{3,7\},\{5,8\}\}$ es máximo, y el argumento es corto: 7 y 9 solo tienen de vecino a 3, así que a lo más uno de los dos queda emparejado, y de R se emparejan a lo más tres vértices.

La forma obvia de construir un emparejamiento es voraz: tomar cada arista que no choque con las ya tomadas. Lo que sale es maximal, pero su tamaño depende del orden: si empieza por $\{1,6\}$ y $\{3,8\}$, se atora en M_a. Lo único que garantiza es la mitad, $|M^*| \le 2\,|M|$, porque cada arista del máximo M^* comparte un extremo con alguna de M, y esos extremos son distintos.

Ojito...

Maximal no es máximo: M_a es maximal y mide 2, y el máximo mide 3. El máximo no es único: la figura 26.8 tiene 10 emparejamientos de tamaño 3, todos máximos. Y máximo no es perfecto: ahí $|L| = 5$ y $|R| = 4$, así que algún vértice de L se queda libre siempre.

Reducción a flujo máximo

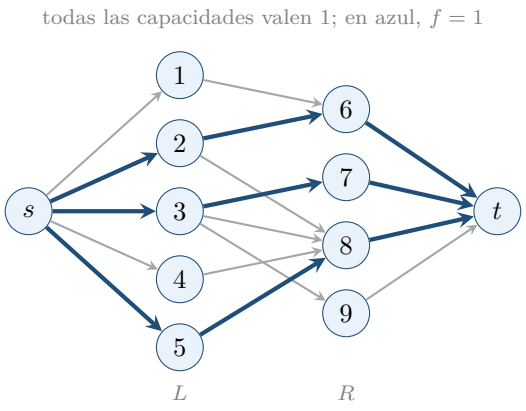
Definición 21.4 (Red de flujo correspondiente)**.** Sea $G = (V, E)$ bipartito con $V = L \cup R$. Su **red de flujo correspondiente** es $G' = (V', E')$ con $V' = V \cup \{s, t\}$ y

$$E' = \{(s,u) : u \in L\} \cup \{(u,v) : u \in L,\ v \in R,\ \{u,v\} \in E\} \cup \{(v,t) : v \in R\},$$

con capacidad $c(u,v) = 1$ en todas las aristas de E'.

Cada arista de G se orienta de L a R, la fuente s apunta a todo L y todo R apunta al sumidero t. La red tiene $|E'| = m + n = \Theta(m)$ aristas. Todo el modelado está en las capacidades de (s,u) y (v,t), que son capacidades de vértice disfrazadas: a $u \in L$ solo le entra (s,u), así que le entra a lo más una unidad, y por conservación esa unidad sale por una sola arista hacia R. Del lado de R pasa lo mismo con (v,t).

Para la figura 26.8 la red tiene 11 vértices y 17 aristas. En azul va el flujo que corresponde a M_b: una unidad por cada uno de los caminos $s,2,6,t$, $s,3,7,t$ y $s,5,8,t$, de valor $|f| = 3$.



Algoritmo 21.1 Emparejamiento-Bipartito(G, L, R)

```
1:  procedimiento Emparejamiento-Bipartito(G, L, R)
2:      V' ← V ∪ {s, t},    E' ← ∅
3:      para todo u ∈ L hacer
4:          E' ← E' ∪ {(s, u)}
5:      fin para
6:      para todo {u, v} ∈ E con u ∈ L, v ∈ R hacer
7:          E' ← E' ∪ {(u, v)}                          // solo de L a R
8:      fin para
9:      para todo v ∈ R hacer
10:         E' ← E' ∪ {(v, t)}
11:     fin para
12:     c(e) ← 1 para toda e ∈ E'
13:     f ← Ford-Fulkerson(G', s, t)
14:     devolver M = {{u, v} : u ∈ L, v ∈ R, f(u, v) = 1}
15: fin procedimiento
```

La línea 13 es el algoritmo 20.1, con cualquier forma de buscar el camino; con BFS es Edmonds-Karp (algoritmo 20.2). Que la línea 14 funcione es justo lo que hay que demostrar: que los flujos enteros son emparejamientos y que el flujo que devuelve Ford-Fulkerson es entero.

Lema 21.5 (Emparejamientos y flujos enteros)**.** *Si M es un emparejamiento de G, existe un flujo entero f en G' con $|f| = |M|$. Recíprocamente, si f es un flujo entero en G', entonces $M = \{\{u, v\} : u \in L,\ v \in R,\ f(u, v) = 1\}$ es un emparejamiento de G con $|M| = |f|$.*

Demostración. Ida. Por cada $\{u, v\} \in M$, con $u \in L$, se pone $f(s, u) = f(u, v) = f(v, t) = 1$, y 0 en las demás aristas. Las capacidades se respetan: cada $u \in L$ está en a lo más una arista de M, así que (s, u) recibe a lo más una unidad, y lo mismo (v, t) del lado de R. Se conserva el flujo: a un $u \in L$ emparejado le entra 1 por (s, u) y le sale 1 por (u, v), y a uno libre no le entra ni le sale nada; en R igual. A s no le entra ninguna arista, así que $|f| = \sum_{u \in L} f(s, u)$, que cuenta los vértices emparejados de L: $|f| = |M|$.

Vuelta. Como f es entero y $0 \le f \le 1$, cada arista lleva 0 o 1. A $u \in L$ solo le entra (s, u), así que le entra 0 o 1; por conservación le sale lo mismo, y como cada arista de salida lleva 0 o 1, a lo más una arista (u, v) lleva 1. Del lado de R, a v solo le sale (v, t), y el mismo argumento dice que a lo más una arista (u, v) que le entra lleva 1. Entonces ningún vértice está en dos aristas de M, y M es un emparejamiento. Por último, sumando la conservación en cada $u \in L$, $|f| = \sum_{u \in L} f(s, u) = \sum_{u \in L} \sum_{v \in R} f(u, v) = |M|$. □

Ojito…

El flujo tiene que ser entero. En $K_{2,2}$, con $L = \{a, b\}$ y $R = \{x, y\}$, poner $1/2$ en las cuatro aristas de en medio y 1 en las de s y de t es un flujo válido de valor 2, máximo, y de él no se lee ningún emparejamiento. Un algoritmo de flujo máximo cualquiera podría devolverlo; lo que salva a la reducción es el teorema siguiente, que vale para los caminos aumentantes.

Teorema 21.6 (Integralidad)**.** *Si todas las capacidades son enteras, el flujo máximo f que devuelve Ford-Fulkerson tiene $|f|$ entero y $f(u, v)$ entero en cada arista.*

Idea de la prueba. Por inducción sobre las iteraciones. Si f es entero, todas las capacidades residuales (definición 20.5) son enteras, $c_f(p)$ es entero y el aumento suma o resta ese entero. Es el argumento

que acota las iteraciones de Ford-Fulkerson en la sección 20. □

Corolario 21.7. *El tamaño de un emparejamiento máximo de G es igual al valor de un flujo máximo en G', y el emparejamiento que devuelve* Emparejamiento-Bipartito *es máximo.*

Idea de la prueba. El flujo máximo f^* que devuelve Ford-Fulkerson es entero, y el lema 21.5 lo convierte en un emparejamiento M con $|M| = |f^*|$. Cualquier otro M' da un flujo de valor $|M'|$, que no pasa de $|f^*|$. □

Costo. Construir G' (líneas 2 a 12) cuesta $\Theta(n+m)$. Cada iteración de Ford-Fulkerson busca un camino en G'_f, que tiene $n+2$ vértices y a lo más $2(m+n)$ aristas: $\mathcal{O}(m)$, usando $n \leq 2m$. Cada aumento sube $|f|$ en exactamente 1, porque todas las capacidades residuales son 0 o 1, y el flujo máximo es $|M^*| \leq \min(|L|, |R|) \leq n/2$: hay a lo más $n/2$ aumentos y una búsqueda final que falla. En total, $\mathcal{O}(n) \cdot \mathcal{O}(m) = \mathcal{O}(nm)$, con BFS o con DFS. Es la cota $\mathcal{O}(m\ |f^*|)$ de Ford-Fulkerson con $|f^*| \leq n/2$.

Ojito...

Las aristas de G' van solo de L a R. Si se dejaran en los dos sentidos, un camino de flujo podría cruzar de L a R, regresar a L y volver a cruzar, y una unidad de flujo ya no correspondería a una arista del emparejamiento: la prueba del lema se cae en el paso que dice que a $u \in L$ solo le entra (s, u).

Caminos aumentantes en el grafo original

Ford-Fulkerson trabaja sobre G', pero lo que hace se entiende mejor en G, y así se escribe una versión que no construye G'. Cormen deja esta correspondencia implícita en el problema 26-6.

Definición 21.8 (Camino aumentante respecto a M)**.** Sea M un emparejamiento de G bipartito. Un camino es **alternante** si sus aristas están, una sí y una no, en $E - M$ y en M. Un **camino aumentante** respecto a M (*augmenting path*) es un camino alternante que empieza en un vértice libre de L y termina en un vértice libre de R.

La primera y la última arista de un camino aumentante están fuera de M, así que tiene k aristas de M y $k+1$ fuera de M, y su longitud $2k+1$ es impar. La **diferencia simétrica** de dos conjuntos es $A \oplus B = (A - B) \cup (B - A)$: lo que está en uno solo de los dos.

Proposición 21.9 (La residual de un emparejamiento)**.** *Sea M un emparejamiento de G y f el flujo entero que le corresponde en G' por el lema 21.5. La red residual G'_f tiene exactamente estas aristas, con $u \in L$ y $v \in R$:*

- *(s, u) si u está libre, y (u, s) si está emparejado;*
- *(u, v) si $\{u, v\} \in E - M$, y (v, u) si $\{u, v\} \in M$;*
- *(v, t) si v está libre, y (t, v) si está emparejado.*

En consecuencia, los caminos simples de s a t en G'_f son exactamente los $s, u_0, v_0, u_1, v_1, \ldots, u_k, v_k, t$ tales que $P = u_0, v_0, \ldots, u_k, v_k$ es un camino aumentante respecto a M, y aumentar el flujo por uno de ellos produce el flujo de $M \oplus P$.

Idea de la prueba. Una arista de G' aparece en G'_f hacia adelante si lleva 0 e invertida si lleva 1, y de ahí salen las tres líneas. Desde L solo se sale por aristas fuera de M, y desde un $v \in R$ emparejado la

única salida es hacia su pareja: un camino simple de s a t alterna solo hasta llegar a un libre de R. Aumentar invierte cada arista del camino, que es aplicar $\oplus P$. □

Lema 21.10. *Si P es un camino aumentante respecto a M, entonces $M \oplus P$ es un emparejamiento y $|M \oplus P| = |M| + 1$.*

Idea de la prueba. $M \oplus P$ quita las k aristas de M que están en P y agrega las $k+1$ de fuera. Cada vértice interior de P cambia una arista de M por otra, y los extremos, que estaban libres, ganan una sola. □

En la figura 26.8, todos los caminos aumentantes respecto a M_a miden 3 y pasan por $\{3, 8\}$, la arista que estorba: ocupa a la vez al único vecino de 7 y 9 y al único vecino de 4 y 5. Con $P = 4$–8–3–7 entran $\{4, 8\}$ y $\{3, 7\}$ y sale $\{3, 8\}$: $M_a \oplus P = \{\{1, 6\}, \{3, 7\}, \{4, 8\}\}$, de tamaño 3. Respecto a M_b no hay ninguno.

Teorema 21.11 (Berge)**.** *Un emparejamiento M es máximo si y solo si no existe ningún camino aumentante respecto a M.*

Idea de la prueba. Si hay un camino aumentante, el lema 21.10 da uno más grande. Al revés, si $|M^*| > |M|$, en $M \oplus M^*$ cada vértice tiene grado a lo más 2, así que se parte en caminos y ciclos que alternan entre M y M^*. Los ciclos están balanceados, así que algún camino tiene una arista más de M^* que de M; sus extremos están libres en M, y es aumentante. □

Con la proposición 21.9, el teorema de Berge para bipartitos es la equivalencia entre flujo máximo y ausencia de caminos aumentantes del teorema 20.14. La prueba de arriba no usa flujo y usa la bipartición solo para nombrar los extremos del camino; por eso sigue valiendo en cualquier grafo.

El algoritmo escrito sobre G

Con la proposición 21.9, Ford-Fulkerson con BFS sobre G' se escribe sin construir G': Emparejamiento-Máximo aplica caminos aumentantes mientras Camino-Aumentante encuentre uno.

Algoritmo 21.2 Emparejamiento-Máximo(G, L, R)

```
1:  procedimiento Emparejamiento-Máximo(G, L, R)
2:      M ← ∅
3:      repetir
4:          P ← Camino-Aumentante(G, M)
5:          si P ≠ nulo entonces
6:              M ← M ⊕ P                          // crece en uno
7:          fin si
8:      hasta que P = nulo
9:      devolver M
10: fin procedimiento
```

Camino-Aumentante es el BFS de la clase del 22 de octubre con dos cambios: arranca con todos los vértices libres de L en la cola, y al llegar a un vértice emparejado de R salta directo a su pareja. El arreglo pareja$[v]$ guarda la pareja de cada vértice, o nulo si está libre.

Algoritmo 21.3 Camino-Aumentante(G, M)

```
1:  procedimiento Camino-Aumentante(G, M)
2:      para todo v ∈ V hacer
3:          visto[v] ← falso,    π[v] ← nulo
4:      fin para
5:      Q ← cola vacía
6:      para todo u ∈ L libre en M hacer
7:          visto[u] ← verdadero;  Encolar(Q, u)
8:      fin para
9:      mientras Q no está vacía hacer
10:         u ← Desencolar(Q)                          // siempre u ∈ L
11:         para todo v adyacente a u con {u, v} ∉ M hacer
12:             si no visto[v] entonces
13:                 visto[v] ← verdadero,    π[v] ← u
14:                 si v está libre en M entonces
15:                     devolver el camino de v hacia atrás por π
16:                 fin si
17:                 w ← pareja[v]                      // {w, v} ∈ M
18:                 visto[w] ← verdadero,    π[w] ← v
19:                 Encolar(Q, w)
20:             fin si
21:         fin para
22:     fin mientras
23:     devolver nulo
24: fin procedimiento
```

Los libres de L son raíces a la vez (líneas 6 a 8), el primer nivel del BFS desde s en G'. El primer libre de R que se descubre cierra el camino (líneas 14 a 16), y desde un v emparejado solo se sigue a su pareja w (líneas 17 a 19). No hace falta preguntar si w ya fue visto: no fue raíz, y solo se llega a él desde v, que se acaba de visitar por primera vez. Por eso en la cola solo hay vértices de L.

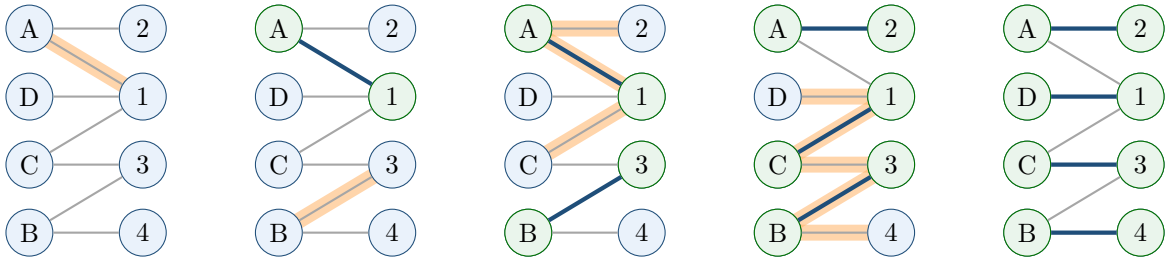
Emparejamiento-Máximo termina con un máximo por el teorema de Berge. Una llamada a Camino-Aumentante cuesta $\Theta(n+m)$, hay a lo más $|M^*|+1 \le n/2+1$ llamadas, y $M \oplus P$ cuesta $\mathcal{O}(n)$ actualizando pareja[ ]. En total $\mathcal{O}(nm)$, igual que la reducción, y no es casualidad: es el mismo algoritmo. Con los vecinos en el mismo orden, elige exactamente los mismos caminos que Edmonds-Karp sobre G'.

Ejemplo: alumnos y proyectos

Ejemplo 21.12 (El emparejamiento crece de 1 a 4)**.** Cuatro alumnos, Ana, Beto, Caro y Dani, forman $L = \{A, B, C, D\}$, y cuatro proyectos forman $R = \{1, 2, 3, 4\}$. Ana acepta los proyectos 1 y 2, Beto el 3 y el 4, Caro el 1 y el 3, y Dani solo el 1. Se abrevia $\{A, 1\}$ como $A1$, así que $E = \{A1, A2, B3, B4, C1, C3, D1\}$, $n = 8$ y $m = 7$. Se corre Emparejamiento-Máximo, que es Edmonds-Karp sobre G', con los vecinos en orden alfabético y numérico.

It.	**Camino en G'_f**	**En G**	**M después**	$\lvert M\rvert$
1	$s, A, 1, t$	A–1	$\{A1\}$	1
2	$s, B, 3, t$	B–3	$\{A1, B3\}$	2
3	$s, C, 1, A, 2, t$	C–1–A–2	$\{A2, B3, C1\}$	3
4	$s, D, 1, C, 3, B, 4, t$	D–1–C–3–B–4	$\{A2, B4, C3, D1\}$	4

En cada instantánea, el trazo naranja es el camino que se aplica en esa iteración, dibujado sobre el M de antes; los vértices van en el orden A, D, C, B y 2, 1, 3, 4 para que ninguna arista se cruce.



it. 1: A–1 | it. 2: B–3 | it. 3: C–1–A–2 | it. 4: D–1–C–3–B–4 | final

Las dos primeras iteraciones son voraces. En la tercera, Caro quiere el 1, que es de Ana, y el BFS encuentra que Ana puede mudarse al 2, que está libre. En la cuarta, Dani solo acepta el 1, que ahora es de Caro: Caro se muda al 3, que es de Beto, y Beto al 4. Cada camino aumentante es una cadena de reasignaciones que termina en un proyecto libre; en la residual son las aristas de M invertidas, de R a L, las que permiten deshacer una asignación.

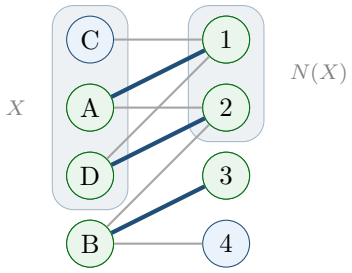
Cuando no hay emparejamiento perfecto: Hall y König

La **vecindad** de $X \subseteq V$ es $N(X) = \{y \in V : \{x, y\} \in E \text{ para algún } x \in X\}$. Si un conjunto X de alumnos acepta entre todos menos proyectos que alumnos tiene, $|N(X)| < |X|$, es imposible acomodarlos a todos. Lo notable es que esa es la única obstrucción.

Teorema 21.13 (Hall)**.** *Sea G bipartito con $V = L \cup R$. Existe un emparejamiento que cubre todo L si y solo si $|A| \leq |N(A)|$ para todo $A \subseteq L$. En particular, si $|L| = |R|$, G tiene un emparejamiento perfecto si y solo si se cumple esa condición.*

Idea de la prueba. La ida es directa: las parejas de los vértices de A son distintas y están en $N(A)$. Para la vuelta, si el emparejamiento máximo M no cubre L, sea S el conjunto de alcanzables desde s en G'_f, el de la prueba del teorema 20.14, y $X = L \cap S$. La proposición 21.9 da tres hechos: (1) todo libre de L está en X; (2) $N(X) = R \cap S$; (3) todo $v \in R \cap S$ está emparejado, con su pareja en X. Entonces $|N(X)|$ es $|X|$ menos el número de libres de L, y X viola la condición. □

Ejemplo 21.14 (Tres alumnos para dos proyectos)**.** Los mismos cuatro alumnos, con otras preferencias: Ana acepta 1 y 2, Beto 2, 3 y 4, Caro solo el 1 y Dani 1 y 2. Emparejamiento-Máximo encuentra $A1$, después $B2$, y luego el camino D–2–B–3: Dani toma el 2 y Beto se muda al 3. Queda $M = \{A1, B3, D2\}$. La cuarta búsqueda arranca de Caro, el único libre de L, llega a 1, de ahí a su pareja A, a 2 y a su pareja D, y se agota. Los alcanzables son $S = \{s, C, 1, A, 2, D\}$, así que $X = \{A, C, D\}$ y $N(X) = \{1, 2\}$, sombreados en la figura: $|N(X)| = 2 < 3 = |X|$. Tres alumnos compiten por dos proyectos, y ningún algoritmo puede acomodar a los cuatro. El subconjunto se llama X y no A porque Ana ya es A.



Del mismo conjunto S sale un segundo certificado. Una **cubierta de vértices** de G es un conjunto $C \subseteq V$ que toca todas las aristas.

Teorema 21.15 (König)**.** *En un grafo bipartito, el tamaño de un emparejamiento máximo es igual al de una cubierta de vértices mínima.*

Idea de la prueba. Cada arista de un emparejamiento necesita su propio vértice en una cubierta, así que $|M'| \leq |C'|$ siempre. Con el máximo M y el S de la prueba de Hall, cuyos tres hechos no usan que M deje libres en L, la cubierta $C = (L - S) \cup (R \cap S)$ toca todas las aristas y tiene exactamente un extremo de cada arista de M. □

Para convencer a alguien de que M es máximo basta enseñarle $|M|$ vértices que tocan todas las aristas, y eso se revisa a mano. En la figura 26.8, $\{3, 6, 8\}$ toca las ocho aristas; en el ejemplo 21.14, la cubierta es $\{B\} \cup \{1, 2\}$. La cubierta es el corte mínimo leído en G: las aristas de G' que van de S a $V' - S$ son exactamente las (s, u) con $u \in L - S$ y las (v, t) con $v \in R \cap S$.

Hopcroft-Karp: varios caminos por fase

Emparejamiento-Máximo hace un BFS completo para agregar una sola arista. Hopcroft y Karp (problema 26-6 de Cormen) aprovechan cada BFS para varios caminos. Sea ℓ la longitud del camino aumentante más corto respecto a M. Una **fase** encuentra un conjunto *maximal* de caminos aumentantes $P_1, \ldots, P_k$ de longitud ℓ, ajenos en vértices, y los aplica todos juntos; el algoritmo repite fases hasta que no queda camino aumentante. Como los P_i no comparten vértices, por el lema 21.10 $M \oplus (P_1 \cup \cdots \cup P_k)$ es un emparejamiento de tamaño $|M| + k$. Basta un conjunto maximal, no uno máximo, y sale con un BFS desde todos los libres de L a la vez, que arma las capas y da ℓ, más un DFS que solo baja de una capa a la siguiente y borra cada vértice que usa o que resulta un callejón sin salida. Cada arista se revisa una vez, y una fase cuesta $\mathcal{O}(m)$.

El número de fases descansa en tres hechos, que Cormen deja como partes del problema. Primero, después de una fase el camino aumentante más corto mide más de ℓ, así que ℓ sube al menos 2 por fase; su prueba es la parte larga y se omite. Segundo, si el más corto mide ℓ, entonces $|M^*| \leq |M| + n/(\ell+1)$, porque $M \oplus M^*$ contiene al menos $|M^*| - |M|$ caminos aumentantes ajenos de al menos $\ell + 1$ vértices cada uno. Tercero, tras $\sqrt{n}$ fases, $\ell > \sqrt{n}$ y faltan menos de $\sqrt{n}$ aristas, una por fase como mínimo. Son a lo más $2\sqrt{n}$ fases y el costo es $\mathcal{O}(\sqrt{n}\, m)$, contra $\mathcal{O}(nm)$. En el ejemplo 21.12 los caminos medían 1, 1, 3 y 5, y Hopcroft-Karp lo resuelve en tres fases, con $\ell = 1, 3, 5$.

Grafos que no son bipartitos

La reducción a flujo necesita la partición en L y R para orientar las aristas. En un triángulo el emparejamiento máximo mide 1, pero no hay manera de repartir sus vértices en dos lados. El teorema de Berge sigue valiendo, con camino aumentante definido como un camino alternante entre dos vértices libres. Lo que se rompe es la búsqueda.

Ejemplo 21.16 (Una búsqueda alternante que se pierde)**.** El grafo es el ciclo a, b, c, d, e de longitud 5 más la arista $\{e, y\}$, con $M = \{\{b, c\}, \{d, e\}\}$. Los libres son a e y, y $P = a$–b–c–d–e–y es un camino aumentante. Una búsqueda como Camino-Aumentante sale de a, descubre b y e y salta a sus parejas, c y d; desde ahí la única arista fuera de M es $\{c, d\}$, entre dos vértices ya vistos, y se detiene sin encontrar nada. El problema es e: se le marcó como visto en el papel de un vértice de R, y el camino necesita llegar a él por su arista de M, desde d, y salir por $\{e, y\}$. En un bipartito cada vértice tiene un solo papel posible; aquí e tiene dos, porque el ciclo es impar.

Por el teorema 13.9, ese ciclo impar no aparece en un bipartito. Edmonds (1965) lo llamó **flor** (*blossom*); su algoritmo, que queda fuera del capítulo 26 de Cormen, la contrae a un vértice, sigue buscando y expande el camino que encuentre, en tiempo polinomial. Por eso, antes de aplicar la reducción hay que verificar que el grafo es bipartito y obtener L y R, con el algoritmo 13.4 de la sección 13, en $\Theta(n + m)$.

22. Empujar y reetiquetar

Complemento: Cormen, cap. 26.4 y 26.5

Resumen rápido. Ford-Fulkerson mejora un flujo buscando caminos completos de s a t. Los algoritmos de empujar y reetiquetar trabajan al revés: saturan de golpe todo lo que sale de la fuente y después cada vértice resuelve, mirando solo a sus vecinos, qué hacer con lo que le sobra. Lo empuja cuesta abajo, o sube de altura cuando no tiene a dónde empujarlo. Durante la ejecución no hay un flujo sino un preflujo, y una función de altura garantiza desde el primer paso que no existe camino aumentante; cuando ningún vértice interno guarda exceso, el preflujo es un flujo y es máximo, y esa corrección se prueba completa. El algoritmo genérico cuesta $\mathcal{O}(n^2m)$ sin importar las capacidades. Reetiquetar al frente fija el orden de las operaciones con una lista que siempre es orden topológico de las aristas por las que se puede empujar, y baja a $\mathcal{O}(n^3)$. La sección cierra con una tabla de todos los algoritmos de la parte de grafos.

La idea: agua que corre cuesta abajo

El método de Ford-Fulkerson (algoritmo 20.1) mantiene un flujo válido en todo momento y, para mover unas cuantas unidades, busca un camino completo de s a t en la red residual. Los algoritmos de **empujar y reetiquetar** (*push-relabel*, de Goldberg y de Goldberg y Tarjan) son locales: operan sobre un vértice a la vez y solo miran las aristas residuales que salen de él. A cambio renuncian, durante la ejecución, a la conservación: lo que sobra en un vértice se queda guardado en él.

Cormen lo explica con fluidos. Las aristas son tuberías y cada vértice tiene un depósito donde se acumula el agua que entró y todavía no sale. Todo está montado sobre plataformas de altura ajustable, y el agua solo corre cuesta abajo. La fuente está fija a altura n y el sumidero a altura 0; los demás vértices empiezan en 0 y solo suben. Al inicio se llenan todas las tuberías que salen de s. Un vértice con agua la empuja hacia un vecino exactamente un nivel abajo, por una tubería con holgura; si no tiene ninguno, se **reetiqueta**: sube a un nivel por encima del más bajo de esos vecinos. Lo que cabe hasta t llega a t. El sobrante se queda en los depósitos, que siguen subiendo hasta rebasar la altura de s, y regresa a la fuente por las tuberías por las que llegó. Al final ningún depósito interno tiene agua y lo que llegó a t es un flujo máximo.

Idea clave

Ford-Fulkerson conserva la **factibilidad** (siempre tiene un flujo) y trabaja hasta alcanzar la **optimalidad** (que no quede camino aumentante). Empujar y reetiquetar conserva la optimalidad desde el primer paso (nunca hay camino de s a t en la red residual) y trabaja hasta alcanzar la factibilidad (que se vacíen los depósitos). Los dos terminan en el mismo punto, un flujo sin camino aumentante, que es máximo por el teorema de flujo máximo y corte mínimo (teorema 20.14).

Preflujo, exceso y alturas

Se trabaja con una red de flujo $G = (V, E)$ como la de la sección 20, de donde $m \geq n - 1$. La red residual G_f es la de la definición 20.5, que solo usa $0 \leq f(u, v) \leq c(u, v)$ y vale tal cual para lo que sigue.

Definición 22.1 (Preflujo)**.** Un **preflujo** (*preflow*) es una función $f : V \times V \to \mathbb{R}$ que cumple la restricción de capacidad, $0 \leq f(u, v) \leq c(u, v)$, y la conservación relajada

$$\sum_{v \in V} f(v, u) - \sum_{v \in V} f(u, v) \;\geq\; 0 \qquad \text{para toda } u \in V - \{s\}\,.$$

El **exceso** (*excess flow*) de u es lo que entra menos lo que sale,

$$e(u) = \sum_{v \in V} f(v,u) - \sum_{v \in V} f(u,v).$$

Un vértice $u \in V - \{s,t\}$ con $e(u) > 0$ está **desbordado** (*overflowing*).

En un preflujo $e(u) \geq 0$ para todo $u \neq s$, y como los excesos suman cero, $e(s) \leq 0$. Por definición s y t nunca están desbordados, aunque t acumule exceso: ahí es justamente donde debe terminar el agua. Un preflujo sin vértices desbordados cumple conservación en todo $V - \{s,t\}$, es decir, es un flujo.

Definición 22.2 (Función de altura)**.** Una **función de altura** (*height function*) es una función $h : V \to \mathbb{N}$ con

$$h(s) = n, \qquad h(t) = 0, \qquad h(u) \leq h(v) + 1 \ \text{ para toda arista residual } (u,v) \in E_f.$$

La condición dice que una arista residual puede subir cuanto quiera, pero baja a lo más un nivel. Leída al revés: si $h(u) > h(v) + 1$, entonces (u,v) no es arista residual (Cormen, lema 26.12).

Las dos operaciones básicas

Empujar

Empujar(u,v) aplica cuando u está desbordado, $c_f(u,v) > 0$ y $h(u) = h(v) + 1$: hay holgura de u a v y v está exactamente un nivel abajo. Mueve $\Delta = \min\big(e(u), c_f(u,v)\big)$ unidades de u a v. Es un **empuje saturante** si después $c_f(u,v) = 0$, y entonces (u,v) desaparece de la red residual; si no, es **no saturante**.

Algoritmo 22.1 Empujar(u,v) (Push en Cormen)

```
1: procedimiento EMPUJAR(u, v)          // aplica si e(u) > 0, c_f(u,v) > 0 y h(u) = h(v) + 1
2:     Δ ← mín(e(u), c_f(u,v))
3:     si (u,v) ∈ E entonces
4:         f(u,v) ← f(u,v) + Δ          // arista original
5:     si no
6:         f(v,u) ← f(v,u) − Δ          // arista de regreso
7:     fin si
8:     e(u) ← e(u) − Δ,    e(v) ← e(v) + Δ
9: fin procedimiento
```

Δ es lo más que se puede mover sin dejar $e(u)$ negativo ni rebasar $c_f(u,v)$. Si $(u,v) \notin E$, la original es (v,u) y empujar de u a v cancela flujo que iba de v a u; sin la prohibición de antiparalelas, la línea 3 no decidiría nada. El procedimiento no revisa alturas: eso lo garantiza quien lo llama. Cuesta $\mathcal{O}(1)$.

Lema 22.3 (Cormen, lema 26.13)**.** *Después de un empuje no saturante de u a v, u ya no está desbordado.*

Idea de la prueba. Si el empuje no saturó, $\Delta \neq c_f(u,v)$, así que el mínimo lo dio $e(u)$: se empujó todo el exceso y $e(u)$ queda en 0. □

Reetiquetar

Reetiquetar(u) aplica cuando u está desbordado y $h(u) \leq h(v)$ para toda arista residual (u, v): tiene agua y no hay nada abajo hacia dónde empujarla.

Algoritmo 22.2 Reetiquetar(u) (Relabel en Cormen)

```
1: procedimiento Reetiquetar(u)        // aplica si e(u) > 0 y h(u) ≤ h(v) para toda (u, v) ∈ E_f
2:     h(u) ← 1 + mín {h(v) : (u, v) ∈ E_f}
3: fin procedimiento
```

La nueva altura es la mayor que permite la condición $h(u) \leq h(v) + 1$, y con ella queda admisible la arista al vecino que dio el mínimo. El mínimo no es sobre un conjunto vacío: como $e(u) > 0$, a u le entra flujo por alguna (v, u), y entonces $c_f(u, v) \geq f(v, u) > 0$. Cuesta $\mathcal{O}(\text{gr}(u))$ con listas.

Inicializar el preflujo y el algoritmo genérico

Algoritmo 22.3 Inicializar-Preflujo(G, s)

```
1: procedimiento Inicializar-Preflujo(G, s)
2:     para todo v ∈ V hacer
3:         h(v) ← 0,    e(v) ← 0
4:     fin para
5:     para todo (u, v) ∈ E hacer
6:         f(u, v) ← 0
7:     fin para
8:     h(s) ← n
9:     para todo v adyacente a s hacer
10:        f(s, v) ← c(s, v)                    // satura todo lo que sale de s
11:        e(v) ← c(s, v),    e(s) ← e(s) − c(s, v)
12:    fin para
13: fin procedimiento
```

Algoritmo 22.4 Empujar-Reetiquetar-Genérico(G) (Generic-Push-Relabel en Cormen)

```
1: procedimiento Empujar-Reetiquetar-Genérico(G)
2:     Inicializar-Preflujo(G, s)
3:     mientras existe una operación Empujar o Reetiquetar que aplique hacer
4:         elegir una cualquiera y ejecutarla
5:     fin mientras
6: fin procedimiento
```

Las líneas 9 a 11 de la inicialización saturan todas las aristas que salen de s, y por eso $h(s) = n$ es una altura válida: las únicas aristas con $h(u) > h(v) + 1$ salen de s, y ninguna es residual. El genérico no fija orden: cualquier secuencia de operaciones válidas termina en un flujo máximo, y el orden solo cambia cuántas operaciones se hacen y cuál flujo máximo se obtiene, nunca su valor.

Ojito...

- **Se empuja exactamente un nivel.** Empujar exige $h(u) = h(v) + 1$, no "cualquier vecino más bajo". De todos modos no existen aristas residuales que bajen más de un nivel.
- **s y t nunca se empujan ni se reetiquetan.** $h(s) = n$ y $h(t) = 0$ quedan fijas. $e(t)$ solo crece.
- **A media ejecución f no es un flujo**, así que hablar de su valor $|f|$ no tiene sentido. El valor del flujo final es $e(t) = -e(s)$.

El genérico en una red de cuatro vértices

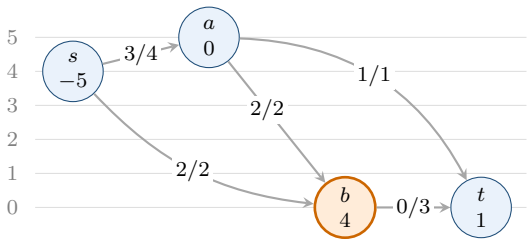
Ejemplo 22.4 (Empujar y reetiquetar genérico)**.** La red tiene $V = \{s, a, b, t\}$, $n = 4$, $m = 5$ y capacidades $c(s, a) = 4$, $c(s, b) = 2$, $c(a, b) = 2$, $c(a, t) = 1$, $c(b, t) = 3$. Su flujo máximo vale 4: el corte $(\{s, a, b\}, \{t\})$ tiene capacidad $1 + 3 = 4$. Como el genérico no fija orden, hace falta una regla para que la traza sea reproducible: *se atiende el desbordado que va primero en el orden a, b; si tiene una arista admisible (residual y un nivel abajo) se empuja por la primera en el orden $\langle s, b, t\rangle$ para a y $\langle s, a, t\rangle$ para b; si no tiene, se reetiqueta.*

Paso	Operación	$h(a)$	$h(b)$	$e(s)$	$e(a)$	$e(b)$	$e(t)$
0	Inicializar-Preflujo	0	0	-6	4	2	0
1	Reetiquetar(a): $0 \to 1$	**1**	0	-6	4	2	0
2	Empujar(a, b): 2, saturante	1	0	-6	2	4	0
3	Empujar(a, t): 1, saturante	1	0	-6	1	4	1
4	Reetiquetar(a): $1 \to 5$	**5**	0	-6	1	4	1
5	Empujar(a, s): 1, no saturante	5	0	-5	0	4	1
6	Reetiquetar(b): $0 \to 1$	5	**1**	-5	0	4	1
7	Empujar(b, t): 3, saturante	5	1	-5	0	1	4
8	Reetiquetar(b): $1 \to 5$	5	**5**	-5	0	1	4
9	Empujar(b, s): 1, no saturante	5	5	-4	0	0	4

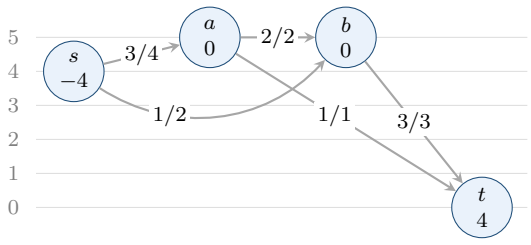
En el paso 1, las aristas residuales de a van a s, b y t, a alturas 4, 0 y 0; ninguna baja, así que se reetiqueta a $1 + \min\{4, 0, 0\} = 1$ y ya puede empujar 2 a b y 1 a t, saturando las dos. En el paso 4 su única arista residual es (a, s) y sube a 5, *arriba de la fuente*; en el paso 5 devuelve 1 a s sin saturar y, como dice el lema 22.3, se vacía. Los pasos 6 a 9 hacen lo mismo con b: sube a 1, manda 3 a t, sube a $1 + \min\{h(s), h(a)\} = 5$ y devuelve 1 a s.

En las instantáneas, la altura de cada vértice es su coordenada vertical, con líneas grises en cada nivel; dentro del círculo va el exceso, en cada arista el par f/c y, en naranja, los desbordados.

Paso 5



Paso 9 (final)



El resultado es $f(s, a) = 3$, $f(s, b) = 1$, $f(a, b) = 2$, $f(a, t) = 1$, $f(b, t) = 3$, con valor $e(t) = -e(s) = 4$.

Edmonds-Karp (algoritmo 20.2) resuelve la misma red con tres caminos completos. Empujar y reetiquetar nunca construyó un camino: mandó 6 unidades y devolvió a la fuente las 2 que no cabían.

Ojito...

Las alturas sí pasan de n. El exceso que no puede llegar a t solo regresa a s si los vértices que lo guardan rebasan $h(s)$: aquí a y b terminan en 5 con $n = 4$. El tope real es $2n - 1$, y se prueba más abajo.

Por qué el resultado es un flujo máximo

La corrección descansa en dos hechos: a un vértice desbordado siempre se le puede hacer algo, y una función de altura impide que haya camino de s a t en la red residual.

Lema 22.5 (Cormen, lema 26.14)**.** *Si f es un preflujo, h una función de altura y u está desbordado, entonces a u le aplica un empuje o un reetiquetado.*

Idea de la prueba. Toda arista residual (u, v) cumple $h(u) \leq h(v)+1$. Si ningún empuje aplica, ninguna cumple la igualdad, así que $h(u) \leq h(v)$ para todas ellas: es la condición de Reetiquetar. □

Lema 22.6 (Cormen, lemas 26.15 y 26.16)**.** *Durante el algoritmo genérico las alturas nunca bajan, cada reetiquetado sube $h(u)$ en al menos 1, y h es una función de altura en todo momento.*

Idea de la prueba. Antes de reetiquetar, $h(u) \leq h(v)$ para todo vecino residual, así que $h(u) < 1 + \min h(v)$: la altura sube. Que h sigue siendo función de altura se prueba por inducción. Al reetiquetar u, sus aristas de salida cumplen por la elección del mínimo y las de entrada porque $h(u)$ solo subió. Empujar de u a v puede crear (v, u), que cumple porque $h(v) = h(u) - 1$. □

Lema 22.7 (Cormen, lema 26.17)**.** *Si f es un preflujo y h una función de altura, no existe camino de s a t en la red residual G_f.*

Demostración. Supóngase que hay uno, $\langle v_0, v_1, \ldots, v_k \rangle$ con $v_0 = s$ y $v_k = t$. Se puede tomar simple, así que tiene a lo más n vértices y $k \leq n - 1$ aristas. Cada (v_i, v_{i+1}) es arista residual, luego $h(v_i) \leq h(v_{i+1}) + 1$. Sumando las k desigualdades, los términos intermedios se cancelan y queda

$$h(s) \;\leq\; h(t) + k \;=\; k \;\leq\; n - 1,$$

lo que contradice $h(s) = n$. □

Teorema 22.8 (Corrección del genérico; Cormen, teorema 26.18)**.** *Si el algoritmo genérico (algoritmo 22.4) termina, el preflujo que calcula es un flujo máximo.*

Demostración. El invariante es "f es un preflujo". Vale tras la inicialización. Reetiquetar no toca f. Empujar lo conserva: $\Delta \leq c_f(u, v)$ respeta la capacidad y $\Delta \leq e(u)$ deja $e(u) \geq 0$. Al terminar no queda ningún vértice desbordado, porque si hubiera uno le aplicaría alguna operación (lema 22.5) y el ciclo no habría terminado. Entonces $e(u) = 0$ en todo $V - \{s, t\}$ y f es un flujo. Por el lema 22.6, h sigue siendo función de altura, y por el lema 22.7 no hay camino aumentante en G_f. Por el teorema 20.14, f es un flujo máximo. □

Falta ver que termina. Eso sale de las cotas de la siguiente subsección, que no dependen de las capacidades: el genérico termina siempre, incluso con capacidades irracionales, caso en el que Ford-Fulkerson con caminos arbitrarios puede no terminar.

Cuánto cuesta

Todo el análisis cuelga de un tope para las alturas, y el tope sale de que todo exceso vino de s y tiene por dónde regresar.

Lema 22.9 (Cormen, lema 26.19)**.** *Si x está desbordado, hay un camino simple de x a s en G_f.*

Idea de la prueba. Si el conjunto U de alcanzables desde x no contiene a s, la suma de los excesos de U, que es positiva, es lo que entra a U desde fuera menos lo que sale. Entonces entra flujo por alguna (v', u') con $v' \notin U$, y la arista de regreso (u', v') es residual: v' sería alcanzable. □

Lema 22.10 (Tope de las alturas; Cormen, lema 26.20)**.** *En todo momento $h(u) \leq 2n-1$ para todo $u \in V$.*

Idea de la prueba. Un vértice interno solo sube al reetiquetarse, y entonces está desbordado. Por el lema 22.9 hay un camino simple de u a s con a lo más $n-1$ aristas, y la misma suma del lema 22.7 da $h(u) \leq h(s) + n - 1 = 2n - 1$. □

Corolario 22.11 (Reetiquetados; Cormen, corolario 26.21)**.** *Cada vértice se reetiqueta a lo más $2n-1$ veces, y hay a lo más $(2n-1)(n-2) < 2n^2$ reetiquetados en total.*

Lema 22.12 (Empujes saturantes; Cormen, lema 26.22)**.** *El algoritmo genérico hace menos de $2nm$ empujes saturantes.*

Idea de la prueba. Tras un saturante de u a v queda $h(v) = h(u) - 1$, y para volver a empujar de u a v primero hay que empujar de v a u, lo que exige que $h(v)$ suba al menos 2. Entre dos saturantes consecutivos del par, uno de los dos vértices sube al menos 2 niveles; con alturas hasta $2n-1$, el par admite menos de $2n$, y los pares que cuentan son las m aristas de E. □

El último conteo domina, y su prueba usa una **función potencial**: un número que nunca es negativo, que algunas operaciones suben y otras bajan, y cuya subida total acota cuántas veces baja.

Lema 22.13 (Empujes no saturantes; Cormen, lema 26.23)**.** *El algoritmo genérico hace menos de $4n^2(n+m)$ empujes no saturantes.*

Idea de la prueba. Sea Φ la suma de las alturas de los vértices desbordados, que empieza en 0 y nunca es negativa. Los reetiquetados la suben en total menos de $2n^2$, y cada uno de los menos de $2nm$ saturantes la sube menos de $2n$. Un no saturante de u a v saca a u de la suma y a lo más mete a v, con $h(v) = h(u) - 1$: baja Φ en al menos 1. Hay menos de $2n^2 + 4n^2m \leq 4n^2(n+m)$. □

Teorema 22.14 (Cormen, teorema 26.24 y corolario 26.25)**.** *El algoritmo genérico hace $\mathcal{O}(n^2m)$ operaciones básicas y se puede implementar en tiempo $\mathcal{O}(n^2m)$.*

Derivación. Los reetiquetados son menos de $2n^2$ y cada uno recorre los vecinos de u; como u se reetiqueta a lo más $2n-1$ veces, el total es $\sum_u (2n-1)\,\mathrm{gr}(u) = (2n-1)\cdot 2m = \mathcal{O}(nm)$. Los empujes saturantes son menos de $2nm$ y los no saturantes menos de $4n^2(n+m)$, que es $\mathcal{O}(n^2m)$ porque $m \geq n-1$; cada uno cuesta $\mathcal{O}(1)$. Para elegir la operación, los desbordados van en una pila o una cola, y el vértice de turno lleva un apuntador al vecino que revisa, como en Descargar (algoritmo 22.5), que recorre su lista una vez entre dos reetiquetados: otra vez $\mathcal{O}(nm)$. Total: $\mathcal{O}(nm + nm + n^2m + nm) = \mathcal{O}(n^2m)$. Domina el conteo de empujes no saturantes, y ninguna cuenta depende de las capacidades. □

Reetiquetar al frente

El genérico deja libre el orden de las operaciones, y el orden importa. **Reetiquetar al frente** (*relabel-to-front*) fija uno: atiende cada vértice hasta vaciarlo, y los toma de una lista L de los vértices internos que se recorre de principio a fin; cada vez que un vértice sube de altura pasa al frente de la lista y el recorrido sigue detrás de él.

Aristas admisibles y la operación Descargar

Definición 22.15 (Arista admisible)**.** Con un preflujo f y una función de altura h, una arista (u, v) es **admisible** (*admissible*) si $c_f(u, v) > 0$ y $h(u) = h(v) + 1$; las demás son **inadmisibles**. La **red admisible** es $G_{f,h} = (V, E_{f,h})$, con $E_{f,h}$ el conjunto de aristas admisibles.

Las aristas admisibles son exactamente aquellas por las que se puede empujar.

La **lista de vecinos** $N(u)$ de un vértice interno contiene cada v con $(u, v) \in E$ o $(v, u) \in E$: los posibles extremos de una arista residual que sale de u, en un orden arbitrario pero fijo. El apuntador actual(u) señala el vecino que se está revisando; empieza en el primero y vale nulo cuando se pasa del último. **Descargar** un vértice desbordado es empujar todo su exceso por aristas admisibles, reetiquetándolo cuando haga falta.

Algoritmo 22.5 Descargar(u) (Discharge en Cormen)

```
1:  procedimiento DESCARGAR(u)
2:      mientras e(u) > 0 hacer
3:          v ← actual(u)
4:          si v = nulo entonces                          // se recorrió toda N(u)
5:              REETIQUETAR(u)
6:              actual(u) ← primer vértice de N(u)
7:          si no si c_f(u, v) > 0 y h(u) = h(v) + 1 entonces
8:              EMPUJAR(u, v)                             // (u, v) es admisible
9:          si no
10:             actual(u) ← vértice que sigue a v en N(u)
11:         fin si
12:     fin mientras
13: fin procedimiento
```

Cada vuelta del mientras hace una de tres cosas. Si v es nulo (líneas 4 a 6), toda la lista resultó inadmisible: se reetiqueta u y el apuntador regresa al inicio. Si (u, v) es admisible (líneas 7 y 8), se empuja y el apuntador *no* avanza: si no saturó, $e(u) = 0$ y la llamada termina; si saturó, la vuelta siguiente la ve inadmisible. Si no (línea 10), se avanza. actual(u) se conserva entre llamadas: la siguiente descarga de u sigue donde se quedó la anterior.

Lema 22.16 (Cormen, lema 26.26)**.** *La red admisible $G_{f,h}$ es acíclica.*

Idea de la prueba. A lo largo de un ciclo de aristas admisibles la altura bajaría exactamente 1 en cada arista y volvería al valor inicial: sumando, $0 = k$ con $k > 0$. □

Lema 22.17 (Cormen, lemas 26.27 y 26.28)**.** *1. Empujar de u a v no crea aristas admisibles, aunque puede quitar (u, v).*

2. *Si u está desbordado y no tiene aristas admisibles de salida,* Reetiquetar(u) *aplica. Después, u tiene al menos una arista admisible de salida y ninguna de entrada.*

Idea de la prueba. (1) La única arista residual nueva es (v, u), que sube. (2) Aplica por el lema 22.5, y la arista hacia el vecino del mínimo queda admisible. Una admisible de entrada (v, u) pediría $h(v) = h(u) + 1$ con la altura nueva, así que antes era $h(v) > h(u) + 1$ y (v, u) no era residual; reetiquetar no cambia la red residual. □

Lema 22.18 (Cormen, lema 26.29)**.** *Cuando* Descargar(u) *llama a* Reetiquetar(u) *en la línea 5, el reetiquetado aplica.*

Idea de la prueba. Basta ver que u no tiene aristas admisibles de salida (lema 22.17). En la pasada completa por $N(u)$ que acaba de terminar, cada arista de salida se encontró inadmisible, y ninguna pudo volverse admisible después: empujar no crea admisibles, y un vértice v que se reetiquetó no tiene admisibles de entrada, en particular no tiene (u, v). □

El algoritmo

Algoritmo 22.6 Reetiquetar-al-Frente(G, s, t) (Relabel-To-Front en Cormen)

```
1:  procedimiento Reetiquetar-al-Frente(G, s, t)
2:      Inicializar-Preflujo(G, s)
3:      L ← V − {s, t} en cualquier orden
4:      para todo u ∈ V − {s, t} hacer
5:          actual(u) ← primer vértice de N(u)
6:      fin para
7:      u ← primer vértice de L
8:      mientras u ≠ nulo hacer
9:          h_antes ← h(u)
10:         Descargar(u)
11:         si h(u) > h_antes entonces                    // u se reetiquetó
12:             mover u al frente de L
13:         fin si
14:         u ← vértice que sigue a u en L
15:     fin mientras
16: fin procedimiento
```

L tiene solo los vértices que pueden desbordarse. h_{antes} sirve para saber si Descargar reetiquetó a u, y si lo hizo, u pasa al frente. La línea 14 sigue con el sucesor de u en su posición *actual*: si u acaba de pasar al frente, el recorrido vuelve a pasar por todos los que quedaron detrás de él.

Teorema 22.19 (Invariante y corrección de reetiquetar al frente)**.** *Cada vez que se evalúa la condición de la línea 8, L es un orden topológico de la red admisible $G_{f,h}$ y ningún vértice anterior a u en L tiene exceso. En consecuencia, al terminar no hay vértices desbordados y f es un flujo máximo.*

Idea de la prueba. $G_{f,h}$ es acíclica (lema 22.16) y al inicio no tiene aristas. Empujar no crea admisibles, y reetiquetar solo crea las de salida del vértice reetiquetado, que se queda sin admisibles de entrada (lema 22.17): ponerlo al frente conserva el orden topológico. Descargar empuja solo hacia adelante en L, así que los anteriores a u siguen vacíos, y el teorema 22.8 concluye. □

Por qué cuesta $\mathcal{O}(n^3)$

Se llama **fase** al tramo de ejecución entre dos reetiquetados consecutivos (Cormen, teorema 26.30).

1. *Llamadas a Descargar.* Hay $\mathcal{O}(n^2)$ fases. Dentro de una, una descarga que no reetiqueta deja a u en su lugar y la siguiente es sobre un vértice más adelante en L, así que una fase tiene a lo más $n-1$ llamadas: $\mathcal{O}(n^3)$ en total.
2. *Reetiquetados y avances del apuntador.* $\mathcal{O}(nm)$, con la cuenta del teorema 22.14: entre dos reetiquetados de u, actual(u) recorre $N(u)$ una vez.
3. *Empujes saturantes.* $\mathcal{O}(nm)$ (lema 22.12).
4. *Empujes no saturantes.* Uno no saturante deja $e(u) = 0$ y termina la llamada, así que hay a lo más uno por llamada: $\mathcal{O}(n^3)$.

El total es $\mathcal{O}(n^3 + nm)$, que es $\mathcal{O}(n^3)$ porque sin antiparalelas $m \leq n(n-1)/2$. Como $m \geq n-1$, $\mathcal{O}(n^3)$ nunca es peor que $\mathcal{O}(n^2 m)$: empatan en grafos dispersos y reetiquetar al frente gana en densos. La mejora está toda en el último renglón: el orden de L topa los empujes no saturantes en uno por llamada.

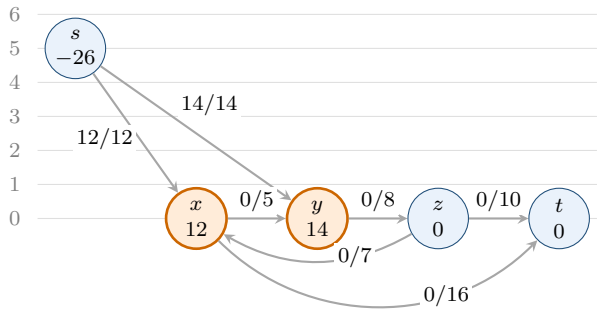
Ojito...

En reetiquetar al frente el apuntador actual(u) **no se reinicia** entre llamadas a Descargar. Si se reiniciara en cada llamada el algoritmo seguiría siendo correcto, pero se perdería la cuenta de avances que da el $\mathcal{O}(nm)$: cada llamada podría volver a recorrer la lista completa.

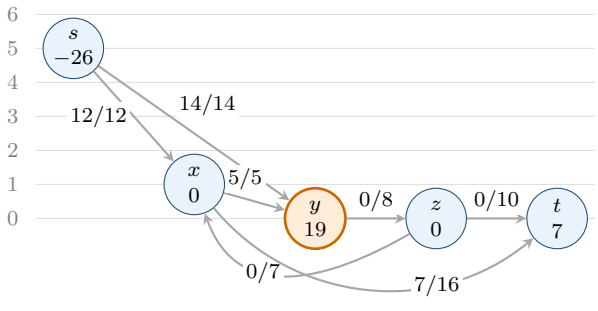
La figura 26.10 de Cormen

Ejemplo 22.20 (Reetiquetar al frente paso a paso)**.** La red tiene $V = \{s, x, y, z, t\}$, $n = 5$ y capacidades $c(s,x) = 12$, $c(s,y) = 14$, $c(x,y) = 5$, $c(y,z) = 8$, $c(z,x) = 7$, $c(z,t) = 10$, $c(x,t) = 16$. Su flujo máximo vale 20. Las listas de vecinos son las de Cormen: $N(x) = \langle s, y, z, t\rangle$, $N(y) = \langle s, x, z\rangle$, $N(z) = \langle x, y, t\rangle$, y la lista inicial es $L = \langle x, y, z\rangle$. En las instantáneas, los desbordados van en naranja.

(a) inicio, $L = \langle x, y, z\rangle$



(b) tras Descargar(x)



De (a) a (b). Se descarga x. No tiene admisibles, sube a 1, empuja 5 a y (satura (x,y)) y 7 a t. Se reetiquetó, así que pasa al frente, donde ya estaba; sigue y. **Después.** Se descarga y con 19 unidades: sube a 1, manda 8 a z, sube a 2, devuelve 5 a x cancelando el flujo de (x,y), sube a 6 (arriba de s) y devuelve 6 a la fuente; pasa al frente. Sigue x, que recibió 5: su apuntador se había quedado en t, y (x,t) es admisible, así que empuja 5 a t sin reetiquetarse. Sigue z con 8: sube a 1, manda 8 a t y pasa al frente. Se recorren y y x sin exceso y la lista se acaba, con flujo de valor 20: $f(s,x) = f(x,t) = 12$ y $f(s,y) = f(y,z) = f(z,t) = 8$. Hubo 5 reetiquetados, 3 empujes saturantes y 4 no saturantes. El

estado tras cada llamada a DESCARGAR, con los renglones nombrados como los paneles de la figura 26.10 de Cormen (arriba se dibujan los dos primeros):

	Descarga	**Reetiq.**	L después	$h(x)$	$h(y)$	$h(z)$	$e(s)$	$e(x)$	$e(y)$	$e(z)$	$e(t)$
(a)	inicio		x, y, z	0	0	0	-26	12	14	0	0
(b)	x	sí, $0 \to 1$	x, y, z	1	0	0	-26	0	19	0	7
(c)	y	sí, $0 \to 6$	y, x, z	1	6	0	-20	5	0	8	7
(d)	x	no	y, x, z	1	6	0	-20	0	0	8	12
(e)	z	sí, $0 \to 1$	z, y, x	1	6	1	-20	0	0	0	20
	y, luego x	no	z, y, x	1	6	1	-20	0	0	0	20

Resumen de la parte de grafos

La tabla junta todos los algoritmos de la parte, en el orden en que aparecen, y leída de arriba abajo cuenta la historia de la parte. Los recorridos lineales, BFS y DFS, son la base de casi todo lo demás: el orden topológico, las componentes, los puentes, el coloreo bipartito, los caminos en un DAG, la cerradura transitiva por recorridos y la búsqueda de caminos aumentantes. Donde aparece un logaritmo hay una cola de prioridad o un ordenamiento; donde hay pesos negativos aparece Bellman-Ford; y el único renglón exponencial, Held-Karp, es el de un problema NP-difícil.

Tres convenciones de la tabla. "Red de flujo" quiere decir grafo dirigido con capacidades $c \geq 0$, fuente y sumidero, sin aristas antiparalelas (sección 20). En restricciones de diferencia, p es el número de incógnitas y r el de restricciones, para no confundirlos con n y m. Los costos son con listas de adyacencia salvo que se diga otra cosa.

Algoritmo	Problema	Requisitos	Estructura	Costo	Secc.
Recorridos y estructura del grafo					
BFS	recorrido por capas; distancias en aristas	ninguno; sin pesos	cola	$\mathcal{O}(n+m)$; $\mathcal{O}(n^2)$ con matriz	14
DFS	recorrido en profundidad; tiempos y tipos de arista	ninguno	pila o recursión	$\Theta(n+m)$	14
Orden topológico con DFS	toda arista hacia adelante	dirigido acíclico (DAG)	DFS e inserción al frente	$\Theta(n+m)$	15
Kahn	lo mismo, quitando vértices sin entrada	dirigido; detecta ciclos	cola y arreglo gr^-	$\Theta(n+m)$	15
Componentes fuertemente conexas	clases de alcanzabilidad mutua	dirigido	dos DFS y grafo transpuesto	$\Theta(n+m)$	15
Puntos de articulación y puentes	vértices y aristas que desconectan	no dirigido, conexo	DFS con entra$[v]$ y low$[v]$	$\Theta(n+m)$	15
Hierholzer	circuito o camino euleriano	conexo; grados pares, o dos impares y salir de uno	pila y apuntador por lista	$\Theta(n+m)$	12
Coloreo bipartito con BFS	decidir si es bipartito; las dos partes	no dirigido	BFS con color por vértice	$\Theta(n+m)$	13
Held-Karp	agente viajero: ciclo hamiltoniano mínimo	completo y pesado; n chico	tabla $C[S][j]$ por subconjuntos	$\Theta(n^2 2^n)$ tiempo, $\Theta(n2^n)$ memoria	13
Árboles de expansión mínima					
Kruskal	árbol de expansión de peso mínimo	no dirigido, conexo, pesado	ordenamiento y conjuntos disjuntos	$\mathcal{O}(m \log n)$	16
Prim	lo mismo, creciendo un solo árbol	no dirigido, conexo, pesado	montículo con decrementar llave	$\mathcal{O}(m \log n)$ binario; $\mathcal{O}(m + n \log n)$ Fibonacci; $\Theta(n^2)$ arreglo	16

Algoritmo	Problema	Requisitos	Estructura	Costo	Secc.
Caminos más cortos					
Bellman-Ford	desde una fuente; detecta ciclos negativos	dirigido, pesos de cualquier signo	arreglos d y π; $n-1$ pasadas	$\mathcal{O}(nm)$	17
Caminos en un DAG	desde una fuente	DAG, pesos de cualquier signo	orden topológico	$\Theta(n+m)$	17
Dijkstra	desde una fuente	pesos no negativos	montículo de mínimos	$\mathcal{O}((n+m)\log n)$ binario; $\mathcal{O}(n^2)$ arreglo; $\mathcal{O}(n\log n+m)$ Fibonacci	17
Restricciones de diferencia	resolver $x_j - x_i \leq b$ o probar que no hay solución	dos incógnitas por restricción	Bellman-Ford en el grafo de restricciones	$\mathcal{O}(p^2+pr)$	17
Floyd-Warshall	entre todos los pares	sin ciclos negativos	matriz $n \times n$	$\Theta(n^3)$	19
Cerradura transitiva	alcanzabilidad entre todos los pares	dirigido	matriz booleana	$\Theta(n^3)$; $\Theta(n(n+m))$ con n recorridos	19
Johnson	entre todos los pares, grafos dispersos	sin ciclos negativos	Bellman-Ford y n veces Dijkstra	$\mathcal{O}(nm\log n)$ binario; $\mathcal{O}(n^2\log n+nm)$ Fibonacci	19
Flujo y emparejamiento					
Ford-Fulkerson	flujo máximo	red de flujo; capacidades enteras	red residual; BFS o DFS	$\mathcal{O}(m\,\lvert f^*\rvert)$	20
Edmonds-Karp	flujo máximo	red de flujo	red residual y BFS	$\mathcal{O}(nm^2)$	20
Emparejamiento bipartito por flujo	emparejamiento máximo	bipartito	red de capacidades 1 y Ford-Fulkerson	$\mathcal{O}(nm)$	21
Hopcroft-Karp	emparejamiento máximo	bipartito	BFS por capas y DFS	$\mathcal{O}(\sqrt{n}\,m)$	21
Empujar y reetiquetar genérico	flujo máximo	red de flujo	alturas, excesos y pila de desbordados	$\mathcal{O}(n^2m)$	22
Reetiquetar al frente	flujo máximo	red de flujo	lista L y listas de vecinos	$\mathcal{O}(n^3)$	22